VU2CPL Shack Changelog

Fixes and changes to non-DXCC tabs of the Node-RED shack automation (SPE amplifier, Power Control, Solar Conditions, Lightning, Rotator, etc.).

The DXCC Tracker has its own doc: see `DXCC.md` / `DXCC_Tracker_README.pdf`. For the umbrella overview of every subsystem in this repo, see `README.md`.

2026-10-15

RBN Skimmer Monitor: skimmer calibration history, per exact SSID (`ccaef77` , `9ed8512`)

Manoj: "why is skew information missing?" `rbnskew.csv` holds only sm7iun's newest day, and VU2OY-9 and VU2CPL-88 had no value that day, although sm7iun rated them a day or two earlier (VU2OY-9 +15.3 ppm on 10-13, VU2CPL-88 +0.0 on 10-12). The lookup was right to find nothing under those calls. Proposals to borrow the plain `VU2OY` row, or strip the SSID to find a base call, were rejected: **every call+SSID is its own receiver and must never be mixed** (Manoj). VU2OY sits near +1 ppm and VU2OY-9 near +14 ppm; they are two rigs. That rule is now written into CLAUDE.md under the RBN node table.

What changed:

- **History source.** New `GET rbnhist.txt` + `Parse Skew History`, on the same 6-h `Fetch Cal` inject as the CSV. `https://sm7iun.se/rbnhist.txt` is sm7iun's plain-text 10-day table, the same one shown in a `<pre>` block on `sm7iun.se/rbn/analytics/`. It's parsed by header column position. Local skimmers (Indian prefixes plus our three) are kept per day in `flow.skew_hist` (file scope) for 30 days, so the history outgrows sm7iun's window.
- **Tiles.** Skew is the SSID's own last value, with "Nd old" when it isn't from sm7iun's newest day; the CSV's spot count and correction factor appear only for a current value. Today: VU2OY-9 +15.3 ppm, 1 d old; VU2CPL-88 +0.0, 2 d old; VU24DX-1 −0.3, current, 16 spots.
- **Skimmer calibration section** on both dashboards (D1 `<details open>`, panel height 12 → 16; Vue v46, open state in `localStorage`). One row per local SSID, ours highlighted: a sparkline on a shared ±3 ppm scale (values beyond it pinned to the edge, the number shows the real value), a solid zero line, dotted ±1.5 ppm guides where amber turns red, gaps on days without data, and the last value with its age. Today it lists VU24DX-1, VU2CPL-88, VU2OY, VU2OY-9, VU2PTT and VU2TUM.
- **`Parse DX Spot`** strips only the trailing `-#` skimmer marker; it used to strip `-<digits>` too. Nothing on the RBN tab reads `spot.spotter`, so nothing displayed changed.
- **VU24DX-1** confirmed by Manoj as VU24DX's only RBN receiver, so its tag stays even though its telnet port prints `VU24DX-#`. Tags therefore stay per feed in the login handlers rather than being read off the spot.

The idea of tracking every local skimmer's calibration over time comes from **VU2PTT's Skimmer Calibration tracker**, a Node-RED flow that scrapes the analytics page's third `<pre>` block and plots `VU2*` calls on a D1 chart. This version reads the plain-text table, covers all Indian prefixes, and doesn't clip: VU2PTT's fixed ±3 ppm chart axis would put VU2OY-9 off the top.

Verified: stored history and MQTT fields on the Pi after deploy. Both dashboards were rendered locally from the real `index.js` and D1 template against today's data, at desktop and 375 px phone width, with no horizontal overflow; `/shack` itself needs a login, so it wasn't screenshotted live. The first cut used 22 px sparklines and read nearly flat (1 ppm under 3 px), so it went to 34 px with the guides (`9ed8512`).

Found on the way (open, HANDOVER #52): the raw-chunk login handlers drop a spot line split across two TCP chunks, about 0.06% of spots on VU24DX and VU2OY.

RBN Skimmer Monitor: VU2OY needs a login now; retagged `VU2OY-9` (`a5c0325`)

Manoj: "vu2oy in dashboard is not updating". The VU2OY tile had been offline since its last spot at 18:59Z on 2026-10-14 (00:29 IST on the 15th). VU2CPL-88 and VU24DX-1 were fine.

Cause. VU2OY (Ranjith, Trivandrum MJ88KN) changed what answers on `vu2oy.ddns.net:7550`. It used to be an open feed that streamed spots without a login. It is now a **Meridian 0.1.25** cluster server that sends a banner and stops at `Please enter your callsign:` (no trailing newline) until it gets one. Our side had two gaps. `Telnet VU2OY` split its input on `\n`, so it buffered the unterminated prompt and never emitted it. And `Login Handler VU2OY` was a bare `DX de` filter with no login path and no reply node. The connection stayed open and silent.

Fix (same shape as VU24DX, added 2026-10-07):

- `Telnet VU2OY :7550`: `newline` `\n` → `''` (raw chunks).
- `Login Handler VU2OY`: now a copy of `Login Handler VU24DX`. It detects the prompt on the last non-empty line and logs in as `VU2CPL-1`, and has 2 outputs.
- New `tcp out` `Reply to VU2OY (login)` (`rbn_vu2oy_tcp_reply`, `beserver:reply`).
- Spots are tagged **`VU2OY-9`** on Manoj's call, matching the feed's `VU2OY-9-#`. The `sk2_call` default in Parse Calibration CSV, Watchdog Check, RBN State Aggregator and Update Spot Counters follows.
- HA: `rbn_vu2oy_h1` (entity id unchanged) reads `skimmers['VU2OY-9']`, renamed "VU2OY-9 spots (1h)"; discovery republished. The Radio dashboard doesn't reference that sensor, so no card change.

Verified after `git pull` + Node-RED restart on the Pi: logged in on the first connection, `VU2OY-9` online with 35 spots ten seconds later; HA sensor read 36.

Side effects. sm7iun's `rbnskew.csv` lists plain `VU2OY` (+1.5 ppm, 44 spots) but has no `VU2OY-9` row for its newest day, so the skew cell read "not found". `VU2OY-9` is a separate receiver, not a rename, so the `VU2OY` row is not its skew; the fix is the calibration history in the next entry. The old `VU2OY` hourly buckets are no longer read and age out within 10 days, so 12H/24H restart from zero. The DXCC tab's VU2OY cluster (tab disabled) needs nothing: it already reads raw chunks, and `login-parse-dedup-v2` matches `enter your callsign:`.

restic: second copy of the backups on the ubersdr box's 1 TB HDD (#50)

Manoj's decision for `.109`'s two hard disks: the healthy 1 TB Seagate (`sda`, 1.3 years powered on) holds a second copy of the backups, and the 500 GB WD (`sdb`, ~9.9 years, the old Ubuntu fallback) was wiped and reused as `/mnt/scratch`. New `restic/restic-copy.sh` runs on the Mac daily at 05:30 and copies both NAS repositories to `/mnt/backups/restic/<host>` over sftp, with the same passwords and `--copy-chunker-params`. On Sundays it also prunes (same 7/5/12 policy) and runs a check. It reports on retained `shack/backup/copy`, and shack-health grades it (`16c9e24`). `install-maint.sh` now installs both Mac agents, both under `caffeinate -i`. First seed: 2026-10-15 from 20:44, 45-55 MB/s.

Off-site decided the same evening (Manoj): the ALPA hosting server, once ALPA's files are moved off it, expected in a month or two. Nothing is built for it yet.

restic: first test restore passed (#50)

Restored to scratch targets only and compared with the live files by sha256:

- **`.109`**, snapshot `b62c8459` (01:33), restored on the box: `/etc`, `~/openwebrx`, `~/ubersdr` and `/var/backups/restic-dumps` (2,153 files, `--verify` clean). The wavelog dump matches live and decompresses complete. 6 files differed from live, all changed after 01:33 (cups, `ld.so.cache`, the Caddy cert record, `receivers.json`).
- **macmini-server**, snapshot `13c012c5` (03:01), restored on the box: `/etc` identical; the ALPA replica dump and the 2026-10-14 Immich DB dump (560 MB, 72 `CREATE TABLE`) match live and decompress complete.
- **From the Mac**, as if the box were gone (`restic-maint macmini-server -- restore`): 25 random Immich originals, 101 MB, 25/25 sha256 OK.

Note: Immich originals live under `/mnt/immich/upload`; `library/` holds a single file.

restic: rest-server survives a NAS reboot (#50 b)

Manoj restarted the DS710+ from DSM at 20:31. The root boot hook `/usr/local/etc/rc.d/S99restic-rest-server.sh` brought both endpoints back by 20:33, running as `admin`, with `:9900` still append-only. `.109` and macmini-server (`:9900`) and the Mac (`:9901`) all listed their snapshots straight away. Since the reboot the NAS no longer answers ping from the LAN, though SSH, DSM and the rest-server ports are fine. Nothing in the shack pings it, so this was left alone.

2026-10-14

AS3935 lightning sensor: three firmware releases in a day (v0.4.1 → v0.4.3)

The bridge's MQTT link flapped from 04:31 to 12:14 (861 keepalive timeouts), the second time after 2026-10-10; a `reboot` command sent through HA's `mqtt.publish` straight after a fresh `ready` fixed it. Firmware changes, all in [`vu2cpl-as3935-bridge`]:

- **v0.4.1** (USB 13:54): restart after 5 MQTT reconnects in 10 min (`boot_cause: mqtt_churn`); `boot_cause` in status (it caught a brownout when the USB cable was pulled at 13:57); disturber/noise events throttled to one a minute with a `count`; rates and reconnects in the heartbeat; HTTP OTA opened by `ota_open`, with bootloader rollback (`ota.sh`); `modem_sleep: none`, now set.
- **v0.4.2** (OTA): `wifi_off_test`. With the radio fully off the chip still counted 398 disturbers/min (405 and 403 with it on). An earlier A/B/A run (throttle on / every event published / on) gave 397.3 / 397.4 / 402.3 per min. So neither the MQTT traffic nor the WiFi radio makes the steady ~400/min; source unknown, hunt parked (the battery check needs a cell that's in poor shape).
- **v0.4.3** (OTA 17:59): a latched IRQ line is released after 1 s; 5 min with no IRQ re-inits the chip and 10 min restarts it (`irq_silence`). That covers the morning's other symptom: the counter froze with heartbeats still arriving.

Shack side: D1 Master Dashboard's Modem sleep dropdown gains `none` (`362745e`).

Home Assistant: TP-Link cameras reload themselves

Entrance, South boundary and Utility area went unavailable 08:13–09:35 with python-kasa `-40105` while their video kept streaming. Camera reboots didn't help; reloading the config entries did, but South boundary dropped again at 14:58. New `automation.reload_tp_link_camera_when_unavailable` reloads a camera after 5 min unavailable, plus a half-hourly sweep; first run brought South boundary back in 5 s. Documented in CLAUDE.md (HassPi section), since HA config isn't in git.

OpenWebRX RSP2: third USB drop; now on the Renesas card

RSP2 fell off Intel port `1-5` at 04:18 (also 09-28 and 10-09). After the replug, a public listener's dead connector blocked `openwebrx-restart.sh` for its full 10 min; it now skips the wait when `openwebrx-watch.sh --stream` confirms the stream dead twice. RSP2 moved to the Renesas card `3-2.1` with the same cable, so a further drop points at the RSP2/cable. Details in the `ubersdr-box` HANDOVER.

2026-10-09

RustDesk: two faults behind "can't connect to .170 / .109"

- **Leftover public-server login (Mac Mini).** Logging in with Google/GitHub while on the public server (2026-10-08) left `access_token` in `RustDesk_local.toml` after switching back. With a token set, RustDesk 1.5.0 waits for an encrypted TCP key exchange from the ID server. The free `hbbs` 1.1.16 never sends one, so every connection the Mac started through the Oracle server failed after ~18 s with `Failed to secure tcp: deadline has elapsed`. Incoming connections were unaffected. Fixed by Account → Log out. Restarting `hbbs` (tried first) made no difference. `rd-mac-switch.sh oracle` now clears the login itself, and `status` prints a `login:` line. The updated script is on both the Mac Mini and the MacBook Air.
- **`.170` key missing its trailing `=`** in both the user and LocalService `RustDesk2.toml`. `hbbs` doesn't check the key at registration, so `.170` showed online and LAN-direct connections from the Mac Mini worked. But `hbbr` rejected every relay join (`Relay authentication failed - invalid key`), so off-LAN clients such as the MacBook on mobile data timed out. Fixed over SSH: service stopped, both files backed up as `.bak-20261009-042603`, `=` added, service started. The MacBook then connected.
- **Verified:** Manoj switched both Macs to public and back to Oracle. Both stuck, and the MacBook then reached the Mac Mini through the Oracle relay. The token-clearing step is not yet tested live (no login during the test).
- **Diagnosing next time:** `sudo docker logs --since 20m hbbr` on the Oracle VM. `invalid key` from the shack's WAN IP means the TARGET host's key is wrong. Details in `~/projects/oracle-rustdesk/HANDOVER.md`; the server address and key stay out of this public repo.

2026-10-07

RBN Skimmer Monitor: VU24DX added as a third skimmer (`fa08ff2`)

- **The feed:** VU24DX (Adersh, Kozhikode MK71UH) runs a Meridian 0.1.25 cluster server on **`vu24dx.ddns.net:7300`**. Of the ports suggested, 7550 is refused and 7330 is filtered. It asks for a callsign with an unterminated `Please enter your callsign:` prompt (CwSkimmer style), so the new **Login Handler VU24DX** is a copy of the VU2CPL one and logs in as `VU2CPL-1`. It sends about 60–70 spots a minute at 02:00Z, nearly all FT8, and the existing Parse DX Spot regexes read every sampled line.
- **Named `VU24DX-1`:** the feed tags spots `VU24DX-#`, but the box uploads to RBN as `VU24DX-1` (seen on `telnet.reversebeacon.net:7001`), and that is its row in sm7iun's `rbnskew.csv` (−0.9 ppm). Tagging it `VU24DX-1` makes the skew lookup match, the same way VU2CPL-88 is keyed by its RBN call.
- **New nodes:** `rbn_vu24dx_tcp_in` (`tcp in`, raw chunks), `rbn_vu24dx_login` (2 outputs: login reply, spots → Parse DX Spot) and `rbn_vu24dx_tcp_reply` (`tcp out`, `beserver: reply`). `sk3_call` (default `VU24DX-1`) joins `sk1` / `sk2` in Parse Calibration CSV, Update Spot Counters, Watchdog Check and the RBN State Aggregator (`st.sk3`), and rides the Vue builder and `shack/rbn/state`.

- **1h counts now come from per-minute buckets.** They used to filter the 10,000-spot `spots_all` ring, but VU2CPL-88 and VU2OY already averaged about 13,700 spots/h together over 24 h (5,270 + 8,400), so the ring covered less than an hour at peak and the counts came out low. VU24DX adds about 3,700/h. Update Spot Counters keeps `minute_buckets` (memory scope, last 60 minutes) and drops three full ring scans per spot. 12h and 24h still sum the file-scoped `hourly_buckets`.
- **Watchdog reads `lastTs`.** Update Spot Counters stamps each skimmer's `lastTs` / `lastSpot` / `lastFreq` from the spot it is handling. The watchdog used to scan the ring, which three busy feeds now cycle in about half an hour, so a quieter skimmer would have read "no spots received yet" instead of its real age.
- **Removed:** `Route CW / FT8` and the two `Format CW/FT8 Spot Row` functions. Their outputs went nowhere (left over from an older RBN panel), yet each formatter filtered the whole ring on every spot.
- **Spot rings removed too (`542b0ff`, on operator request):** with no reader left, Parse DX Spot no longer keeps `spots_all` (10,000) or the per-skimmer `spots_<call>` rings (500). Every spot passed an `unshift` on both rings, and once a ring was full the trim back to its cap copied the whole array. A 2-minute sample showed no visible change (Node-RED 24.7% → 24.6% of one core), because the before-sample ran ~~12 min after a restart, while the ring was still filling. The copying only starts once the 10,000 ring is full (~~35 min at ~17k spots/h), so the saving is that steady-state cost, which this sample couldn't show.
- **D1:** the RBN Skimmer Panel draws a third card (`[d.sk1, d.sk2, d.sk3]`) and grew from height 8 to 12.
- **Vue `v44`:** the RBN card already lists whatever skimmers arrive, so VU24DX-1 needed no template change. The skew line was wrong, though: it said `Hz`, but sm7iun's values are ppm, and its colour bands (>2 amber,

  > *5 red) didn't match D1. It now reads ppm with D1's bands (<0.5 green, <1.5 amber, else red), so the two dashboards agree on VU24DX-1's −0.9.*

- **Home Assistant:** new discovery sensor `rbn_vu24dx_h1` → entity `sensor.rbn_skimmers_vu24dx_1_spots_1h` (93 configs, republished). The Radio dashboard's RBN card loops over every skimmer in `sensor.rbn_skimmers_state`, so it showed VU24DX-1 with no dashboard edit (state "3 online").
- **Verified live** after the Pi pull + restart: `Telnet VU24DX :7300` connected, `shack/rbn/state` carried `sk3: VU24DX-1` with status online, 24 FT8 spots in its first half-minute and skew −0.9; no `Unmatched` warnings in the journal; the Pi serves Vue `v44`.

2026-10-06

RustDesk: `.164` and `.169` moved to the Oracle Cloud server

- **New server:** self-hosted `hbbs` + `hbbr` (rustdesk-server 1.1.16, `-k _` so only clients with the key can register or relay) on an Oracle Cloud Always Free **VM.Standard.E2.1.Micro** in Hyderabad (ephemeral public IP). Ubuntu 24.04, SSH keys only, fail2ban, unattended-upgrades, 2 GB swap; Oracle security list and host iptables open TCP 21115–21117 and UDP 21116 only. Setup, ports and the key backup: `~/projects/oracle-rustdesk/HANDOVER.md` (not a git repo: it holds the server's private key). The server's address and client key are kept there, not here, because this repo is public.
- **Clients:** on `.164` (`pi`) and `.169` (`vu2cpl`) the `rustdesk` service was stopped, both `/root/.config/rustdesk/RustDesk2.toml` and the user's copy backed up as `.bak-2026-10-06`, and `custom-rendezvous-server`, `relay-server` and `rendezvous_server` set to the Oracle server with its key. Restarted; the server logged `update_pk` for `1252487319` (.164) and

`21464059` (.169) within seconds. `.170` (`1062710720`) had already registered there, switched by Manoj.
- **Also on the Oracle server:** `.109` 's own client (`1733915999`) and one more (`446570374` , the Mac Mini), switched by Manoj.
- **Home server retired:** with no clients left, the `.109` server (`vu2cpl.ddns.net`) was stopped and its containers removed (data kept; see ubersdr-box). The UniFi "Rustdesk" forward (21115–21117) was deleted by Manoj the same day; closed from outside. Phone/Mac clients need the Oracle server + key to reach these Pis.
- **Roll back** a Pi: `sudo systemctl stop rustdesk` , copy each `RustDesk2.toml.bak-2026-10-06` back, `sudo systemctl start rustdesk` .

2026-10-03

Pi 3B+ (`.142`) stripped to an idle box; Pi 5 bootloader updated

- **`.142` :** on operator instruction, removed `varnish` + `varnishncsa` (the `:6081` cache the old nginx site forwarded to), `codecserver` (+ drivers), and the `sdrplay` API service (a hand-installed unit, `/opt/sdrplay_api` kept). Then **purged `openwebrx`** with `--auto-remove` : 190 packages, all its decoders and their libraries, after a dry run showed nothing else on the box needed them. The `openwebrx` user went too. Archives are in `~vu2cpl` on the box. RAM in use went 425 → 176 MiB. Left: the fleet agent on :7799, `monitor.sh` , and nothing else. A last pass removed the remaining SDR stack: 43 packages (two `sdrplay3` Soapy modules had to be unheld; OpenWebRX's installer held them) and ~1,800 hand-built files in `/usr/local` + `/opt/sdrplay_api` , found by diffing against every package's file list (archive `sdr-local-removed-20261003.tar.gz`). The empty-dir sweep also took standard `/usr/local/share/{man,ca-certificates,…}` , which were recreated so `/usr/local/man` resolves again.
- **TODOs:** #48 and #40 dropped and #39 closed (operator); #36 closed on evidence (VU2CPL-88 delivers ~860 FT8 spots/h on the telnet path).
- **`.164` Pi 5:** bootloader 2025-05-08 → **2026-09-25** (`rpi-eeprom-update -a` , flashed and verified, then a reboot). The same headless "No displays" issue as `.169` under X11 was fixed the same way: `video=HDMI-A-1:1920x1080@60D` on `cmdline.txt` plus the live sysfs/xrandr step.
- **`.169` Pi 4 (this Pi):** after Manoj's `apt upgrade` finished, bootloader 2025-05-08 → **2026-09-23** (staged with `rpi-eeprom-update -a`) and kernel 6.18.39 → **6.18.50**, both through one reboot at 00:40 IST, with Open-Meteo state `cold` and no storm. Back in ~50 s with every service active, the flows guard OK, and lightning state + the AS3935 heartbeat flowing. X came back at 1024×768 (no EDID), so both headless Pis got `/etc/X11/xorg.conf.d/10-headless-hdmi.conf` with a preferred 1080p CEA mode. Verified on `.169` by a lightdm restart.

RustDesk on this Pi (`.169`), desktop switched to X11

- **Install:** RustDesk 1.5.0 (official arm64 deb, SHA-256 checked) plus `libayatana-appindicator3-1` for the tray icon, run through `systemd-run` so an SSH drop couldn't interrupt dpkg. The service is on.
- **Desktop:** `raspi-config nonint do_wayland W1` → `rpd-x` / `pi-greeter-x` (backup `lightdm.conf.bak-pre-x11-20261003`), then `systemctl restart lightdm` . Only the desktop shell ran in that session; Node-RED and Mosquitto stayed `active` throughout. This was done because RustDesk under labwc Wayland shows no screen (the `.164` finding).
- **Server:** pointed at `vu2cpl.ddns.net` with the key, and it registered as `21464059` . It's a fresh install, so it has **no permanent password** until one is set with `sudo rustdesk --password` .

- **"No displays":** the Pi is headless, so both HDMI connectors read `disconnected` and RustDesk logged `displays len: 0`. Fixed live by forcing `card1-HDMI-A-1` on through sysfs plus an xrandr 1080p CEA mode, and made permanent with `video=HDMI-A-1:1920x1080@60D` on `cmdline.txt` (backup `cmdline.txt.bak-pre-hdmi-force-20261003`). No reboot was needed.
- REBUILD_PI.md has a new optional step with the recipe.

Same session, other boxes (recorded in their own repos): `.170` (Win10) and `.109` moved to RustDesk 1.5.0 on the self-hosted server. The `.109` reboot for its new kernel + X11 left the **RX888 missing from USB**. Manoj power-cycled it on the rear port, and after one UberSDR restart it was back to 12 bands at 5000M (`ubersdr-box` HANDOVER).

2026-10-02

Self-hosted RustDesk server on .109; `.164` on it

RustDesk clients in the shack used the public `rs-ny.rustdesk.com` server (~185 ms). There is now a self-hosted one.

- **Server:** `hbbs` (ID) + `hbbr` (relay) from `rustdesk/rustdesk-server:1.1.16`, host-networked, as a compose stack in `~/rustdesk-server` on `.109`. The compose file is tracked in the private `ubersdr-box` repo. Both run with `-k _`, so only clients holding the generated public key can use the server. The key is kept out of this repo because it's public.
- **Router:** UniFi forward "Rustdesk", 21115–21117 TCP/UDP → `.109`, so the server is `vu2cpl.ddns.net` from anywhere (LAN clients go via hairpin NAT). It was first entered as `2115-2117`. The symptom was a dead hairpin test while `:7550` passed, with nothing at all reaching `.109`'s `eno1`.
- **`.164` (`meridianpi5`):** RustDesk upgraded 1.4.5 → 1.5.0 (official arm64 deb, SHA-256 checked). It connected but showed no screen: under labwc Wayland the portal granted the screencast, but the pipewire pipeline stalled at `Paused`. It was the same on 1.4.5. The desktop was switched to X11 (`raspi-config nonint do_wayland W1`, backup `lightdm.conf.bak-pre-x11-20261002`), and `libayatana-appindicator3-1` was added for the tray icon. It was then pointed at `vu2cpl.ddns.net` with the server key (config backups `RustDesk2.toml.bak-pre-selfhost-20261002`), and the server logged `update_pk 1252487319`.
- **Still to do by hand:** the phone and Mac clients need the same ID server + key. A client on the public server can't see peers registered here.

Network monitor: OpenwebRX+ tile pointed at the wrong box

The "OpenwebRX+" tile on the Internet and network monitor tab pinged `192.168.1.158`. That address is `gpsntp` (SSH `hostname` → `gpsntp`), so the tile had been showing gpsntp's reachability under the OpenWebRX+ name. The other tile addresses checked out: `.148` FlexRadio, `.164` `meridianpi5`, `.109` `ubersdr`.

- **Change:** the `OpenwebRX+` ping node's `host` and the `stamp OpenwebRX` function's `addr` were both changed `.158` → `.142`, the host named `openwebrxplus` (`openwebrxplus.local` resolves there). The stamp owns the address both dashboards display, so it needs the same edit as the ping node. Two field edits, no nodes or wires touched, `flows_guard` OK.
- **Correction, later the same evening:** `.142` is the *old* OpenWebRX+ Pi. Its `openwebrx` service is disabled (1.2.123 still installed), and it has run `meridian.service` (`~/meridian/meridian-server`) since 2026-09-25, with nginx on 80/443. **OpenWebRX+ runs on `.109`** as the `openwebrx` container (`slechev/openwebrxplus-softmbe:1.2.125`, compose in the `ubersdr-box` repo). So the tile now shows whether the old Pi is up, not whether OpenWebRX+ is serving.

- **Resolved the same night (operator decision):** the tile is relabelled **Pi 3B+** (ping node name, stamp name + `label`). The internal key stays `OpenwebRX`, because it's also in both dashboards' `ORDER` arrays. The meridian process on the 3B+ turned out to be idle, with no SDR attached and only propagation refreshes in its log. On operator instruction it was stopped and removed, together with the nginx `openwebrx` site; nginx is disabled. (Its `:6081` upstream was varnish, not "empty"; see the 2026-10-03 cleanup.) The archive is at `~vu2cpl/meridian-nginx-removed-20261002.tar.gz` on the box. The `shack-health` host list stopped expecting the `meridian` unit there (shack-health `5ab246c`, deployed).

Home Assistant's own Mosquitto add-on stopped (unused)

HassPi was running the Mosquitto broker add-on alongside the shack broker. Nothing used it. HA's MQTT integration connects to the shack broker on `.169` (its connection shows in `.169`'s client list). The add-on had no logins and no custom config (so no bridge). It had sent about 16 KB in total since it started, and nothing in a 60-second sample, so no client held a session. The IoT VLAN couldn't reach it anyway, because the firewall only allows IoT → `.169:1883`.

- **Change:** add-on `boot` set to `manual`, then stopped, both through the Supervisor API over HA's WebSocket. It stays installed, so undoing it is a click: Settings → Add-ons → Mosquitto broker → Start, with "Start on boot" switched back on.
- **Verified:** `.36:1883` and `.36:8883` now refuse connections. HA's MQTT entry is still `loaded`, and `sensor.rpi_noderedpi4_cpu` kept updating through the shack broker afterwards.

Found while planning HANDOVER #48 (Node-RED to a container on .109).

2026-10-01

RBN Skimmer tab: the VU2CPL skimmer is now VU2CPL-88 (the .109 meridian container)

Manoj moved the shack to container-only meridian: the Pi 5's meridian is stopped and disabled, .170's is disabled, and the Web-888 is retired temporarily. The only VU2CPL skimmer left is the .109 container, which uploads to RBN as `VU2CPL-88`.

- **Flows (5 lines, no nodes or wires):** `Login Handler VU2CPL` tags spots `skimmer: 'VU2CPL-88'` (status text too), and the `sk1_call` default in Parse Calibration CSV, Watchdog Check, RBN State Aggregator and Update Spot Counters is `'VU2CPL-88'`. The panels read `sk1` dynamically, so no template changed. The 12 h / 24 h counters are keyed by call, so VU2CPL-88 starts from zero.
- **HA:** the sensor (discovery id `rbn_vu2cpl_h1`, entity `sensor.rbn_skimmers_vu2cpl_spots_1h`) keeps its entity id but is now named "VU2CPL-88 spots (1h)" and reads `skimmers['VU2CPL-88']` (`ha_discovery_publish.py`, republished).
- **Feed:** unchanged — the tab still dials `vu2cpl.ddns.net:7550`, and the router's `:7550` forward already points at the container (`192.168.1.109:7550`): Node-RED logs in through hairpin NAT from `192.168.1.1`, and remote users (VU2OY, VU24DX, VU2WJ, VU2PTT) were connected over the WAN. Verified `shack/rbn/state` → `VU2CPL-88` online, and HA's `sensor.rbn_skimmers_vu2cpl_spots_1h` ("VU2CPL-88 spots (1h)") reading it.

2026-09-25

Daily shack health check now runs on this Pi (07:15), with a 07:30 dead-man on .109

A read-only check of every shack system, about 95 checks in 8 s, now runs daily at 07:15 IST on noderedpi4 as `shack-health.timer`. Coverage: internet/DDNS, LAN, this broker's retained state, the Pis, UberSDR +

RX888, OpenWebRX+, Meridian/RBN, NTP, the AS3935, Node-RED, HA, macmini-server.

It sends a short Telegram summary with the Pi's existing creds, and publishes a retained `shack/health/summary` `{ts, overall, fail, warn, checks, telegram, report}` as `svc` (inside the `shack/#` grant, so no ACL change). The ubersdr box checks that topic at 07:30 and Telegrams on its own if today's run is missing or unsent. That covers the one failure this Pi can't report: itself or the broker being down. It first ran on the Mac and moved here the same day: the Mac sleeps and travels; this Pi is always on and already holds the Telegram creds.

Supporting changes: this Pi's SSH key was added on .109/.164/.142/.158/.213 (recorded in vlan-setup), and `HA_URL` / `HA_TOKEN` were added to `~/.config/vu2cpl-shack.env` (mode 600). The code lives in the private `vu2cpl/shack-health` repo and is deployed from the Mac. First runs: all green, plus one real catch, a 2.5-minute Wi-Fi drop of the South boundary camera.

MQTT: passwd owner warning fixed; `svc` can read `lightning/#`

Two broker changes on noderedpi4, both with the operator's go and both applied with a SIGHUP reload (no client disconnects):

- **`/etc/mosquitto/passwd` → `mosquitto:mosquitto 600`** (was `root:mosquitto 640`, backup `passwd.bak-20260925-owner`). Every reload logged `owner is not mosquitto. Future versions will refuse to load this file.` — mosquitto 2.0 checks the owner against the user it runs as, so the 2026-08-21 note in `MQTT_AUTH.md` that root ownership "satisfies the ownership check" was wrong. `aclfile` was already `mosquitto:mosquitto 600` and never warned. After the reload the log was clean and a **fresh** `svc` login succeeded (the failure mode to fear is new logins breaking while existing sessions carry on). `MQTT_AUTH.md` corrected, and its password-rotation recipe now re-applies the owner, because `mosquitto_passwd` rewrites the file.
- **`user svc` gained `topic read lightning/#`** (backup `aclfile.bak-20260925-svclightning`). The Mac's shack health check authenticates as `svc` and could not see the AS3935's own retained `lightning/as3935/{status,hb}` — it had to go through HA's mirror. Read only: `svc` still can't publish to the sensor or its `cmd` topic. Verified: `status {"event":"ready",...,"fw":"v0.4.0","ip":"192.168.30.215"}` and `hb {"alive":true,...,"vbat_mv":4200}` read back as `svc`.

MQTT: VUKEYER status silently dropped since the rename — broker ACL fixed

Found by the new shack health check: the only retained keyer state on the broker was a stale `shack/esp32-winkeyer/status` `{"event":"offline"}`, while `vukeyer.local` answered ping. The keyer was renamed from "ESP32 WinKeyer" on 2026-09-17 and publishes `shack/vukeyer/status` as `iot`, but `/etc/mosquitto/aclfile` still granted `iot` only `topic write shack/esp32-winkeyer/#`. Mosquitto accepts the login and drops every publish the ACL doesn't cover without an error, so the keyer's status had never reached the broker under its new name. (The keyer repo's docs made it worse by naming `shack/esp32-vukeyer/#`, the MQTT *client id*, not the topic.)

Fix, with the operator's go:

- `aclfile` on noderedpi4: `topic write shack/esp32-winkeyer/#` → `topic write shack/vukeyer/#` under `user iot` (backup `aclfile.bak-20260925-vukeyer`), applied with `systemctl kill -s HUP mosquitto` — config reload, no client disconnects.
- Stale `shack/esp32-winkeyer/status` retained message deleted (empty retained publish as `svc`).
- Verified: a live heartbeat read back as `svc` — `{"event":"heartbeat",...,"wpm":25,"busy":false,"backend":"flex",...}`.

- `MQTT_AUTH.md` now lists the keyer under `iot`; the `vukeyer` repo's README/HANDOVER were corrected (`3fcc3f8`).

Nothing in `flows.json` subscribed to either keyer topic, so no flow change. Seen during the reload, pre-existing and not touched: mosquitto warns `/etc/mosquitto/passwd owner is not mosquitto` (future versions will refuse to load it).

RPi Fleet: IPs showed as "192.168", uptimes as "0m" — parseFloat mangling in Store Uptime/IP

Operator: "ip adresses are incomplete" — screenshot showed every fleet card reading `IP 192.168`, and every monitor.sh host `Up 0m` while HassPi alone showed a real uptime.

The publishers were innocent — `monitor.sh` sends the full IP and `uptime -p`'s human string ("6 hours, 34 minutes"); `monitor_redpitaya.sh` sends the full IP and its own compact "3d 2h" style. The mangling was flow-side, in two steps on the RPi Fleet Monitor tab:

- **`Parse Topic → Device/Metric`** coerces every payload with `msg.value = parseFloat(value) || value`. `parseFloat("192.168.1.169")` parses up to the *second* dot and returns the number `192.168` — truthy, so the `|| value` fallback never engages. Same for uptime: `parseFloat("6 hours, 34 minutes")` → `6`.
- **`Store Uptime/IP`** then wrote `String(msg.value)` into the `devices` flow context, overwriting the correct raw string the parser itself had stored one line earlier.

HassPi was the tell: HA's automation publishes uptime as plain seconds, which survives `parseFloat` intact — so its card alone formatted correctly (`6h 34m`), while every monitor.sh host's string arrived as `"6"`, was read as 6 *seconds* by the Vue card's `fmtUptime`, and rendered `0m`.

Fixes (one line + two formatters):

- **`Store Uptime/IP`** now stores `String(msg.payload).trim()` — the raw MQTT payload. Full IPs on both dashboards from the next telemetry minute.
- **Vue `fmtUptime`** (build `v43`, cache-buster `?v=43`) treats the value as seconds only when it is all digits (HassPi); anything else is compacted as a human string — "6 hours, 34 minutes" → `6h 34m`, "1 week, 2 days" → `1w 2d`, the Zynq boards' already-compact "3d 2h" passes through unchanged.
- **D1 `RPi Fleet Panel`** got the same helper (`fmtUp`) so its meta line reads `Up: 6h 34m` instead of the raw sentence.

Function/template text edits only — no nodes or wires, guard green. The formatter was verified against all three publisher formats (human string, compact string, plain seconds) before deploy. Same pass: README.md's fleet host list was three hosts and long stale — now lists all seven (HassPi, gpsntp, meridianpi5, noderedpi4, openwebrxplus, rp-f02054, web-888).

Zynq boards: the `.100` collision can no longer recur

This closes the recurrence risk left open below. Line 41 of `/etc/dhcpcd.conf` (`profile static_eth0`) was changed on each board from the shared `static ip_address=192.168.1.100/24` to that board's own address: `192.168.1.241/24` on the Red Pitaya and `192.168.1.235/24` on the Web-888 (backups `dhcpcd.conf.bak-20260925`). Both were saved with `lbu commit -d`; `lbu status` was clean afterwards, and each board's saved overlay on `mmcblk0p1` contains the new line. No dhcpcd restart was needed: the config is read at start, which is exactly when the fallback applies. The operator had already added the UniFi reservations for both MACs. Together, a boot that beats DHCP now leaves each board on its normal address, which is also its reserved address. Caveats: the fallback path itself hasn't been exercised live (that needs DHCP blocked during a boot), and re-imaging either board restores the stock `.100`. Line 51, a `192.168.1.101/24` fallback for `mvl0`, is also shared but was left alone: neither board has that interface.

Web-888 MAC ("distro"), findings for the remaining hygiene item. The kernel's `64:69:73:74:72:6f` is not in the device tree (no `*mac-address*` property), and it isn't the EEPROM's either. The 24c64 at I²C `0-0050` holds a u-boot env (`fw_printenv` works) with `ethaddr=02:00:11:22:33:44`, itself a placeholder, plus `hw_rev=Web-888.1`, `serial=26030080`. The Web-888 boots from `boot.bin` alone, with no `uEnv.txt` or `devicetree.dtb` to edit. On the Red Pitaya, by contrast, the EEPROM's `ethaddr=00:26:32:F0:20:54` is exactly what the kernel uses. The practical fix is a Linux-side override in `/etc/dhcpcd.enter-hook` on `reason=PREINIT`, which dhcpcd 10.0.6's run-hooks supports. Not done yet.

Web-888 mDNS fixed: `web-888.local` now survives reboots

Operator approved the fix proposed in the entry below. On the Web-888, line 31 of `/etc/avahi/avahi-daemon.conf` went from the commented default `#enable-dbus=yes` to `enable-dbus=no` (backup `avahi-daemon.conf.bak-20260925`), then `lbu commit -d`. `lbu status` was clean afterwards, and the saved overlay `/media/mmcblk0p1/web-888.apkovl.tar.gz` contains the new line.

Why it works: avahi only needs D-Bus to serve on-board clients such as `avahi-browse`, and nothing here uses them. Publishing its own name doesn't touch D-Bus, so the boot race described below can't happen.

Reboot-verified. After a cold reboot with no manual touch, avahi logged `Server startup complete. Host name is web-888.local` at `06:08:22` (pre-chrony clock), the same second it used to fail in. All three services (SSH, SFTP-SSH, HTTP) were published, and `noderedpi4` resolves `web-888.local` → `192.168.1.235`.

That boot also logged one `chroot.c: open() failed: No such file or directory` from avahi's chroot helper. To rule out the change, avahi was restarted by hand on the original config and then on the new one. Both started clean with no such line, so it is boot-only and independent of the change. The likely cause is a file avahi reads (probably `/etc/resolv.conf`, which dhcpcd rewrites on lease) not existing yet that early. Harmless, since the name and all services publish.

Docs now say to address both boards by `.local` name (`DEPLOY_PI.md`, `monitor_redpitaya.sh` header, `CLAUDE.md`). HANDOVER follow-up #46 (c) is closed. (a) distinct dhcpcd fallbacks, (b) UniFi reservations and (d) Web-888 MAC hygiene remain open.

Zynq-board IP conflict: root cause confirmed on the boards, cleared by reboot

Follow-up to the 2026-09-24 entry. Yesterday's diagnosis was inferred from the network side. Today I logged into both boards as `root` (the Mac's key is accepted) and confirmed it from the inside.

Root cause. `/etc/dhcpcd.conf` is byte-identical on the Red Pitaya and the Web-888, and both contain `fallback static_eth0` → `static ip_address=192.168.1.100/24` plus `noarp`. When DHCP doesn't answer at boot, both fall back to `.100`. `noarp` disables dhcpcd's probe/defend, so neither backs off. Both `/var/lib/dhcpcd/` were empty (no lease) and `ip addr` showed `192.168.1.100/24` on each. The trigger was almost certainly the 18:12 IST mains loss on 2026-09-24.

Fix applied. Rebooted each board once, Red Pitaya first, with DHCP up. Each got a real lease (`scope global dynamic`, fresh `eth0.lease`): Red Pitaya → `.241`, Web-888 → `.235`. Verified: `.100` answers nothing, 4 flushed-ARP probes of each address return one MAC every time, and the `RBN_SDR` target (`rp-f02054.local` → `.241`) went from 50% loss to 0%.

Corrections to the 2026-09-24 entry.

- The "distro" MAC was *not* the cause. The MACs differ, so the boards got separate leases the moment DHCP answered. It is hygiene only.
- The "trading one lease" explanation of the August `.241` / `.235` moves is retracted. Those were deliberate tile retargets.
- "Address both boards by `.local` name" was wrong for the Web-888, as below.

New finding: `web-888.local` dies at every Web-888 reboot (fixed later the same day, see "Web-888 mDNS fixed" below). It's a readiness race with D-Bus; the dependency itself is declared. avahi's init script has `need dbus`, and OpenRC starts dbus first, but the Web-888's dbus script runs `dbus-daemon --nofork` with `command_background="yes"`. So dbus is marked "started" before its socket exists. avahi starts in the same second, fails with `Failed to connect to socket /var/run/dbus/system_bus_socket`, and is never retried. I started it by hand after the reboot, so the name resolves now, until the next reboot. Proposed fix, awaiting the operator's OK: `enable-dbus=no` in `/etc/avahi/avahi-daemon.conf` + `lbu commit -d`. The Red Pitaya's dbus isn't readiness-aware either. It just wins the race.

Access technique recorded (in `CLAUDE.md`): while both boards share one IPv4, reach each one by its MAC-derived IPv6 link-local through `noderedpi4`, using `ProxyCommand` with the zone written `%%eth0`. ProxyJump drops the zone, and a bare `%e` is an ssh percent-token.

Still open: distinct dhcpcd fallbacks per board (`.241` / `.235`) + UniFi reservations, each followed by `lbu commit -d`. Until then, the next power cut that beats DHCP will cause the same conflict. Also the Web-888 avahi fix above, and (hygiene) a real MAC for the Web-888.

Docs corrected: `CLAUDE.md` (host inventory + `RBN_SDR` note), `DEPLOY_PI.md`, `monitor_redpitaya.sh` header.

2026-09-24

RBN_SDR tile: ping the Red Pitaya by mDNS name, not by a drifting IP

The `RBN_SDR` network-monitor tile had gone red. Root cause was not the tile: the Red Pitaya skimmer `rp-f02054` had lost `192.168.1.241` altogether — the address answered nothing, not even ARP.

The 2026-08-25 entry for this tile already carried the warning that `.241` was a DHCP lease and that a reservation, or pointing the ping node at `rp-f02054.local`, "would be sturdier". That prediction came true, so the tile now uses the name:

- **ping node `RBN SDR`** — `host` `192.168.1.241` → `rp-f02054.local`
- **`stamp RBN_SDR` function** — `addr` `'192.168.1.241'` → `'rp-f02054.local'`

Both edits are required, as that entry warns: the stamp owns the *displayed* address, so changing only the ping node leaves both dashboards showing a stale IP while silently pinging the new target. `flows_guard.py` passes (field-value change only, no structural change, so the guard constants are untouched). Verified `noderedpi4` can resolve the name: `getent hosts rp-f02054.local` → `192.168.1.100` (avahi-daemon active; `avahi-resolve` is not installed there, use `getent`).

Why the lease drifted: the two Zynq boards are fighting over .100

The UDM Pro Max logged *"Multiple devices are using the same 192.168.1.100 IP address"* at 12:59:59 UTC (18:30 IST). Both Alpine/Zynq SDR appliances had left their documented addresses — `.241` (Red Pitaya) and `.235` (Web-888) were both dead — and both were claiming `.100`.

Confirmed by flushing the ARP cache and re-probing `.100` six times: the replying MAC **alternates** between `00:26:32:f0:20:54` (the Red Pitaya, genuine Red Pitaya OUI) and `64:69:73:74:72:6f` (the Web-888), both serving `<title>Red Pitaya Apps`. `tcpdump` shows the UDM (`1c:6a:1b:17:42:e2`) re-ARPing `.100` repeatedly without settling.

`64:69:73:74:72:6f` decodes as ASCII **"distro"** — a hardwired placeholder, so the Web-888 is not reading a real MAC from EEPROM. `.100` is the Red Pitaya image's fallback when DHCP doesn't answer, so a simultaneous boot with no DHCP reply puts both boards on it. ~~This also retro-explains the `.241` → `.235` → `.241` "moves" of 2026-08-21/25: the two boards were trading a single lease the whole time.~~ **Retracted 2026-09-25** — the boards have different MACs and got separate leases as soon as DHCP answered, and those moves were deliberate tile retargets. See the 2026-09-25 entry.

Docs updated for name-based addressing: the host inventory, the tile table and narrative in `CLAUDE.md`, the Zynq/Alpine recipe in `DEPLOY_PI.md`, and the header comment of `monitor_redpitaya.sh`.

Still open (both need the UniFi console or the board itself, so they were not done in this change):

1. UniFi fixed reservations — `00:26:32:f0:20:54` → `.241`, `64:69:73:74:72:6f` → `.235`.
2. Give the Web-888 a real MAC. While it presents "distro", any second board from that image collides with it, and a reservation keyed on a placeholder MAC is fragile.

Until those land the tile will still flap, because the name resolves to the contested `.100` — but it is correct rather than permanently dead.

2026-09-16

UberSDR card: band noise & voice collapsible on both dashboards

Operator: "band noise and voice can be a click to open type just like the ft8 decodes". The per-band Noise/Voice/Dec-Mon table on the UberSDR card now starts collapsed on both dashboards, using the exact FT8-decodes pattern:

- **D1** (`ubersdr_panel`): the pane became a native `<details>` (closed by default). The summary line shows "Band noise & voice · `N` active" — N = bands with `voice > 0`, green when any. Only the tbody and the summary span repaint on each state message, so the open/closed state survives repaints (same trick as the FT8 pane).
- **Vue** (`UberSdrCard`): chevron-toggled section (`bandsOpen` ref, default closed) with the same green " `N` active" hint in the label. Build `v42`, cache-buster `?v=42`.

Housekeeping found on the way: `window.__shackBuild` had been stuck at `v38` while the `index.html` cache-buster advanced to `?v=41` across the two OpenWebRX sessions — the on-screen stamp under-reported the running build. Resynced at `v42` (the pair must move together, per CLAUDE.md's Vue-card conventions).

Network monitor: "Display results" startup TypeError guarded

`Display results` (Internet and network monitor tab) threw `TypeError: Cannot read properties of undefined (reading 'status')` once on every Node-RED restart, weeks back in the journal. Its trigger inject fires every 2 s from flow start, and the first tick lands before the first ping cycle seeds `flow.flow_data` — every sibling function on the tab guards with `flow.get('flow_data')||{}`; this one didn't. Fixed with an early `if(!flow_data)return null;` rather than the `||{}` idiom, so the dashboard skips the empty tick instead of painting a false "down" for 2 s. Spotted in the journal while verifying the RBN counter deploy (same day, below).

RBN 1H/12H/24H counters: real windows via per-skimmer hourly buckets

Operator spotted that the three spot-count windows on the RBN skimmer cards always read identical (1480/1480/1480 for VU2CPL). Root cause: `Parse DX Spot` capped `spots_all` at **2000 entries** — a memory guard from the flow's first commit, sized for CW-only rates — and the FT8 flood on the `:7550` feed (~~1480 spots/h from VU2CPL alone, ~2000/h combined) meant the ring only ever held **~~1 hour** of history. The window math in `Update Spot Counters` was correct all along; the 12H and 24H filters just ran over the same ≤1 h dataset as 1H. (The screenshot's 1480 + 520 = exactly 2000 was the giveaway.) Even at old CW rates the cap never truly covered 24 h, so the 24H tile was likely never honest — the FT8 rate just collapsed all three into one.

Fix, two function-only edits (no nodes/wires, guard green):

- **`Parse DX Spot`**: `spots_all` cap 2000 → **10000**. Needed for the 1H tile alone — at current rates the old cap sat exactly on the 1-hour clip edge, so 1H was about to silently undercount too. ~3–4 MB of flow context worst case; the Format CW/FT8 Spot Row consumers filter-then-slice(30), so a longer ring only costs them scan time.
- **`Update Spot Counters`**: 12H/24H no longer filter the ring (which can never affordably hold 24 h at ~2000 spots/h ≈ 48k entries). `hourly_buckets` — previously written but **never read** — is now per-skimmer (`{"YYYY-MM-DDTHH":{CALL:{total,cw,ft8}}}`), **file-scoped** so 12H/24H survive Node-RED restarts (the memory-scoped ring wiped every window on restart before), and summed for the two long windows (hour granularity; buckets whose UTC hour-start falls inside the window). 1H stays exact from the ring. Old-format global bucket entries (`{cw,ft8}` numbers at top level) are dropped on sight, so a context-preserving redeploy can't mix schemas. Retention unchanged at 240 hourly keys (10 days).

Expected settling behaviour after deploy: buckets start empty (the old ones were memory-scoped and global-format), so all three tiles read equal again at first and diverge as hours accrue — 12H honest after 12 h, 24H after 24 h. From then on, restarts only reset the 1H tile.

OpenWebRX card: health of both receivers

The OpenWebRX box now runs two SDRs — the RSP2 (VHF/UHF) and a Mirics MSi2500 dedicated to FM broadcast, so an FM listener no longer retunes the RSP2 for everyone — and its watcher checks each separately. `shack/openwebrx/status` gained `msi_result` / `msi_detail` alongside the RSP2's `result` / `detail` (`OK`, `BAD`, or `SKIP` when the container itself is down).

- `Aggregate OpenWebRX` keeps the two new fields (`msiHealth`, `msiDetail`).
- **D1 panel:** the single "Receiver" tile became **"Receivers"** — one colour-coded line each, `RSP2 OK` / `MSi2500 OK`; the sub-line shows the detail of whichever receiver isn't healthy.
- **Vue card:** the same two-line tile, plus a red `⚠ receiver` in the collapsed header when either is unhealthy (a collapsed card hides its tiles). Build `v40`.
- `SKIP` renders as a muted `—`, not red: a container outage is already reported by the RSP2 line, so one outage shows one red receiver.

No new nodes or wires (587 nodes, guard unchanged) — two existing nodes edited.

Follow-up (build `v41`): the Vue OpenWebRX card now **starts collapsed**, like the other 13. It was created expanded by mistake, contradicting README's "all collapsed by default". The collapsed header still shows the listener count, profile and the `⚠ receiver` marker. README's card count corrected 13 → 14.

OpenWebRX+ VHF listeners on both dashboards

The VHF receiver (OpenWebRX+ on the RSP2, `192.168.1.109:8873`) now reports who is listening, and both dashboards show it. The publisher side lives in the `ubersdr-box` repo: `openwebrx-watch.sh` samples the receiver every 2 min and publishes retained JSON as the `svc` account to `shack/openwebrx/listeners` (count, shared SDR profile, and per listener IP, city, country, MHz, mode, connect time) and `shack/openwebrx/status` (that run's health verdict). The daily/weekly/monthly Telegram summaries also publish their stats to `shack/openwebrx/summary/{daily,weekly,monthly}`. Cities come from an offline DB-IP Lite database on that box, so listener IPs never leave the shack for a geo lookup.

Seven nodes added, all on `ubersdr_tab` (the existing UberSDR flow tab): two `mqtt in` on the live topics → `Aggregate OpenWebRX` (merges the two payloads into one flow-context state, marks it `stale` when the publisher has been quiet for 6 min) → the D1 `ui_template` **OpenWebRX Listeners Panel** in a new `ui_group` "OpenWebRX VHF" (Shack Monitoring tools tab, order 31, next to UberSDR) **and** `Build OpenWebRX state for Vue` → `link out → Shack Vue`. The Vue side gets an `OpenWebRxCard` on topic `openwebrx` with the same tiles and table, `CARDS.openwebrx` flag.

Per CLAUDE.md rule #7 the Vue hop is a **link pair**, not a cross-tab wire. The guard constants needed no change — `MIN_UIB_FEEDERS` (10) and `MIN_NODE_COUNT` (300) are floors, and this change moves the live numbers up, 580 → 587 nodes and one more feeder; `flows_guard.py` passes.

2026-09-09

UberSDR host tiles + FT8/FT4 decode stream on the UberSDR card

Follow-on from the topic inventory done after the who-table landed (the flow used 2 of ~30 `ubersdr/#` topic families): operator picked two additions — "add the system load cpu load and temp and a collapsible ft8 decode stream". First cut put the decode stream on the **RBN tab**; the operator corrected the placement ("this should be in ubersdr") — it's UberSDR's own decoder output, so it belongs on the UberSDR card. Moved the two decode nodes onto `ubersdr_tab`, hung the `ft8` payload off `ubersdr_agg`, rendered the collapsible in the UberSDR panel/card, and reverted the RBN panel + `RBN State Aggregator` + `vue_rbn_builder_01` to stock. (RBN skimmer data was never actually affected — VU2CPL was online with 16 FT8 spots/h throughout; the brief "no RBN data on /ui" was the mid-flight RBN panel edit, undone by the revert.)

Host tile (UberSDR card, both dashboards). New mqtt-in `ubersdr_mqtt_sysload` (`ubersdr/metrics/system_load`, ~25 s cadence) → `ubersdr_agg` caches `usdr_sysload` and emits `host: {cores, load1, load5, load15, tempC, tempThr}` in the flat payload — null when the cache is >5 min stale, so a dead publisher blanks the tile rather than freezing it. The UberSDR host (16-core x86, not in the RPi fleet) was previously monitored nowhere. D1: tiles grid 5→6 columns, new Host tile — temp headline (green <65 / amber <75 / red ≥75, against the feed's own 80 °C threshold), load bar (load1/cores, same colour ramp as the CPU bar), `load N.N · 16 cores` sub. Vue: 4th tile on the second row (`.tiles` is auto-fit, no CSS change), same colouring.

FT8/FT4 decode stream (UberSDR card, both dashboards). The RBN telnet feed on `:7550` is CW/RTTY-only by design (TODO #36), but UberSDR's own FT8/FT4 decoders publish every decode on `ubersdr/metrics/digital_modes/<mode>/<band>` — callsign, country, exact freq, SNR, raw message. New on the UberSDR tab: mqtt-in `ubersdr_ft8_mqtt_in` (`digital_modes/#`, so FT4/WSPR ride along) → `ubersdr_ft8_parse_fn` keeps a 300-entry ring in memory flow context (`flow.ft8_decodes`) and pokes `Aggregate UberSDR` so decodes paint promptly instead of waiting for the next tick. `ubersdr_agg` attaches `ft8: {decodes: last 30, h1, h1capped}` to its flat payload. D1 panel: native `<details>`>

collapsible (closed by default; only the tbody is rewritten on repaint, so the open/closed state survives) with UTC/Band/Mode/Call/Country/SNR/Message columns and a decodes-per-hour counter in the summary line (`N+` when the ring can't see the full hour). Vue UberSdrCard: chevron-toggled section, columns trimmed to UTC/Band/Mode/Call/SNR for the ~330 px card — message + country ride the row tooltip. Build `v37` → `v38` , `index.js?v=38` .

Verified by simulation before deploy: system_load + sessions payloads through `ubersdr_agg` (host object correct), synthetic FT8+FT4 decodes through parse→aggregator (ring + h1 count correct), and live on the broker at deploy (CQs from San Marino/Lithuania/Italy several per 15 s FT8 cycle). flows_guard green (+3 nodes on `ubersdr_tab` , all wires same-tab).

TODO #36 note: this puts live FT8 decodes on the UberSDR card *now*, via MQTT, from the same UberSDR decoders the telnet Aggregator never carried — #36 remains about getting FT8 onto the RBN/cluster telnet path specifically.

UberSDR: who-is-listening table; LISTENERS now counts humans only

Operator: "in ubersdr dashboard can we add listener statistics? like which user is tuned to what?" The data was already on the wire — every `ubersdr/metrics/sessions` publish carries per-session `client_ip` , `reverse_dns` , geo (`country` / `country_code` / `city` / `region`), `frequency` , `mode` , `user_agent` , `created_at` , `daily_time_used_secs` , and a `user_session_id` that pairs a user's audio session with their waterfall session. `ubersdr_agg` was flattening all of it into counts.

Found on the way (and fixed per operator's "only humans" call): the LISTENERS tile was counting ~26 when 1–2 were real. UberSDR's own docker-network service clients (`cwskimmer` , `hfdl` , `lightning` , ...) report `is_internal: false` , so the aggregator's human/internal split put ~24 of them in the listeners bucket. They live on the 172.16/12 docker range, which is now the discriminator: a new `isDocker()` check routes them into a separate `services` count. LAN humans (192.168.x, also `ip_bypass`) still count as listeners — deliberate, so the operator sees his own sessions.

`ubersdr_agg` changes (function only — no node/wire changes, flows_guard green):

- Non-internal, non-docker sessions are merged per `user_session_id` into a `who` array: `{ip, rdns, cc, country, city, region, freq, mode, wf, since, daily}` — freq/mode from the audio session (waterfall-only users show `wf`), `since` = earliest `created_at` , sorted by frequency.
- `listeners` = unique human users (was: raw non-internal session count); `viewers` = humans with a waterfall open; `countries` and `listenersByBand` now count per user and cover all humans (byBand was previously waterfall sessions only — labels dropped the "(waterfall)" qualifier accordingly). New `services` count in the payload; `who` rides it too. Offline default carries empty equivalents.
- Verified by executing the new function body against a live captured payload: 72 sessions → 1 human (Wakayama JP, 7324.0 kHz LSB + wf), 24 services, 33 decoders, 13 monitors — and the human count matched the payload's own `non_bypassed_users` .

Both dashboards get a "Who's listening" table (Where — flag emoji

- city/country · IP with rDNS as hover title · Freq + mode, `+wf` when the waterfall is open · Time connected), capped at 12 rows with a "+N more" line. D1: new full-width pane in `UberSDR Panel` + `flag` / `fmtF` / `dur` helpers; header chip and Listeners tile switch to the human count, tile subs now read `N wf · M svc` . Vue: same table in `UberSdrCard` , build `v35` → `v36` + `index.js?v=36` .

Ripples: the UberSDR Telegram back-online alert now reports the human count (`p.listeners` , was `p.viewers`). HA's `sensor.ubersdr_listeners` had the same inflation baked into its value_template (`count - internal_sessions` = 26) — `ha_discovery_publish.py` now uses the payload's `non_bypassed_users` (external humans; a LAN listener won't show there, unlike the dashboards' table — noted in the script).

2026-09-08

Rotator: OVERLAP LED on both dashboards (gateway-derived)

Operator asked for an overlap LED on the rotator. Clarified up front (it changed the design): the rotor behind the Rotor-EZ interface is a **Yaesu that physically rotates past 360° of travel into an overlap zone** (end stop at South) — but the DCU-1 wire protocol only ever reports azimuth mod 360, so there is nothing to read; the flag has to be **derived**.

rotator-remote `b24bc32` (the data source): new `OverlapTracker` (`rotator/protocol.py`) keeps an unwrapped-travel accumulator fed by the 1 Hz poll — each sample moves it by the signed shortest-path delta, and travel past either end of the `[180, 540]` normal zone flips `overlap` (2° hysteresis so parking exactly on the boundary doesn't flicker; "both ways" per the operator, so the CCW side is handled symmetrically). The flag rides every WS state broadcast and `/healthz` . A throttled snapshot file (`~/.rotator-remote.state.json` , atomic tmp+rename, `tracker:` config section) restores it across service restarts iff the heading hasn't moved >15° while down — only front-panel-knob use can do that, since every remote path runs through the gateway. Also new: rate-limited INFO log for digit frames parsing >360, in case this Rotor-EZ variant ever reports overlap directly (none observed; that would let the flag be read instead of derived). Unit tests in `tests/test_overlap_tracker.py` (CW/CCW entry+exit, north-crossing non-event, boundary jitter, snapshot restore) — green on Mac and Pi.

This repo (5 function/SVG edits, no node/wire changes — flows_guard clean): `rotator_ws_parse` caches `flow.rotator_overlap` and rides `msg.overlap` downstream; `Rotator fmt + SVG update` drives a new amber `rot-ovl-led` + `OVL` label on the D1 HUD compass (top-left, mirroring the PWR LED top-right; unlit = dark ring, lit = `#e3b341`); Vue **v35** RotatorCard adds an `⟳ OVL` header chip (only when lit) and an always-visible `● OVERLAP` LED in the aside column (dim when off, amber

- glow when on), matching a front panel's always-there lamp. `vue_rotator_builder_01` and the HA bridge `rot_state_fn_01` both carry `overlap` now — the HA discovery entity itself is deferred on operator scope call ("both dashboards now, HA later" → follow-up #45), as is the `:8090` standalone compass page.

Deployed both halves (gateway restarted clean; Node-RED restarted after pull). **Verified on-air the same day**: the operator swung the rotor across South into the overlap zone — LED lit on both dashboards (after a page reload to pick up the new SVG/JS) and **matched the controller's front-panel OVERLAP LED**, with `shack/rotator/state` reading `heading:189, overlap:true` . The derived flag agrees with the hardware.

...and the HA overlap entity, same evening (#45, HA half)

Operator: "add the HA overlap entity too". `ha_discovery_publish.py` grew `binary_sensor.rotator_overlap` on the Rotator device (92 retained discovery configs now; template maps the bridge's `overlap` — including its pre-first-frame `null` — to ON/OFF, `expire_after` 30 s like the other rotator sensors), republished idempotently through HA's own `mqtt.publish` service and verified live (`state: off` , matching the rotor having been swung back out of overlap after the earlier test). The Radio dashboard's compass card got a conditional amber `mdi:rotate-360` icon at the rose's top-left (mirroring the D1/Vue LED position), rendered only while the sensor is `on` — saved over the WS API after confirming with the operator that no dashboard editor was open (the edit-mode-clobbers gotcha), with the pre-change config backed up to `~/Documents/vu2cpl-ha-backups/dashboard-193-radio-pre-overlap-20260908.json` and the element verified present on re-fetch. Post-landing fix the same evening (operator hit "no entity provided for more info dialog" clicking the compass): HA's default tap on entity-less picture-elements is more-info, so the 16 transparent full-card needle overlays — error-prone since 2026-08-25 — got `tap_action: none`, and the overlap icon got its entity + explicit more-info (tapping it now opens the sensor history). Backup `dashboard-193-radio-pre-tapfix-20260908.json`.

...and the `:8090` chip, closing #45 — plus a persistence bug it flushed out

Operator: "add the :8090 page too". rotator-remote `468d12b`: the standalone compass page now renders the flag it was already receiving — an ⟳ **OVERLAP chip** next to SLEWING, reusing the page's `.chip` pattern (invisible off, amber on); deploy is just `git pull` on the Pi thanks to the no-cache static handler. Verified in the browser with the operator live-slewing: chip lit while the rotor sat past South, healthz agreeing (`heading 189, overlap true`). All four surfaces (D1, Vue, HA, :8090) now show the flag — **#45 closed**.

The live session also flushed out a real bug: the overlap **snapshot had never once been written**. The unit's `ProtectHome=read-only` hardening makes the service's `~` unwritable, so every persist failed (`[Errno 30]`) and retried at the poll rate, spamming the journal. Fixed in rotator-remote `f313f19`: `StateDirectory=rotator-remote` in the unit (systemd creates `/var/lib/rotator-remote` for the service user), `tracker.state_file` default `auto` (resolves to `$STATE_DIRECTORY/state.json` under systemd, the home path for bare `./run.sh`), failed persists throttled to one attempt per 5 s and one WARNING per minute. Redeployed via `sudo ./install-service.sh`; the state dir was verified created and the journal is quiet. The first real snapshot write lands on the next heading sample (the controller was powered off at deploy time), so restart-survival of a live overlap flag remains covered by the unit tests until the rotator's next outing.

2026-09-02

Sweep #44 refined on operator feedback: radio rules, UI follows, tune in CW

Three same-evening amendments after the operator test-drove the sweep (all landed in `spe-remote ddfc449` + `25f6c3e`; dashboards `v34` + D1 copy in this repo):

- **The radio rules the band** — the first cut's mismatch-FAIL lasted about an hour. Operator: "whatever is on radio rules". The orchestrator now sweeps the band derived from the radio's slice freq whenever it's readable, overriding a stale menu pick with a note in `BAND_CHECKED` instead of refusing; the explicit pick is only trusted when the radio's band can't be read. Also the safest interpretation — the antenna follows the radio.
- **The band picker follows the radio** — both dashboards (Vue `v34`, D1 `SPE Panel (WS)`, and spe-remote's bundled web UI) now auto-select the sweep band from the amp's `band` field, which tracks the radio via CAT, on every state update. Manual clicks still work but the next band change re-follows; the pick only matters server-side in the radio-unreadable fallback anyway.
- **Tune runs in CW** — operator-observed: in DIGU/DIGL the tune carrier keys at the mode's TX-offset freq, and near a band edge that lands out of band, where the Flex refuses to key and the cycle fails. Their workaround ("switch to CW and tune") is now automated: the slice switches to CW before tuning (new `MODE_SET` phase) and the existing VFO snapshot/restore puts the original mode back.

Fake harness grown to 46 checks; radio-rules verified live by the operator from the Vue dashboard (correct band tunes; the earlier mismatch-refusal behaviour is gone). Both dashboards' sweep hint/ confirm updated again ("Band follows the radio; STBY/OPERATE and mode (CW) are automatic").

SPE sweep tuning: auto-STBY, band verification, OPERATE restore (#44)

TODO #44, closed the same day it was scoped. All three gaps landed in `spe-remote` (`a1c1252` , docs `ed3e0c5`), not this repo:

- **Auto-STBY.** The sweep (and `tune_single`) no longer FAILs unless the amp is pre-STBY'd — `TuneOrchestrator` drops it to STBY itself. CMD_OPERATE is a *toggle*, so the switch is verified against the CSV `op_status` (4 s timeout) rather than assumed, and the prior mode is remembered. The old "SPE must be in STBY to tune" preflight stays as a safety net against mid-sweep front-panel flips.
- **Band verification.** The requested band is checked against the radio before anything moves: the Flex slice freq (waiting up to 2 s for the post-connect slice-state events) is mapped to its ham band via the new `spe_band_table.band_for_freq()` ; a mismatch FAILs the sweep — wrong-antenna protection. `tune_band:auto` (also `current` / empty) sweeps whatever band the radio is on; an underivable radio band trusts an explicit request with a note.
- **OPERATE restore.** OPERATE returns at the end iff it was on at the start, on every exit path (success / FAIL / ABORT) — the restore runs after carrier-off + VFO restore and ignores the stop flag, so an aborted sweep still hands the amp back.

New `tune_event` phases `BAND_CHECKED` / `STBY_SET` / `OPER_RESTORED` render generically on all four sweep UIs, so this repo's change is copy-only: both dashboards' sweep hint + confirm dialog no longer instruct "Amp must be in STBY" (D1 `SPE Panel (WS)` template strings; Vue build `v33` , cache-buster `?v=33`). Verified with a 41-check fake-serial/fake-flex harness (toggle + restore, already-STBY, mismatch refusal, auto band, STBY-switch failure, stop-mid-sweep restore, tune_single wrap); deployed — the Pi pulled `ed3e0c5` and `spe-remote.service` restarted clean.

Verified on-air the same evening. First, the band check proved itself unprompted: the radio had been QSY'd to 17m (18.130 MHz, an AetherSDR client) in the meantime, so `tune_band:40m` was refused with the mismatch FAIL and nothing keyed, amp untouched. After an operator-approved QSY to 7.100, the full 40m sweep ran 12/12 sub-bands in 47 s: STBY confirmed via op_status 0.5 s after the toggle and before any TUNE keycode; every cycle LED_ON → CARRIER_ON 10 W → LED_OFF with the ATU taking 1.4–2.1 s per sub-band; VFO restored (7.100 LSB); OPERATE back 0.5 s after SWEEP_DONE. End state: amp OPERATE / RECEIVE / no warnings, TUNE LED off, and the slice returned to 18.130 where the Flex's own band memory restored USB. #44 fully closed — nothing left open.

Smarteefi #43 fully closed — actuation confirmed, all 5 IPs reserved

Operator toggled a relay from HA and the device switched: `set-status` works over the same routed-unicast path as `get-status` , through the patched `switch.py` → `cli_target()` chain. Before closing it, the session re-verified the patch set on the Samba share — all three fixes (`config_flow.py` options-flow, `__init__.py` UDP/unload, the unicast map) still live, `smarteefi_hosts.json` intact, all five backups untouched, so no HACS revert has happened: the switches work *because of* the patches, not despite them.

Loose end (a) closed the same day: the operator pinned all 5 devices as UniFi static reservations at their mapped addresses (`.112` / `.120` / `.149` / `.206` / `.249`), so a lease drift can no longer invalidate `smarteefi_hosts.json` . If a device is ever deliberately re-addressed, edit the map to match and fully restart HA.

That closes every actionable item on #43. The one permanent caveat: device push (UDP :8890 broadcast) still can't cross the VLAN, so state freshness stays bounded by the 5-minute poll.

2026-09-01

Rotator: LP/SP now powers on; the power indication stops lying

Operator report: LP/SP doesn't power the rotator, and the Vue card gives no power indication — "in NR dashboard, we had the whole display dimming for power off". All three complaints traced to real defects:

- **LP/SP had no power path.** `rotator_lpsp_fn` was literally `return {payload:'toggle_180'}` → serial. GO has had the ON-then-200 ms pattern since the auto-power feature landed; LP/SP was simply never given it, so with the outlet off it wrote serial into a dead rotator. Now mirrors GO exactly, through the same mqtt-out. The auto-off timer needs nothing — it arms off the `stat/` echo, so every power-on path gets it for free.
- **The Vue power pill was stuck ON since the card was built.** It read `global.rotator_power` — written by *nothing*, anywhere. The fallback was `heading != null` against a heading cache that is never cleared, which is permanently true after the first poll. The fix is one line in the D1 dim function (which already receives `stat/powerstrip1/POWER2`): cache the state into the global.
- **Both dashboards were blind after a Node-RED restart** — `stat/` isn't retained, so until someone toggled the outlet, D1 rendered undimmed and the global stayed unset. A new startup inject sends an **empty** payload to `cmnd/powerstrip1/POWER2` (empty = query, not toggle); Tasmota's stat reply seeds everything ~3 s after boot. Unknown now renders as OFF rather than a false ON.

Vue `v32` : compass, manual-heading row and DXCC presets dim to 0.25 when power is off — same value as the D1 SVG dim — while the header pill and the aside's power button stay full-brightness, so switching back on stays reachable. And `shack/rotator/state` now carries `power` (`null` until first known this NR run) so the HA rotator card can show it too.

Verified on the Pi post-restart: the bridge published `power:false` within seconds (the bootstrap query answered — the outlet really was off), v32 served, flows_guard green through the pre-commit hook.

Smarteefi HA integration — three HA-2026 breakages patched on HassPi

Reported as "smarteefi integration fails to load". The config *entry* was not the problem — it read `state: loaded` with all 17 `switch.*` entities registered. What failed was the **Configure** dialog:

```
File "/config/custom_components/smarteefi/config_flow.py", line 114, in __init__
    self.config_entry = config_entry
AttributeError: property 'config_entry' of 'SmarteefiOptionsFlowHandler'
object has no setter
```

The familiar HA break: `OptionsFlow.config_entry` became a read-only property in 2024.11 and the setter was removed in 2025.12. HassPi runs **2026.8.3**. It had fired twice, from two of the operator's own browsers.

Finding it needed a detour worth recording. `GET /api/error_log` **404s on 2026.8.x** — the log is no longer served over REST. The WebSocket command `system_log/list` still works and is now the fastest way to read an HA box's errors from a script; a ~60-line stdlib WebSocket client (handshake, masked client frames, `auth` → `auth_ok`) is enough, no `websockets` package required. `config/entity_registry/list` over the same socket is what mapped those 17 unremarkable entity ids (`switch.porch` , `switch.gate1` , ...) back to the `smarteefi` platform. Config-entry **data** is exposed by neither API — that needs `.storage/core.config_entries` over the share.

The component is HACS-installed from `coreembedded/smarteefi-homeassistant` at `3a8ae3f`, which **is** upstream HEAD — last pushed June 2025, three stars, unmaintained. Nothing to pull, so it was patched in place over the mounted Samba share, originals kept beside it.

Verifying the first fix surfaced two more:

Bug	Symptom	Fix
`SmarteefiOptionsFlowHandler.__init__` assigns `self.config_entry`	Configure 500s	drop the `__init__`, return `SmarteefiOptionsFlowHandler()`
`async_forward_entry_unload(entry, [4 platforms])`, twice	**Reload** throws — that API takes one domain, not a list	`async_unload_platforms(entry, [...])`
UDP listener is a bare class, and sets `SO_REUSEADDR`/`SO_REUSEPORT` inside `connection_made`	close raises `AttributeError: … no attribute 'connection_lost'`; re-bind fails `[Errno 98] Address in use`, leaving the push listener dead until a **full restart**	subclass `asyncio.DatagramProtocol`; build the socket by hand, set the flags **before** `bind()`, pass it as `sock=`; null the transport and `await asyncio.sleep(0)` on unload

That third one is the instructive one: the flags were being set in `connection_made`, which asyncio calls *after* it has already created and bound the socket. They could never affect the bind they existed for.

Verified end-to-end: the options flow now returns `type: form` instead of a 500 (opened over the API and aborted), and two consecutive entry reloads produce no `Address in use`, no `connection_lost` `AttributeError`, and no "UDP server failed to start".

Custom-component edits need a restart, not a reload — `POST /api/services/homeassistant/restart`. A config-entry reload re-runs setup but does not re-import the module; Python has it cached in `sys.modules`.

...and why the switches still don't work (it isn't the code)

All 765 status polls since install had failed with `Device Offline` — five devices, every five minutes, 100%. Reading the bundled `smarteefi-ha-cli` ARM binary settles what that string means: the CLI is **pure LAN UDP broadcast**, no cloud path anywhere. It does `socket(AF_INET, SOCK_DGRAM)` → `setsockopt(SO_BROADCAST)` + `SO_RCVTIMEO` → `sendto`, and `recvfrom() <= 0` is literally the branch that prints `Device Offline`. It is a receive timeout, nothing more.

The destination comes out of `get_baddr()`, which stores `0xd9270002` into the global `sockaddr_in`: family `0x0002` = AF_INET, port `0x27d9` big-endian = **10201**, address `inet_addr(ip) | ~inet_addr(netmask)` — the subnet broadcast of whatever interface the integration auto-detected. That detection follows the default route, so HA broadcasts to `192.168.1.255`.

The Smarteefi devices are on `192.168.30.x`. Broadcast does not cross VLANs, so the request never arrives.

The tempting shortcut — lie about the subnet so it aims at `192.168.30.255` — was tested and ruled out:

```
ping 192.168.30.255   → 3 sent, 0 received      (from 192.168.10.226)
ping 192.168.10.255   → replies + duplicates    (control: method works)
```

The router drops directed broadcasts between VLANs, so that would only move the timeout. Three real options, all needing a network decision, are carried as HANDOVER follow-up #43: move the devices onto `192.168.1.x`; a router-side UDP broadcast relay (10201 in, 8890 back); or a tagged VLAN-30 interface on HassPi plus a patch to `_get_active_interface_ip_and_netmask`.

Until then, do not trust those 17 entities. `async_turn_on/off` decides success from the CLI's **exit code**, which is `0` even when it printed `Device Offline` — so toggling a Smarteefi switch in HA flips the UI state while nothing physically happens.

Since none of this is in git, CLAUDE.md gained a **"Household HA integrations (patched in place — invisible to git)"** section naming the patched files, the backups, and the fact that HACS will silently revert them if the integration is ever reinstalled.

Same day, part two: fixed without moving anything — the devices answer unicast

The operator's first call was "move the smarteefi devices to 192.168.1.x", followed twenty minutes later by the better question: *"this is against our principle?"* It is — the IoT VLAN exists so that a compromised closed-firmware cloud device can't sit on L2 with the broker, HA and the radios. And with UniFi ruling out a router-side broadcast relay, the remaining "clean" option was a VLAN-trunked relay host: a real project.

The cheap idea won instead. The CLI computes its destination as `ip | ~netmask` — hand it a device's **own IP** with netmask `255.255.255.255` and that arithmetic collapses to plain routed unicast, which crosses VLANs like any other traffic (it is exactly how HA polls the Deye). Nobody knew whether the devices would *answer* a unicast request, so that was the test: the arm64 CLI ran natively on the Pi against every live VLAN-30 host. `192.168.30.112` answered `get-status` with `31`. Four more probes mapped the rest — `ns2110001071` needed a full-subnet sweep because it answers its own port but not ICMP:

Serial	IP	First unicast answer
`se5110002011`	192.168.30.112	31 (all 5 relays on)
`ns4110003532`	192.168.30.120	15 (all 4 on)
`ns2110001047`	192.168.30.149	0
`ns2110001071`	192.168.30.206	0
`ns4110003068`	192.168.30.249	0

The patch (on top of the morning's fixes): `__init__.py` gains a `cli_target()` helper and loads `/config/smarteefi_hosts.json` — a serial → IP map kept **outside** the component so a HACS reinstall doesn't delete it. A mapped serial gets the CLI invoked with its IP + `/32`; unmapped serials keep the stock broadcast, so the patch is inert for anyone else. `switch.py` applies the same target to `set-status`. (`fan` / `light` / `cover` untouched — no such devices.)

First poll after restart: **all 17 entities snapped to true relay state, bit-for-bit against the probes**, the smarteefi error count stayed at zero, and the morning's false `switch.pooja` ON — a UI toggle that had "succeeded" against a dead socket at 06:58, the optimistic-state bug demonstrating itself live — self-corrected.

Loose ends, carried in HANDOVER #43: the five IPs are DHCP leases until UniFi static reservations are pinned (`.206` is ICMP-silent — find it by MAC); `set-status` awaits one deliberate flip of a harmless relay; and device push updates (UDP :8890 broadcast) still can't cross the VLAN, so state freshness is bounded by the 5-minute poll.

VLAN isolation: fully intact. Nothing moved, nothing bridged, no new L2 adjacency — the principle held and the switches work anyway.

2026-08-30

The overnight experiment answered — and the answer is negative

Night 29→30 Aug was the test the whole exercise was built for: the first full night with the stabiliser back in circuit. **480 of 480 samples, zero interruptions.** The nightly outage pattern did not return.

Night	Samples	Interrupted	Stabiliser
25→26 Aug	480	1	bypassed
26→27 Aug	480	2	bypassed
27→28 Aug	480	0	bypassed
28→29 Aug	480	2	bypassed
29→30 Aug	**480**	**0**	**in circuit**

(The 1–2 "interrupted" counts are isolated single-sample blips, not outages — no episode reached the 2-sample floor.)

This substantially weakens the stabiliser-trip hypothesis. The reasoning that motivated it was: nightly cuts stopped when the stabiliser was bypassed, so the stabiliser was probably what tripped. The test of that is to put it back and see if they resume. They did not. Restoring the suspected cause did not restore the effect, which is the outcome that most cleanly disconfirms it.

What survives is the alternative already flagged on 08-29: the pattern stopped around 25 Aug for reasons outside the shack, and the four-night bypass run was confounded with that same calendar period. One in-circuit night is a small sample and does not *prove* the stabiliser innocent — but the hypothesis no longer has evidence behind it, and it should stop being the headline of the BESCOM case.

What the evidence does still support, and should be the case instead: chronic over-voltage on true mains (24.5% of the sealed window's samples above 253 V, peak 264.2 V) and **interruptions that continue by day** — six across 29–30 Aug alone (3, 6, 8, 9, 12 and 15 min). The supply is demonstrably unreliable; it is the *nightly* framing that the data has stopped supporting.

Two sags were logged, both explicable: 30 Aug 08:45 with L1 alone down to 167.6 V (isolated), and 09:55 with all three phases collapsing to 103 / 98 / 142 V in the minute immediately before the 09:56–10:07 cut — a transition artefact of the grid falling over, not a separate event.

No updated voltage report was generated, deliberately. Every row since 29 Aug 10:09 is the stabiliser's regulated output, so a combined report would mix regulated output with true mains in the same daily min/mean/max lines and misrepresent the evidence. The sealed bypass-window report stands as the voltage record; the interruption findings above are the only part of the post-switchover data that means anything to the dispute.

Stabiliser watchdog retired (and validated on the way out)

Operator: "stop monitoring". The cron entry was removed; `stabiliser_watch.py`, its state file and the env-file credentials were all left in place, so re-enabling is a single crontab line. `flows_guard.py` and `solar_inverter_mqtt.py` crons were deliberately **not** touched — the latter is the evidence log itself, the former is a critical-rule safety guard.

It performed correctly across ~34 hours: **7 outage alerts + 7 recoveries, all corresponding to real interruptions**, with the one-minute `read_fail` blips correctly suppressed by `CONFIRM_OUTAGE = 2`. Fourteen messages in a day and a half is also a fair guess at why "stop monitoring" was asked for — daytime supply is bad enough that an outage alerter is noisy when the operator is awake to see it anyway.

Follow-up #42(b) closes with real numbers. Across the full regulated era (1966 live samples, including a night with no PV export) vmax peaked at **242.7 V** and **no sample crossed 245 V** — so no false 🟠 fired, and none would have fired at the old `k=3` either. The 08-29 retune was precautionary rather than necessary, but the ceiling is now measured rather than guessed: **regulated output tops out ≈243 V**.

2026-08-29

Grid voltage — stabiliser back online, bypass window closed and sealed

Operator brought the servo stabiliser back online at **10:09 IST**, which closes the only period in which the inverter's grid input reads true incoming mains. The transition is unmistakable in the log: the 10:09 sample was **253.2 / 251.0 / 256.7 V** (6 V spread between phases), the 10:10 sample **232.3 / 232.7 / 232.0 V** — a ~20 V drop with all three phases converging to within 0.7 V. That tight convergence is the signature to look for, because `grid_voltage.csv` carries **no stabiliser-state column**; the boundary is recoverable only from the timestamp recorded here and in README / CLAUDE.md.

Bypass window, final: 2026-08-25 13:11 → 2026-08-29 10:09 IST (5553 valid samples, 99.6% coverage, 3 d 21 h). Everything from 10:10 on 29 Aug is the stabiliser's regulated ~232 V output and is **not** supply evidence. Sealed-window findings:

- **1334 of 5445 grid-present samples (24.5%) above the 253 V (+10%) limit**, peak **264.2 V on L3** (25 Aug 15:38).
- **Zero samples above the 270 V stabiliser trip point** — across four nights, the closest approach left ~6 V of headroom.
- **Eight supply interruptions, none of them overnight**: 25 Aug 16:05–16:37 (32 min), 26 Aug 10:23–10:51 (29 min), 26 Aug 12:51–12:57, 26 Aug 13:07–13:09, 27 Aug 19:32–19:41, 28 Aug 17:34–17:38, 28 Aug 20:33–20:50 (17 min), 29 Aug 09:28–09:32.

The finding that matters for the dispute. The Solarman cloud export (`deye_history_report.py`, follow-up #39) put **89% of 49 interruptions in the 22:00–06:00 window** over the preceding 32 days. Across four consecutive nights with the stabiliser bypassed, that pattern did not recur even once — while the chronic over-voltage did continue, never coming near 270 V. Two readings are consistent with this, and the data so far cannot separate them: either the nightly cuts were the stabiliser tripping (removed from the circuit, they stop), or BESCOM's behaviour changed independently over the same days. Four nights is a small sample and the confound is real, so neither is claimed as established here.

Now that the stabiliser is back in circuit, the **next few nights are the natural experiment**: if the nightly outages resume, that points at the stabiliser rather than the supply, and the sealed window above is the control period. Worth noting the stabiliser is rated to hold 220 V across an 180–270 V input, and the measured supply stayed inside that range throughout — so a trip, if one occurs, would be happening below the rated limit.

`grid_voltage_report.py` was run with `--bypass` carrying the closed window; the sealed report and chart live outside the repo (session scratchpad) since the CSV they derive from is Pi-side and append-only.

New: `stabiliser_watch.py` — overnight trip alerting

Operator asked to be alerted if the stabiliser trips overnight. That rules out a chat-side watcher: the answer comes due while he is asleep and the session may not outlive the evening, so it goes on the Pi as a 1-min cron pushing to the same Telegram bot as the lightning and UberSDR alerts.

Three conditions are separated because they implicate different things — 🔴 **outage** (grid absent 2+ samples: the nightly-cut pattern under test), 🟠 **dropout** (stabiliser no longer in circuit), ⚪ **stale** (CSV stopped growing, so the watchdog is blind and silence would otherwise read as calm) — plus a ✅ recovery. `~/.stabiliser_watch.state` makes each transition alert once instead of every minute.

Phase spread is not the discriminator, though it looks like it should be. Across the bypass window spreads ran 0.6–22 V and overlapped the regulated range entirely — 10:08 on 29 Aug was bypassed with a 2.6 V spread, while a regulated sample five minutes later had 5.1 V. Absolute voltage separates them cleanly (regulated 230–236 V vs raw mains 250–264 V), so `RAW_MAINS_V = 245` keys on vmax, confirmed over 3 samples so one spike cannot trip it. Revisit that constant if the stabiliser is ever set to regulate higher.

Verified before trusting it: the classifier was replayed over all 5580 historical rows, and it reads the entire bypass window as `dropout`, flips to `regulating` at exactly the 10:10 sample, and recovers both real outages with the correct start times (26 Aug 10:23, 28 Aug 20:33). The alert path was then exercised end-to-end against a synthetic grid-absent CSV — real `compose()`, real send, message delivered.

Credential gap, raised and then closed the same day. As first written, Telegram credentials came only from the running Node-RED process (`/proc/<pid>/environ`, the same trick `flows_guard.py` used) so no token sat on disk — which meant **alerts were skipped whenever Node-RED was down**, i.e. precisely when the shack is least healthy. That was surfaced as a judgement call rather than decided unilaterally, and the operator's answer was "put the token in the env file".

Telegram creds moved to the env file (both watchdogs)

`~/.config/vu2cpl-shack.env` did not exist on the Pi — the MQTT creds live in `/etc/default/vu2cpl-shack` (`root:vu2cpl 640`). The new file was created in the already-`700` home directory, opened `0600` via `os.open` from the outset rather than `chmod`-ed afterwards, so there is no window in which it is world-readable.

The copy was done entirely Pi-side and the value was never printed — a short Python heredoc read Node-RED's environ and wrote the file, emitting only string lengths (46 / 9). That matters because **this repo is public** and session transcripts can leak; a token echoed once is a token to rotate. Anyone repeating this should keep the same discipline.

Both `stabiliser_watch.py` and `flows_guard.py` now resolve credentials env → `/etc/default/vu2cpl-shack` → `~/.config/vu2cpl-shack.env` (wins — `monitor.sh`'s precedence) → Node-RED `/proc` fallback, via a small quote-aware parser. It is duplicated in both scripts rather than shared: `flows_guard.py` also runs as a git pre-commit hook from an arbitrary cwd in two clones, so it has to stay import-free.

Verified by sabotaging the `/proc` lookup and sending a real Telegram, which arrived — not merely by checking that the lookup returned something. `flows_guard.py` was re-run in both normal and `--stdin` (pre-commit) modes afterwards, both OK, since a broken guard would have made the repo uncommittable.

`flows_guard.py`'s docstring had justified the old behaviour twice over — "so no secret is copied to disk" and "the failure mode this guards is a bad DEPLOY, during which Node-RED is by definition running". The first half is now simply false, and the second holds only for the main case: a wipe arriving by git operation or manual edit while Node-RED is stopped, and the restart in the documented recovery path, both fall outside it. Rewritten accordingly.

The env file is not in the repo and `rebuild_pi.sh` does not create it — a rebuilt Pi has no token until it is written by hand. REBUILD_PI.md now carries that step.

Dropout threshold retuned — the confirm count, not the voltage

The 11:17 hourly check caught the calibration concern raised the same morning turning real. Regulated output had peaked at **242.2 V** within its first hour — 2.8 V under `RAW_MAINS_V`, and only 3.9 V of margin across a 3-sample run. Not enough to stake a 3 a.m. alert on, and a false 🟠 would have cost more than the feature is worth.

Both levers were measured against the actual bypassed-vs-regulated data rather than guessed:

Setting	Worst sustained regulated	Margin	Night coverage
245 V, k=3 *(shipped)*	241.1 V	+3.9 V	96.7%
245 V, k=5 *(now)*	**238.0 V**	**+7.0 V**	**95.5%**
250 V, k=3	241.1 V	+8.9 V	75.9%
252 V, k=5	238.0 V	+14.0 V	46.8%

So `CONFIRM_DROPOUT` went 3 → 5 and the threshold stayed: it nearly doubles the margin for a 1.2-point coverage loss, because genuine dropouts lasted many minutes while regulated excursions are brief. Raising the threshold buys the same margin at four times the cost.

The tradeoff is 5 minutes' slower dropout detection, which is acceptable — the *urgent* case is a trip that cuts supply, and that fires on the outage branch at 2 samples, untouched.

Still not fully settled: all 68 regulated samples are **daytime with PV exporting**. A night's readings should confirm the ceiling, and both constants need re-measuring if the stabiliser is ever set to regulate higher.

Otherwise quiet: supply uninterrupted 10:17–11:17 (61 samples, no failed reads, no grid-absent), 222.4–242.2 V, mean 234.1 V. Worst phase spread 9.0 V at 11:05 — further confirmation that spread is not a usable discriminator, since that is squarely inside the range raw mains also produces.

2026-08-26

Grid voltage — first bypass-night report; chart gets 200–300 V scale + BESCOM-off bands

Operator confirmed the servo stabiliser is in **bypass, and has been since logging began** (2026-08-25 13:11 IST) — so every row of `grid_voltage.csv` is true incoming mains, and the report's `--bypass` note now says so. First full night on record (25 Aug 22:00 → 26 Aug 06:00, 480/480 samples): **no supply interruption at all** (off-pattern against the Solarman export's 89%-overnight finding), battery held 100% throughout — but **at least one phase sat above the 253 V (+10%) limit for 320 of the 480 minutes**, peaking 255.9 V (L1, 01:39) and 255.5 V (L3, 02:20). Nothing approached the 270 V stabiliser trip point, which is worth watching: if the nightly outages return with no preceding 270 V excursion, the trips-on-over-voltage hypothesis weakens and the outage cause points at BESCOM's side. One daytime interruption 25 Aug 16:05–16:37 (32 min).

On operator ask, `grid_voltage_report.py` 's SVG chart was reworked: **fixed 200–300 V y-scale** (was ~180–280 auto-ranged, which wasted half the plot below any real reading), and supply interruptions are now drawn as red **"BESCOM off" bands** — grid-absent samples are excluded from the phase traces entirely (a new `M` segment resumes after each gap), so an outage no longer plots as a misleading plunge toward 0 V. `svg_chart()` grew an `outages` parameter fed from the same `outage_episodes()` (≥ `--min-outage`) the Markdown table uses; out-of-range values clamp to the plot edge. Legend footer notes the band meaning.

Operator screenshots drove three fixes. **(1)** The compass hub read `353°°` — a `state-label` renders the sensor state *with its unit*, so the explicit `°` suffix doubled it; suffixes removed (v16). **(2)** The Batt/RSSI/Strikes/Disturbers glance sat at *Unavailable*: the AS3935 bridge publishes heartbeats every 30 s (firmware `HEARTBEAT_MS` , retained) but the outdoor unit runs `modem_sleep: max` and its radio naps can hold messages up for minutes (watched its 4.5-day-old MQTT session flap offline→ready live at 00:09:42); `expire_after` on the four hb sensors raised 180 → 600 s — HA now flags only real outages, and Node-RED's 3-min Telegram alert stays the authoritative liveness alarm. **(3)** The Distance tile showed `0 km` for a *disturber* (reads like "lightning overhead"): template now yields a distance only when the last event is actual lightning, `out of range` for the 63 marker, `–` otherwise. **MQTT-discovery gotcha worth remembering:** updating a retained discovery config does NOT re-evaluate the entity against the retained state message (same topic ⇒ no re-subscribe) — the new template only runs on the *next* publish; force it by echoing the retained payload back via `mqtt.publish` . Used twice tonight. Follow-on operator asks, same pass: the Sensor tile + Batt/RSSI/ Strikes/Disturbers glance merged into one **state-coloured** markdown line (sensor dot green/red; battery 🟢 ≥3.9 V / 🟡 ≥3.5 / 🔴 below; RSSI 🟢 ≥-67 dBm / 🟡 ≥-75 / 🔴 below; strikes red only when >0) and the **16A Master button got a confirmation dialog** (v17). The Solar section's battery glance followed (v18): SOC dot banded to the load-shed ladder (🟢 ≥90 / 🟡 ≥50 / 🔴 below), charge state as ⚡ CHG green / ▼ DIS amber / dim otherwise (Vue convention), temp banded 40/50 °C, volts plain.

HA Radio dashboard — rotator compass (third pass, the real one)

Operator on the slider ("how can you use a volume slider to turn a compass rotator?") — correct, and the stepper variant was no better. Stock HA has no compass input, so one was built from primitives: a **picture-elements card** over `/local/rotator/rose.svg` (generated into HassPi's `config/www/rotator/` via the mounted Samba share — GitHub-dark rose, degree ticks, 8-wind labels) with **16 pre-rotated needle SVGs shown via conditional elements** keyed on a new `sensor.rotator_sector` (16-wind discovery sensor off `shack/rotator/state` ; 91 configs now) — a live rotating needle with zero custom cards. The rim carries **16 tap points (every 22.5°)** firing `script.shack_rotator_go` with that bearing; the hub shows the exact heading and an amber `→ target°` while turning. Below: a compact "Exact" stepper + GO/STOP for precise aims. Backup `dashboard-193-radio-shack-v15.json` (v13/v14 were the interim text layouts; one was also lost to the operator's open dashboard editor — see the edit-mode gotcha in HANDOVER).

HA Radio dashboard — rotator card rework (compass + controls)

Operator, screenshot in hand: "the rotator ui has to be changed, its useless now in its current state." Fair — an HA gauge maps 0-360° onto a linear arc (not a compass), and the Target tile sat at "Unknown" whenever idle. Replaced in place (matched by entity, not section, since the operator had regrouped cards): a compass markdown line (`🧭 353° NNW · → 270°` while moving, 16-wind cardinal), a target slider (`input_number.shack_rotator_target` helper created over WS, tile with the `numeric-input` slider feature), and a **GO · STOP · N · E · S · W** button row. GO uses a passed `heading` (presets) or the slider value, via script `shack_rotator_go` → `rest_command.shack_rotator_go` . Two facts worth keeping:

the `/rotator/go` body key is **`hdg`** (not `heading`) and the endpoint's handler powers the rotator on before moving; and since the `rest_command` domain was already loaded, `rest_command.reload` picked the new commands up without an HA restart. STOP verified 200 end-to-end (safe no-op while idle); GO deliberately left for the operator to fire first — it turns a physical antenna. Backup `dashboard-193-radio-shack-v12.json`.

HA Radio dashboard — phase 2: all six gateway cards (project complete)

Operator: "go ahead with the bridge for remaining cards" — the rule-#1 approval for extending the lightning bridge pattern to the six subsystems HA couldn't see. **17 new nodes across 6 flow tabs** (`89f6211`), every one a read-only tap publishing retained JSON:

- `shack/flex/state` — `flow.flexState` trimmed (model, callsign, txstate, active slices, PA readings), 5 s tick
- `shack/spe/state` — the one **stream tap**: the SPE tab keeps no context, so `spe_state_pub_01` hangs off `ws_format_state`'s output fan-out with an internal 5 s throttle and forwards the whole flat panel payload (all keys known: mode/rxtx/band/pwr/pwr_avg/pwr_peak/atuswr/antswr/tune/temps/alarms/…)
- `shack/lp700/state` — `global.lpState` (avg/peak/swr/range), 5 s
- `shack/rotator/state` — `flow.rotator_heading` + `target_hdg`, 5 s
- `shack/dxcc/state` — `workedStats` + last-5 `alertList` (key whitelist) + `clusterStatus`, 30 s
- `shack/rbn/state` — `flow.rbn_dash` (**file scope** — the aggregator file-backs it) with nested arrays trimmed to 5, 10 s

flows_guard OK; deployed (push → Pi pull → restart) and all six topics sampled live before touching HA — which also confirmed real operating data: FLEX-6600 connected/READY, SPE Operate on 12M (Middle), heading 353°, DXCC 320 entities / 2474 confirmed / 3 of 4 clusters up, both RBN skimmers online at 13 spots/h.

HA side: **23 new discovery entities** (90 configs total in `ha_discovery_publish.py` — per-subsystem devices with the history-worthy scalars as real sensors and the full payload as attributes on one state sensor each), then six consolidated one-card sections on the Radio tab: FlexRadio markdown (connected dot · TRANSMIT/RECEIVE · slice lines), SPE markdown (mode/band/ power-level · W · pk · SWR · PA °C · ⚡ TUNE chip · alarms line, with an OFFLINE fallback when the sensor expires), LP-700 glance (Avg/Peak/SWR), Rotator needle gauge 0–360° + target tile, DXCC markdown (stats line + last-5 alerts), RBN markdown (per-skimmer dot · spots/h · last frequency). All entities verified live before `lovelace/config/save`; backup `dashboard-193-radio-shack-v11.json`.

Liveness convention across all bridges: HA `expire_after` ≈ 3× the publish tick, so a stopped Node-RED or dead gateway shows *unavailable* rather than a stale number. Reference table for all seven bridges (incl. lightning): CLAUDE.md "HA state bridges". **This closes HANDOVER #41 — the HA Radio dashboard is now the full Vue-dashboard equivalent**, in one day: phase 1 (7 data-ready cards) → lightning controls → state-lit toggles + lightning bridge → layout polish + tabs split → gateway bridges + cards.

HA Radio dashboard — Vue-dashboard equivalent, phase 1

Operator asked for "an equivalent of the Vue dashboard in Home Assistant". Scoped up front (extend the existing **Radio** satellite dashboard · phase-1 data-ready cards · full controls · stock HA cards), then built entirely API-side — no broker ACL edit, no flows.json change, no file touched on any Pi.

What's live at `/193-radio`: 8 sections / 108 entities mirroring the 7 data-ready Vue cards — Shack Power (the existing 7 switch buttons + Flex Radio/PTT + the PS2/PS3/spare-relay outlets + the 16A energy row), Solar·Grid·Battery (the Utopia-2.0-approved needle gauges with the utility-case voltage thresholds, battery SOC graph + state/power/temp/volts glance, PV/Load/Grid W, today import/export), House Loads (4 toggle tiles + watts/today glances), Lightning (AS3935 state, last event/distance/when, battery/RSSI/counters, plus display-only Antenna/Radio tiles until rest_command lands), RPi Fleet (7 hosts × cpu/temp/mem/disk), GPS NTP (stratum/ref/fix/ offsets/sats), Network (6 ping tiles + RTT glance), UberSDR (listeners/decoders/sessions).

The data plumbing trick: HA's MQTT integration is connected to the *shack* broker (proved by watching `rpi/+` arrive over HA's WS `mqtt/subscribe`, which also captured every payload shape), and the `ha` broker account can write `#`. So the 61 missing entities (AS3935, chrony, RPi fleet, UberSDR) were created by publishing retained MQTT-discovery configs to `homeassistant/sensor/#` **via HA's own `mqtt.publish` REST service** — HA creates its own entities. Publisher checked in as **`ha_discovery_publish.py`** (repo root; idempotent, re-run to adjust). The network tiles use HA's native ping integration — 6 entries created over the REST config-flow API (its only field is `host`; passing `count` 400s), then the entity registry labelled them and enabled only the RTT-average sensors. All 108 referenced entities were verified against `/api/states` before `lovelace/config/save`; the prior config is backed up as `dashboard-193-radio-pre-shack.json` and the new one as `dashboard-193-radio-shack-v1.json` (both in `~/Documents/vu2cpl-ha-backups/`, outside this public repo).

Gotchas for the next session: discovery `object_id` is ignored on HA 2026.8 — entity_ids come out device-name-prefixed (`sensor.as3935_lightning_*`, `sensor.gps_ntp_gpsntp_*`, `sensor.rpi_<host>_*`); the RPi fleet sensors carry `expire_after: 180` so a dead publisher shows *unavailable* rather than a stale number; and `switch.tasmota` / `_2` / `_3` are the house loads GF/FF/Utility (Tasmota autodiscovery's default naming), not shack strips.

Lightning controls — landed the same evening. The operator mounted HassPi's Samba `config` share (credentials read from the Samba add-on's Configuration page — direct URL `/hassio/addon/core_samba/config`; SSH is closed and reading the password via the supervisor API from here was blocked, correctly). A `rest_command:` block went into `configuration.yaml` — 5 services (`rest_command.shack_ant_on / ant_off / radio_on / bypass_on / bypass_off`) calling the Node-RED lightning endpoints with basic auth: `username: vu2cpl`, `password: !secret nodered_http_password` (the secret line typed into `secrets.yaml` by the operator, never seen or handled here). **Gotcha:** `homeassistant.reload_all` does NOT set up a YAML domain that was never loaded — `check_config` passed but the services only appeared after a core restart. Four confirm-gated buttons (ANT ON / ANT OFF / BYP ON / BYP OFF, nested `grid columns: 4`) were appended to the Lightning section. End-to-end verified: `rest_command.shack_ant_on?return_response` → `{"content": "OK", "status": 200}` through httpNodeAuth, plus the ✅ ANTENNA ON (manual) Telegram. Note HA has no bypass-*state* entity yet (bypass lives in Node-RED flow context) — the BYP buttons fire blind until the phase-2 bridge publishes it. Backups: `configuration.yaml.pre-restcommand` and `dashboard-193-radio-shack-v2.json` beside the others.

Indications round (operator: "there are no indications..."). The four blind buttons became two **state-lit toggles**. ANTENNA: an entity-bound button on the antenna relay (icon lights + state text, repainted within ~1 s of any change from either side) tapping HA script `shack_ant_toggle`, which branches on the relay state and calls the matching NR endpoint — Vue-toggle semantics exactly. BYPASS needed state that existed only in NR flow context, so the **first NR→MQTT bridge brick** landed with operator approval (rule #1): 3 nodes on the Lightning tab — `light_state_tick_01` (inject, 10 s) → `light_state_fn_01` (read-only context tap: bypass_active/ expires_at, antenna_off, radio_off, manual_off [file scope], om_state, threshold_km, reconnect_min) → `light_state_mqtt_01` (retained `shack/lightning/state`). flows_guard OK — its constants are minimums, so +3 nodes needs no constant edit. Deployed Mac-edit → push → Pi pull → restart; six new discovery entities under a "Lightning Protection" device (`ha_discovery_publish.py` now

67 configs, `expire_after: 60` so a dead bridge reads *unavailable*); dashboard v4 = ANTENNA + BYPASS toggle pair (BYPASS branches on `binary_sensor.lightning_protection_bypass` via script `shack_byp_toggle`) + a protection glance (Manual hold · Storm · Threshold · Reconnect · Byp left). **Verified live, including by accident:** the test bypass-ON propagated MQTT→HA in ~10 s, then 14 s later the state flipped off — briefly chased as a bug until `nr_lightning_events.jsonl` showed `bypass_off source:operator` : the operator tapping their new toggle mid-verification, branch correct. Lesson recorded: when a state "mysteriously" reverts on a live system, check the event log for a concurrent human before suspecting the code.

Deferred (HANDOVER #41): Flex/SPE/LP-700/Rotator and DXCC/RBN cards need Node-RED→MQTT state bridges (rule #1 proposal per tab; copy the lightning pattern) before HA can see them. Also found: `/lightning/bypass` existed in flows.json but was missing from CLAUDE.md's endpoint table (fixed), and web-888 is fully offline — its fleet tiles honestly read unavailable.

HA: geyser countdown timers + backups to the Synology NAS

Geyser countdowns. Operator asked for a countdown on the geyser switches. Five `timer.geyser_*` helpers (30:00) created over WS (`timer/create`); two automations start/cancel them on switch on/off, deriving the timer from the switch via one template (`timer.geyser_{{ trigger.entity_id.split('.')[1].split('_geyser')[0] }}`). The dashboard's geyser row became timer **tiles** (live countdown when heating, tap toggles the switch via `perform-action`). Deliberately display-only: the existing "Geyser timer" automations still do the actual switching, so a timer failure can't leave a geyser on. Buttons also renamed to first names at card level (the Tuya sockets all call their relay "Socket 1", which dominated the labels).

Backup chain to the NAS. A fresh full backup (806 MB) was generated over WS, downloaded via `/api/backup/download/<id>?agent_id=hassio.local` , checksummed, and copied to the Synology share alongside a `RESTORE.md` runbook, the emergency-kit key files, and a `granular/` folder holding every automation/dashboard snapshot plus the `ha_backup.py` / `ha_ws.py` scripts. Then made permanent: operator added the share as HA network storage (credentials theirs alone), the new `hassio.nas_backups` agent was added to the nightly automatic (daily 05:15, retention 3, local + NAS), and a live test backup confirmed an 807 MB tar landing on the DiskStation.

Worth remembering: HA encrypts copies sent to off-box agents even when the local copy is unencrypted — the NAS tars need the emergency-kit key, the local ones don't. The kits sit beside the tars on the NAS; a copy belongs off-NAS too (printed, or in the password manager).

HA dashboards reorganised: one console + four satellites, original hidden

Continuation of the "193, Utopia 2.0" work, driven card-by-card by the operator with screenshots. End state: **193, Utopia 2.0** = At a Glance (clock + live weather glyph + nav buttons) and one **Power** section (grid tile, battery + 24 h graph, voltage gauges, geysers, load graphs, 4 switches with watts beneath); satellites **Radio / Security / Lights / Energy**, each with a full-width Home button; original **193, Utopia** hidden from the sidebar (config intact, restorable). All six backed up as `dashboard-*-settled.json` beside the automation snapshots.

Grid gotchas that cost real iterations, recorded so they're not re-paid:

- A section's internal grid is 12 columns × its **effective** `column_span`, and the span is **clamped to the view's `max_columns`** — a span-3 section on a 2-column view has a 24-unit grid, not 36. Sizing against the wrong denominator made every card a third narrower than intended, and only operator screenshots exposed it.

- A `sensor` card with `grid_options.rows: 3` **saves fine and then silently doesn't render**. Use `rows: auto` or `2`.
- N cards that don't divide the grid (5 geyser buttons, 7 shack switches) only get true equal widths from a nested `grid` card with `columns: N` — per-card grid_options can't express sevenths.
- Lovelace dashboards are WebSocket-only (`lovelace/config`, `config/save`, `dashboards/create|update`); `ha_ws.py` in the backups folder wraps auth + calls.

HA dashboard: "193, Utopia" → "193, Utopia 2.0"

Operator asked for a copy of the house HA dashboard, optimised, with one new feature: grid voltages, battery state, and the four house-load switches in one place. ("193, Utopia" is the house — which finally explains the Telegram bot name `Utopialertbot`.)

Lovelace configs are WebSocket-only — no REST route exists (`/api/panels` 404s). New `ha_ws.py` (kept with the other HA tooling in `~/Documents/vu2cpl-ha-backups/`, needs the `websockets` package) covers auth + `lovelace/dashboards/list`, `lovelace/config`, `lovelace/dashboards/create`, `lovelace/config/save`.

The copy is a **separate dashboard** at `/193-utopia-2` ("193, Utopia 2.0" in the sidebar) — the original is untouched and its config is backed up beside the helper. Dashboard JSON stays out of this repo for the same reason as the automation dumps: household names throughout, and the repo is public.

New **Grid · Battery · Loads** section, placed right after At a Glance:

- **3 phase-voltage needle gauges** banded to the utility case: amber below 207 V, green to 243.8 V (+6%), amber to 253 V (+10%), red above — so the condition that tripped the stabiliser for a month is visible at a glance, on the same instrument HA already reads.
- **Battery SOC gauge** banded to match the load-shed ladder (red <50, amber <90, green ≥90) plus state / power / temperature / volts tiles.
- **The four load switches** (GF, FF, Utility, Dryer Kitchen) as tap-to-toggle tiles with live-watts tiles beneath. Gotcha kept from the automation work: the Utility *circuit* is `switch.tasmota_3`; `switch.utility_area` is a camera.

Also fixed on the copy: one button card had a stray `{'type':'device'}` **object** as its `name` (the UI renders the dict literally), and the unlabelled energy section gained a "Energy Flows" heading. All 17 entities referenced by the new section were verified live before the config was saved.

Home Assistant under management via REST — and a load-shed threshold fixed

Operator asked whether HA automations could be handled from Claude Code and whether an MCP was needed.

No MCP. HA's own MCP Server integration isn't installed here (404 on `/mcp_server/sse`), and it would not have helped: it exposes Assist *intents* — device control and state queries — not automation authoring. The REST API was already live and does all of it (`/api/` returned 401, i.e. enabled and token-gated). The operator created a long-lived token into `~/.config/vu2cpl-shack.env` (`HA_URL` / `HA_TOKEN`, mode 600, Mac-side, never committed), closing the "HA REST API Bearer token" item that had been pending since the fleet work.

Worth recording for next time: the opaque 32-hex `entity_id` values in HA **device triggers** are entity-registry ids, and the registry is only listable over the WebSocket API. They can be resolved without WebSocket by POSTing Jinja to **`/api/template`** and using `device_attr()` / `device_entities()` — which is how the load-shed switches were mapped to `switch.tasmota` (GF_LOAD), `switch.tasmota_2`

(FF_LOAD), `switch.tasmota_3` (Utility) and `switch.dryer_kitchen` (Dryer_Kitchen). Note `switch.utility_area` is a *camera*, not the Utility circuit.

The fix. `load shedding 90` triggered on `below: 96`, not 90, while its own alert text said "<90%". Combined with `Load restore 95` firing only on an *upward* crossing of 95, that produced a trap: a shallow dip to ~95.5% shed all four house circuits, and they could not come back until SOC first fell below 95 and then recovered — so after a small discharge and a recharge to full, the loads stayed off indefinitely. Changed to `below: 90`, giving a closed 90/95 hysteresis band: any shed now necessarily takes SOC under 90, so the recovery crossing through 95 always happens on the way back up.

The 80 and 70 tiers are **deliberate redundancy**, not a staged ladder — they re-fire the same all-four-circuit shed in case HA misses the 90 crossing (restart, offline). Operator-confirmed; left as they are.

A correction made mid-task. The first inventory reported the tiers switching only 2 of the 4 house circuits. That was wrong — the summariser used to render the configs dropped `target.device_id` from service-call actions, so half of each action list went unseen. All four tiers have always covered all four circuits. The lesson is narrow and worth keeping: when summarising HA configs, device actions carry `device_id` at the top level while service calls carry it under `target`, and a reader that checks only one shape silently under-reports.

Backups deliberately outside this repo. Automation dumps carry household member names, per-person geyser schedules and notification targets, and **this repo is public** (`gh repo view` — the CLAUDE.md header claiming "private" was wrong and has been corrected). Snapshots therefore live in `~/Documents/vu2cpl-ha-backups/` alongside `ha_backup.py` (`dump` / `show` / `restore`, pure stdlib, credentials from the same env file), with pre-change and post-change copies of all 14 automations. `.gitignore` gained `ha_automations*.json` and `automations-*.json` so a stray copy in the working tree cannot be committed by accident.

Left open (follow-up #40): HA's `binary_sensor.inverter_grid` fired Disconnected at 16:51:10 and Restored at 16:58:41 during the feed transfer, while the per-minute inverter log has grid absent 16:05–16:37 and normal voltage throughout HA's window. Both poll the same few-client-tolerant Solarman logger. This decides which source defines the outage windows in the utility evidence, so it needs resolving.

Solarman cloud export → 32 days of supply-interruption evidence

The operator exported the month from the Deye/Solarman cloud — 33 daily `PlantsDetails-History*.xlsx` files. New **`deye_history_report.py`** turns them into a report for the supply utility.

Reading them without a dependency. These sheets store every value inline (no `sharedStrings` table), so a ~20-line regex reader over `xl/worksheets/sheet1.xml` is enough — no openpyxl, no pandas, matching the pure-stdlib idiom of `rpi_agent.py` and the rest. The daily files overlap on a 00:00 boundary row and carry all-zero padding rows; both are dropped (538 of 8,502) rather than being left to skew the numbers.

What the export does not contain: voltage. The channels are Production / Consumption / Grid / Battery / SOC / PV / Generator / Grid-tied Inverter Power, all kW, 5-minute resolution. So this evidences *interruptions*, not the over-voltage causing them — and the inverter reads downstream of the servo stabiliser anyway, so even a voltage channel would have shown the stabiliser's regulated ~220 V for any period it was engaged. The over-voltage half of the case rests entirely on the forward log started the same day.

The methodological point that makes the report defensible. `Grid = 0 kW` is *not* by itself an outage: a hybrid inverter with a full battery and PV covering the load also imports nothing, and looks identical in that column. An episode is only counted where the battery had to carry the property — SOC fell ≥ 1% — or there was no PV to do so. **71 zero-import episodes were excluded on that test.** The report says so explicitly and prints the basis for every episode it does count, because that is the first thing a utility will challenge. The battery's state of charge falling is independent physical corroboration that supply was genuinely absent.

Result over 25 Jul → 25 Aug 2026 (32 days):

- **49 interruptions ≥ 10 min, 74.3 hours without supply** — 139 min/day average.
- **63% began between 22:00 and 06:00, accounting for 89% of all lost hours** — matching the operator's account exactly.
- Worst single event: **445 minutes on 24 Aug (22:50 → 06:10)**, battery 99% → 39%.
- Lost minutes peak at 02:00-04:00 (895 and 965 minutes across the month); 10:00-14:00 is almost entirely clear.
- A weekly roll-up dates the onset without the report having to assert one: **0.9 h and 0.2 h** in the two weeks to 2 Aug, then **24.4 h, 29.5 h, 10.8 h, 8.5 h**. Something changed around 4 August.

Outputs: Markdown report, a CSV of the interruption table, and a day-by-day timeline SVG (one row per day, 24 h across, interruptions in red, the 22:00-06:00 window shaded) — the timeline is the one that makes the overnight clustering obvious at a glance.

Telemetry gaps are listed separately and **not** counted as interruptions: the inverter reporting nothing could equally be a loss of internet connectivity, and claiming them would weaken everything else.

Grid voltage logging — building an evidence file for the utility

Operator asked for two weeks of L1/L2/L3 to hand the supply company, which he is disputing over a **nightly outage around midnight**: a servo stabiliser holds the house at 220 V across an 180-270 V input range, and when mains climbs past 270 V at night it can't hold and trips — which presents as an outage.

There was no such data. Worth recording exactly what was checked, so nobody re-runs the search: the inverter feed publishes a *retained* MQTT message (last value only, by definition); no InfluxDB, Grafana, Telegraf, Prometheus, or any DB runs on the Pi; `mosquitto.db` holds only retained payloads; Node-RED writes exactly one data file (`nr_lightning_events.jsonl`) and it isn't energy; and `solar_inverter_mqtt.py` itself first ran that morning at 06:37. The honest answer was "it was never recorded", not a reconstruction.

Forward logging (`solar_inverter_mqtt.py`): one CSV row per run to `~/grid_voltage.csv` — `ts_iso, ts_epoch, status, l1_v, l2_v, l3_v, grid_on, batt_soc, batt_p_w` . Written inside the existing read cycle, so there is no second poller and no extra TCP client on a Solarman logger that tolerates only a couple (HA holds one). Failed reads emit `status=read_fail` instead of leaving a bare timestamp gap, because "collision with HA's poll" and "the site lost power" must stay distinguishable in something being used as evidence. The whole logging path is wrapped so it can never break publishing. Live from 13:11 IST.

Reporting (`grid_voltage_report.py` , new, pure stdlib — no matplotlib, no pandas): daily per-phase min/mean/max with time-of-peak, a night-window section, over-voltage episodes, and supply interruptions, plus a hand-written SVG chart with nominal / +10% / trip reference lines and the night hours shaded. Details that took a second pass: voltage statistics exclude samples taken while the grid was absent (a 0 V reading is not a supply voltage, and letting it into `min` understates the case); episodes crossing midnight print the end date; sub-3-minute excursions are summarised rather than listed, or threshold noise fills the table with one-minute rows.

Two caveats the report states on its own front page, because both change what the numbers prove:

1. **The inverter's grid input is downstream of the servo stabiliser** (operator-confirmed). The log is true incoming mains only while the stabiliser is in **bypass** — which it is now, and which is why the 13:05 reading was 254.6 / 253.8 / 262.1 V. With the stabiliser engaged the same registers would read its regulated ~220 V, so historical data from an engaged period evidences *interruptions*, not over-voltage.
2. **PV export lifts the voltage at the inverter's own terminals**, so daytime figures are arguable and the utility will say so. The night window has neither PV nor export — and is the disputed period anyway.

First live reading, 13:05 IST: **L1 254.6 · L2 253.8 · L3 262.1 V** — all three above +10% of 230 V nominal, in the middle of the afternoon, with L3 about 8 V under the stabiliser's trip point.

The retrospective fortnight is coming from the Solarman cloud export (HANDOVER follow-up #39).

Network monitor: RBN SDR tile back on the Red Pitaya (`.241`)

Operator repointed the `RBN SDR` **ping** node `192.168.1.235` → `192.168.1.241` in the editor and deployed, then reported the Vue card still showing `.235`. Not a cache miss or a stale build — both dashboards render `pings.RBN_SDR.addr`, and that string is owned by the **`stamp RBN_SDR` function**, not the ping node (Vue: `uibuilder/shack/src/index.js`, `addr: p?.addr || ''`; D1's `Network Monitor Panel` does the same). So the flow was pinging `.241` while both UIs displayed `.235`. Fixed by editing the stamp's `addr` to match — committed Pi-side as `2d58ba7` ("ping + stamp addr"). The 08-21 `.241` → `.235` hop needed the same pair of edits; the tab's "Please Read!!" comment node and the CLAUDE.md network-monitor section both already said so, which is why this was one grep rather than a debugging session.

What `.241` is: the Red Pitaya skimmer `rp-f02054` — the SDR feeding the RBN chain (operator-confirmed 2026-08-25). `.235`, the Web-888 receiver, keeps its fleet telemetry (`rpi/web-888/*` via `monitor_redpitaya.sh`) but no longer has a network-monitor tile.

Standing risk worth closing: `.241` is a DHCP lease, not a reservation. If it drifts, the tile reds out *and* the displayed address goes stale. A static reservation — or pointing the ping node at `rp-f02054.local` rather than a literal IP — removes the failure mode for good.

Power card: solar inverter + house-load rows (both dashboards)

Operator request: grid on/off + battery % under the 16A master's voltage/current/power/today line, on D1 and Vue. Scope grew mid-build (operator steers): battery power + charge state, the grid **input** voltages L1/L2/L3, and status for the four house-load Tasmota circuits.

Solar data path — direct, not via Home Assistant (operator's explicit choice; HA does read the same inverter but is not in this loop). The Deye SG0*LP3 LV 3-phase hybrid's Solarman V5 WiFi logger (serial `2924751994`, MAC `40:2A:8F:84:C1:E4`) was located at `192.168.30.193:8899` on the IoT VLAN — the discovery broadcast (`WIFIKIT-214028-READ`) went unanswered and the `.1` subnet had no trace of the MAC; the operator supplied the VLAN-30 address. New **`solar_inverter_mqtt.py`** (repo root → `/home/vu2cpl/`, 1-min user cron like `monitor.sh`) does a quick connect→read→disconnect of two register blocks via `pysolarmanv5` and publishes one retained JSON to **`shack/solar/inverter`** (`svc` account; creds parsed from the same `/etc/default/vu2cpl-shack` / `~/.config/vu2cpl-shack.env` files monitor.sh sources). Quickness matters: these loggers tolerate only a couple of concurrent TCP clients and HA polls it too — on a collision the script retries once then exits silently; the retained message stands and both dashboards dim the row when `ts` goes >5 min stale.

Register semantics verified live against the operator's HA screenshot (SOC, battery power −1986 W vs HA's −1968 W moments earlier, current, temp all matched): `586` - `591` = battery temp (offset 1000, ×0.1 °C) / voltage (×0.01 V) / SOC % / power (signed W, **negative = charging**) / current (signed ×0.01 A); `598` - `600` = **grid-input** phase voltages L1/L2/L3 (×0.1 V) — confirmed input-side by contrast-reading the load-side outputs at `644` - `646`, and cross-checked against the house Tasmota meters' per-phase voltage readings taken at the same moment. `grid_on` = any phase > 80 V.

House loads. The four remaining `iot` -account Tasmotas from MQTT_AUTH.md's "9 devices" turned out to be exactly the requested circuits: `FF_LOAD`, `GF_LOAD`, `GF_Dryer_Kit`, `GF_UTILITY` — found via a retained-LWT grep of `mosquitto.db` plus a 6-minute passive tcpdump of broker traffic (their web UIs are password-protected, and none of our readable accounts may subscribe to `tele/#`). They are **HA's battery-SOC load-shedding relays** (the "load shedding 70/80/90" / "Load restore 95" automations) with full per-circuit `ENERGY` blocks at TelePeriod 60 s. Node-RED shows them **read-only** — deliberately no toggle path; HA owns those relays.

Flows (7 new nodes on All Power Strips, `flows_guard` clean, node count 546→553): `solar_inv_mqtt_in` → `solar_inv_parse_fn`, and wildcard `tele/+/STATE` / `tele/+/SENSOR` / `tele/+/LWT` + `stat/+/POWER` (a filter that matches only single-relay devices — exactly the house set) → `house_parse_fn`; both feed the existing **Energy Aggregator**, which gains `energy/solar` (stored whole) and `energy/house` (per-device merge) branches. `vue_pwr_builder_01` passes `energy.solar` / `energy.house` on its existing 3-s tick.

Displays. D1's card retitled "Energy — 16A Master · Solar · House Loads" (height 2→6): solar row = Grid ON/OFF with `L1 231 · L2 232 · L3 233 V` beneath, Battery % (green ≥50 / amber ≥20 / red), Batt Power W, Batt State (`⚡ CHG` / `▼ DIS` / `IDLE`); house row = per-circuit relay ON/OFF with `W · kWh today` beneath, "offline" + dimmed on LWT Offline or >5 min silence. Vue PowerCard mirrors both rows (`energy-tile__sub` CSS added; build `v24`, then `v25` when the operator asked for explicit L1/L2/L3 labels — register identity re-verified before labelling).

Verified: publisher end-to-end (retained payload read back from the broker), cron installed (3rd line alongside monitor.sh + flows_guard), Node-RED restarted clean (no errors from the new nodes), and the whole parse chain unit-simulated with the captured live payloads (house filtering, relay merge, solar passthrough, Vue payload shape). Visual check on the auth-protected dashboards left to the operator (HANDOVER #38). Side finding for the operator: the 4 house Tasmotas have no `Timezone 5:30` set — their `Today` rolls at 05:30 IST (HANDOVER #37). `rebuild_pi.sh` Stage 9 + REBUILD_PI.md now deploy the publisher + pysolarmanv5 + cron on a rebuild.

Same-day follow-ups: house tiles = switches · IST on house Tasmotas · plain volts

Operator verified both dashboards ("look good") and steered three changes:

1. **House tiles are the switches now.** Each of the four house-load tiles toggles its relay: click → `confirm()` → `POST /power/house-toggle {dev}` — fetch+http-in per the repo's dashboard convention (never `send() / ng-click`) — → `House Toggle HTTP` validates against the house allow-list (400 otherwise) → the existing dynamic `MQTT Publish` node → `cmnd/<dev>/POWER TOGGLE`. Deliberately no optimistic flip: the device's `stat/<dev>/POWER` echo repaints the tile within ~1 s through the normal parse path. D1 uses an inline `onclick` + `window._houseToggle` (the SPE pill-toggle pattern); Vue adds `energy-tile--btn` (cursor + hover) and a `toggleHouse()` handler — build `v26`, cache-buster `?v=26`. 3 new nodes (http in / function ×2 outputs / http response), tab at 58, `flows_guard` green. Caveat stated in CLAUDE.md: HA's load-shedding automations still own these relays and may override a manual toggle at the next SOC threshold.

2. **`Timezone 5:30` stamped on the 4 house Tasmotas** (closes HANDOVER #37, operator-approved): a TEMP inject+function pair fired once 5 s after deploy, publishing `cmnd/<dev>/Timezone 5:30` to all four via the same dynamic MQTT Publish node; a packet capture over the restart showed all four devices ack `{"Timezone":"+05:30"}` on `stat/<dev>/RESULT`; the TEMP nodes were then removed in the follow-on commit. All 9 Tasmotas now report IST — `ENERGY.Today` (displayed on the house row) rolls at local midnight everywhere.

3. **L1/L2/L3 labels dropped** from the grid-voltage sub-line on both UIs (operator preference after seeing them; the values are still L1/L2/L3 in that order — register identity documented in CLAUDE.md).

4. **Grid-voltage sub-line forced onto one line** (v27, operator: "lots of space and it wraps to second line"), then **sized to actually fill the tile** (v28, operator: "now its tiny, just drop V and use the full width, max font will take"). The v27 pass fixed the wrap by shrinking the font, which traded one complaint for another — the real constraint was the tile, not the string.

   Measured it properly in a throwaway browser harness (tile markup + the app's real CSS vars, rendered at 330/420 px card widths, probing computed font-size and `scrollWidth > clientWidth`): the masonry `column-width: 320px` means **a card is ~330 px wide no matter the screen**, so a 4-across row leaves each tile **61 px** inside its padding. At 61 px an 11-character string cannot exceed ~10 px without clipping — and it *was* clipping (`233/234/23…`), on the house `W · kWh` lines too. No amount of font tuning fixes that.

   Fix: **the two new rows go 2-across** (`.energy-row--wide`; D1's inline `repeat(4,1fr)` → `repeat(2,1fr)`, card `height` 6→8). Tile inner width becomes ~145 px, the volts line renders at its **19 px** cap with nothing clipped, and the house lines sit at ~13.8 px. The sub-line now sizes itself in **container query units** (`cqw` against the tile, which becomes a `container-type:inline-size` query container) rather than `vw` — a quarter-of-a-card tile bears no relation to viewport width — with a plain-px first declaration in each rule as the fallback for engines without `cqw`. Unit `V` dropped (implied by the Grid tile above it), bare `/` separator, `nowrap`. The 16A row above stays 4-across: short values, no sub-line.

5. **Final layout, operator-specified (v29).** The 2-across compromise was rejected ("i dont like this layout") in favour of an explicit shape: the solar data on **two half-row tiles**, the house loads **all four across one row**.
   - **Grid** tile: ON/OFF + `233V 234V 232V` — the wider tile has room for the per-phase `V` again (14 chars ≈ 16 px at ~145 px inner).
   - **Battery** tile: all three battery values merged — SOC % as the headline, `⚡ CHG 2022W` on the sub-line (state colour-coded; wattage as a magnitude, since CHG/DIS already carries the sign). `Batt Power` and `Batt State` cease to exist as separate tiles.
   - **House** row: four tiles across, with watts and today's kWh as **two short stacked lines** (`1234W` / `12.5kWh`) — a quarter-row tile is ~61 px inside its padding, where the joined `123 W · 0.25 kWh` clips at any readable size.
   - Tile **labels** are now `nowrap` + `clamp(10px,12cqw,14px)`: `DRYER KIT` was wrapping at the 330 px card width and knocking that tile's contents out of alignment with its neighbours.
   - **D1 CSS scoped to `.en-card`.** Caught on review: `ui_template` CSS is injected page-wide, so the `.gh-card .gh-box` rule added in v28 was setting `container-type: inline-size` on **every box on the D1 dashboard**, not just this card. Now every rule in the template is scoped to a card-specific class.

    - D1 card `height` 8→7 (solar row back to a single line). Verified by re-rendering the tiles in the harness at 330/380/430 px and asserting no label and no sub-line clips or wraps at any width. **`index.css` cache-buster `?v=5` → `?v=8`** across the passes — the fixes live in the stylesheet, so bumping only `index.js` would have served stale CSS.

  Lesson worth keeping: v25→v28 each tried to fix a too-small / clipped sub-line by tuning the font, and each traded one complaint for another. The constraint was never the font — it was that the masonry pins a card at ~330 px, so tile width is fixed and *content has to be laid out to fit it*. Measuring the rendered tile (rather than reasoning about it) surfaced that in one pass.

Also closed HANDOVER #38 (the operator's visual check).

2026-08-24

VE7CC cluster diagnosed down (server-side) + `your call:` prompt fragment added

Operator asked why VE7CC shows disconnected. Diagnosis (all probes Pi-side): **not our setup — VE7CC's login processing is dead.** TCP is stable (4 brief drops in 26 h), the node config is correct, and the socket counters proved the login went out: since the last reconnect Node-RED had received exactly 1345 bytes (the banner: 222+1116+7-byte chunks) and sent exactly 10 bytes (`VU2CPL-1\r\n`), then silence. Interactive probes with a throwaway `VU2CPL-9` confirmed: banner → `Please enter your call:` → a bare `login:` re-prompt 200 ms later → then the server ignores every callsign sent (immediate, delayed, retried — 45 s of silence), identically on **both** port 23 and its native user port 7373 (7300/7000/8000 closed). A healthy CC Cluster answers a login instantly. Nothing to fix locally; watch the CC-User groups.io list if it persists. The tile is red because `clusterStatus` only marks a source connected after ≥1 parsed spot.

One real (minor) local gap the diagnosis surfaced: the login handlers' prompt-fragment list matched `login: / callsign:` variants but not CC Cluster's primary `…your call:` prompt (`call:` ≠ `callsign:` under `endsWith`). Today VE7CC's `login:` re-prompt masks it, but that's luck. Added `lastLine.endsWith('your call:')` to **both** handlers (`login-parse-dedup-v2` on the DXCC tab and `Login Handler VU2CPL` on the RBN tab — kept mirrored per convention). Not a fix for the outage — replying to the first prompt was probed and equally ignored — just robustness for any CC Cluster-style host.

Also checked (operator request): **DXClusterAggregator** (the macOS Swift app) does *not* share the gap — its `promptSuffixes` (`DXClusterClient.swift`) already include `call:` and `please enter your call:` , and its buffered line-splitter skips empty lines, so the 2026-07-31 `\r\n` -terminated-prompt bug class doesn't apply either.

DXCC band row — 70CM button (operator follow-up)

After confirming 60M/2M work on both dashboards, operator asked for the 70CM button too (it's in the Club Log download) and confirmed the default selection: 160–6 **excluding 60M, 2M and 70CM** — which the existing defaults already satisfied, so 70CM just defaults off like the other two. The plumbing was already in place from the entry below (`meterToBand` maps `'70'→'70CM'` , `getBand()` classifies 420–450 MHz) — this is purely buttons + filter map + presets:

- `dxcc_fn_filters_01` `bm` map: added `b70:'70CM'` .
- D1 `DXCC Dashboard` : `70cm` button after `2` (13 band buttons), `b70:false` in `def.b` , localStorage backfill for `b70` , VHF preset now sets `b6+b2+b70` .
- Vue: `70CM` in `bandKeys` , `'70CM':'b70'` in `bandKeyMap` , `70CM` in `VHF_BANDS` . Build `v23` , `index.js?v=23` (CSS unchanged).

- 13CM stays button-less: worked-table only, no spot can classify there (`getBand()` tops out at 70CM).

DXCC band row — 2M added, Vue gets All/HF/VHF presets, Club Log map completed

Follow-up to the 60m fix below, operator-steered: base the button set on what the Club Log download actually contains, add the missing presets to Vue, and set a sane default.

What Club Log returns (probed live with the account's own creds — probe ran Pi-side so secrets never left the Pi): band keys `160/80/60/40/30/20/17/15/12/10/6/2` plus single-entity `70` and `13` (cm bands). The parse function's `meterToBand` map only accepted 160–6 **without 60** — so 60M/2M/70CM/13CM worked-slots were silently dropped from the worked table, meaning a 60m spot of an already-worked-on-60m entity would have false-alerted as NEW_BAND.

Changes:

- **`Fetch All Modes + Parse lotw only`** (`aa7434df62b95ebc`): `meterToBand` extended with `'60':'60M'` , `'2':'2M'` , `'70':'70CM'` , `'13':'13CM'` . Labels match `getBand()` in `login-parse-dedup-v2` (which already classified spots up to 2M/70CM — no freq-table change needed). `bandSlots` stat grows accordingly (~+83).
- **D1 `DXCC Dashboard`** : `2` button added after `6` (12 band buttons total); default `def.b` is now **160–6 all on, 60M + 2M off** (`b160` and `b6` flipped to true per operator's "default = 160-6 without 60m"); localStorage backfill lines add missing `b60` / `b2` keys as false so the All preset covers them on existing browsers.
- **Vue `/shack` DxccCard**: `2M` added to `bandKeys` ; new **All / HF / VHF preset pills** (accent-blue `.pill--preset`) mirroring D1 semantics exactly — replace the whole selection: All = all 12, HF = 160M–10M incl 60M, VHF = 6M+2M. Build `v22` , `index.js?v=22` , `index.css?v=5` (new class).
- Server-side `bm` map already had `b2` and gained `b60` in the previous entry — no change needed.

Also fixed the Mac's `npx md-to-pdf` (used for the PDF pairing rules #3/#6): its npx-cached puppeteer 24.42.0 had no browser in `~/.cache/puppeteer` — installed the pinned Chrome for Testing 147.0.7727.57 via md-to-pdf's own `puppeteer browsers install chrome` , so the plain `npx md-to-pdf` invocation works again with no env-var workaround.

DXCC Tracker — 60m band now selectable (both dashboards)

Operator report: 60m couldn't be selected in the DXCC Tracker band filter. Two distinct symptoms, one root cause chain:

- **D1 `/ui` :** no 60 button existed at all — the BANDS row in the `DXCC Dashboard` ui_template (`38a6451a95a57685`) is hand-written HTML listing 160/80/40/30/20/17/15/12/10/6 only, and the default filter-state object had no `b60` key. (Oddly the HF preset handler *did* already include `'b60'` in its list.)
- **Vue `/shack` :** the 60 button existed (`bandKeys` has `60M` , `bandKeyMap` has `'60M':'b60'`) but wouldn't stay selected — the click optimistically set state and POSTed `b60:true` to `/dxcc/filters` , but the server-side `Save Alert Filters HTTP` function (`dxcc_fn_filters_01`) translates UI keys through a `bm` map that had **no `b60` entry** and silently drops unknown keys, so `60M` never entered `flow.filterBands` ; the next `dxcc` bridge echo (`Object.assign(state, msg.payload)`) overwrote the optimistic toggle and the button reverted.

Consequence beyond the UI: with band filtering active (non-empty `filterBands` , the default), the classifier's `!filterBands[band]` check dropped **every 60m spot** before the alert table / Telegram — the frequency table already knew 5250–5450 kHz → `60M`, so spots parsed fine and then always filtered.

Fix — flows.json only, no Vue change (build stamp unchanged):

1. `dxcc_fn_filters_01`: added `b60: '60M'` to the `bm` map.
2. D1 `DXCC Dashboard` template: added the `data-band="b60"` button (between 80 and 40) and `b60:true` in the `def` state object. Button render + click wiring are generic (`querySelectorAll('[data-band]')`), so no other JS touched. Existing browsers with saved localStorage state simply see the new button unselected until clicked (missing key = falsy).

DXCC.md Dashboard Features row updated + PDF regenerated (rule #3).

2026-08-22

Web-888 joins the fleet too — same script, zero changes

The `.235` mystery box from the Red Pitaya detour (below) is the **Web-888 receiver** (operator-confirmed) — which also resolves what the 08-21 RBN_SDR tile repoint pointed at: intentional and correct. It turned out to be another Zynq/Alpine/BusyBox platform (XADC at `iio:device0`, `lbu` persistence, recent Alpine with OpenSSH 9.7), so `monitor_redpitaya.sh` deployed **unchanged**: `rpi/web-888/*` topics live (61.8 °C XADC at install, 2d5h uptime), `svc` creds piped machine-to-machine, `apk add mosquitto-clients` (unlike the RP, not pre-installed), `crond` enabled (also stopped here — apparently the Alpine-appliance default), `lbu commit -d` clean, autonomous cron publish verified on the broker. Script header + DEPLOY_PI.md special case now cover both boxes; fleet count: `noderedpi4`, `openwebrxplus`, `rp-f02054`, `web-888` (+ HassPi pending).

Red Pitaya joins the RPi Fleet Monitor (telemetry only)

The Red Pitaya skimmer `rp-f02054` (`rp-f02054.local` — `192.168.1.241` via DHCP; Zynq-7010, 2020-era Alpine image, OpenSSH 8.3, no `/opt/redpitaya` vendor tools) now publishes fleet telemetry — live same day, verified end-to-end (all 7 topics on the broker, cron firing autonomously, `lbu status` clean).

Wrong-box detour, for the record: the install first targeted `192.168.1.235` — the address the RBN_SDR ping tile was repointed to on 08-21 — on the assumption that tile = this Red Pitaya. SSH key auth kept failing there despite a byte-perfect `authorized_keys`; the tell was version skew (port 22 on `.235` banners OpenSSH 9.7, while a debug `sshd -d` run in the operator's actual shell reported OpenSSH 8.3), and host-key fingerprinting settled it: `.235` is a **different machine**, and `rp-f02054.local` still answers at `.241`. Identity of the `.235` box: operator to confirm (the RBN_SDR network-monitor tile currently pings it). Fleet docs reference the mDNS name, not the DHCP address. Also caught during install: busybox `crond` was neither running nor enabled on this image — `rc-update add crond default` is part of the runbook. New repo script **`monitor_redpitaya.sh`**: same `rpi/<hostname>/*` topics and 1-min cadence as `monitor.sh`, so the host auto-appears on the fleet tab (the flow derives its device list from `rpi/#` — no flow edit needed), but every probe is platform-specific: **temp from the Zynq's on-chip XADC** via IIO sysfs (`(raw+offset)×scale/1000` °C — `/sys/class/thermal` doesn't exist on Zynq), cpu% from a 1-s `/proc/stat` delta (BusyBox `top` format differs), mem% from `MemAvailable`, IP via `ip route get` (no `hostname -I` in BusyBox), prettified `/proc/uptime`. MQTT auth via the `svc` account from `/root/.config/vu2cpl-shack.env`. Telemetry-only like HassPi — no control agent, reboot/shutdown buttons unwired for this host. Install runbook in DEPLOY_PI.md "Special cases", including the **Alpine-diskless gotcha**: the image runs from RAM, so package + script + creds + crontab all need `lbu commit -d` to survive a reboot. Note: the Zynq idles warm (45–65 °C) — the fleet's shared >75 °C Telegram alert may need a per-host threshold if it turns out noisy under skimmer load in summer.

2026-08-21

MQTT broker authentication + credential encryption + broker consolidation

Part of the shack-wide MQTT migration off anonymous access (broker `192.168.1.169`, tracked in the `vlan-setup` repo's SECURITY-AUDIT.md). Three related changes:

- **Node-RED now authenticates** to the broker with the dedicated `nodered` account (ACL-scoped once enforcement lands). The `iot` account covers the Tasmota fleet + the as3935 bridge; `ha` and `svc` accounts remain to be wired on their respective hosts.
- **Duplicate broker config removed.** The project carried *two* `mqtt-broker` config nodes both pointing at `192.168.1.169:1883` ("Tasmota MQTT Broker" `f4785be9863eab08` with 35 nodes and "shack-mqtt" `mqttbroker.shack` with 4). The 4 nodes (Rotator Power, gpsntp/chrony, two UberSDR metrics) were repointed to the single keeper and `shack-mqtt` deleted.
- **`flows_cred.json` is now encrypted.** ⚠️ It is git-tracked in this **public** repo, and `credentialSecret` was `false` (plaintext). The `nodered` password was briefly committed in plaintext **locally** but caught before push. Fix: set a project `credentialSecret` (stored in `~/.node-red/.config.projects.json` on the Pi — **outside** this repo; back it up, losing it makes creds undecryptable), re-encrypted `flows_cred.json`, and verified the public history is plaintext-free. Later that day `flows_cred.json` was also `git rm --cached` + gitignored, so no credential blob remains in the repo tree at all.

MQTT enforcement complete + Pi publishers on `svc`

- **All clients migrated, then enforcement flipped.** The `svc` account was wired onto the Pi telemetry publishers — `monitor.sh` (rpi metrics, 4 Pis) and `gpsntp-mqtt-publish.sh` (chrony) both gained optional MQTT auth; `ha` was set in Home Assistant, ubersdr in its own UI. The broker then had `password_file` + `allow_anonymous false` + an `acl_file` applied and verified in stages (each account authenticates, anonymous refused, `iot` topic-scoped away from services topics, all telemetry still flowing). **The broker no longer accepts anonymous connections** — full detail in `MQTT_AUTH.md`.
- **`monitor.sh`** now passes `-u/-P` to `mosquitto_pub` when `MQTT_USER` / `MQTT_PASS` are set (empty ⇒ anonymous). Reads them from `/etc/default/vu2cpl-shack` or a new no-sudo `~/.config/vu2cpl-shack.env` override.
- **`rebuild_pi.sh`** (the Pi installer) now prompts for the MQTT username/password in the mandatory broker inventory and writes them to `/etc/default/vu2cpl-shack` (`root:<user>`, `chmod 640` since it holds a password). Empty username keeps the previous anonymous behaviour, so forks are unaffected.
- **New doc `MQTT_AUTH.md`** — the operational reference: the accounts, the ACL, per-client credential locations, and how to onboard or rotate a client.

AetherSDR panadapter status display — external humanizer flow

Show live shack status on the AetherSDR panadapter overlay (`Antenna: ON`, `Lightning: Disturber, 3 min ago`, `AS3935: Active`, `GPS: S1 +9 ns`, `Power: 100 W`, `SWR: 1.20:1`) **without any AetherSDR code changes, now or on future releases** — the operator's hard constraint. All formatting is done outside the app.

- **New Node-RED flow `nodered/aether-display/`** — subscribes to the raw shack topics (`stat/powerstrip1/POWER5`, `lightning/as3935/{last_event,status}`, `shack/gpsntp/chrony`), formats each into a human-readable one-liner, and re-publishes them **retained** under a dedicated `aether/*` tree. AetherSDR only subscribes to `aether/*` and prints

`leaf: payload` verbatim — it never parses a payload, so a raw-format change is a one-function edit in Node-RED, never an app rebuild. Importable JSON + full README in that folder.
- **LP-700 power/SWR** aren't on MQTT (they arrive over `ws://lp700-server/ws` into `flow.lpState`, tab-scoped). One line added to `LP State Aggregator` — `global.set('lpState', st)` — mirrors that state to global context so the new flow can read it and publish `aether/Power` + `aether/SWR`.
- **New read-only `display` MQTT account** — ACL `topic read aether/#` only, so AetherSDR holds a least-privilege credential in its on-disk QSettings rather than the broad `nodered` / `ha` one. See `MQTT_AUTH.md`.
- **Gotcha caught same day:** `chown root:root chmod 600` on `/etc/mosquitto/passwd` makes it unreadable to the broker (runs as user `mosquitto`) → *all new auth fails* while existing persistent connections keep working, so it looks fine until a device reconnects. Correct is `root:mosquitto 640`. Documented in `MQTT_AUTH.md`.

Stale-tab wipes #3 and #4 — Vue bridge re-wiped twice in one day; flows_guard tripwire added

Third and fourth occurrences of the 2026-07-13 failure mode, both today, and the fourth landed in git history — which is what finally forced a real guard instead of a protocol note.

- **Wipe #3 (morning):** a Deploy (`41696b7`, during the MQTT-auth broker consolidation) reverted the Vue-bridge wiring (`uib_shack_01` full-text refs 15→1). Fixed at 14:57 in `7910cab` by rebuilding from the last healthy commit and re-applying only the broker consolidation.
- **Wipe #4 (evening):** the AetherSDR Display deploy (`200c836`, 17:20) came from an editor tab that predated the 14:57 fix — the operator had **only one tab open**, which is the key lesson: this was never a multiple-tabs problem. A single tab goes stale the moment an out-of-band flows.json change lands underneath it (git pull + `systemctl restart nodered`); the editor keeps its old in-memory model across the restart, and Deploy writes back the ENTIRE model. The wipe rode into `nrsave` → commit → merge (`7b5b72e`), so `git restore` could no longer fix it. Noticed when a later small edit (RBN SDR ping repoint) "broke the Vue dashboard" — the repoint was innocent; HEAD itself was broken.
- **Recovery (`f6df05d`):** per-node semantic rebuild — last-healthy `afe8981` base + `200c836`'s intended changes only (9 `aetherdisp_*` nodes, which are fully self-contained, + the one-line `global.set('lpState')` mirror in LP State Aggregator) + the operator's pending RBN SDR ping repoint `192.168.1.241` → `.235`. Collateral restored: `uib_shack_01` wires/ `okToGo`, all 13 Vue builders' wires, Master Dashboard + Parse+route + Raw HDG wires, AS3935 cache/replay wires, dashboard dark-mode CSS (the same catalog as 07-13). Follow-up commit completed the repoint's other half — the `stamp RBN_SDR` display addr (the stamp function owns label/addr/maxMs per the 07-31 convention).
- **Permanent guard (`flows_guard.py`, same day):** validates flows.json structural invariants (≥10 nodes wiring into `uib_shack_01` — healthy is 14 — ui_base CSS non-empty, sane node count). Installed two ways: **(a)** git `pre-commit` hook in both the Pi and Mac clones — a wiped flows.json is now uncommittable (`nrsave` fails loudly), so HEAD stays healthy and recovery is always `git checkout -- flows.json` + restart, never another forensic rebuild; **(b)** 1-minute cron on the Pi (`--cron`) — Telegram ⚠️ within 60 s of a live wipe (with the recovery one-liner), ✅ on recovery, alerts only on state transitions. The bot token is read from the running Node-RED process environment (`/proc/<pid>/environ` — same user), so no secret is copied out of the root-only systemd drop-in. Hook block-test and Telegram path verified live. `rebuild_pi.sh` installs both (Stage 7 hook, Stage 9 cron); manual steps in `REBUILD_PI.md` Step 7. New CLAUDE.md **critical rule #7** documents the protocol: reload any open editor tab after every Pi-side pull+restart, and

never deploy over the "flows changed on server" dialog — review/merge instead. If a deliberate refactor changes the invariants, update the `flows_guard.py` constants in the same commit.

Wipe #5 same evening — client hygiene defeated; server-side deploy rejection added

Within the hour, a fifth wipe — and the forensics changed the diagnosis. The operator deployed again ~18:36 after being told to reload the editor tab first. The guard cron caught it in under 2 minutes (first real ⚠️ Telegram alert), HEAD was still healthy (pre-commit hook doing its job — the wipe stayed uncommitted), recovery was the documented one-liner. But the per-node fingerprint of the broken file was the smoking gun: **513 of 518 nodes byte-identical to the 200c836 wipe model** (including `uib_shack_01`'s `okToGo:true`, which the healthy lineage doesn't carry) — *yet it also contained server-side content that only existed since 18:25* (the `stamp RBN_SDR` `.235` fix). A freshly-reloaded editor cannot produce that file. Conclusion: the deploy went through the editor's **conflict/"review changes" dialog**, which merged the newer server-side edits into a client model whose base was still the broken one — carrying the wiped wiring straight through. Client-side hygiene (reload first, close tabs) therefore cannot fully close this hole.

Third guard layer — server-side rejection (`flows_guard_middleware.js` , commit `358d9c7`): wired as `httpAdminMiddleware` in the Pi's `settings.js` (backup kept at `settings.js.bak-flowsguard`), it intercepts `POST /flows` and 400-rejects any config failing the flows_guard invariants **before anything is written or restarted** — the editor surfaces it as an error toast telling the operator to close the tab and start fresh; the runtime keeps running the healthy flows. Any client, any device, any browser state: a wiped model can no longer be deployed at all. Implementation notes: parses the request body itself and hands it to the admin API via `req.body` + `req._body` (body-parser honors the flag and skips re-reading); Node-RED POSTs the complete config regardless of deploy type, so partial deploys are covered; "reload"-type POSTs (no flows array) and all other admin routes pass through untouched. Unit-tested against the healthy and wiped configs (v1 array / v2 `{flows,rev}` / reload / garbage bodies) and probe-verified live on the Pi (garbage POST → our 400). Installed by `rebuild_pi.sh` Stage 5 on a rebuild (idempotent, inserted after `module.exports = {` ; safe that the repo isn't cloned until Stage 7 — the snippet's try/catch no-ops until the first restart after the clone). Keep the constants in sync across `flows_guard.py` and `flows_guard_middleware.js` .

ROOT CAUSE FOUND — cross-tab wires; link-node refactor ends the wipe saga

The middleware immediately earned its keep — six rejected deploy attempts at 18:54 — but the decisive clue came when the operator opened a **brand-new** editor tab and the Shack Vue node *still* showed no incoming wires. That forced a check nobody had made in three months: **which tab does each Vue feeder live on?** Answer: all 14 of them on *other* tabs — `Build Flex state for Vue` on FlexRadio, `Build DXCC…` on DXCC Tracker, etc. — wired straight across tabs into `uib_shack_01` on Vue Dashboard. Full inventory: **30 cross-tab wires** (14 Vue data fan-in, 6 Vue cmd fan-out, 9 AS3935 Tuning ↔ Lightning, 1 Master Dashboard → AS3935 Cmd via router) **+ 1 dead wire** (`Raw HDG to display` → a long-deleted node), plus a `css` field on `ui_base` that dashboard 3.6.6's node schema doesn't contain at all.

The Node-RED **runtime** executes cross-tab wires happily; the **editor** cannot represent them — each tab renders separately, so the wires have been *invisible in the editor since May*. Consequence: ANY Deploy from ANY editor session normalized them away. Every wipe date was simply an editor-deploy date, five for five. The stale-tab narrative (and the day's conflict-merge theory, hereby retracted) was misdirection: stale tabs only added the field reverts (cluster port, template sizes) seen in the early incidents. All these constructs came from the May 2026 Vue migration being written directly into flows.json — valid for the runtime, illegal for the editor, and never editor-round-tripped until 07-13.

Fix (`9ad16f7`) — link-node refactor, 28 new nodes: every cross-tab hop replaced with `link in` / `link out` pairs (the sanctioned mechanism, pure pass-through): 13 per-tab `link out`s + one `Builders →`

`Shack Vue` `link in` for the data fan-in (the Vue wiring is now *visible in the editor* for the first time); one `Shack Vue cmds →` `link out` + 6 `link in`s for the command fan-out; shared pairs for the 7 AS3935→Master Dashboard feeds, →AS3935 Cmd (2 sources), and →TEST publish. Dead wire deleted; `Rotator Controls` ui_template given real canvas coordinates (it had none — the editor stamped 0,0 on every deploy); the dark-scrollbar CSS moved from the inert `ui_base.css` field into a proper site-`<head>` `ui_template` ("D1 global CSS", GPS NTP (card) tab). All 30 original logical paths machine-verified reachable through the link hops before deploying.

Guards updated in the same commit: feeder count now traverses link pairs; the css check is replaced by a **zero cross-tab/dead wires** invariant, so the editor-illegal construct can never be silently reintroduced by a future hand edit — the pre-commit hook refuses to commit it and the middleware refuses to deploy *around* it. Verified: guards pass the refactored file, reject the pre-refactor file (31 cross-tab) and the wiped model (0 feeders). Editor deploys are round-trip-safe from here on; rule #7 rewritten accordingly.

2026-07-31

Stale Node-RED editor tab reverted `flows.json` — second occurrence

Same failure mode as 2026-07-13 (see that date's entry below and HANDOVER.md "Known weirdness"): a browser tab left open since before recent changes got Deployed from, silently pushing its stale in-memory flow model to disk and clobbering everything done since via other tabs or direct edits. The diff stat — **3016 insertions(+), 3069 deletions(-)** — is an exact match to the original incident's numbers, which makes it near-certain this was the *same* tab, never closed in the intervening weeks.

Diagnosis, this time faster: operator reported "minor Pi flow change, then `/shack` stopped working, but `/ui` still works." That D1-fine/Vue-broken split is itself now a documented triage signal — D1's widgets wire directly to their own data, no shared choke point, while every Vue card funnels through one node (`uib_shack_01`). Confirmed via `grep -c uib_shack_01 flows.json` (1) vs `git show HEAD:flows.json | grep -c uib_shack_01` (15) — matching the 2026-07-13 symptom (every Vue-bridge wire but one reverted) exactly.

Fix: same as last time — `cp flows.json /tmp/flows.json.broken.*` as a safety copy, `git restore flows.json`, `sudo systemctl restart nodered`.

Casualty: the operator's actual intended edit — repointing the VU2CPL RBN skimmer's telnet client off `vu2cpl.ddns.net` to a LAN IP — was uncommitted, so the `git restore` discarded it along with the corruption. Not yet redone as of this entry; needs the operator to confirm the right target host first (see RBN Skimmer Monitor note below).

Follow-up: "hard-refresh the tab before deploying" was already the documented mitigation after the first incident and evidently wasn't enough — the operator's own call was to route small edits through Claude editing `flows.json` directly on the Mac instead of the browser editor whenever the edit doesn't need the visual canvas, sidestepping the whole stale-tab class of bug rather than relying on remembering to refresh.

RBN Skimmer Monitor — VU2CPL telnet target repointed to LAN

Operator reported the VU2CPL tile showing red (down) despite believing it was pointed at `192.168.1.109`. Two things needed reconciling before fixing this: (1) the currently-committed flow still had `Telnet VU2CPL :7550` (`df7d1786eab4d5a2`) pointed at `vu2cpl.ddns.net`, not `.109` — almost certainly the edit lost in the stale-tab revert above. (2) `.109` is the "ubersdr box," which per `~/projects/meridian/HANDOVER.md` runs `meridian` (an SDR/IQ tool), not CW Skimmer — the operator had separately said CW Skimmer itself is currently running on the Windows box (`192.168.1.170`) as a test, which didn't obviously square with pointing the telnet client at `.109`.

Flagged both points to the operator; **confirmed `.109` is correct** — and the follow-up probe (next entry) showed why both facts are true at once: meridian *includes a DX-cluster aggregator server* (`meridian-core/src/dxcluster/`), so `.109:7550` is meridian aggregating the Windows box's CW Skimmer feed. `df7d1786eab4d5a2` 's `host` field changed `vu2cpl.ddns.net` → `192.168.1.109` (port unchanged, still `7550`), taking VU2CPL's RBN-monitoring connection off the public DDNS + router-port-forward path onto a direct LAN hop. CLAUDE.md's `df7d1786eab4d5a2` entry updated with the new host

- history.

RBN + DXCC — VU2CPL feed requires login now; prompt detection fixed in both handlers

The real reason the VU2CPL tile was red — found by asking the obvious question the host-swap above didn't answer: if `vu2cpl.ddns.net:7550` resolves and connects (it does — verified, DDNS + port-forward both fine), why were there no spots?

Live probe of `.109:7550` :

```
Welcome to Aggregator. You are client #19.\r\n
Please enter your callsign:\r\n
```

The feed is **not CwSkimmer's raw no-login port anymore** — it's meridian's DX-cluster aggregator server, and it **requires a login**. Sending `VU2CPL-1\r\n` gets the cluster prompt (`VU2CPL-1 de SKIMMER via Aggregator >`) and a working session.

The bug: both login handlers (`Login Handler VU2CPL` on the RBN tab, `login-parse-dedup-v2` on the DXCC tab — both tcp-ins use `newline:""` raw chunks) detect the prompt on the **literal last element** of `chunk.split(/\r?\n/)` . CwSkimmer's prompt has no trailing newline, so that element used to *be* the prompt — the 2026-05-17 design assumption. Meridian terminates its prompt with `\r\n` , so the last split element is the **empty string**, no `endsWith` fragment matches, and the login is never sent. Both connections sat at the prompt forever — connected (so nothing looked "down" at TCP level) but with zero spots ever flowing. The "client #19" count is consistent with weeks of such zombie sessions piling up.

Fix: both handlers now take the last **non-empty** line for prompt detection. Verified against all four wire cases (meridian's banner+prompt concatenated, meridian's prompt alone, CwSkimmer's unterminated prompt, and a `DX de` spot line → no false positive). All existing guards (<40 chars, `endsWith` fragment list — which already contained `enter your callsign:`) unchanged. Checked meridian's server for duplicate-callsign handling: sessions are independent, no kick — both tabs logging in as `VU2CPL-1` concurrently is fine.

Doc corrections that ride along: CLAUDE.md's claims that "on `:7550` there's no prompt at all, so the login handlers no-op" (written 2026-07-13, still true then) are now wrong — rewritten in the RBN node table, the DXCC node table, and the DX Clusters note. The earlier "repointed to LAN" entry above initially said "no other node needed changes" — this entry supersedes that.

Deployed + operator-verified same day: after `git pull` + Node-RED restart on the Pi, both VU2CPL connections went green with spots flowing — the RBN Skimmer tab tile and the DXCC tab's VU2CPL cluster.

Correction (same day, operator): the server on `.109:7550` is **UberSDR's RBN-style Aggregator**, not meridian's. The meridian attribution above was an over-conclusion from a source-code match — meridian's `dxcluster/server.rs` reproduces the same wire contract because it was *wire-trace modeled on* this Aggregator (its own test comments say "the wire-traced Aggregator contract lines survive untouched"), but it is not the process listening there. Everything else in the entry (the prompt bytes, the `\r\n` terminator, the handler bug, the fix) stands unchanged — the wire evidence was live, only the name of the software behind it was wrong.

FT8 follow-up (same day): operator asked why no FT8 spots. By design, not a bug: the Aggregator feed is **CW/RTTY-only** — its own help text offers just CW/RTTY display filters, and the live spots carry the classic CW-skimmer `dB` / `WPM` fields. UberSDR *is* decoding FT8/FT4 (its MQTT metrics show decoders on 10 bands) but those decodes don't feed this telnet port, and no separate FT8 spot port is open on `.109` (probed 7000/7001/7373/7300 — connection refused; only 8073, the web UI, is open). UberSDR's FT8 decodes most likely upload to PSK Reporter — check pskreporter.info to confirm. FT8 DX spots still reach the DXCC tab via the full clusters (N2WQ/VE7CC); the RBN Skimmer tab's sources are skimmer-telnet feeds only, so it stays CW-only unless UberSDR grows an FT8 spot output someday.

Power Control — powerstrip1/POWER3 relabeled "UberSDR"

The UberSDR box now draws power from powerstrip1's plug 3 (was a spare, labeled "PLUG3"). Label updated in both dashboards — D1 `Power Control Panel` template + Vue PowerCard plug list (build `v21`, `?v=21`) — and the CLAUDE.md hardware map. Labels only; no topic/behavior change. (Done via the Mac-side flows.json workflow rather than the browser editor — the label lives in both UIs, so an editor edit would have covered only `/ui`.) **Deployed + operator-verified:** label showing correctly after pull + restart.

Editor red triangles on 3 ui panels — group widths aligned to widgets

Operator asked why some ui panels show red in the editor, suspecting a link to the two flows.json revert incidents. Diagnosis: three widgets — `eee1a8b8552aa21f` (Header — Clocks + Weather, 18×2), `1edeb9703cae2fcb` (RBN Skimmer Panel, 18×8), `38a6451a95a57685` (DXCC Dashboard, 18×12) — are 18 units wide inside `ui_group`s declared width 8 (`vu2cpl_grp_header`, `1bcbc2eb8f2124aa`, `grp_dxcc_stats`). The editor red-flags any widget wider than its group.

Not incident damage: a sweep of the last 40 flows.json commits (back to 2026-05-26) shows the 8-vs-18 mismatch unchanged the whole time — it predates both stale-tab incidents and neither caused nor was caused by them. D1 renders fine regardless because the card auto-grows to fit the widget, which is why nothing ever looked wrong on `/ui`. The one real cost: invalid nodes make the editor pop the "workspace contains invalid nodes — deploy anyway?" confirmation on every Deploy, which habituates click-through on exactly the kind of warning dialog that a stale-tab Deploy rides in on.

Fix: the three groups' width set 8 → 18, matching their widgets (the repo convention everywhere else — Lightning 12/12, gpsntp 10/10). Render-neutral in practice; clears the red triangles and the deploy-time warning. **Deployed + operator-verified same evening:** red marks gone after pull + restart.

VU2CPL feed — back on `vu2cpl.ddns.net`; router forward becomes the server switch-point

Evening follow-up to the entries above. Operator decided to test meridian by **repointing the router's `:7550` port-forward** at it — and asked whether Node-RED would pick that up automatically. It wouldn't have: the morning's LAN repoint had taken the RBN tab's `tcp in` straight to `.109`, bypassing the router. So, per operator's call, the RBN node (`df7d1786eab4d5a2`) went back to `vu2cpl.ddns.net:7550` — hairpin NAT verified working from inside the LAN — making the router forward the single switch-point for which server (UberSDR's Aggregator vs meridian) feeds both tabs, with no Node-RED edits needed per switch.

Discovered while making the change: the DXCC tab's `cf2f9b095d6f1624` **had been on `vu2cpl.ddns.net` all along** — the morning's "repointed to LAN" commit (`4b17770`) only changed the RBN node, and the doc claims that both connections went LAN-direct were wrong (corrected in

CLAUDE.md's DX Clusters note + node tables). Both nodes now uniformly on the DDNS name. TODO #36 (HANDOVER) updated: the meridian cutover needs no flow change at all now — just the router forward + verification once meridian's decoders are done.

Deployed + operator-verified same evening: pull + restart on the Pi, both VU2CPL connections green with spots flowing over the DDNS path.

Network monitor — device list de-duplicated, repointed, one new tile

Immediate change: the "Mac RBN" ping target (`4a4d2801d848f882` , `192.168.1.245`) is a Mac that's no longer active. Repointed + renamed to **RBN PC/PI** (`192.168.1.164` — the Pi running `meridian`). Added a new **Ubersdr** tile (`192.168.1.109` , new `ping` node `netmon_ubersdr_ping` + `function` node `netmon_ubersdr_stamp`) for the ubersdr box, also running `meridian` . `b05f8c028b368ae9` node count 28 → 30.

Underlying fix — the actual ask. The operator's complaint wasn't just "change this one host," it was "I have to edit this in multiple places every time." True: each device's label, display address, and LAN-latency color threshold (`maxMs`) were hardcoded independently in **three** spots — the D1 `Network Monitor Panel` ui_template's `HOSTS` array, the Vue `NetworkCard` 's `hosts` array (`uibuilder/shack/src/index.js` , previously `// FORK:` -marked), and implicitly via the ping node's own host field. A relabel meant editing JavaScript in two different files in two different runtimes (Node-RED backend, browser frontend) just to keep them agreeing.

Consolidated so **each `stamp X` function is the single owner** of its device's `label` , `addr` , and `maxMs` , written straight into `flow.net_state.pings[key]` alongside the existing `ms` / `up` fields — colocated on the canvas right next to that device's `ping` node, so changing a target and its label/threshold happens in one place you're already looking at. Both dashboards now render whatever's in `pings` dynamically:

- **D1** (`Network Monitor Panel` , `4cf44df9ecf7a0b2`): the hardcoded `HOSTS` array became an `ORDER` array of keys only; the render loop looks up `label` / `maxMs` from each ping's own data (falling back to the key name / `maxMs=50` if a brand-new device hasn't reported in yet).
- **Vue** (`NetworkCard`): the hardcoded `hosts` array (with baked-in label/addr) became a `computed()` deriving `{key, label, addr}` from `state.pings` off the same kind of `ORDER` array. `upCount` / `anyDown` / `avgMs` computeds updated to read `hosts.value` accordingly (`hosts` is no longer a plain array). Build stamp → `v20` , `index.html` cache-buster → `?v=20` .

What still needs a two-place edit: the `ORDER` array itself (which keys to show, in what order) — D1's lives in the panel's own `<script>` , Vue's in `NetworkCard` . Adding a genuinely new device still means adding a `ping` + `stamp` pair in Node-RED regardless (inherent to one-ping-node-per-device), so bumping both tiny `ORDER` lists at the same time is a minor addendum, not the full label/addr/threshold triplication this fix removes. The "Please Read!!" comment node on this flow tab documents the convention for next time.

Syntax-checked (`node --check`) on all 6 stamp function bodies, the D1 panel's embedded `<script>` , and the full Vue `index.js` ; JSON round-trips clean. CLAUDE.md gained a new "Internet and network monitor" flow-specific-notes subsection with the current device table (FLOW TABS node count 28→30 too).

Mac SwiftUI app — shelved (closes HANDOVER #6)

Operator decision: not proceeding with the native macOS menu-bar app for now. It was never scaffolded — no Xcode project, no code, `~/projects/vu2cpl-shack-app/` doesn't exist. Closed as **shelved** rather than deleted: CLAUDE.md's "MAC APP" section (5-tab spec, build order, CocoaMQTT + `Network.framework` library choices) stays in the doc, clearly marked as a shelved plan rather than active or in-progress work, in case it's picked back up later. No code changes in this repo.

With this, HANDOVER #6, #1 (AS3935 outdoor install), and #35 (UberSDR alerting) are all closed — no open follow-ups remain as of this entry.

UberSDR — Telegram offline/back alerting (closes HANDOVER #35)

New nodes on `ubersdr_tab`: `ubersdr_tg_xition` (function) → `ubersdr_tg_http` (http request) → `ubersdr_tg_debug` (debug). Tab node count 6 → 9.

The UberSDR dashboard (landed 2026-07-01) already computes an `online` flag in `ubersdr_agg` — `now - sess.ts < 60000`, re-evaluated every 10 s by the `ubersdr_replay` inject regardless of new MQTT traffic. That flag was deferred as HANDOVER #35 pending two decisions: what triggers an alert, and who receives it. Picked back up today and closed both:

- **Trigger:** offline/back only. The 60 s staleness window already absorbs a single missed MQTT publish, so no extra debounce layer was added — CPU >85%, listener-count milestones, and per-band noise spikes were considered and deliberately deferred (noisier, each needs its own threshold tuning, lower value for a receiver-uptime alert).
- **Recipient:** Manoj-only, via the existing shack Telegram bot — same `TELEGRAM_TOKEN` / `TELEGRAM_CHAT_ID` systemd env vars Lightning's `Init Defaults` and DXCC's Credentials node already read. No new HA path, no new MQTT topic (`shack/alerts/ubersdr` was the alternative sketched in the original TODO — not built).

`ubersdr_tg_xition` compares `msg.payload.online` against the last value in tab-local `context` (Node-RED `flow` context doesn't cross tabs, so this reads `env.get()` directly rather than a shared Credentials node — env vars are process-global, unlike flow context). First sample after a Node-RED restart is suppressed, since there's no way to tell an initial steady state from a transition — same pattern as the Lightning tab's `as3935_health_xition`. On a real flip it builds an HTML-formatted Telegram message (🔴 `UberSDR OFFLINE` / ✅ `UberSDR BACK ONLINE`, the latter including live listener count + SDR CPU%) and sets `msg.url` / `msg.method` / `msg.headers` / `msg.payload` for the downstream `http request` node — the same blank-URL-on-the-node, `msg.url`-set-upstream convention as `tg_lightning_http`.

Wiring: tapped off `ubersdr_agg`'s existing single output, alongside `ubersdr_panel` and `ubersdr_vue_bridge` — no change to the aggregator itself, since the `online` field it needed already existed.

Edited directly in `flows.json` on Mac (per CLAUDE.md's "Claude Code on Mac ... Flow JSON review and planning" — this was a small, precisely scoped addition of 3 nodes + 1 wire, verified via a JSON round-trip against the untouched file before editing to confirm no formatting drift). Diff is exactly the new wire + 3 new node objects, nothing else touched. Pulled + Node-RED restarted on the Pi same day — **live and operator-verified on the dashboard**.

AS3935 sensor — outdoor install complete (closes HANDOVER #1)

The ESP32 bridge (`vu2cpl-as3935-bridge`) had been bench-verified since 2026-05-12 (v0.2.0 — full MQTT cmd channel, NVS-persisted tunables, on-device TUN_CAP sweep), with the physical field install as the one remaining piece of HANDOVER #1. That install is done: enclosure sealed, 18650+TP4056+solar power chain live, shade-mounted, and TUN_CAP retuned for the new outdoor environment (the sweep is location-dependent — the bench value doesn't carry over).

Sensor is off the bench and back to its rated ~40 km detection range, instead of the indoor few-km limit noted since the original bench bring-up. Hardware-only change — no firmware or Node-RED flow changes required; the MQTT contract (`lightning/as3935`, `lightning/as3935/status`, `lightning/as3935/cmd` + `/ack`) is unaffected by sensor location.

Docs updated: CLAUDE.md TODO #1, HANDOVER.md (Open follow-ups #1, Current system state, #21's trailing note), README.md Lightning Antenna Protector section.

2026-07-13

`/shack` Vue dashboard stopped populating — uibuilder upgrade + stale-editor-tab flows.json corruption, VU2CPL cluster reverted to :7550

Symptom: operator reported `/shack` cards all opened but showed no live data, and had seen a Node-RED Palette Manager notification about a new `node-red-contrib-uibuilder` version shortly before. Two unrelated root causes turned out to be stacked on top of each other; resolving the first one uncovered the second.

Cause #1 — uibuilder upgraded on disk but the runtime was never restarted. Node-RED's journal showed the Palette Manager had upgraded `node-red-contrib-uibuilder` 7.6.2 → 7.7.4 at 19:37:

```
[info] Upgrading module: node-red-contrib-uibuilder to version: 7.7.4
[info] Upgraded module: node-red-contrib-uibuilder. Restart Node-RED to use the new version
```

Node-RED logs this explicitly but does **not** auto-restart. Static front-end files (`uibuilder.iife.min.js` etc.) are served fresh from disk on every request, so the browser was immediately getting 7.7.4 client code — while the live server process (running since 2026-07-04) was still executing 7.6.2 server logic. That client/server split is almost certainly what the operator's "new uibuilder version" notification was pointing at, and is consistent with the dashboard shell loading but never receiving data.

First restart attempt collided with a concurrent `apt upgrade`. The operator restarted Node-RED at 19:46, not realising `apt upgrade` was mid-flight on the Pi (`dpkg --configure --pending` was actively relinking `/usr/bin/npm` / `/usr/bin/node` at that exact moment). uibuilder's own startup runs a `package-mgt.cjs` `pkgCheck()` that shells out to `npm list --json`; it got malformed/empty output at that instant:

```
[error] npm list installed packages FAILED. Unexpected end of JSON input
TypeError: Cannot create property 'dependencies' on string ''
  at UibPackages.pkgCheck ... at UibPackages.setup ... at pluginDefinition (uib-runtime-plugin.js)
```

which cascaded into the `Shack Vue` uibuilder node instance (`uib_shack_01`) throwing `TypeError: Cannot read properties of undefined (reading 'RED')` at flow-start — a *different* failure from the version mismatch above, coincidentally surfacing in the same troubleshooting window. Waited for `apt upgrade` to fully finish (confirmed no `apt` / `dpkg` processes left, `npm --version` / `node --version` both clean), restarted again at 20:01 — uibuilder initialised cleanly this time (`🌐 uibuilder v7.7.4 initialised`, zero `uib_shack_01` errors). Cards rendered — but still carried no live data.

Cause #2 — the real bug: `flows.json` had picked up a stale, uncommitted overwrite. `git status` on the Pi showed `flows.json` modified with a huge diff (3016 insertions / 3069 deletions against commit `39f6e25`) — file mtime 19:50:22, sandwiched exactly between the two restarts above. A raw `git diff --stat` that size looks like a rewrite, but diffing **by node id** (`{id: node}` dict comparison, ignoring Node-RED's own key-reordering noise) showed the true picture: **same node count, 0 added, 0 removed, 32 nodes with content changes** — all of it a reversion to an older state, not new work:

- Every "Build `<subsystem>` state for Vue" bridge/builder function across every card (SPE, GPS-NTP, UberSDR, Solar, RBN, Network, LP-700, DXCC, Rotator, Power, Lightning, RPi, Flex) lost its output wire into the `Shack Vue` uibuilder node (`uib_shack_01`). A full-text search for `uib_shack_01` across `flows.json` dropped from **15 references to 1** (its own node definition) — nothing fed the Vue app any data at all. This is the direct cause of "all cards open, no data."
- `df7d1786eab4d5a2` / `cf2f9b095d6f1624` (the two VU2CPL cluster `tcp in` nodes, RBN Skimmer Monitor + DXCC Tracker) reverted from port **7300 back to the pre-2026-05-17 7550**.
- `f925c3107b5f407e` (`ui_base` config node) had its dark-mode `css` field wiped to `null`.
- `38a6451a95a57685` (DXCC Dashboard `ui_template`) `width` / `height` reset from `18` / `12` (numbers) to `"0"` / `"0"` (strings).
- Two more wire drops: `48d90f1e7b0172b0` ("Raw HDG to display", lost its wire to `27a6286454485995`) and `as3935_evt_cache_last` (lost its wire to Master Dashboard `557083037f168b22`).

The uniform, interleaved nature of the reversion — some fields old, some already-fixed, spanning entirely unrelated subsystems, all landing in one write at 19:50 — points to a **stale Node-RED editor browser tab**, open since before several of these fixes shipped (likely weeks), getting a Deploy click sometime during the troubleshooting session. Node-RED Deploy always writes the *deploying* tab's full in-memory flow model to disk — if that tab's model predates work done via other tabs, `nrsave`, or direct file edits, Deploy silently clobbers all of it. There is no warning dialog for "this flow changed on disk since you last loaded it."

Fix: `git restore flows.json` on the Pi (clean discard back to `39f6e25` — confirmed 14 wires into `uib_shack_01` restored, VU2CPL back on port 7300), then `sudo systemctl restart nodered`. Clean boot, uibuilder v7.7.4 initialised with no errors, Vue cards populated with live data — operator-confirmed.

Follow-up, deliberate this time: with the dashboard fixed, VU2CPL's cluster connections on port 7300 were still throwing continuous `ECONNREFUSED 103.203.39.165:7300` — the auth-required CwSkimmer port isn't currently reachable from outside. Operator explicitly requested reverting both VU2CPL `tcp in` nodes (`df7d1786eab4d5a2` "Telnet VU2CPL", RBN Skimmer Monitor; `cf2f9b095d6f1624`, DXCC Tracker) to the old "raw" no-login port **7550** — this time permanently, until 7300 comes back up on their end. Edited `port` + `name` on both nodes directly in `flows.json`. Both connections came up clean on restart; no changes needed to either tab's login-handler code, which already no-ops gracefully when there's no login prompt to answer (same as the existing VU2OY `:7550` connection, which never needed login handling either). Committed from the Pi as `98daacb`.

Process gotcha caught mid-fix: the first pass at the port edit used `json.dump(d, f, indent=4)` with Python's default `ensure_ascii=True`, which escapes every non-ASCII character (emoji in node names, en-dashes in comments) into `\uXXXX` sequences — Node-RED itself never does this, so the diff inflated to 146 lines across the whole file for what should have been a 4-line change. Caught before committing; fixed by reloading the file and re-dumping with `ensure_ascii=False`, which collapsed the diff back to exactly the intended `port` + `name` fields on the two nodes.

Lessons for next time:

1. Palette Manager module upgrades never auto-restart Node-RED — a module can sit "Upgraded" on disk (front-end assets serving immediately) while the live process keeps running old server code indefinitely, until someone restarts it.
2. Don't restart Node-RED while `apt upgrade` / `dpkg` is actively running — binaries can be transiently relinked mid-upgrade, and a restart at that instant can catch spawned child processes (like uibuilder's internal `npm list` package-check) in a broken PATH/binary state that resolves itself once the upgrade finishes.

3. Long-lived Node-RED editor browser tabs are a real hazard. A tab left open across multiple `nrsave` / direct-file-edit sessions holds a stale in-memory flow model; deploying from it silently reverts everything done via other paths since that tab last synced. Hard-refresh any editor tab that's been open a while before deploying from it.
4. When a `git diff --stat` on `flows.json` looks implausibly large, don't assume it's all real — diff **by node id**, ignoring Node-RED's own re-serialization noise, before deciding what actually changed.
5. `ensure_ascii=False` (and matching Node-RED's `indent=4`) is mandatory for any script that hand-edits `flows.json` — the default Python JSON encoder's unicode escaping produces a diff Node-RED itself would never write.

No flows.json node count change (501 → 501, restore + 2-field edit only). See HANDOVER.md "What landed this week" 07-13 for the condensed version.

2026-07-01

UberSDR — receiver-monitor dashboard (new flow tab, both `/ui` + `/shack`)

New flow tab: `UberSDR` (`ubersdr_tab`, 6 nodes) · **D1 group:** `ubersdr_grp` on the Shack Monitoring tools tab · **Vue:** `UberSdrCard` (topic `ubersdr`), `CARDS.ubersdr`, build `v16` / cache-buster `?v=16`.

An UberSDR multi-channel receiver publishes metrics to the shack broker (192.168.1.169); added a dashboard for it on both UIs.

MQTT consumed (read-only):

- `ubersdr/metrics/sessions` — `count` + `sessions[]` (per-session frequency, mode, channel, bandwidth, per-thread CPU, audio/waterfall/total kbps, country, and `is_internal` / `is_spectrum` flags).
- `ubersdr/metrics/voice_activity/<band>` — 12 bands (2200m→10m): noise floor, threshold, detected-voice `activities[]` + `total_activities`.

Aggregator (`ubersdr_agg` function): topic-routes each input — caches the session roll-up (`flow.usdr_sess`) or one band's voice-activity (`flow.usdr_va[band]`) — then emits one flat payload merging both. Sessions are categorised into **real listeners** (`!is_internal`), **decoders** (`is_internal` + id contains `decoder`), and **spectrum / noise-floor monitors** (`is_spectrum`); listeners are bucketed to a ham band by frequency, with an `Other` catch-all for out-of-band tuning (which is actually the largest bucket). Per-thread CPU and total kbps are summed (→ Mbps). A 10 s replay inject re-emits from cache so the panels rehydrate on page open. Snapshot at build time: 64 sessions = 30 listeners · 21 decoders · 13 monitors, ~121 Mbps egress, ~50 % SDR CPU, across 12 bands.

Both dashboards render the same flat payload — D1 `ubersdr_panel` ui_template directly; Vue `UberSdrCard` via `ubersdr_vue_bridge` → `uib_shack_01` (topic `ubersdr`). Four sections: summary tiles (listeners / sessions breakdown / egress Mbps / SDR CPU / band count), a listeners-by-band bar list, a per-band noise-floor + voice-activity table (noise colour-graded; 0/absent → "—"), and a decoders/monitors count line.

Verified before wiring any UI: the aggregator was run standalone against a live captured `sessions` payload + all 12 `voice_activity` bands, driven exactly as Node-RED does (topic routing, string-payload parse), confirming the counts (64/30/21/13), egress, and per-band merge. `flows.json` + `clublog_dxcc_tracker_v7.json` (Rule #4) committed together.

Same-day enhancement — user vs internal-channel split (build `v17`). The reliable discriminators are UberSDR's own flags, not mode/frequency: `is_internal = false` → real user (UUID id, `ubersdr-<uuid>` channel); `is_internal = true && !is_spectrum` → decoder (the mode lives in the `decoder-<band>-<mode>` channel string); `is_internal = true && is_spectrum = true` → noise-floor monitor; and the lone `is_internal = false && is_spectrum = true` = a real user watching the waterfall (not audio). `ubersdr_agg` now also emits **`decodersByMode`** (parsed `ft8|ft4|cw|wspr|rtty|js8|psk` counts — live: FT8 10 · FT4 8 · WSPR 3) and a **`viewers`** count (spectrum-only real users). Both cards gained a "Decoders by mode" line and split the Listeners tile into audio vs waterfall. Aggregator re-verified against live data; `?v=17`.

Emphasis flip (build `v18`). Operator clarified that the **waterfall session** (`is_spectrum && !is_internal`) is the actual *human* listener — the audio/IQ streams (`!is_internal && !is_spectrum`) are automated consumers, not people. So both cards now **headline the waterfall count** as the big bold "Listeners (waterfall)" number (`state.viewers`, 28 px / green), demote the total-sessions and IQ-stream counts to small sub-text, and relabel the bar list "Sessions by band" (it buckets all user sessions, not humans). The collapsed header's "● N listening" also uses the waterfall count now. `?v=18`.

Bar graph scoped to waterfall-only (build `v19`). The "by band" bar list still bucketed *all* real-user sessions (audio/IQ streams included), so it disagreed with the new headline number. `ubersdr_agg`'s `byBand` now only buckets sessions where `is_spectrum && !is_internal` — verified against live data: `listenersByBand` drops from an 11-band spread to `[{"band":"40m","n":1}]`, matching the single real waterfall listener. Both cards relabeled back to "Listeners by band (waterfall)" now that the label is accurate again. `?v=19`.

Repo tooling — retired the `clublog_dxcc_tracker_v7.json` DXCC-tab extract

The file was a filtered copy of the DXCC Tracker tab's nodes out of `flows.json` — a fossil from when the DXCC Tracker was developed as its own standalone flow before being merged into this repo (its earliest commit is literally titled "v7 clean start", 2026-04-14). It carried no information not already in `flows.json`, and existed only to be kept in sync — via CLAUDE.md rule #4 and a matching block in the `nrsave` Pi function — every single time `flows.json` changed. Pure upkeep cost, no benefit.

Removed: `clublog_dxcc_tracker_v7.json` (`git rm`). **Retired in place:** CLAUDE.md rule #4 (kept its number — 11 historical changelog/handover entries reference "rule #4" by number; renumbering would've silently repointed them at rule #5 instead). **Simplified:** the `nrsave` bash function (both the doc in `REBUILD_PI.md` and the live generator in `rebuild_pi.sh`) now just stages + commits `flows.json`, no extract-regen step. **Also cleaned:** `DXCC.md`'s "Files on Pi" table, its "Standard Commit Sequence" (now just a PDF-regen reminder per rule #3, which is unaffected), a stray "Flow file" row that mislabeled the extract as *the* flow file, `README.md`'s repository-layout tree, and `HANDOVER.md`'s "Key files" list. `DXCC_Tracker_README.pdf` regenerated per rule #3 since `DXCC.md` changed.

Resolved 2026-07-01, same day. The *live* `nrsave` function in `~/.bashrc` on the Pi had the old extract-regen body (Claude Code can't reach the Pi directly to fix it) — Manoj hand-swapped it on the Pi via a byte-exact Python replace (backed up to `~/.bashrc.bak.YYYYMMDD_HHMM` first, matching the HANDOVER #17 convention), verified by `sed`-printing the function back out. Confirmed simplified to just stage + commit `flows.json`, no extract step.

Repo tooling — SHACK_CHANGELOG.pdf caught up + rule added (new rule #6)

Found `SHACK_CHANGELOG.pdf` (6.7 MB, git-tracked) **12 commits stale** (last regenerated 2026-06-12, commit `9f11bb6`) while `SHACK_CHANGELOG.md` had substantial edits since — unlike the `DXCC.md` / `DXCC_Tracker_README.pdf` pairing (CLAUDE.md rule #3), nothing enforced this one, so it had quietly drifted. Regenerated (`npx --yes md-to-pdf SHACK_CHANGELOG.md` , ~3s, no rename needed) and added **CLAUDE.md rule #6** mirroring rule #3, so this can't silently lapse again.

2026-06-27

SPE Amplifier — RAW / AVG / PEAK power-meter modes in both `/ui` and `/shack`

Repos: `spe-remote` (server) + `vu2cpl-shack` (flows + Vue) **Nodes touched:** `ws_format_state` , `ws_panel_node` ; `uibuilder/shack/src/index.js` SPECard (+ `index.html` cache-buster)

Added a power-meter mode toggle to the SPE Output Power bar on **both** dashboards. RAW = the live ~25 Hz reading; AVG = a ~1 s EMA; PEAK = a peak-hold that pins the crest ~2.5 s then decays back toward live.

Server (`spe-remote` , commit `f5537ec` on main): `AmplifierState` (`protocol.py`) gains `p_out_avg` + `p_out_peak` floats, both computed in `SerialHandler._consume_csv_frame` and reset across an op/tx transition; raw `p_out` untouched. EMA α=0.15; peak-hold decays at a constant 600 W/s. **`p_out_peak` is a sampled (25 Hz) peak, not true envelope PEP** — documented in the field comment + README.

Flow (`ws_format_state`): maps `p_out_avg` → `pwr_avg` and `p_out_peak` → `pwr_peak` , **null when the server field is absent** (so an old gateway is distinguishable from a genuine 0 W and the UI hides AVG/PEAK instead of offering a dead button). The single flat payload already fans out to both D1 (`ws_panel_node`) and Vue (`vue_spe_bridge_01`), so one edit feeds both dashboards.

D1 (`ws_panel_node`): RAW/AVG/PEAK pill group in the Output Power header; `speRenderPwr()` picks the source with fallback-to-raw, toggles the buttons, and caches the last payload (`speLastD`) so a pill click re-renders without a new message; `window.speSetMode()` is the global the inline `onclick`s call. Mode persists in `localStorage` `speMeterMode` .

Vue (`/shack` SPECard): RAW/AVG/PEAK `.pill-group` pills, a `pwrDisplay` computed every meter element reads from, and `pwrHasAvg` / `pwrHasPeak` gating so a pill shows only once the server is sending its field. Shares the same `speMeterMode` key — same origin as `/ui` , so the two dashboards stay in sync. Build stamp → `v15` , `index.html` cache-buster → `?v=15` .

Decay-bug fix vs the source package. Landed as a contributed package (adersh fork) whose peak-decay math subtracted the *total* elapsed `decay_s` from the running peak every frame — an accelerating, sample-rate-dependent collapse (1500→0 in ~0.46 s) rather than the documented constant 600 W/s. The version landed here uses a per-frame `dt` -scaled decrement (simulated 2.50 s for 1500→0, frame-rate independent). If the package is ever re-imported, re-apply the fix.

Graceful degradation: every layer is independent — server, flow, and each dashboard fall back to RAW if the others aren't updated, so they deploy in any order.

Documentation — stale-reference sweep

Audited every doc against the live `flows.json` / code and fixed the staleness found:

- **CLAUDE.md FLOW TABS table:** SPE row → the real tab `spe_ws_tab_01` ("SPE (WS)", 12 nodes; was the long-deleted serial-owning `648eb83c2566c7b6`); SPE group `vu2cpl_grp_spe` → `vu2cpl_grp_spe_ws` ; Lightning group `grp_main` → `8b723cd03854ac2c` ; All Power Strips group trimmed to `vu2cpl_grp_power` (the `vu2cpl_grp_energy` group is gone); all node counts re-counted live (with a note that counts drift).

- **CLAUDE.md Dashboard tabs table:** dropped the two `ui_tab`s that no longer exist (`dd11372f9c492be8` Lightning-detect, `tab_ui_dxcc`) — both folded into the VU2CPL Shack tab.
- **CLAUDE.md AS3935 Control Panel:** `223cb2ce733c5d3f` was deleted as a standalone node 2026-05-27 (HANDOVER #26, `38bf3fb`) and absorbed into the Lightning Master Dashboard (`557083037f168b22`); the three references that still presented it as a live AS3935-Tuning-tab node now say so.
- **CLAUDE.md rotator timer note:** dropped the obsolete "change 60s → 5min for production" — the node has been `5 * 60 * 1000` since 2026-05-10.
- **CLAUDE.md header:** "Last updated" April → June 2026.
- **HANDOVER.md header:** period end → 2026-06-27, last commit → `ea70bf0` .
- **README.md:** AS3935 bridge v0.1.1/05-11 → v0.2.0/05-12; SPE power-on corrected to the WebSocket/gateway path (`power_spe_on.py` is the fallback); rotator auto-off 60 s → 5 min.
- **DXCC.md:** VU2CPL cluster port 7550 → 7300 (moved 2026-05-17); `DXCC_Tracker_README.pdf` regenerated to match (Rule #3).

`spe-remote/handover.md` cross-references were fixed in that repo separately (`8e85cfb`).

2026-06-19

SPE Amplifier — ATU band-sweep UI in both `/ui` and `/shack`

Tabs: SPE (WS) (`spe_ws_tab_01`) + Vue Dashboard tab (uibuilder) **Nodes touched:** `ws_btn_router` , `ws_parse_node` , `ws_panel_node` , `vue_spe_cmd_router_01` ; `uibuilder/shack/src/index.js` SPECard

spe-remote's Phase 2 work (orchestrated TUNE + ATU band sweep via Flex 6000) needed a control surface inside the shack dashboard. Added SWEEP buttons to both dashboards — the legacy Node-RED `/ui` SPE Panel and the Vue `/shack` SPECard — sharing the same Pi-side `tune_event` broadcast.

Routing changes in `flows.json` :

- `ws_btn_router` (Button → WS command) gained a pass-through for tune-orchestration commands. The existing `map` lookup still translates UPPERCASE button names (TUNE, MODE, OFF_SPE …) to lowercase; commands starting with `tune_band:` or equal to `tune_single` / `tune_stop` now fall through unchanged.
- `ws_parse_node` (Parse + route) widened from 3 to 4 outputs. The new 4th output handles incoming `tune_event` JSON: sets `msg.topic = 'spe/tune'` and fans out to both `ws_panel_node` (Node-RED panel) and `uib_shack_01` (Vue dashboard via uibuilder).
- `vue_spe_cmd_router_01` learnt two new payload types: `speTuneBand` (translates to `tune_band:<value>`) and `speTuneStop` (translates to `tune_stop`). Both go through `ws_btn_router` 's pass-through.

UI changes:

- `ws_panel_node` (Node-RED `/ui` SPE Panel template): 4-button controls row widened to 5 columns with a SWEEP button after Tune. SWEEP toggles a small panel below the controls — band picker (6-col grid, 11 bands), status/phase display, message line, progress bar, Start/Stop. JS hooks use the captured `scope` (existing `speAmpToggle` pattern) so `scope.send()` actually reaches the router. A `scope.$watch("msg", …)` watches for tune_event payloads delivered via the new 4th wire.
- `uibuilder/shack/src/index.js` SPECard: primary-controls grid widened from 3 to 4 cells, gains a SWEEP button. Setup adds reactive sweep state (`sweep.show / running / phase /`

`message / current / total`), `sweepPct` + `sweepPhaseColor` computeds, `startSweep` / `stopSweep` methods that emit `{topic:'spe/cmd', payload:{type:'speTuneBand', value:band}}` over uibuilder, and a second `uibuilder.onTopic('spe/tune')` subscription that calls `handleTuneEvent(p)` to drive the panel.

First-pass scope-binding bug: the initial Node-RED panel JS used `window.scope.send(...)` — that path doesn't exist; the dashboard passes `scope` as an IIFE parameter, not on `window` . Symptom: buttons rendered but did nothing. Fixed in commit `b4c8c11` by wrapping the script body in `(function(scope){ … })(scope)` and switching to `scope.send` . The Vue side never had this issue because uibuilder is a normal Vue component.

Backup of the original flows.json from the first patch is at `/tmp/flows.json.bak` should rollback be needed.

Related commits: `9eb615f` (Node-RED first cut), `b4c8c11` (scope fix), `945f96c` (Vue add).

2026-04-22

SPE Amplifier — Power meter scales with selected power level

Tab: SPE (`648eb83c2566c7b6`) **Node:** SPE → Dashboard Formatter (`8f7d96af1a4c24a9`)

The output-power bar was always scaled to a 1500 W full-scale regardless of the amp's selected power level. It now auto-scales based on `col10` (L / M / H):

Level	Full scale
L (Low)	500 W
M (Middle)	1000 W
H (Maximum)	1500 W

The bar text, percentage width, and amber/red thresholds (70 % / 90 %) all follow the new max automatically. Dashboard template (`e5b15d8e700bf002`) was already reading `d.pwrMax` — no UI change needed.

Power Control Panel — Fix stale "ON" defaults + reliable state refresh

Tab: All Power Strips (`b76a5310767803b4`)

Three related bugs resolved:

1. **HTML defaults lied.** Rotator, USB2, and USB3 plug tiles had `class="pw-plug on"` hardcoded in the template (`780b75182df31634`) — they showed ON on every page load until an MQTT `stat/.../POWER<n>` arrived. Since Tasmota only publishes `stat/.../POWER<n>` on a *state change*, the stale classes could persist indefinitely. All three now default to `off` .

2. **Dashboard ignored Tasmota's query responses.** `Power State → Dashboard` (`f8c3c072b381bd1c`) was filtering out `/RESULT` topics entirely, so responses to state queries (which Tasmota delivers on `stat/<device>/RESULT` as `{"POWER<n>":"..."}`) never reached the panel. The function now parses the RESULT JSON and extracts every `POWER<n>` key into the shared `power_states` object.

3. **Startup poll only queried POWER1.** The four `Poll <device>` function nodes (powerstrip1, powerstrip2, powerstrip3, 4relayboard) emitted a single empty-payload query to `cmnd/<device>/Power`, which Tasmota interprets as a POWER1 query only. They now loop over every outlet on the device, emitting one query per POWER:

   ```
   for (var i = 1; i <= 5; i++) {
       node.send({ topic: 'cmnd/<device>/POWER' + i, payload: '' });
   }
   ```

   Combined with fix #2, this means every 30 s (and on deploy) the dashboard receives a fresh state for every outlet — the "always on" bug cannot persist more than ~30 s.

Solar Conditions — Geomagnetic K/A column indices

Tab: Solar (`590e889d44815afb`) **Node:** Parse K and A (`6e22622d54f653b1`)

The daily geomagnetic indices text file from NOAA (`services.swpc.noaa.gov/text/daily-geomagnetic-indices.txt`) has three index groups per data row:

Columns	Group
3	Middle-Latitude A
4–11	Middle-Latitude 8 × K
12	High-Latitude A
13–20	High-Latitude 8 × K
21	**Planetary A (Ap)**
22–29	**Planetary 8 × Kp (fractional)**

The parser was reading `cols[19]` for Ap and scanning `cols[20]..cols[27]` for Kp — that's the high-latitude K range plus the first six planetary Kp values, off by two columns in every case. Corrected to `cols[21]` for Ap and `cols[22]..cols[29]` for Kp. The length guard was also bumped from `< 28` to `< 30`.

Observed effect: yesterday's real values (2026-04-21) are A = 19, K ≈ 2.00. Before the fix the dashboard was showing A ≈ 3, K ≈ 4.00.

Solar Conditions — Aggregator branch discriminators

Tab: Solar (`590e889d44815afb`) **Node:** Solar State Aggregator (`d8ed9693432963df`)

The aggregator's SFI, K-Index, and A-Index branches all accepted any numeric `msg.payload` with a `msg.label`. In practice:

- The **SFI branch** matched every K / A message and happened to survive only because the SFI HTTP response arrived *last* in each poll cycle.
- The **K branch** had no discriminator against A, so the A-Index message (second output of Parse K and A) would overwrite `st.k` with the A value.

This was invisible while the parser was producing bogus near-identical K and A values (pre-column-fix). Once the column-index fix started producing a correct A = 19, the K gauge began showing 19.0 with the A's "Unsettled" label.

The three branches now discriminate by their gauge's `control.sectors` shape:

Metric	Discriminator
SFI	`sectors[0].val === 50` (gauge min is 50, not 0)
K	`sectors[0].val === 0.0 && sectors[1].val < 15` (K's 4.5)
A	`sectors[0].val === 0.0 && sectors[1].val > 10` (A's ≥ 35.5)

Rotator Auto-Off Timer — 60 s countdown reset loop

Tab: All Power Strips (`b76a5310767803b4`) **Node:** Rotator Auto-Off Timer (`05f0ddeb566a90fc`)

Symptom: rotator turned on, 60 s timer counted down to 0, then reset itself back to 60 s; physical switch stayed ON indefinitely. Cause: the timer unconditionally restarts whenever a `stat/powerstrip1/POWER2` = `"ON"` message arrives, and something (Tasmota echo, state-poll response, possibly a SetOption-related republish) keeps sending `"ON"` for an already-on outlet.

Fix: make the timer **idempotent** and add a post-OFF cooldown:

- If a timer is already running, additional `"ON"` messages are ignored (they do not reset the countdown).
- After the timer fires OFF (or an OFF is received), a 10 s cooldown begins. Any `"ON"` during that window is ignored — this breaks the reset loop regardless of its source.

Also fixed the status string from "Timer running — 5 min" to "Timer running — 60 s" (it has always been 60 s in code; the label was stale from an earlier intended value).

2026-05-01

AS3935 Lightning Sensor — Service revived after 10-day silent outage

Service: `as3935.service` (systemd) → `/home/vu2cpl/as3935_mqtt.py`

The lightning sensor daemon had been **dead since 2026-04-21 09:14 IST** and went unnoticed for 10 days because the dashboard had no liveness indicator. Discovered when a real storm fired no MQTT events.

Root cause of the original crash: at boot, `paho-mqtt` 's `client.connect()` called before the network was ready and threw `OSError: [Errno 101] Network is unreachable` . systemd hit `StartLimitBurst` and gave up. The script had no retry loop and no liveness signal, so silence looked identical to "no nearby strikes."

Hardening applied to `as3935_mqtt.py`

Change	Why
`mqtt_connect_with_retry()` (2 → 60 s exponential backoff)	Survives boot-time network race

Last Will & Testament on `lightning/as3935/status`	Broker auto-publishes `offline` if script dies
Startup retained publish to `lightning/as3935/status` (`event:"ready"`)	Visible in MQTT Explorer immediately
30-second heartbeat to `lightning/as3935/hb` (retained, includes `uptime_s` + per-event counters)	Silence > 60 s now unambiguously means dead
`on_connect` / `on_disconnect` callbacks log broker state	Easier diagnosis
I²C self-test on startup — reads `REG_CFG0` and logs the value	Catches dead-bus cases before the IRQ loop
Clean SIGTERM handler — publishes `offline` + GPIO cleanup	systemd `stop` no longer leaves a stuck GPIO claim
`try/except` around IRQ handler body	One bad I²C read can't crash the daemon
Event counters (lightning / disturber / noise / irq) in heartbeat	Observable from MQTT Explorer without reading logs

Antenna mode parametrization

The original script's `set_outdoor_mode()` wrote `0x0C` to `REG_CFG0`, which actually maps to `AFE_GB = 0x06` — *less* sensitive than the datasheet's "outdoor" value (`AFE_GB = 0x0E` → write `0x1C`). For the current indoor antenna placement, gain should be higher still: `AFE_GB = 0x12` → write `0x24`.

Replaced with `set_antenna_mode("indoor" | "outdoor")` controlled by a top-of-file `ANTENNA_LOCATION` constant. Currently `"indoor"` since the ferrite antenna is indoors. The mode is also published in the retained status payload so the dashboard can surface it.

Note: an indoor AS3935 has effective range of a few km at most (walls block the H-field) and is prone to false disturbers from local RF. Open-Meteo CAPE is the primary detection layer until the antenna is moved outdoors.

Required systemd unit override

`Environment=PYTHONUNBUFFERED=1` must be in `/etc/systemd/system/as3935.service.d/override.conf` — without it, `print()` output is block-buffered and never reaches `journalctl`. This is what made the running-but-silent state indistinguishable from "crashed."

Lightning Antenna Protector — Open-Meteo: `lightning_potential` is null in IN

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes:** Build Open-Meteo URL (`3d61a4f561d40c70`), Parse Open-Meteo → Strike (`593f22a507b46335`)

Symptom: dashboard's OPEN-METEO MONITOR badge stuck at "Waiting for data..." indefinitely; auto-disconnect never fired even during real storms.

Root cause: the flow polled the `lightning_potential` parameter, which Open-Meteo's ICON-Global model returns as **all- `null` for India** (verified: 0/24 non-null entries for 13.065°N 77.806°E). Every poll hit `index === null`, the parser silently returned `null`, and nothing reached the dashboard or strike chain.

A second issue compounded it: the parser's mapping `km = (1 − index/100) × 200` assumed `lightning_potential` was a 0–100 percentage. Open-Meteo actually returns it as J/kg (LPI, an ICON-model quantity that typically ranges 0–25 even for severe storms). Even if the data had been present, the formula would have mapped a 25 J/kg severe storm to 150 km — well above any practical threshold — so disconnection would still never trigger.

Fix

Build Open-Meteo URL now requests `cape`, `weather_code`, and `precipitation_probability` (hourly + current weather_code). All three have continuous coverage globally.

Parse Open-Meteo → Strike rewritten with two outputs:

Decision	Synthetic km
`current.weather_code ∈ {95, 96, 99}` (TS now)	0 (overhead)
Hourly `weather_code ∈ {95, 96, 99}`	0
Showers (80–82) + CAPE ≥ 1000	20
CAPE ≥ 2500	10 (severe)
CAPE ≥ 1500	40 (strong)
CAPE ≥ 800	100 (moderate)
else	calm — no strike emitted

Output 1 → existing Parse Strike chain (only when storm-relevant). Output 2 → reserved for Master Dashboard live status (wire pending).

CAPE is *predictive* — it catches the convective setup hours before the storm peaks, which is what we want for a pre-emptive antenna disconnect. weather_code 95/96/99 is *deterministic* — when the model reports actual thunderstorm at the grid cell, force overhead.

Lightning Antenna Protector — Strike source label propagation

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes:** Parse Strike (`26ddff0cbbfe5fc1`), Trigger Disconnect (`d62fb0c3c40f03b7`)

Event log lines for Open-Meteo strikes were appearing as `DISCONNECT: Blitzortung strike N km` because:

1. **Parse Strike Case 1** (object payloads from Parse Open-Meteo and test injects) constructed `msg.strike` without copying the upstream `payload.source` field. Source was lost on every object-shaped strike.
2. **Trigger Disconnect** had a fallback default of `'Blitzortung'` when `msg.strike.source` was missing — a leftover from when Blitzortung TCP was the original real-time source (Cases 2 and 3 in Parse Strike, which parse a binary/string TCP feed and are currently dead code — no upstream node feeds them).

Fix

```
// Parse Strike Case 1
msg.strike = { lat, lon, time, pol, region, source: d.source };  // added source

// Trigger Disconnect
const src = msg.strike && msg.strike.source ? msg.strike.source : 'unknown';
```

Cases 2/3 left intact — if Blitzortung TCP is wired in later, set `source: 'Blitzortung'` there at the same time.

Verified: log now reads `DISCONNECT: Open-Meteo strike 0 km`.

Lightning Antenna Protector — Dashboard refresh persistence

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes:** Strike → Dashboard (`51367d456e71fb3e`), Master Dashboard (`557083037f168b22`), Refresh Stats (`61dca3d98a0e4c28`), Clear map every 30 min (`55f94d9dde0dc893`), new node Strikes Replay, new node Clear All

Symptom: refreshing the dashboard browser tab wiped the **event log** and **strike map markers** until the next event arrived. Both data sets were held only in dashboard-local JS variables (`var strikes=[]` and the `logLines` innerHTML), with no server-side persistence or replay path.

Fix — persist on the flow side, replay on the existing 30 s tick

1. **Strike → Dashboard** now appends each emitted strike to a flow-context array (`flow.strikes`), capped at 800 entries (FIFO). The `Format Log` node was already persisting log lines to `flow.event_log` .
2. **New `Strikes Replay` function node** — reads `flow.strikes` and emits `{type:'strikes_replay', list:[...]}` to Master Dashboard.
3. **`Refresh Stats`** (the existing 30 s ticker) now also fans out to:
    - `Log → Dashboard` (re-emits cached event log)
    - `Strikes Replay` (re-emits cached map markers)
4. **Master Dashboard** picks up a new message type:

```
if (d.type === 'strikes_replay') {
    strikeLayer.clearLayers();
    strikes = [];
    d.list.forEach(s => {
        strikes.push(s);
        L.circleMarker([s.lat, s.lon], {
            radius: 5, color: s.color, fillColor: s.color,
            fillOpacity: 0.9, weight: 1
        }).addTo(strikeLayer);
    });
    document.getElementById('lzCnt').textContent = 'Strikes: ' +
strikes.length;
    return;
}
```

5. **New `Clear All` function** sits between the 30-min clear-map inject and Master Dashboard:

   ```
   flow.set('strikes', []);
   flow.set('event_log', []);
   return { payload: { type: 'clear' } };
   ```

   Clears both the server cache and the dashboard simultaneously, so the next replay tick doesn't re-paint old strikes onto a freshly cleared map.

Behavior after fix

Action	Result
Page refresh	Log + strike markers re-appear within 30 s
New strike	Immediate update; also persisted to `flow.strikes`
30-min auto-clear	Map cleared on dashboard *and* server cache
Node-RED restart	Flow context lost (default). Add `flow.context.persistent=true` for cross-restart durability — separate change.

2026-05-06

AS3935 — IRQ pin was GPIO17 in software, GPIO4 in hardware

Script: `as3935_mqtt.py` **Helper:** `as3935_tune.py` (new)

Symptom after the May 1 revival: heartbeat counters stayed at `{lightning:0, disturber:0, noise:0, irq:0}` indefinitely. MQTT, I²C, and `status: "ready"` all worked, but no IRQ events ever fired.

Root cause

The original script declared `IRQ_PIN = 17`, but the AS3935 board's INT line was physically wired to **GPIO4** (Pi physical pin 7). Since the chip's I²C path was healthy, every diagnostic short of an end-to-end IRQ test passed. The mismatch had been there since first deploy; it was hidden because the chip-self-test only exercised I²C.

How it was found

A two-step diagnostic:

1. **Pull-test on GPIO17** — with the as3935 service stopped, sampled GPIO17 with `PUD_DOWN` / `PUD_OFF` / `PUD_UP`. Pin tracked the pull setting → genuine floating input → not connected to anything.
2. **SRCO scan** — set `DISP_SRCO=1` in chip register `0x08[6]` to route the chip's internal RC oscillator onto whatever pin its INT line is bonded to. Polled every accessible Pi GPIO (4, 5, 6, ...,
   27. for transitions over 200 ms. **GPIO4 showed 12,162 transitions; every other pin showed 0.** Conclusive.

`IRQ_PIN = 4` change applied. Heartbeat counters started incrementing on the next NF=0 test (see below).

AS3935 — LC tank tuning to 500 kHz

Helper: `as3935_tune.py` (new)

The AS3935 has 16 internal tuning capacitor settings (0–15, ~8 pF per step) at register `0x08[3:0]`. The script wasn't using any of them — it relied on whatever stray capacitance the antenna+wiring happened to give. For accurate sferic detection, the tank should resonate at 500 kHz ± 3.5 %.

`as3935_tune.py`

New helper that:

1. Configures `LCO_FDIV = 3` (÷128 divider — slow enough for Python event-detection to count edges reliably; ÷16 at ~31 kHz overruns the user-space callback)
2. Sweeps `TUN_CAP` 0..15, samples 2 s of edges per setting, calculates frequency = `edges × 128 / 2 s`
3. Reports the table + recommends the cap value closest to 500 kHz
4. Flags out-of-spec results (>±3.5 %) with diagnostic hints

Must be run with `as3935.service` stopped (otherwise the daemon holds GPIO4) and the antenna in its **final mounting position** — moving the antenna afterwards changes stray capacitance and invalidates the tuning.

Result for current installation

TUN_CAP	pF	Hz	err %
0	0	516,480	+3.30
9	72	501,504	+0.30
10	**80**	**499,904**	**−0.02**
11	88	498,560	−0.29
15	120	493,056	−1.39

All 16 cap settings happened to be within spec (typical of a clean antenna circuit); `TUN_CAP=10` lands at −0.02 % err. Baked into the script as `TUN_CAP = 10`.

AS3935 — Add `CALIB_RCO`, INT flush, retained-status enrichment

Script: `as3935_mqtt.py`

Three additions to the chip-init sequence in the daemon, all chasing the same goal of making "is the chip happy?" answerable from MQTT without journal grep:

1. **`CALIB_RCO` at every startup** — the AS3935 has internal TRCO and SRCO oscillators that benefit from a one-time post-power-on calibration. The original script never called it; some chips work without, but timing-sensitive code paths can misbehave. Added: write `0x96` to register `0x3D`, wait 5 ms, verify `CALIB_DONE` (bit 7) set and `CALIB_NOK` (bit 6) clear in both `0x3A` (TRCO) and `0x3B` (SRCO). Result logged + published.
2. **Pending-INT flush after configuration writes** — register writes during init can transiently spike the IRQ line. Reading `0x03` (INT) clears it. Without this flush, the first real event sometimes arrived as a no-op rising edge that the `add_event_detect` callback then ignored because INT bits read 0.

3. **Retained `lightning/as3935/status` payload now includes** `tun_cap`, `irq_pin`, `calib_trco`, `calib_srco`. Status is re-published *after* calibration completes so the retained message reflects the actual result (the `on_connect` callback fires before init finishes).

Final init log

```
[init] AS3935 CFG0=0x24 (i2c bus 1 addr 0x03)
[mqtt] connected to 192.168.1.169:1883
[init] AFE_GB set for indoor antenna (CFG0=0x24)
[init] TUN_CAP set to 10 (80 pF)
[init] CALIB_RCO  TRCO=OK (0xA3)  SRCO=OK (0xA5)
[init] Cleared pending INT: 0x0
AS3935 ready — interrupt mode on GPIO4, noise_floor=4, antenna=indoor, tun_cap=10
```

Field verification

During the May 1 storm window (NF=0 momentarily), the chip caught **209 noise events** in roughly 30 s, then went silent as the storm moved off — the expected behaviour for an indoor narrow-band 500 kHz antenna. Subsequent tests with NF=2/NF=4 in clear conditions remained at 0 events: not a bug, just no nearby sferics for the selectively-tuned tank to resonate with. Until the antenna is moved outdoors, AS3935 is a confirmation-only secondary; Open-Meteo CAPE is the primary detection layer.

Lightning Antenna Protector — AS3935 dashboard liveness panel

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes added:** AS3935 Status (mqtt in), AS3935 Heartbeat (mqtt in), Format AS3935 State (function), Replay AS3935 State (function) **Node modified:** Master Dashboard (`557083037f168b22`) — three handler inserts inside `scope.$watch('msg', …)`

Symptom: even with the AS3935 daemon healthy and publishing `lightning/as3935/status` (retained, on connect) and `lightning/as3935/hb` (retained, every 30 s), the dashboard's AS3935 panel header was permanently stuck at the static placeholder "Waiting..." because nothing in Node-RED forwarded those topics into Master Dashboard, and the panel only updated on event-driven `as3935_status` messages (which only fire when the chip detects something — never indoors with a 500 kHz selective antenna).

Fix — surface daemon liveness in the panel header

1. **Two new `mqtt in` nodes** subscribe to `lightning/as3935/status` and `lightning/as3935/hb`, datatype JSON, feeding a single `Format AS3935 State` function node.
2. **`Format AS3935 State`** caches each payload in flow context (`as3935_status`, `as3935_hb`) for refresh-replay and forwards typed payloads `{type: 'as3935_ready', …}` and `{type: 'as3935_hb', …}` to Master Dashboard.
3. **`Replay AS3935 State`** (new, fed by the existing 30 s `Refresh Stats` ticker) re-emits cached `flow.as3935_status` and `flow.as3935_hb` so a refreshed dashboard tab repopulates within 30 s — same pattern as the strike-replay fix from May 1.
4. **Master Dashboard** gained two new message handlers and a guard line on the existing `as3935_status` handler:
    - `as3935_ready` → sets the status LED green/amber/red based on calibration (TRCO + SRCO) and `event` field, writes `✓ READY · NF=4 · TUN=10` (or `⚠ CALIB?` / `OFFLINE`) to the header right-side text, and caches the last ready snapshot in `window._as3935Ready` for the heartbeat handler to render.
    - `as3935_hb` → updates the header text every 30 s with current uptime + IRQ count: `✓ READY · NF=4 · up 14m · irq=0`.

  - The existing `as3935_status` handler (disturber/noise) now sets `window._as3935EvtActive = true` for 60 s so a fresh event display isn't immediately overwritten by the next heartbeat; after the timeout it auto-reverts to ready/hb display.

Resulting panel states

Daemon state	Header text	LED
Up, calib OK, idle	`✓ READY · NF=4 · up 14m · irq=0`	🟢 green
Up, calib failed	`⚠ CALIB? · NF=4 · TUN=10`	🟡 amber
Offline (LWT)	`OFFLINE`	🔴 red
Disturber/noise event (60 s display)	`⚠ Disturber 23:18:01`	🟡 amber
Lightning strike	`⚡ N km (energy=…)`	red/amber/green by km
Page refresh while idle	re-populates within 30 s	🟢 green

Layout cleanup

While editing the dashboard template, the `<!-- Event Log -->` block was moved to sit immediately above `<!-- Map -->`. The log is more useful adjacent to the operator's eye-line for the lightning panel than buried at the bottom.

2026-05-08

Lightning Detect — Leaflet map removed

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** Master Dashboard (`557083037f168b22`)

The map was dead weight. Both active strike sources only know *distance*, not direction:

- **Open-Meteo** queries CAPE/ `weather_code` at the home grid point, so `Parse Open-Meteo → Strike` always emits with `lat=home_lat` , `lon=home_lon` .
- **AS3935** chip provides a distance estimate; no azimuth.

Every strike circle therefore stacked on top of the green home marker and was invisible. The Leaflet map was originally designed for a Blitzortung TCP feed (real lat/lon for every strike worldwide), but that integration was never wired up.

What was deleted from the ui_template

- `<div id="leafletMap">` block + the legend bar / `lzCnt` counter
- Leaflet 1.9.4 CSS + JS CDN imports
- CSS rules: `#mapBox` , `#lzWrap` , `.lz-bar` , `.lz-d` , `.lz-home-tip`
- JS state: `lmap` , `homeMarker` , `thrCircle` , `strikeLayer` , the `strikes` array + `MAX=800` cap
- `initMap()` function (Leaflet init, tile layer, threshold ring, home marker, strike layer group)
- `setTimeout(initMap, 400)` boot call + the `resize` listener
- `strikeLayer` / `lzCnt` manipulation in 4 handlers: `strike` , AS3935 `as3935` , `clear` , `strikes_replay`
- The `thrCircle.setRadius()` follow-up in the threshold-update path

Template trimmed from 1,047 lines → 943 lines.

The payload `lat` and `lon` fields in `Strike → Dashboard` are intentionally retained — harmless dead data today, but they let Blitzortung be wired in without re-touching that function (HANDOVER follow-up #7).

Lightning Detect — Nearest Strike gauge persists across page refresh

Nodes: `Strike → Dashboard` (`51367d456e71fb3e`), `Replay on lightning tab` (`f1c57672ba7c0ec1`), Master Dashboard (`557083037f168b22`), `clear all` (`61516f75d2343aae`)

Symptom

The half-circle "Nearest Strike" gauge updated correctly when a strike arrived live, but on every dashboard reload it reverted to the boot value (200 km) and stayed there until the next live strike. The existing 30 s replay tick (which had restored the strike map and event log since 05-01) didn't touch the gauge.

Why

`Strike → Dashboard` was pushing only `{lat, lon, color, ts}` into `flow.strikes` — no `km` field — so even if the replay path had been wired to the gauge, there was nothing to feed it. And the replay emitter (`Replay on lightning tab`) was producing `{type:'strikes_replay', list}` for the map only; no `lastKm` . The boot script also painted `drawGauge(200)` immediately, overwriting the `—` placeholder before any data arrived.

Fix

With the map gone (above), the cache no longer needs a list at all. Simplified to a single value:

- `Strike → Dashboard` : replaced the array push with `flow.set('last_strike_km', km)` .
- `Replay on lightning tab` : now reads `flow.get('last_strike_km')` and emits `{type:'strikes_replay', lastKm:N}` (or returns `null` when there is nothing to replay).
- Master Dashboard: `strikes_replay` handler reduced to `if(typeof d.lastKm === 'number') drawGauge(Math.min(d.lastKm,200));` . Boot-time `drawGauge(200)` removed — gauge now starts at "—" until first strike or replay.
- `clear all` : clears `last_strike_km` (and the legacy `strikes` key, in case anything still reads it).

Verified: trigger synthetic strike → gauge updates → page refresh → gauge redraws to that km within the next 30 s tick. With no cached strike, gauge stays at "—".

Lightning Detect — moved to Shack tab + bypass switch + last-activity recap

Tab moved: Lightning Detect dashboard tab (`dd11372f9c492be8`) — deleted. **New group:** `vu2cpl_grp_lightning` "Lightning Protection" on the Shack tab (`vu2cpl_ui_tab_shack`), width 12, order 9.

The lightning UI is no longer a separate dashboard tab — it lives at the bottom of the main Shack tab. The Master Dashboard ui_template (`557083037f168b22`) was relocated to the new group and trimmed:

- Internal header card (`#hdr` with callsign + clock) removed — Shack tab's existing Header (`eee1a8b8552aa21f`) provides clocks + weather.
- Weather card removed — same reason.
- The wire from `Parse Weather → Header` output 2 → Master Dashboard is no longer needed; weather flows only to the existing Header template now.

Bypass switch

A new vertical `BYPASS` button lives in the switchBox between the ANTENNA and RADIO controls. Off = grey. On = amber with a live `MM:SS` countdown starting at `120:00`. Auto-expires after 2 hours; can be deactivated manually at any time.

While bypass is active:

- A yellow banner above the alert reads `🔕 BYPASS ACTIVE — strikes will alert & log only · no auto-disconnect · expires in MM:SS`.
- Strikes still fire the alert banner (in amber, with text `STRIKE N km — BYPASS · alert only`) but **Trigger Disconnect** (`d62fb0c3c40f03b7`) early-outs without publishing MQTT off, without flipping `flow.antenna_off`, and without starting the reconnect timer.
- The dashboard ANTENNA + RADIO switches stay green ON.
- Activating bypass also force-reconnects (cancels any pending Reconnect Timer + sends an ON command to ant + radio) — designed for "I'm on the air, the storm is far enough away, don't drop me".

State variables: `flow.bypass_active` (bool), `flow.bypass_expires_at` (ms timestamp). Init Defaults (`ec1fd4dece8c4dc0`) clears both on every deploy/launch — bypass never survives a Node-RED restart, by design.

Server side, three new nodes (Lightning Antenna Protector tab):

- `POST /lightning/bypass` http-in — receives `{action:'on'|'off'}`
- `Bypass Handler` function (3 outputs):
  1. → Master Dashboard — emits `{type:'bypass_state', active, expiresAt}` plus a log line
  2. → Reconnect Timer — `{payload:'cancel'}` to kill any pending timer
  3. → Execute Reconnect — force ant + radio ON immediately on activation
- `200 OK` http response (parallel wire from http-in for immediate ack)

The `Replay on lightning tab` function (`f1c57672ba7c0ec1`) now emits `bypass_state` on every Refresh Stats tick (30 s), so the banner + countdown survive page refresh. Includes a server-side expiry safety net in case the in-process `setTimeout` is somehow lost.

Reconnect buttons relabelled

The two ↻ icons next to ANTENNA and RADIO were tiny 36×36 squares with no text. Now they stretch to match the switch row height with `↺ RECONNECT` text and a 110 px minimum width — much easier to spot.

Last-activity recap (replaces auto-clear)

Old behaviour: 30 s after every alert, the banner displayed `✔ No recent activity` — misleading, since plenty of activity had just happened. New behaviour:

- On boot, before any alert: `⏱ Awaiting first event` (muted).
- During an alert: full colour (red / amber / green / yellow-bypass).
- 30 s after the alert: banner transitions to muted recap form `⏱ Last: STRIKE 12 km — ANTENNA + FlexRadio OFF · 4m ago`.
- The `Nm ago` text auto-refreshes every 30 s. Hours roll into `2h 17m ago`, days into `1d ago`.

Two `window.lastAlert` tracking lines + a `setInterval` driver in the ui_template; no flow node changes required.

Strike → Dashboard simplification

Removed the `flow.set('last_strike_km', km)` line — the gauge that consumed it was already gone in `bf1b941`, so the write was dead.

Verified end-to-end

- Bypass-OFF + TEST inject (6 km strike) → red alert + ant/radio → OFF + reconnect timer starts. ✓
- Bypass-OFF + Open-Meteo poll (CAPE → synthetic 0–20 km strike) → same as TEST. ✓
- Bypass-ON + TEST inject → amber alert "BYPASS · alert only", switches stay green, no MQTT publish. ✓
- Bypass-ON + Open-Meteo poll → same as bypass-ON TEST (verified during a moderate-CAPE day where polls fired every 5 min and TD's status badge stayed `BYPASS · Open-Meteo Nkm — disconnect skipped` for the full 2 h). ✓
- Page refresh during bypass → banner + button + countdown all replay within 30 s via Refresh Stats. ✓

2026-05-09

LP-700 — switched from direct USB-HID to lp700-server WebSocket gateway

Tab: LP-700-HID ws (`18fb42443172f33c`) — renamed from `LP-700-HID`, trimmed from 25 nodes to 18.

VU3ESV's LP-700-Server is now running as `lp700-server.service` on the Pi at `ws://192.168.1.169:8089/ws`. Pure-Go single binary; owns `/dev/hidraw*`; multi-client WebSocket fan-out.

Why migrate

`@gdziuba/node-red-usbhid` only allows one process to hold the LP-700's HID handle at a time, which would block any other consumer (browser, phone, the planned Mac SwiftUI app). The Go gateway moves ownership to a dedicated service; Node-RED becomes one of N clients on the bus, same as everything else.

Deployment was already done — the user installed `lp700-server` on the Pi and confirmed `/healthz` before the Node-RED migration. This entry covers only the Node-RED side.

What was deleted from the LP-700 tab

11 nodes, all direct-HID plumbing:

- `getHIDdevices`, `HIDdevice (HID-LP)` — HID node config + worker
- `Poll Meter Values`, `LP Dice and Slice` — poll trigger + binary parser
- `inject Poll Devices`, `inject Kickstart`, `trigger` — startup machinery
- `Raw Buffer`, `Errors` — debug nodes
- `CH cmd`, `RNG cmd` — HID-buffer button payloads (b[1]=56 / b[1]=57)

What was added (imported from `examples/node-red/lp700-websocket-flow.json`)

13 nodes from VU3ESV's example flow, all namespaced `lp7…` to avoid ID collisions:

- `ws://lp700-server/ws` — websocket-in (URL `ws://192.168.1.169:8089/ws`, `wholemsg=false` so payload arrives as message). One websocket-client config (`lp7wsclient00001`).
- `ws://lp700-server/ws` — websocket-out (same client config, send side)

- `Parse frame` — JSON decoder; routes telemetry to output 1, heartbeat to output 2, ack to output 3
- `Reshape (KD4Z-compatible)` — maps the gateway's verbose snapshot to the flat `power_avg / power_peak / swr / scale / mode / channel / range` keys that the existing `LP State Aggregator` (and KD4Z's upstream nodes) expect. Zero changes downstream.
- `Connection state` — exposes link status as a status badge
- `ack` debug — shows server replies to control verbs (`{ok:true}` or `control disabled` etc.)
- 7 `inject` nodes labelled with each verb (peak/channel/range/alarm/ mode/setup/freeze) — kept for canvas-side testing; deletable later
- 2 comment nodes labelling the inbound/outbound halves of the gateway

What was kept and rewired

- `LP State Aggregator` — function body unchanged; now fed by `Reshape` output (was `LP Dice and Slice` output)
- `LP-700 Panel` — ui_template unchanged
- `LP Button Router` — function body rewritten. Old version produced HID buffers (`Buffer.alloc(64)` with `b[1]=56` for channel, `b[1]=57` for range). New version emits JSON command frames:

  ```
  if (msg.topic === 'lp700_ch')
    return { payload: JSON.stringify({type:'command', id:'…',
  action:'channel_step'}) };
  if (msg.topic === 'lp700_rng')
    return { payload: JSON.stringify({type:'command', id:'…',
  action:'range_step'}) };
  ```

  Wired directly into the websocket-out node (no link-in/out indirection since the gateway and dashboard live on the same tab).

Configuration gotcha

First import attempt produced a fake-green connection — the imported `websocket-client` config had a stale client ID that didn't match the URL after editing. Resolved by deleting the imported config and creating a fresh one pointed at `ws://192.168.1.169:8089/ws` . Telemetry flowed within 2 s of redeploy.

Deps that became optional

- npm: `@gdziuba/node-red-usbhid 1.0.3` — still installed but no longer used by any flow. Keep for now; uninstall after a week of stable WS operation.
- apt: `libudev-dev` , `librtlsdr-dev` , `libusb-1.0-0-dev` — only needed to *build* the HID node. Same retention guidance.

Verified

- Telemetry frames arriving at ~25 Hz (matches LP-700 LCD update cadence)
- Power/SWR/channel/range labels all live on the dashboard
- CH and RNG buttons step the meter; `ack` debug shows `{ok:true}`
- Service status: `systemctl status lp700-server` clean; no Node-RED HID errors after restart

Post-deploy fix (08c907f)

After the migration deployed, the dashboard panel was stuck displaying stale `flow.lpState` values (AVG/PEAK 14 W, SWR 1.25 — the last numbers the old `LP Dice and Slice` had cached before the migration). Telemetry was flowing fine through the gateway → Reshape pipeline; the bug was that **`LP State Aggregator` couldn't see the new shape**.

KD4Z's `LP Dice and Slice` returned `{power_avg, power_peak, swr, …}` at the **top level of msg**. The aggregator was hand-written to read `msg.power_avg` directly. VU3ESV's `Reshape (KD4Z-compatible)` puts those same keys on **`msg.payload`** (per his README, matching how KD4Z's downstream nodes consume them). So `msg.power_avg` was always `undefined`, every conditional in the aggregator was false, and `flow.lpState` never updated — the panel kept emitting whatever values were in flow context at deploy time.

Fix: aggregator function body now reads from `msg.payload` first, falling back to the top of `msg` if payload is absent. Backward- compatible with the legacy KD4Z shape.

```
const d = (msg.payload && typeof msg.payload === 'object') ? msg.payload : msg;
// …then read d.power_avg, d.power_peak, d.swr, etc.
```

Lesson for future migrations on this flow: when swapping a parser, audit every consumer that reads from `msg` directly (not `msg.payload`) — the ui_template and the aggregator can both quietly drift out of sync with the new payload shape.

Pi-side scripts — check in `rpi_agent.py` + `monitor.sh` + systemd unit

Discovered during a CLAUDE.md audit that the "RPi Fleet Monitor" docs described `rpi_agent.py` as both the HTTP reboot/shutdown handler AND the MQTT telemetry publisher. In reality the script is HTTP-only — telemetry comes from a separate shell script (`monitor.sh`) running once a minute via the `vu2cpl` user crontab. Both scripts existed only on the Pis themselves, not in this repo. If a Pi ever needed re-imaging the operational glue would have been lost.

Files added (root level)

- **`rpi_agent.py`** — 21-line stdlib-only `BaseHTTPRequestHandler` bound to `:7799`. Two routes: `POST /reboot` and `POST /shutdown`, each shells out to `sudo reboot` / `sudo shutdown -h now`. Returns `{"status": "rebooting"}` JSON 200 then forks the syscall.
- **`monitor.sh`** — bash, executable. Reads CPU (`top -bn1`), mem (`free`), temp (`/sys/class/thermal/thermal_zone0`), disk (`df /`), uptime (`uptime -p` with `/proc/uptime` fallback), IP (`hostname -I` first non-blank). Publishes each as a separate MQTT topic `rpi/<hostname>/{cpu,mem,temp,disk,uptime,ip,status}` to broker `192.168.1.169` via `mosquitto_pub`.
- **`rpi-agent.service`** — systemd unit. Runs as user `vu2cpl`, `Restart=always`, `After=network.target`. Standard install path `/etc/systemd/system/rpi-agent.service`.

CLAUDE.md changes

- Replaced the 7-line "RPi Fleet Monitor" subsection with a split "HTTP control agent" + "Telemetry publisher" + "Home Assistant Pi (special case)" structure. Each agent gets explicit per-Pi install commands (cp, daemon-reload, enable --now, sudoers, crontab line).
- Backup section: extended `tar -czf` glob to include the not-yet- checked-in scripts (`fetch_clublog.sh`, `enable_file_context.sh`, `power_spe_on.py`) flagged with HANDOVER follow-up reference.

- Added a "Pi-side scripts already in this repo" table mapping each repo file to its canonical deployment path under `/home/vu2cpl/`.

Sudoers entry confirmed

```
vu2cpl ALL=(ALL) NOPASSWD: /sbin/reboot, /sbin/shutdown
```

documented at `/etc/sudoers.d/rpi-agent` in the install commands.

Pi-side scripts — `power_spe_on.py` checked in, `fetch_clublog.sh` retired

Follow-up to the earlier "Pi-side scripts" entry above.

- **`power_spe_on.py`** added to the repo. Tiny FTDI DTR/RTS toggling helper used to soft-power the SPE Expert 1.5 KFA. Already referenced in CLAUDE.md ("Power-on requires external Python: `python3 ~/power_spe_on.py`") but never checked in. Now in repo root with the rest.
- **`enable_file_context.sh`** was discovered to already be tracked in the repo — the earlier "not yet checked into repo" annotation in CLAUDE.md was wrong. Annotation removed.
- **`fetch_clublog.sh`** has been deleted from the Pi entirely. The output file `nr_dxcc_live.json` it produced is referenced by zero nodes in `flows.json` (verified by full-flow grep). Live DXCC fetch happens in-flow via `Build Club Log API Request` → `Fetch All Modes + Parse` → writes to `nr_dxcc_seed.json`. The shell script and its output were a vestige of an older workflow. Removed: `~/fetch_clublog.sh`, `~/bin/`, the orphaned `nr_dxcc_live.json`. Cron entry checked and absent. Club Log password + API key rotated since they had been pasted in plaintext during the audit.

CLAUDE.md updates:

- "Pi-side scripts already in this repo" table extended to include `power_spe_on.py` and `enable_file_context.sh`.
- BACKUP section's tar glob trimmed: `fetch_clublog.sh` and `enable_file_context.sh` no longer listed (one's deleted, one's now in repo). The `power_spe_on.py` line drops the "not yet checked into repo" annotation.

HANDOVER follow-up #9 closed (the deferred Pi-side script migration).

`DEPLOY_PI.md` — full Pi onboarding runbook

Drafted while preparing to deploy the agent + telemetry to the two remaining Pis (and eventually the HA Pi). The CLAUDE.md "RPi Fleet Monitor" subsection has the right copy-paste blocks but lacks verification, troubleshooting, and the special-cases / removal flows.

`DEPLOY_PI.md` covers the full per-Pi lifecycle:

1. Prerequisites (SSH, hostname uniqueness, broker reachability)
2. `scp` the three files from the Mac repo
3. Install at canonical paths with the `sudo cp + chown` ownership gotcha (the 2026-05-07 quirk) called out
4. mosquitto-clients install + smoke-test telemetry with `mosquitto_sub`
5. Idempotent crontab append (`grep -v 'monitor.sh'` first)
6. `/etc/sudoers.d/rpi-agent` with `visudo -c` syntax check
7. `systemctl enable --now rpi-agent` + `curl /probe` smoke-test
8. Add hostname to `httpDevices` in Node-RED → deploy → `nrsave`

9. Dashboard verification

Plus: HA Pi special case (HA-side automation, not script-based), troubleshooting table (8 common failure modes), and a clean decommission flow.

CLAUDE.md gets a one-line cross-reference at the top of the RPi Fleet Monitor section pointing to the new doc.

Pi GPS NTP Server — new standalone repo, stratum 1 in service

Standalone repo: vu2cpl/pi-gps-ntp-server

The shack now has a dedicated GPS-disciplined NTP server. A previously unused Raspberry Pi 3B was repurposed as `gpsntp.local` (192.168.1.158) running stratum-1 NTP for the LAN.

Pivot from the original ESP32 plan

The project directory was originally `~/projects/ESP32 GPS NTP/` and scoped as an embedded firmware build (ESP32 + NEO-M8N + custom NTP code). After re-evaluation the path was changed to a Raspberry Pi running chrony + gpsd + kernel PPS. Reasoning recorded in detail in the new repo's `HANDOVER.md`; short version:

- chrony is what most public stratum-1 servers actually run; ESP32 hobby code has to re-implement leap seconds, holdover, multi-source comparison, and stratum honesty — chrony already does this correctly.
- Linux kernel timestamps NTP packets at NIC level; ESP32 lwIP does this in userspace. Client-visible accuracy ends up better on the Pi.
- Wired Ethernet by default vs Wi-Fi NTP jitter swamping ESP32's local PPS lock.
- Debug with SSH + `chronyc` + `gpsmon` rather than serial console + logic analyzer.

Local directory renamed `~/projects/ESP32 GPS NTP/` → `~/projects/Pi GPS NTP Server/`.

How the GPS is sourced

No new GPS hardware was purchased. The existing **QRP Labs QLG1** (MediaTek chipset, 10 ns RMS PPS) that already feeds the U3S beacon was tapped via its unused **6-way connector**. The U3S retains its own connection on the 4-way header and is electrically undisturbed.

QLG1 outputs are 5 V (74ACT08 buffers from +5 V rail). Pi GPIO is not 5 V tolerant. Each tapped signal (TXD, PPS) goes through a 2.2 kΩ + 3.3 kΩ voltage divider that drops 5 V → 3.0 V — comfortably above the Pi's input-high threshold (~2.3 V) and well below the 3.6 V absolute max. Three wires from QLG1 6-way pins 3 (TXD), 5 (1PPS), 6 (GND) to Pi physical pins 10 (GPIO15 / UART RX), 12 (GPIO18), 6 (GND).

Software stack

- **Pi OS Lite, 64-bit, Trixie** (Bookworm successor — current Imager default). Headless install via Imager OS Customisation with hostname `gpsntp` and SSH public-key auth.
- `/boot/firmware/config.txt` adds: `enable_uart=1`, `dtoverlay=disable-bt` (frees PL011 from BT for GPIO14/15), and `dtoverlay=pps-gpio,gpiopin=18`.
- Serial login console disabled via `raspi-config`.
- **`gpsd 3.25`** parsing NMEA from `/dev/serial0`, exporting PPS via SHM-2. `GPSD_OPTIONS="-n"` so it polls without waiting for clients (critical — without this chrony never gets data).
- **Kernel PPS** via `pps-gpio` overlay on GPIO18 → `/dev/pps0`.
- **chrony** with:

  - `refclock SHM 0 refid NMEA offset 0.0 delay 0.2 noselect` — gpsd's coarse second-of-time, used only to label which integer second the PPS edge belongs to.
  - `refclock PPS /dev/pps0 lock NMEA refid PPS` — kernel PPS device, locked to NMEA's second-numbering. The actual time source.
  - `allow 192.168.1.0/24` and `allow fd00::/8` (IPv6 ULA, so `sntp gpsntp.local` from the Mac doesn't get a "recv failure" on its first IPv6 attempt before falling back to IPv4).

Achieved performance (first lock)

Metric	Value
Reference ID	PPS
Stratum	1
PPS error	±152 ns
System clock vs GPS truth	35 ns slow
Skew	0.009 ppm
Root dispersion	18 µs

Mac (`MiniM4-Pro`) configured as client via `sudo systemsetup -setnetworktimeserver gpsntp.local` . `timed` disciplines the Mac continuously; observed offset 1–4 ms on the LAN (limited by macOS scheduling jitter, not the Pi or the network).

SkimServer Mac, the U3S, and any other shack PCs all gain a precise, LAN-local, internet-independent time source by way of the Mac (or by pointing them at `gpsntp.local` directly).

Repo contents

- `BUILD.md` — full step-by-step procedure with per-stage verification and a detailed troubleshooting section.
- `HANDOVER.md` — context, decisions made, ops-checks block for future diagnosis.
- `README.md` — landing page with perf headlines and the why-Pi-not-ESP32 case.
- `LICENSE` — MIT, © 2026 Manoj Kumar R (VU2CPL).

Known risk parked

When the U3S transmits, the QLG1 sits next to the transmitter and may RFI-desensitize. This is a pre-existing condition of the U3S+QLG1 combination; the new NTP server is downstream of it. If `chronyc tracking` shows fix-loss during transmissions over the next week or two, fall back to a dedicated NEO-M8N + antenna for the Pi (path documented in `pi-gps-ntp-server/HANDOVER.md`). HANDOVER follow-up #10.

---

gpsntp — first Pi onboarded via `DEPLOY_PI.md`

Companion to the GPS NTP server above, and the first real-world exercise of the new `DEPLOY_PI.md` runbook. The `gpsntp` Pi was added to the RPi Fleet Monitor agent set:

- `rpi_agent.py` deployed to `/home/vu2cpl/` . Active on :7799, HTTP 404 probe verified.
- `monitor.sh` deployed to `/home/vu2cpl/` . Per-minute crontab line `* * * * * /home/vu2cpl/monitor.sh` publishes 7 MQTT topics

(`cpu/mem/temp/disk/uptime/ip/status`) under `rpi/gpsntp/` to broker @ 192.168.1.169.
- `rpi-agent.service` deployed to `/etc/systemd/system/`. Enabled + active.
- Sudoers entry `/etc/sudoers.d/rpi-agent` : `vu2cpl ALL=(ALL) NOPASSWD: /sbin/reboot, /sbin/shutdown`.

Smoke test (`mosquitto_sub -h 192.168.1.169 -t "rpi/gpsntp/#" -v -C 7`) caught all 7 messages cleanly. Important note for `DEPLOY_PI.md` follow-up: `monitor.sh` publishes without `-r` (retain), so the smoke-test subscriber must be subscribed *before* `monitor.sh` runs — otherwise the messages fly past the broker and the sub hangs waiting for retained data that isn't there. Worth folding into the runbook so the next Pi onboarding doesn't get tripped by it.

The Node-RED `httpDevices` map (function `a0695975fec84e2c`) still needs `'gpsntp':'http://gpsntp.local:7799'` added so the Reboot/Shutdown buttons surface for the new host. Tracked as HANDOVER follow-up #9.

2026-05-10

Lightning Antenna Protector — Tier 1 cleanup (drop dead nodes/wires/vars)

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`)

Audit pass after the Shack-tab merge, the bypass switch landing, and yesterday's "minor cleanup of unused nodes" cleared visible deadwood. 80 → 78 nodes; flows.json -33 lines net.

Deleted nodes

Node	Why dead
`Save Reconnect` (`c95abd88`)	Listened for `msg.topic === 'reconnect_change'`. No node anywhere emits that topic. The real handler is `Save Reconnect HTTP`, fed by `POST /lightning/reconnect`.
`UI Stats` (`a870e611`)	Body was literally `return msg;`. Pure passthrough with no inbound wires — orphan.

Deleted wires

Wire	Why dead
`Parse Weather → Header` output 2 → `Master Dashboard`	The Shack-tab merge stripped the weather card from Master Dashboard. The dashboard's `type:'weather'` handler was deleted in that pass. Operator went one further and reduced the function's output count from 2 → 1, so `msg2` is no longer constructed.
`AS3935 within threshold?` output 1 → `Send Radio Command`	At that point in the chain `msg.payload` is the strike object (not `'ON'`/`'OFF'`), so Send Radio Command's guard returned `null`. The intended OFF for the radio comes via `Trigger Disconnect → Send Radio Command` with proper payload — that path is unchanged.

Stripped flow-context vars

`map_count` / `connectCount` / `wssStatus` were initialised in `Init Defaults` and incremented in `Strike → Dashboard` and `AS3935 → Dashboard`, but had no readers — the consumers (the strike map and an old WS status panel) were both removed earlier this week. Three lines out of `Init Defaults` plus the two-line `count = …` / `flow.set('map_count', …)` block removed from each of the two dashboard handlers.

Verified

- Bypass toggle still works (banner + countdown + force-reconnect)
- TEST inject still triggers full disconnect chain
- AS3935 panel still updates from heartbeat / status frames
- Header weather card still renders (only the duplicate Master Dashboard wire was removed)
- Reconnect Timer still cancels on bypass-on

Pending (deferred to Tier 3)

- Rename `Replay on lightning tab` → `Replay Bypass State` (post-Shack-tab the old name is misleading)
- Demote Init Defaults `node.warn` to `node.log`
- Merge `Strike → Dashboard` and `AS3935 → Dashboard` (90% duplicated payload construction)

Lightning Antenna Protector — Tier 2 cleanup (drop passthroughs)

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`)

Two more middleman nodes deleted; tab 78 → 76 nodes; flows.json -32 lines net.

`Check Threshold` (`b9d407d9`) — deleted

The strike chain ran `… → Haversine Distance → Check Threshold → Within threshold? → Trigger Disconnect`. Check Threshold computed `msg.shouldDisconnect` and `msg.thresholdKm` and updated its own status badge — but **no downstream node read either field**. The following `Within threshold?` switch evaluates `msg.strike.distance_km` against `flow.threshold_km` directly, bypassing the computation.

Pure indirection node. Deleted; Haversine Distance output 1 now wires straight to `Within threshold?` (and continues to wire to `Strike → Dashboard` for the dashboard alert).

Also drops one msg-mutation per strike (≈12 strike events/hour during active storms — small, but it's still N less work).

`Refresh Stats` (`61dca3d9`) — deleted

Body was literally `return msg;`. Existed only to fan out the 30 s tick from the `Stats refresh every 30s` inject to 5 destinations: `Sync Switch State`, `Stats → Dashboard`, `Log → Dashboard`, `Replay on lightning tab`, `Replay AS3935 State`.

Deleted. The inject node's wires array now contains those 5 destinations directly — Node-RED's inject natively supports multi-destination fan-out, no helper needed.

Verified

- TEST inject fires full disconnect chain
- `TEST ✅ 120 km safe` correctly skips disconnect
- Bypass banner + countdown still replay across page refresh
- AS3935 panel "Last seen" updates on the 30 s tick
- Stats box numbers update after strikes

Lightning Antenna Protector — Tier 3 cleanup (rename + dedup + trim)

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`)

Three follow-up cleanups + one consistency fix; tab 76 → 75 nodes.

Renames + small tweaks

- `Replay on lightning tab` → **`Replay Bypass State`** . The old name was a relic from when the lightning UI had its own dashboard tab. After the 05-08 Shack-tab merge, that label was misleading. New name describes what the function actually does: emit `bypass_state` on the 30 s tick so the bypass banner survives page refresh.
- `Init Defaults` `node.warn(...)` line **deleted**. The existing `node.status({...})` already shows broker, callsign, threshold, reconnect-min in the node's badge — the warn was just spamming the in-editor debug sidebar on every deploy.

Strike → Dashboard / AS3935 → Dashboard merge

Two functions, ~25 lines each, ~90 % duplicated payload construction. Merged into a single `Strike → Dashboard` that handles both paths.

Detection inside the function:

```
const isAS3935 = (msg.strike.source || '').toLowerCase().indexOf('as3935') !== -1;
```

If true: payload gets `source:'as3935'` and `energy:<n>` fields. If false: those fields are omitted (matches old OM payload shape exactly).

Both upstream nodes (`Haversine Distance` for OM/TEST, `AS3935 Threshold Check` for AS3935) already populate `msg.strike` with `lat` , `lon` , `distance_km` , `time` , and `source` . AS3935 Threshold Check additionally sets `msg.strike.energy` . So the unified function only reads from `msg.strike` regardless of source.

Wiring:

- `Strike → Dashboard` (the unified function) is now fed by `Haversine Distance` AND `AS3935 Threshold Check` (the function *before* the AS3935 threshold switch).
- `AS3935 → Dashboard` deleted.

AS3935 banner consistency fix

The first wiring attempt fed `Strike → Dashboard` from `AS3935 within threshold?` output 1 (within-threshold strikes only), which was a behavior change from the old `AS3935 → Dashboard` (fed by both switch outputs). That made AS3935 distant strikes silently log to the event log without firing the alert banner — inconsistent with the OM path, where `Haversine → Strike → Dashboard` runs unconditionally and the banner colours by km.

Corrected: rewired to feed `Strike → Dashboard` from `AS3935 Threshold Check` (one node *earlier*, before the switch). All AS3935 strikes now fire the banner; the colour distinguishes within-threshold (red) vs. nearby (amber) vs. far (green), exactly like OM.

`Within threshold?` switch trim

Operator noticed that the OM-path `Within threshold?` switch had two output ports, only one of which was wired. The `else` branch was genuinely unneeded:

- The alert banner for distant OM strikes already fires upstream via `Haversine Distance → Strike → Dashboard` (no threshold filter on that branch).
- The event log for OM polls is already populated by `Parse Open-Meteo → Strike` output 2, regardless of distance.

Switch reduced from 2 rules / 2 outputs to 1 rule (`≤ threshold_km`) / 1 output. (The `AS3935 within threshold?` switch is unchanged — its out2 is genuinely used by `AS3935 Warn Log` .)

Tier-by-tier audit summary

	Start	After T1	After T2	After T3
Nodes on Lightning tab	80	78	76	**75**

Net for this audit pass: -5 nodes, several dead wires removed, duplicated payload-construction merged, naming brought into line with current architecture. No behavioral changes for the operator beyond consistency (AS3935 distant strikes now banner-render).

Documentation — README split into umbrella + DXCC reference

The repo's `README.md` predated everything except DXCC and was effectively the DXCC reference doc with a station footer. Reflowed:

- **`README.md`** — new umbrella overview. Hardware table, an 11-subsystem summary (Lightning / SPE / FlexRadio / Power / Solar / LP-700 / Rotor / DXCC / RPi Fleet / Internet & Network / RBN Skimmer), a repo-layout file tree, and a documentation map pointing at each detailed reference. ~290 lines.
- **`DXCC.md`** — new file. The previous README content moved here verbatim (retitled and reframed as a subsystem reference, with a link back to `README.md`). All node IDs, alert types, startup sequence, persistence logic preserved.
- **`DXCC_Tracker_README.pdf`** — regenerated from `DXCC.md` instead of `README.md` . Same name, same role.
- **CLAUDE.md rule #3** updated: the "regenerate the PDF when you edit the doc" pairing now refers to `DXCC.md` ↔ `DXCC_Tracker_README.pdf` , not the old README pairing.
- Header line in this changelog updated to point at `DXCC.md` .

Future per-subsystem deep-dive docs (e.g. a hypothetical `LIGHTNING.md`) can now slot in cleanly without bloating the front page on GitHub.

Rebuild runbook + missing systemd unit

The repo had `rpi-agent.service` checked in (since 2026-05-09) but **not** the AS3935 daemon's systemd unit (`as3935.service`). Disaster- recovery had a real gap: if the SD card died, you'd be reconstructing the install path from CLAUDE.md fragments and memory.

`as3935.service` checked in

Pulled verbatim from `/etc/systemd/system/as3935.service` on the Pi. Standard `User=vu2cpl` , `Restart=always` , `WorkingDirectory= /home/vu2cpl` . Added to the "Pi-side scripts in this repo" table in CLAUDE.md alongside `rpi-agent.service` .

`REBUILD_PI.md` runbook added

A new top-level doc — linear, copy-pastable, "blank SD card to working shack" sequence in 12 steps:

1. OS install (Pi OS Lite 64-bit), hostname, SSH, DHCP reservation
2. System packages (`mosquitto` , `python3-paho-mqtt` , `python3-rpi.gpio` , `i2c-tools` , build tools); enable I²C + UART
3. Mosquitto LAN-only config + autostart
4. Node-RED via official installer + 9 palette packages + Projects feature in settings.js
5. GitHub SSH key → clone the project (Projects-feature path or manual git clone) → `nrsave` git alias setup
6. `enable_file_context.sh` for persistent flow context
7. Pi-side scripts deploy: `as3935_mqtt.py` , `as3935.service` , `rpi_agent.py` , `rpi-agent.service` , `monitor.sh` (cron), `power_spe_on.py` . Sudoers entry. Ownership reset (the 05-07 quirk explicitly called out).
8. Hardware setup: udev rules for Telepost / LP-700, AS3935 wiring confirmation (IRQ on **GPIO4** not GPIO17)
9. `lp700-server` install via VU3ESV's `redeploy.sh`
10. Manual paste of DXCC + Telegram credentials into the `⚙️ Credentials` node (these aren't in the repo — were rotated after the 05-09 plaintext-leak audit)
11. Tasmota broker pointer check (no-op if the new Pi keeps `192.168.1.169`)
12. 12-point final-verification checklist (ping, dashboard, MQTT, AS3935 heartbeat, RPi telemetry, LP-700 telemetry, FlexRadio TCP, Tasmota state sync, DXCC alerts, lightning auto-disconnect)

Plus a 6-row common-failure-modes table.

CLAUDE.md and README.md cross-link the new runbook. README's "Documentation map" now distinguishes:

- `REBUILD_PI.md` = **the shack Pi** (this repo, full install)
- `DEPLOY_PI.md` = **a different Pi** as a fleet member (telemetry + reboot agent only)

DXCC — N2WQ disconnect cycle + secrets out of flows.json

Tab: DXCC Tracker (`d110d176c0aad308`) **Nodes:** `⚙️ Credentials (edit once)` (`08dcd537`), `Login + Parse + Dedup` (`login-parse-dedup-v2`)

The disconnect cycle

N2WQ was kicking us in a tight loop with `Another login with your call. This session is being closed (multiple logins are not allowed)` . tcp-in auto-reconnect produced a new session every ~10 s and the loop never broke. Two interlocking causes:

1. Login regex matched the welcome banner. The original detector was

```
line.toLowerCase().includes('login:')
```

N2WQ-2's welcome banner contains `Last login: (first login) from <ip>` . That triggered a *second* `VU2CPL\r\n` send after we'd already authenticated. AR-Cluster treats the second send as a command, replies `Unknown command: VU2CPL` , and ~10 s later the duplicate-login policy kicks the session.

Fixed by tightening the detector to match a *prompt*, not a banner mention:

```
var lc = line.trim().toLowerCase();
if (lc.length < 40 && (
    lc.endsWith('login:') ||
    lc.endsWith('please login') ||
    lc.endsWith('please login:') ||
    lc.endsWith('callsign:') ||
    lc.endsWith('callsign please:') ||
    lc.endsWith('your callsign:') ||
    lc.endsWith('enter your callsign:')
)) { ... }
```

The `length < 40` guard kills false positives in the 412-char welcome banner. `endsWith` confirms we're at a *prompt* waiting for input, not a mid-banner reference.

2. Another VU2CPL client was on the LAN. With the regex fix in, we *still* got kicked. Confirmed via `nc cluster.n2wq.com 8300` — even with Node-RED stopped, manual `VU2CPL` login was kicked, but a different test callsign stayed connected. Some other LAN device (unidentified — not Mac, not Pi; possibly an iOS app, logging program, or forgotten `nc` session) was also logged in as `VU2CPL`. AR-Cluster's "no duplicate logins" enforcement was firing.

Sidestepped using a packet-radio SSID:

- **Credentials node** got a new `cl_login_ssid: '-1'` config
- **Login + Parse + Dedup** now sends `(flow.get('cfg_cl_callsign') || 'VU2CPL') + (flow.get('cfg_cl_login_ssid') || '') + '\r\n'`

So Node-RED logs in as `VU2CPL-1` and the other unidentified `VU2CPL` client coexists peacefully. AR-Cluster treats `-1` – `-15` SSIDs as distinct users (standard packet-radio convention).

Verified `VU2CPL-1` accepted by N2WQ-2 via manual `nc` test before deploying.

Secrets out of flows.json

While the Credentials node was being touched anyway, the API key, password, and Telegram token were moved from inline-in-function to systemd environment variables. The motivation isn't repo exposure (repo is private) — it's rotation friction. Previously each rotation required: edit node → Done → Full Deploy → `nrsave` → push. With env-vars: edit `/etc/systemd/system/nodered.service.d/secrets.conf` → `systemctl restart nodered`. One step, no commit needed.

Setup on the Pi:

```
sudo mkdir -p /etc/systemd/system/nodered.service.d/
sudo tee /etc/systemd/system/nodered.service.d/secrets.conf <<'EOF'
[Service]
Environment="CLUBLOG_API_KEY=<key>"
Environment="CLUBLOG_PASSWORD=<pwd>"
Environment="TELEGRAM_TOKEN=<token>"
EOF
sudo chmod 600 /etc/systemd/system/nodered.service.d/secrets.conf
sudo systemctl daemon-reload
sudo systemctl restart nodered
```

The Credentials node now reads each via `env.get('VAR_NAME')` with a pre-flight validation block — if any of the three are missing it fires `node.error()` and a red status badge so misconfiguration is loud, not silent.

`cl_email`, `cl_callsign`, `cl_login_ssid`, `tg_chat_id`, and `cfg_flows_dir` stay inline in the Credentials node — they aren't sensitive and rarely change.

Diagnostic notes for future-self

- **AR-Cluster forks (like N2WQ-2) don't have a `WHO` command.** `HELP` showed only spot/filter commands; no way to query who else is logged in as your call. `SH/USER`, `WHO`, `LIST USERS` all returned `Unknown command`.
- `tcpdump` on the Pi only sees the Pi's own packets — switch doesn't mirror unicast LAN traffic to the Pi's NIC. Not useful for finding which LAN device is talking to a remote IP. Brute- force quit / power-down was faster than network capture for identifying the offender (though we never positively identified it; the SSID workaround sidestepped the need).

Pending follow-ups

- Remaining ghost VU2CPL session on the LAN — never identified. Currently harmless thanks to the `-1` SSID workaround, but worth tracking down eventually (search `tmux` / `screen` sessions, iOS apps, logging programs).
- API key + Telegram token historical commits — rotation recommended if/when the repo is ever made public. Currently private.

DXCC — startup-fetch trigger fixed + Bootstrap unblocks tracker without Club Log

Tab: DXCC Tracker (`d110d176c0aad308`) **Nodes:** `Load Club Log on startup`, `Retry Club Log (90s)`, `Bootstrap Worked Table (manual seed)` (`1a13cd6d`)

Symptom

After deploying the secrets-to-env-var change, the dashboard stopped showing alerts and `DXCC Prefix Lookup + Alert Classify` was stuck on `Paused — reference data loading...`. Cluster spots were arriving fine (cluster-status panel green, counters incrementing), but no prefix lookup, no alert classification.

Root cause #1 — startup injects had `once: false`

`Load Club Log on startup` and `Retry Club Log (90s)` both had the "Inject once after start" checkbox **unticked**. Their `onceDelay` was set (12 s and 90 s respectively) but ignored. Result: the only nodes that triggered the Club Log fetch were the `0200` cron and the manual UI button. After every Node-RED restart the tracker stayed paused for the entire day until the cron fired or the operator manually re-triggered.

This had been latent since some earlier deploy — surfaced today after the secrets-to-env-var redeploy made all the symptoms acute.

Fix: ticked the once-on-startup checkbox on both injects. Now every deploy fires Club Log fetch at +12 s, with a +90 s safety retry.

Root cause #2 — `dxccReady` only set by live Club Log fetch

`DXCC Prefix Lookup + Alert Classify` gates on `flow.get('dxccReady') === true`. That flag was set ONLY by `Fetch All Modes + Parse` (or its lotw-only sibling) on a successful Club Log API response. `Bootstrap Worked Table` loaded cached data from `nr_dxcc_seed.json` (which is git-tracked and always fresh on a clone) but never flipped the flag.

Result: any time Club Log was unreachable / auth failed / rate-limited, the tracker stayed paused even though all the worked-data was already loaded from cache. Brittle.

Fix: `Bootstrap Worked Table` now sets `flow.set('dxccReady', true)` in all three of its success branches:

1. **File store branch** — when `flow.get('workedTable', 'file')` returned cached worked-table from the file context store.
2. **Already-loaded branch** — when `flow.workedTable` was already populated from a previous run (idempotent re-set).
3. **Seed-file branch** — when `nr_dxcc_seed.json` was read from disk.

Now the tracker is operational within ~2 s of any restart, regardless of Club Log availability. Live fetch (when it works) overlays fresh data on top later — idempotent.

Verified

- Manual trigger of `Load Club Log on startup` post-secrets-deploy → `Fetch All Modes + Parse lotw only` turned green with entity count; `dxccReady` flipped true; Prefix Lookup unpaused
- Inject once-on-startup checkbox now persisted across re-import
- Bootstrap fix: even with Club Log fetch artificially blocked, Prefix Lookup stays unpaused on a fresh restart (cached data alone is sufficient for operation)

Diagnostic notes for future-self

- AR-Cluster forks (N2WQ-2) don't have a `WHO` command — can't query who else is logged in as your call. Brute-force device-by-device was faster than network capture for finding LAN duplicates.
- Bootstrap's status colours: blue = "Restored from file store", green = "Loaded from nr_dxcc_seed.json", grey = "Already loaded". All three are success states — only red = "File not found" indicates a real problem.
- "No status" on a function node means it hasn't processed a message since last deploy. Not an error. Wait for the next message.

Correction (same day)

The "fix" of ticking the once-on-startup checkbox on `Load Club Log on startup` and `Retry Club Log (90s)` was wrong. Those injects had been deliberately set to `once: false` as an anti-ban measure — Club Log had previously rate-limited / banned the API key for over-eager re-fetches, and the operator's design was to fetch ONLY via the 02:00 cron (1 API call/day) and rely on `Bootstrap` reading from `nr_dxcc_seed.json` for startup operation. CLAUDE.md TODO #10 ("verify Club Log API ban status + re-enable nodes if lifted") had been the trail; it was missed during the diagnosis.

The Bootstrap-sets- `dxccReady` fix turned out to be the entire correct answer — it makes the tracker functional from cached data on every restart with zero API calls. The once-on-startup checkboxes have been reverted.

Final design (post-correction):

Trigger	What runs	Club Log API hits
Pi restart / Node-RED redeploy	Bootstrap loads `nr_dxcc_seed.json` → flips `dxccReady=true`	**0**
Daily 02:00 (cron)	`Daily club log refresh (0200)` → fetch + refresh seed	1 / day

Operator manually clicks `Load Club Log on startup` inject	One-shot fetch	1 / click

Lesson for future-self: **TODO #10's "Pending" status was a real constraint, not a stale note**. When in doubt about a node that looks "broken" but is intentionally disabled, check CLAUDE.md TODOs / HANDOVER follow-ups before flipping it.

DXCC — `Login+Parse+Dedup` was emitting wrong spot field names (real regression)

Tab: DXCC Tracker (`d110d176c0aad308`) **Nodes:** `Login + Parse + Dedup` (`login-parse-dedup-v2`), `DXCC Prefix Lookup + Alert Classify` (`b981643f`)

Symptom

User reported "I was getting alerts a few per hour earlier — now nothing" despite all 4 cluster cards green and spots flowing. The `DXCC Prefix Lookup + Alert Classify` node had **empty status** — never updating, never firing alerts.

Root cause — field-name mismatch between two functions

`Login + Parse + Dedup` was emitting spots as:

```
spot: { seq, dxCall, freqKHz, freqMHz, mode, spotter, comment, src }
```

But `DXCC Prefix Lookup + Alert Classify` was reading:

- `spot.call` — for blacklist check, DXCC resolution, status text, alert payloads
- `spot.band` — for band filtering and band-worked classification

Neither field was being set by the upstream parser. So Prefix Lookup hit this line near the top:

```
var band = (spot.band || '').toString().toUpperCase().trim();
if (!band) return null;   // silent return — no status update
```

...and silently returned on every real spot. Empty status, no alerts, forever.

The `TEST P5ABC` / `TEST C6ABC 160M` / etc. inject buttons that feed Prefix Lookup directly **were setting `call` and `band` correctly** in their payloads, which is why manual-test alerts fired but real cluster spots never did. That masked the bug for some unknown duration.

Fix

Added a `getBand(khz)` helper inside `Login + Parse + Dedup` (standard amateur band ranges, kHz → band name) and updated the emitted spot object:

```
spot: {
    seq:      seq,
    call:     dxCall,         // ← NEW — what Prefix Lookup reads
    dxCall:   dxCall,         //           kept for any legacy consumer
    freqKHz:  freqKHz,
    freqMHz:  freqMHz,
    mode:     mode,
    band:     getBand(freqKHz),   // ← NEW
    spotter:  spotter,
    comment:  comment,
    src:      src
}
```

`call` is added alongside `dxCall` (rather than replacing) so nothing else that reads `spot.dxCall` (e.g. SmartSDR command construction earlier in the function) breaks.

Verification

After deploy, `DXCC Prefix Lookup + Alert Classify` immediately started showing per-spot activity:

- Grey ring `K1ABC worked (United States 20M)` for already-worked
- Red/blue/yellow on actual alerts

Sequence of today's diagnostic missteps (worth recording)

1. Symptom appeared after the secrets-to-env-var redeploy → wrongly attributed to Club Log fetch failure
2. Investigated `dxccReady` flag → made Bootstrap unilaterally set it (correct fix, but for a different problem)
3. Ticked once-on-startup checkboxes → wrong fix; reverted
4. Verified all clusters green / spots flowing → realised the issue is downstream of the parser
5. Read both function bodies side-by-side → found the field-name mismatch

Lesson: when a node has empty status and the symptom is "downstream silence despite upstream activity", suspect a field-name mismatch between producer and consumer before suspecting flag/state issues. Should have read both function bodies first.

RPi Fleet — Chrony / GPS Time Server card

Tab: RPi Fleet Monitor (`d5fec2fea3dd37f4`) — eventual home **Source of truth:** `vu2cpl/pi-gps-ntp-server` , `dashboard/` folder

What

A single `ui_template` widget showing live status of `gpsntp.local` (the stratum-1 GPS-disciplined NTP server installed 2026-05-09). Replaces a transitional seven-widget version on a dedicated `GPS NTP` dashboard tab.

Data source

Broker	`192.168.1.169:1883` (the existing shack MQTT broker)
Config node	reuses existing `mqttbroker.shack` — **do not duplicate**
Topic	`shack/gpsntp/chrony`
QoS	retained, JSON, refreshed every minute
Publisher	`/usr/local/bin/gpsntp-mqtt-publish.sh` on `gpsntp.local` (Pi-side cron, NOT in this repo)

Because the topic is retained, any new subscriber gets the latest snapshot the moment it connects.

Payload

```
{
  "host": "gpsntp",
  "ts": 1747032600,                 // Unix epoch seconds
  "ref_id": "50505300",             // chrony hex ref id
  "ref_name": "PPS",                // PPS / NMEA / pool host etc.
  "stratum": 1,                     // 1 healthy, 16 unsynced
  "system_time_offset_s": -3.5e-08,
  "last_offset_s": 1.52e-07,
  "rms_offset_s": 2.1e-07,
  "freq_ppm": 0.142,
  "skew_ppm": 0.009,
  "root_delay_s": 0.0,
  "root_dispersion_s": 1.8e-05,
  "leap": "Normal",
  "fix_mode": 3,                    // 0/1/2/3 = none/nofix/2D/3D
  "sat_used": 9,
  "sat_seen": 12
}
```

Architecture

- One `mqtt in` → one `ui_template`. **No function node in between** — all formatting + threshold logic lives in the template's AngularJS `<script>`.
- CSS namespaced under `.gpsntp-card` so it doesn't bleed into other widgets.
- Footer: UTC clock (`HH:MM:SS UTC`, derived from `ts`) plus a `setInterval` ticker that updates the relative "N s ago" stamp every 5 s without re-rendering the card. Interval cleared on `$destroy` so navigating between tabs doesn't leak.
- `ui_group` width 6 to match other dashboard panels; inner card uses `width: 100%` (no max-width) so it fills whatever group width is assigned.

Attention thresholds

Value rendered in orange when crossed:

- `|system_time_offset_s|` > 1 ms
- `rms_offset_s` > 1 ms
- `root_dispersion_s` > 5 ms
- `|skew_ppm|` > 1
- `ref_name` ≠ PPS / PPS2
- `fix_mode` ≠ 3

Updating the widget — preferred path

1. **In-place edit** — open the existing Chrony status card `ui_template` node, replace the format box with `dashboard/chrony-card-template.html` from `pi-gps-ntp-server`, Done → Deploy. Doesn't disturb the group / position.
2. **Re-import the flow** — only if you also want to replace the broker / mqtt-in nodes. Use Import → "Copy" (not "Replace") and move the new `ui_template` into your group, otherwise the import resets the widget's position.

Gotchas

- The core MQTT input node's type string is `"mqtt in"` (with a space), not `"mqtt-in"`. Only the broker config node uses the hyphen (`mqtt-broker`). Hand-built flows that get this wrong are

rejected on import as "unknown types".
- The publisher publishes Unix epoch in `ts`. The widget formats it as **UTC** for the footer (ham radio convention). Do **not** switch to `toLocaleString()` — that's a local-time bug we already fixed elsewhere.

Required palette

`node-red-dashboard` (classic Angular dashboard, **not** Dashboard 2.0). The widget uses `ui_template`, `ui_group`, `ui_tab`.

Current state

- `743a0d8` pushed the transitional seven-widget version on a dedicated `GPS NTP` flow tab (id `4cac0c07e2686c33`).
- `286348e` migrated to the single `ui_template` design — the new card lives on flow tab `GPS NTP (card)` (id `4590ed80de4873b1`) with just two nodes: `mqtt in shack/gpsntp/chrony` and `Chrony status card` ui_template (id `38e130c3`). Dashboard tab is `Shack Monitoring tools`, group `Network Monitor` (width 6).
- The orphan `GPS NTP` flow tab was subsequently deleted by the operator (closes HANDOVER follow-up #13).

`rebuild_pi.sh` — automated rebuild script

Disaster-recovery just got faster. `REBUILD_PI.md` (Pi-rebuild runbook from 2026-05-09) was a manual copy-paste sequence taking ~90 minutes. Wrapped it in a single bash script that automates Stages 2–13 (everything after the SD-card burn).

`rebuild_pi.sh` highlights:

- **Stage-based, 1:1 with REBUILD_PI.md numbering.** Each stage has its own function (`stage_01_apt_packages`, …, `stage_13_verify`), prints a banner, runs commands with `set -euo pipefail`, marks itself done in a state file.
- **Resumable.** State persisted in `/tmp/rebuild_pi.state`. After Ctrl-C or unexpected reboot, re-run and it skips completed stages. `--reset` wipes the state file. `--stage N` re-runs a single stage. `--status` lists what's done.
- **Idempotent.** Every operation safe to re-run. apt installs are no-ops once present; npm install skips already-installed packages; clones detect existing repo and `git pull` instead; systemd `enable --now` is idempotent; sudoers entry uses `tee` with overwrite; cron is grep+tee idempotent.
- **Two interactive pauses** (necessarily so):
  1. Stage 6 — generate ed25519 keypair, print pubkey, wait for operator to paste it into GitHub Settings → SSH keys
  2. Stage 12 — `read -s` for Club Log API key, password, Telegram token (no echo); written to `/etc/systemd/system/nodered.service.d/secrets.conf`
- **Fail-fast pre-flight.** Refuses to run as root (sudo only when needed). Verifies hostname, internet, sudo auth.
- **Built-in verification** at Stage 13 — 10-point checklist matching REBUILD_PI.md Step 12 (ping, Node-RED editor + UI, Mosquitto, as3935 service, rpi-agent, lp700-server `/healthz`, three MQTT topic smoke-tests). Pass/fail summary; pointer to manual diagnosis on failures.
- **Coloured output** (red/green/yellow/blue helpers) for at-a- glance progress.

Wall-clock target: ~30 min vs. ~90 min manual. Uses include the obvious "shack Pi died, rebuild" plus quicker iteration on test Pis or VMs.

The script lives next to REBUILD_PI.md as a peer source-of-truth. The runbook stays as the **manual fallback** when the script breaks (which it eventually will, e.g. when Pi OS bumps a major version). The two must stay in sync — script banners reference the runbook's section numbers.

REBUILD_PI.md updated to point at the script as the faster path, with a one-paragraph summary at the top before the manual steps.

Power meter panels — auto-scale + 10 px bars + stable colours

Tabs / nodes: LP-700-HID ws (`18fb42443172f33c`) `LP-700 Panel` (`c638a0991ad0b768`); SPE (WS) (`spe_ws_tab_01`) `SPE Panel (WS)` (`ws_panel_node`).

Both power-meter panels grew an auto-scaling display bar so single visual works across QRP through high-power operation without manual fiddling.

Common — auto-scale steps

`5 / 25 / 50 / 100 / 500 / 1000 / 1500 / 2000 / 5000 W` . The bar's full-scale auto-picks the smallest step ≥ current power. As power crosses a step boundary, the scale snaps up to the next step and the bar redraws (typical auto-ranging-meter behaviour).

A small `scale: NW` indicator alongside the relevant header tells the operator what the bar's full-width represents — separate from the meter's own range setting (LP-700) or the amp's rated power (SPE).

Common — bar thickness

6 px → **10 px**, with a 1 px border on the track for definition and an inset shadow on the fill for depth. Conspicuous without being overwhelming. CSS transitions on `width` and `background-color` both eased to 0.3 s for smooth movement.

LP-700 panel

- Header gets `scale: <auto-picked> W` indicator next to the title.
- Both AVG and PEAK bars share the same auto-scale, computed from `max(avg, peak)` . Keeps the visual relationship between AVG and PEAK meaningful — peak fills more of the bar than avg.
- **Stable bar colours** — AVG always green (`#3fb950`), PEAK always amber (`#e3b341`). The earlier dynamic 70/90 % thresholds were rejected after first deploy: with auto-scale, the bar nearly always sits in the upper portion of its current band, so the threshold colour-shift triggered constantly and stopped meaning anything. Solid colours preserve the panel's pre-auto-scale visual identity (AVG=green / PEAK=amber).
- SWR bar **keeps** its original threshold logic (1.0–3.0 mapping, thresholds at 1.5 / 2.0). SWR colour transitions remain meaningful because SWR has fixed safety zones independent of the bar's auto-scale.

SPE WS panel

Only the Output Power bar got the treatment — SWR bars unchanged.

- Inline `style="height:10px"` on the Output Power track + fill (preserves `gh-track` / `gh-fill` defaults at 6 px so ATU SWR and ANT SWR stay thin).
- Auto-scale logic identical to LP-700.
- Small `(scale NW)` indicator inline next to the "Output Power" label.
- Removed the redundant `currentW / pwrMax W` text on the right side of the row. The bar shows the live value visually; the scale label shows the bar's full-width; the amp's rated power level is

already shown in the separate `PWR Lvl` badge above the bar. The `currentW / 500W` text was just sitting at "0W / 500W" between TXs and adding noise.

Diagnostic note for future-self

The LP-700 first attempted the dynamic-threshold colours pattern (red/amber/green based on % of the current scale step). Looked good in theory; visually broken in practice because auto-scale always picks the tight-fitting step → bar always near top → colour stuck at amber. Lesson: **dynamic colours and auto-scale don't compose well** — one or the other carries the magnitude info, not both. We chose to keep auto-scale and drop the dynamic colours.

Follow-up — SPE scale label legibility (`0d10af3`)

The first SPE WS scale label was styled `opacity:0.55;font-size:9px` on top of the dim `gh-lbl` colour, which made it nearly invisible in practice. Bumped to `font-size:11px;color:#c9d1d9` (the panel's bright text colour) and dropped the opacity entirely. Operator also tried bold and rejected it ("too thick") — final version is unbold.

DXCC — alert-table dedup on `call+band+mode+alertType`

Tab: DXCC Tracker (`d110d176c0aad308`) **Node:** `Format Alert for Dashboard Table` (`2286f0a512733e92`) **Closes:** HANDOVER follow-up #15

The 60 s dedup in `Login + Parse + Dedup` catches back-to-back duplicate cluster spots, but identical spots arriving 60+ s apart (e.g. XE1RK on 15M USB twice within ~75 s — observed today on the dashboard) re-fired alerts and the alert table rendered the same NEW MODE / NEW BAND row twice within minutes. Visual noise; no functional impact.

Added a second-tier dedup at the alert-table level. The existing TTL-expiry filter on `alerts` was extended in the same pass:

```
alerts = alerts.filter(function (r) {
    if ((now - (r.ts || 0)) >= ttlMs) return false;
    if (r.call      === row.call      &&
        r.band      === row.band      &&
        r.mode      === row.mode      &&
        r.alertType === row.alertType) return false;
    return true;
});
alerts.unshift(row);
```

Net behaviour: at most one row per `(call, band, mode, alertType)` combination while inside the spot-lifetime TTL window. New sighting replaces the old row at the top — its timestamp / freq / spotter / `time` field reflect the latest hit, so the operator always sees current info.

This complements the upstream 60 s spot dedup (per-spot suppression) without conflicting with it. The 60 s window is short enough to catch packet duplicates from the cluster network; the alert-table window is the configured spot-lifetime (default 20 min) so alerts don't pile up when the same DX is repeatedly spotted by different spotters.

No `REBUILD_PI.md` update needed — flows.json change, restored via git clone.

HANDOVER #3 — Open-Meteo dashboard placeholder — closed as obsolete

The follow-up referred to the `OPEN–METEO MONITOR · Waiting for data…` badge in the Master Dashboard's `#hdr` block. That entire header block was stripped during the 2026-05-08 Shack-tab merge (Master Dashboard moved to a new group on the Shack tab; the dashboard's own header was removed in favour of the Shack tab's existing Header card).

The badge therefore no longer exists, so the follow-up is moot. OM polls continue to produce `type:'log'` messages every 5 min via `Parse Open–Meteo` output 2 → Master Dashboard, which flow into the event log normally.

No code change. Closing the entry rather than deleting so future-self sees the rationale.

Format Log — `0 km` strikes now render the distance segment

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** `Format Log` (`5bfc6db2af9dd24c`) **Closes:** HANDOVER follow-up #4

One-character bug fix in the event-log formatter. Old line:

```
const dist = msg.distance ? ' | ' + msg.distance + ' km' : '';
```

Truthy check on `msg.distance`. When the AS3935 chip reports an overhead strike (or an out-of-range strike that we treat as 0 per the chip's `dist=63 → null → 0` mapping in `AS3935 Threshold Check`), `msg.distance === 0` is falsy, so the `| 0 km` segment was dropped from the log line. Fixed with explicit null check:

```
const dist = (msg.distance != null) ? ' | ' + msg.distance + ' km' : '';
```

Loose `!= null` catches both `null` and `undefined` but not `0` — exactly the desired semantics.

No `REBUILD_PI.md` update needed — flows.json change, restored via git clone.

AS3935 systemd hardening

Unit: `/etc/systemd/system/as3935.service` **Repo source:** `as3935.service` **Closes:** HANDOVER follow-up #2

The original unit had `After=network.target` (kernel-level only) and `Restart=always`. The 2026-04-21 silent outage that left the daemon dead for 10 days was caused by the boot-time race where `paho–mqtt`'s `client.connect()` ran before the network was actually routable. The May 1 hardening added a Python-side retry loop to the script; this commit hardens systemd itself as belt-and-braces.

Four directive changes:

Before	After	Why
`After=network.target`	`After=network-online.target` `mosquitto.service`	Wait until network is *fully* up AND broker is ready — not just kernel-level reachability
(no `Wants`)	`Wants=network-online.target`	Pulls `network-online.target` into the boot sequence so `After=` actually delays

`Restart=always`	`Restart=on-failure`	Don't restart on clean SIGTERM during shutdown
(no `RestartSec`)	`RestartSec=5`	Default 100 ms is too aggressive; 5 s lets transient errors clear

Verified post-deploy:

```
$ systemctl show as3935 -p Restart,RestartSec,Wants
Restart=on-failure
RestartSec=5s
Wants=network-online.target
```

Directives are baked into the base unit (not a `.service.d/override.conf` drop-in) since we own this unit file. Drop-ins are the systemd-recommended approach for overriding **distro-provided** units we don't own; for self-owned units, editing the file directly is cleaner.

The script's own MQTT retry loop (added 2026-05-01) handles broker disconnects mid-run. This systemd hardening handles the boot-time case and the post-broker-restart case.

No `REBUILD_PI.md` update needed — `rebuild_pi.sh` Stage 9 already copies `as3935.service` from the repo, and Stage 13 verifies the service is active.

Two more follow-ups closed

Brief admin entry — operator-confirmed both items today:

- **CLAUDE.md TODO #4** — RPi agent deployed on remaining Pis + HA Pi (Bearer token). Closes the multi-host fleet onboarding thread that started 2026-05-09 with `DEPLOY_PI.md`.
- **HANDOVER follow-up #9** — `gpsntp` added to `httpDevices` map in `Route CMD: HTTP or MQTT` (function `a0695975fec84e2c`). Reboot/shutdown buttons on the Chrony / GPS Time Server card now route correctly through the existing fleet HTTP-control pipeline (POST `/reboot` / `/shutdown` on `:7799`).

Rotator Auto-Off Timer — 60 s → 5 min for production

Tab: All Power Strips (`b76a5310767803b4`) **Node:** `Rotator Auto-Off Timer` (`05f0ddeb566a90fc`)

Two-character config change. The timer that auto-cuts power to the rotator (Tasmota `powerstrip1/POWER2`) was set to 60 s back when this flow was first iterated; the production target was always 5 minutes. Surfaced by HANDOVER follow-up #5 / CLAUDE.md TODO #3.

```
// before
var duration = 60 * 1000;
node.status({fill:'yellow', shape:'dot', text:'Timer running — 60s'});

// after
var duration = 5 * 60 * 1000;
node.status({fill:'yellow', shape:'dot', text:'Timer running — 5 mins'});
```

The idempotent retrigger guard + 10 s post-OFF cooldown (added 2026-04-22 to break the reset loop from spurious Tasmota stat-republishes) remain unchanged. Both still operate independently of the duration value.

`flow.rotatorTimerEnd` (the dashboard countdown anchor) is also unchanged — it was already computed as `Date.now() + duration` so it auto-scaled.

Closes CLAUDE.md TODO #3 and HANDOVER follow-up #5.

Lightning — Blitzortung integration dropped from roadmap

`HANDOVER.md` follow-up #7 had been carrying "wire Blitzortung TCP-in to Parse Strike" since the 2026-05-08 map removal. The parser's Case 2 (binary frame) and Case 3 (string frame) were left intact specifically for this future integration.

Verified today on `map.blitzortung.org` — **zero coverage at MK83TE (13.06°N 77.63°E)**. Southern India has very few contributing stations and the TOA triangulation algorithm needs ≥4 stations seeing the same strike for usable accuracy. Bengaluru sits in a gap where Blitzortung would either miss strikes entirely or report them with kilometre-scale position errors.

The two-tier coverage we already have is sufficient:

Tier	Source	Range
Close	AS3935 sensor	~few km indoors / ~40 km outdoors
Regional	Open-Meteo CAPE + weather_code	5-min poll, 13 km grid

The mid-range gap (10–80 km, real-time strike-by-strike) that Blitzortung would have filled isn't actionable for the auto-disconnect decision (close range is) and isn't a planning question (regional CAPE is).

Closed in HANDOVER.md as "dropped" rather than deleted, so future- self can see why we chose not to pursue this. Parser Cases 2/3 are now permanently dead code in `Parse Strike` — flagged for stripping in a future flow cleanup pass (low priority; the dead branches cost nothing at runtime).

2026-05-13

Lightning audit — fix silent disconnect regression + 11 follow-up cleanups

Audit pass on the Lightning Antenna Protector flow tab triggered by a "check if this can be optimised and check all errors" prompt. Found one **critical regression** introduced yesterday by `76d60e5` (the distance-graded disconnect commit), plus ten consistency / hygiene issues worth fixing in one pass.

Critical: AS3935 disconnects were silently broken

`Trigger Disconnect` (`d62fb0c3c40f03b7`) yesterday gained a source filter:

```
if (source !== 'AS3935') {
    node.status({fill:'grey', shape:'ring',
        text:'no-op · ' + source + ' (corroboration only)'});
    return null;
}
```

But `AS3935 Threshold Check` (`ad80b86a672a9ec6`) sets `msg.strike.source = 'AS3935 (local)'` — note the trailing `' (local)'`. Strict equality `'AS3935 (local)' !== 'AS3935'` is **true**, so every real

AS3935 lightning event was getting early-rejected. The chain *looked* fine on the dashboard because `AS3935 within threshold?` (`af1b4512527ffb45`) fan-outs its output-0 to both `Trigger Disconnect` **and** `AS3935 Disconnect Log` in parallel — so the log + JSONL kept writing `"DISCONNECT triggered"` lines while the MQTT publish to the antenna switch was suppressed. Dashboard lied; antenna stayed connected.

Why the 2026-05-12T16:00:06 real-strike disconnect cited in yesterday's changelog worked anyway: the strike was at 16:00; the regression commit landed at 16:40. The verification used pre-regression code.

Fix: loosened the filter to reject only `Open-Meteo` (the source that genuinely should never directly fire DC) and let AS3935 / TEST / any future source fall through the matrix. AS3935-specific detection moved inside the function via `isAS3935 = source.indexOf('AS3935') !== -1`, which gates the only AS3935-specific behaviour (pushing into the sliding `recent_as3935` corroboration window). TEST injects now exercise the matrix without polluting the corroboration counter — so clicking "TEST ⚠ 35 km" three times can't manufacture a fake "2 hits in 5 min" disconnect.

Follow-up fixes bundled into the same patch

1. **Removed dead `Within threshold?` switch** (`12c029b533385153`). It pre-gated `strike.distance_km <= flow.threshold_km` before `Trigger Disconnect`, but the matrix already does its own zone filtering (`cfg_close_km` / `cfg_medium_km`). Worse, if the user slid `threshold_km` below `cfg_medium_km`, valid medium-zone strikes would never reach the matrix. Haversine now wires directly to Trigger Disconnect. 76 → 75 nodes (matches the count CLAUDE.md already stated — we were briefly at 76).
2. **`AS3935 Disconnect Log` is now bypass-aware.** Previously wrote "DISCONNECT triggered" lines into event_log + JSONL even when the bypass switch suppressed the actual disconnect (the log wire is parallel to `Trigger Disconnect`, not downstream of it). Now reads `flow.bypass_active` and writes `"BYPASS · disconnect suppressed"` with `event_record.type = 'disconnect_suppressed'` + `bypass: true` when in bypass mode. JSONL stays truthful for post-mortem analysis.
3. **Reconnect-timer cancellation from manual paths.** Previously a pending reconnect timer could keep running after the user did `Manual Override`, `Force Reconnect`, or `POST /lightning/ant-on` — firing later and re-toggling the antenna behind the user's back. All three nodes now have an extra output wired to `Reconnect Timer` with `payload: 'cancel'`. `Manual Override` outputs 1→2, `Force Reconnect` outputs 1→2, `HTTP → Antenna ON` outputs 2→3. `HTTP → Radio ON` does **not** cancel — the timer governs antenna, not radio.
4. **Stripped three `node.warn` debug lines.** `Stats → Dashboard` had a leftover 5-line `node.warn` block firing every 30 s into the debug sidebar (from filter-persistence debugging on 05-11). `Reconnect Timer` had `node.warn('Reconnect Timer FIRED → ...')`. `Bypass Handler` had `node.warn('Bypass Handler: action=...')`. All three were leftover instrumentation.
5. **Dropped unused `flow.recent` push from `Haversine Distance`** — 100-entry rolling list nothing read. The corroboration window (`flow.recent_as3935`) is maintained by Trigger Disconnect and is the only history any node consumes.
6. **Fixed stale comment in `Parse AS3935`** — `// AS3935 reports 1 = overhead, 40 = out of range` claimed `40` but the actual sentinel the code checks (correctly) is `63` (0x3F). Comment now matches.
7. **Cleaned `Replay AS3935 State` return form.** Was returning `[out]` where `out` is an array of payloads (relies on Node-RED's fan-out-on-array-output behaviour). Replaced with explicit `out.forEach(m => node.send(m)); return null;` — same behaviour, reads obviously.

What did NOT change

- **Decision matrix logic** — close < 10 km always fires; medium 10–25 km needs corroboration (2 AS3935 hits in 5 min OR OM lit); far ≥25 km only fires on OM severe. All seven `cfg_*` keys unchanged.
- **Bypass behaviour** — still early-exits in Trigger Disconnect (`flow.bypass_active === true`); still resets on every deploy via Init Defaults; still expires after 120 min.
- **Init Defaults clobbering `threshold_km` / `reconnect_min` on every deploy** — flagged in the audit but the author's "always overwrite from config" comment is intentional. Slider/HTTP changes still don't survive a Deploy. Left alone; revisit if it becomes irritating.
- **TEST inject button labels vs actual haversine distance** — the three test inject buttons' lat/lon coordinates produce distances that don't match their labels (e.g. "TEST ⚡ 6 km" actually computes to ~23 km from MK83TE). Pre-existing; tangential to this audit.

Net node count

Lightning tab: 76 → 75 nodes (one switch deleted). CLAUDE.md flow tab table line for Lightning Antenna Protector showed 71 — that's been stale since at least the 2026-05-08 bypass + Shack-tab merge; not touched in this commit, but worth noting if anyone audits it next.

Diagnosis lesson

When a downstream log node fan-outs in parallel with an action node (rather than being downstream of it), the log will lie if the action node short-circuits. Two design choices to prevent recurrence:

- Put the disconnect log **downstream** of `Trigger Disconnect` , so the log only fires when the disconnect actually fires. Tradeoff: loses the "AS3935 hit but matrix said no" log entries which are useful for tuning the corroboration thresholds.
- Or keep parallel logs but always read the action-node's state (bypass flag, source-filter result) into the log line, so the log is honest about what actually happened. This is what (2) above implements for bypass.

The matrix-said-no case is now logged truthfully via the `AS3935 Warn Log` branch of `AS3935 within threshold?` (which fires when km > threshold) — the existing wiring there was already correct.

Files touched

- `flows.json` — 11 function bodies + 3 outputs/wires + 1 switch deletion. Diff: +24 / −40 lines.
- `clublog_dxcc_tracker_v7.json` — no change (DXCC tab not touched); regen produces identical bytes.

REBUILD_PI.md / `rebuild_pi.sh` impact

None — flows.json is git-tracked; Pi just `git pull && sudo systemctl restart nodered` .

Lightning dashboard: ALL CLEAR boot state replaces "Awaiting first event"

Master Dashboard `ui_template` (`557083037f168b22`) used to render `⏱ Awaiting first event` in muted grey on first paint and after the 30-second event-recap timeout when no prior alert existed. Operator preference: the post-relaunch / no-events state should show a positive `✔ ALL CLEAR` in green, matching the styling used by the `> 50 km` strike path. Reads as reassuring rather than pending.

Two surgical changes inside the `format` string:

1. **Initial HTML render** — `<div id="alertBox">` now ships with inline `background:#0d2818; border-color:#238636;` , `<span id="alertIcon">✔</span>` , and `<span id="alertTxt"`

`style="color:var(--green);">ALL CLEAR</span>`. Boot state is green from the first paint.

2. **`clearAlert()` no-prior-event branch** — the `if (!lastAlert)` path now calls `paintAlert('✔','ALL CLEAR','#0d2818','#238636', '#3fb950')` instead of `paintAlert('⏱','Awaiting first event', '#161b22','#21262d','#8b949e')`. Returning-to-idle after Node-RED restart (no `lastAlert` cached) re-paints to green.

The post-first-event recap path (`Last: <text> · Nm ago` in muted grey) is untouched — that one still kicks in 30 s after every real strike and continues to refresh its relative-time string every 30 s.

Net: the dashboard reads "all clear" on every relaunch, then if a strike fires it goes red/amber/green per zone for 30 s, then settles into a muted recap line for the rest of the session.

Files touched

- `flows.json` — 1 ui_template `format` field; 1 insertion, 1 deletion (the HTML block became a one-line replacement inside the big format string).
- No node count change.
- DXCC tab untouched; extract bytes-identical.

Pre-flight audit for making `vu2cpl-shack` public. Five fixes applied:

1. Removed hardcoded Club Log API key. `Build cty.xml URL` function had a literal `7cc2e40298a9da3f173bd06118f6cb08cc3131f3` as fallback when `flow.get('cfg_cl_apikey')` returned empty. Defeated the entire env-vars-via-systemd pattern the rest of the secrets follow. Replaced with `|| ''`. **Operator must rotate the key on Club Log side** — it's been in `origin/main` history for some time, public-OK only if rotated.

2. Telegram chat ID externalised. Was inline as `tg_chat_id: '784711092'` in the Credentials node. Not a secret in the cryptographic sense (the bot token is what matters, and that's already env'd), but it does identify the operator's personal Telegram chat. Moved to `env.get('TELEGRAM_CHAT_ID')` for consistency with the other three credential keys. Pi-side needs a new line in `/etc/systemd/system/nodered.service.d/secrets.conf`:

```
Environment="TELEGRAM_CHAT_ID=784711092"
```

`REBUILD_PI.md` Step 10 updated to include this env var in its `secrets.conf` template.

3-4. Untracked two runtime data files. `git rm --cached` for `nr_dxcc_seed.json` (306 KB, full per-band/per-mode worked-DXCC matrix, auto-refreshed daily by the Club Log fetch cron) and `nr_dxcc_blacklist.json` (operator's per-callsign mute list — small but socially-revealing in a public repo). Files stay on disk; the daily fetch repopulates the seed after any `git clone` + first cron tick. Blacklist starts empty on a fresh clone and gets populated by the `POST /dxcc/blacklist-add` HTTP endpoint as needed.

5. `.gitignore` expanded from one line (`*.backup`) to cover:

```
*.backup
.DS_Store
.claude/
nr_dxcc_seed.json
nr_dxcc_blacklist.json
nr_lightning_events.jsonl
```

The last one is the JSONL historic store we added today — runtime data, not source-controlled.

Audit findings deliberately NOT changed:

- **LAN IPs** (`192.168.1.148/158/169/170/241`) stay hardcoded. RFC1918, no external risk, and most ham-shack repos ship their LAN IPs so forkers see the structure. Genericising would touch dozens of nodes in flows.json + four .md files for low payoff.
- **Email address** (`vu2cpl@gmail.com` in the Credentials node) stays — already in CLAUDE.md and on every QSL card. Same applies to the callsign, grid, and DXpedition history.
- **MQTT broker is plain / no-auth** (`192.168.1.169:1883`) — LAN-only, documented as such. Anyone inside the LAN could already see it.

Git history pollution — even after this scrub, the literal API key + chat ID sit in past commits. Accepting that; the key rotation makes the historical disclosure dead. `git filter-repo` to rewrite history would be overkill for a hobby repo with hardware-side secondary auth (the broker is LAN-only, the bot token is env'd, the API key will be rotated).

Sequence on the Pi to deploy:

```
# 1. Add the new env var to secrets.conf:
sudo nano /etc/systemd/system/nodered.service.d/secrets.conf
# Add: Environment="TELEGRAM_CHAT_ID=784711092"
# Also update CLUBLOG_API_KEY to the freshly-rotated value.

# 2. Pull + restart:
cd ~/.node-red/projects/vu2cpl-shack
git pull
sudo systemctl restart nodered

# 3. Sanity-check Credentials node status badge in editor:
#    should show: "Config loaded: VU2CPL-1 / tg:784711092"
#    (anything red means an env var is missing)
```

Then it's safe to flip the GitHub repo Settings → Visibility → Public.

Lightning: historic event store — strikes + actions persisted to JSONL

`flow.event_log` is great for the dashboard's snappy Event Log card but it's in-memory, capped at 50 rows, and wipes on every deploy. Operator wanted a long-running record for post-storm analysis, false-positive trend tracking, and general "what happened on the night of …" recall.

Design (C2 — separated concerns):

- Dashboard keeps its in-memory `flow.event_log` exactly as before (50 rows, fast, no I/O on the render path).
- A new function node **`Append Lightning JSONL`** (`bf480be`) appends one JSON-Lines row per event to `/home/vu2cpl/.node-red/projects/vu2cpl-shack/nr_lightning_events.jsonl` . Append-only, no read-modify-write race, grows unbounded (logrotate when it gets big).
- File path is set by `Init Defaults` on every deploy as `cfg_events_jsonl` so the append function reads it from flow context.

Coverage (in two waves):

1. **First wave** (`bf480be`) — the three existing log-writer functions (`Format Log` , `AS3935 Warn Log` , `AS3935 Disconnect Log`) each now also emit `msg.event_record = {ts, source,`

`type, km, energy, …}` alongside their existing dashboard output, and have a parallel wire to the new append node. Captures: disconnects, reconnects, manual overrides, AS3935 below-threshold warnings, AS3935 above-threshold disconnects.

2. **Second wave** (`a93a1e3`) — operator noticed a pre-existing gap: above-threshold OM strikes and TEST injects above the 25 km disconnect threshold silently drop at the `Within threshold?` switch (single-output `lte` rule, anything `>` just falls off the end). Added a third tap, `Log all strikes (to JSONL)` , hanging off `Haversine Distance` 's output as a 3rd parallel target. Now every strike — AS3935 / Open-Meteo / TEST — gets a `type:"strike"` JSONL row regardless of whether it triggers a disconnect.

The two waves emit different `type` values intentionally — `"strike"` captures the inbound event, `"disconnect"` / `"reconnect"` / `"warn"` capture the action. Both arrive for events that fire the chain (one strike, one action), only the strike row arrives for events that don't. Plenty of data for jq-side analysis.

Two bugs hit during bring-up — instructive enough to record:

Bug	Symptom	Cause	Fix
Init Defaults silently broken	`Append Lightning JSONL` warned "cfg_events_jsonl not set" every event; file never created	`os.homedir() + …` threw `ReferenceError: os is not defined` at line 83 of Init Defaults — `os` is **not** a free global in Node-RED function nodes unless the node's `libs:` array declares it. DXCC functions all have `libs: [{var:'os', module:'os'}, …]`; Init Defaults didn't.	Hardcoded `/home/vu2cpl/.node-red/projects/vu2cpl-shack` (784a634) — this Pi only
Append node also broken	`JSONL append failed: fs is not defined` once Init Defaults was fixed	Same root cause one layer down: `Append Lightning JSONL` referenced `fs.appendFileSync` without a `libs:` declaration.	Added `libs: [{var:'fs', module:'fs'}]` to the new function node (c8fbcb4)

Mental model worth pinning (recording so future-self stops re-learning this):

> *In Node-RED 1.3+ function nodes,* `fs` */* `os` */* `path` */* `https` */ etc. are **per-node libs**, not project-wide globals. If a function uses one of them, its node config must include* `libs: [{var:"<name>", module:"<module>"}]` *. Symptom of a missing declaration is* `ReferenceError: <name> is not defined` *from the first line that references it. Audit existing nodes via* `grep -l "libs.*[\"']fs[\"']"` *in flows.json before assuming a name is available.*

Test injects gained `source:"TEST"` (20d0b16) — the three test injects (`6 km DISCONNECT` , `35 km warning` , `120 km safe`) previously had no `source` field in their payload, so JSONL rows came out as `source:"unknown"` . Added `"source":"TEST"` to each so analysis filters can cleanly separate test from real:

```
# real events only
jq -c 'select(.source != "TEST")' nr_lightning_events.jsonl
# test events only
jq -c 'select(.source == "TEST")' nr_lightning_events.jsonl
```

Once trusted, the `Log all strikes` function has a commented "suppress TEST entries from JSONL" guard that can be uncommented to stop polluting the historic record with bench tests.

Deferred (operator's call): Open-Meteo 5-min poll cadence rows and Bypass switch on/off events are not yet captured — both are small-volume / high-value adds. Disturber / noise from AS3935 not captured either (high-volume — would 10× the file size with an indoor sensor; deferred until either someone wants the data, or the sensor moves outdoors and disturber rate drops).

REBUILD_PI.md impact: none. `jq` is already in Step 2's `apt install` list — this Pi predates the runbook and needed a one-time `sudo apt install -y jq` separately, but a fresh rebuild gets it. `nr_lightning_events.jsonl` is runtime data (like `nr_dxcc_seed.json`); not in `.gitignore` but `nrsave` only stages `flows.json` so it doesn't accidentally land in a commit.

SPE: stale-state lying fix — gateway-side presence-heartbeat + panel offline-wipe

Operator noticed the Node-RED SPE panel + Macexpert SPE app both kept showing the last-known amp state after physically powering the amp off — only a fresh page-load / WS reconnect picked up the truth. Identical symptom across two independent clients, so the bug had to be on the gateway. Multi-hour debug session, fixed in two commits on the `vu2cpl/spe-remote` repo plus several follow-on Node-RED panel changes here.

Diagnosed root cause (in `spe-remote/spe/websocket_handler.py` + `spe/serial_handler.py`): `broadcast_state` only fires when `SerialHandler.on_state_update` calls it, which only fires when serial frames arrive and parse cleanly. When the amp powers off, no frames → no broadcast → connected clients never learn the amp went down. Macexpert's defence was to reconnect every 5 s on silence — visible in `journalctl -u spe-remote` as continuous connect/disconnect spam, each fresh client just refetching the same stale snapshot.

Gateway fix (cross-ref `spe-remote@248b922` and `spe-remote@ab6d94d`):

- New asyncio task `presence_heartbeat_loop` emits `{"heartbeat": true, "serial": "up"|"down", "ts": ..., "clients": …}` every `polling.presence_heartbeat` seconds (default 5).
- `serial` is **amp liveness**, not USB-FTDI link state. The SPE Expert 1.5 KFA's FTDI cable is USB-powered from the Pi end — the link stays open even when the amp's CPU is fully off. Real signal: SerialHandler now tracks `_last_state_at` (monotonic timestamp of last successful CSV parse); `serial: "up"` iff `last_state_age < polling.amp_alive_threshold` (default 3 s).
- New `AmplifierWebSocket.broadcast_raw` classmethod bypasses the state-dedup gate so heartbeat cadence is exact.
- Worst-case amp-off detection latency: 5 + 3 = 8 s.
- Wire-compatible: existing state / power_result / RCU paths unchanged. The Node-RED `Parse + route` function already dispatched on `d.heartbeat` and `d.serial === "up"` — those keys finally exist.

Panel-side follow-ups in this repo — drag the dashboard into parity with the new gateway truth:

1. **Single state-aware Power button** (`b6640ea`) — replaced the separate `Power Off` (primary row) and `Power On` (buried in "More Details") buttons with one toggle at the top of the panel. Green `ON` when amp alive, red `OFF` otherwise. Click confirms then publishes `OFF_SPE` (→ existing WS `power_off`) or `ON_SPE` (→ new exec node running `python3 /home/vu2cpl/power_spe_on.py` — WS `power_on` can't wake an amp whose CPU is fully off). Router gained a 2nd output for the script path.

2. **Wipe stale state to em-dash when amp goes offline** (`86d4b1b`) — when `d.usb` flips false, every cached field (Mode / RX-TX / Band / Input / TX Ant / PWR Lvl / Warnings / Alarms / V PA / I PA / all Temps / Bank / RX Ant / Model / both SWR values) drops to `—` , and all three bars reset to 0%. Real state msgs flow on amp-return and repopulate every field via existing per-field handlers — no "re-enable" logic needed. Panel stops lying.
3. **Output Power gets a numeric readout** (`2585b25` , reformatted in `652f033`) — previously the row had only a bar fill + a small "(scale …W)" sublabel. Now matches the SWR rows: label on the left, live `247/500W` value on the right (actual / current band-max), bar underneath. Same threshold colouring (green / amber / red at 70 / 90 % of band-max).

Hard-earned debug lesson — `scope` inside a Node-RED dashboard 1.x `ui_template` inline `<script>` is a **transient** top-level reference, not a free closure variable. Code at the top level of the script can call `scope.$watch(…)` immediately, but functions defined there that **later** reference `scope` (e.g. button handlers) silently break in Safari with `ReferenceError: can't find variable: scope` . We chased three wrong fixes (ng-click scope routing → `addEventListener` on the button → `onclick="window.X()"`) before the Safari console finally spelled out the failure. The working pattern (now confirmed in two panels — chrony card + AS3935 Control Panel — plus the SPE panel here):

```
// 'scope' at top level is transient. Capture it via IIFE parameter
// for any function that fires later (button handlers, async
// callbacks, etc.).
(function(scope){
  window.speAmpToggle = function(){
    /* … uses scope.send(…) here … */
  };
})(scope);
```

Recorded in this changelog so future-self stops re-discovering it.

Other notes:

- Macexpert SPE app reconnect-on-5-s-silence loop should be replaced with reconnect-on-30-s-of-nothing once the Mac-side update lands. Standalone debug-handover written into the Macexpert SPE repo on the same date covers what the Mac client needs to consume.
- Pi `git pull` in `spe-remote` always needs a stash dance because the live `temperature_unit` toggle writes back to `config.yaml` at runtime. Worth flagging in that repo's `handover.md` (already done in the entries pasted from this session).

No REBUILD_PI.md / `rebuild_pi.sh` impact — `spe-remote` isn't installed by either; it's a separate project with its own install lineage. Flows.json changes flow through `git clone` automatically. DXCC tab extract regenerated per rule #4 (no diff — DXCC tab not touched).

2026-05-12

Lightning protection: distance-graded disconnect + AS3935 runtime cmd channel

Big day. Two related changes landed, plus a runtime control surface for the ESP32 sensor bridge:

1. Decision matrix — `Trigger Disconnect` now grades by zone × OM state

`Trigger Disconnect` (`d62fb0c3c40f03b7`) used to fire unconditionally on any strike that passed the upstream threshold switch — regardless of source, regardless of corroboration. With Open-Meteo synthesising a 0 km "strike" on any current-hour `weather_code ∈ {95, 96, 99}` and a 10 km "strike" on any CAPE ≥ 2500, the chain was firing preemptively on high-CAPE-no-storm days. Conservative, but noisy.

New logic: AS3935 lightning events drive the chain; OM is a probability / severity signal that modulates the corroboration threshold per zone. **Open-Meteo never directly fires the disconnect** anymore.

Decision matrix:

OM state	AS3935 close (<10 km)	AS3935 medium (10–25 km)	AS3935 far (≥25 km)
cold	single hit → DC	2 hits in 5 min → DC, else log only	log only
lit	single hit → DC	single hit → DC (corroborated)	log only
severe	single hit → DC	single hit → DC	single hit → DC

OM state derivation (computed every 5-min poll in `Parse Open-Meteo → Strike`):

OM state	Condition
cold	CAPE < `cfg_om_cape_thresh` (default 800 J/kg) OR wmo ∉ {95, 96, 99}
lit	CAPE ≥ 800 AND wmo ∈ {95, 96, 99}
severe	CAPE ≥ `cfg_om_cape_severe_thresh` (default 2500 J/kg) AND wmo ∈ {95, 96, 99}

State persists `cfg_om_lit_window_min` (default 20 min) after each poll, so a transient calm-CAPE reading mid-storm doesn't immediately drop OM back to cold.

Invariants chosen deliberately during planning:

- **Close zone always fires on single hit.** The user accepted the trade-off: this means single misclassified-disturber-as-lightning events at distance 1 km still trigger DC. Cost of a false positive (≤20 min radio offline) ≪ cost of missing a real close strike (FlexRadio + SPE + antennas exposed).
- **OM alone never disconnects.** Even with `severe` state, if AS3935 has seen nothing, no DC fires. OM is pure probability — it can amplify trust in an AS3935 hit but cannot manufacture one.
- **Disturber events don't feed the chain.** Only `lightning`-typed AS3935 events go through `Trigger Disconnect`; disturbers still increment counters and log for diagnostics.

Config keys (all live-tunable from `Init Defaults`):

```
cfg_close_km              = 10    // AS3935 close-zone radius (km)
cfg_medium_km             = 25    // AS3935 medium-zone radius (km)
cfg_med_window_min        = 5     // sliding strike window
cfg_med_count             = 2     // hits in window for OM-cold medium DC
cfg_om_lit_window_min     = 20    // OM state persistence
cfg_om_cape_thresh        = 800   // "lit" CAPE threshold
cfg_om_cape_severe_thresh = 2500  // "severe" CAPE threshold
```

Strike history lives in `flow.recent_as3935 = [{ts, km}, …]`, trimmed to the sliding window on every call. Reset on every `Init Defaults` run (deploy / restart).

Implementation (`76d60e5`): three function-body changes, no wire changes.

- `Init Defaults` (`ec1fd4dece8c4dc0`) — 7 new cfg keys + `recent_as3935 / om_state` reset.

- `Parse Open-Meteo → Strike` (`593f22a507b46335`) — derive OM state from CAPE + wmo; persist with TTL. Strike-emission path (Output 1) kept intact for dashboard / event log compatibility — the actual filter happens downstream.
- `Trigger Disconnect` (`d62fb0c3c40f03b7`) — full rewrite. Source filter, matrix decision, sliding history. Status badge now reports the decision (`close 6km` , `medium 18km · uncorroborated` , `far 30km · OM severe` , …) for live observability.

Net behaviour shift vs pre-today: OM-only DCs stop happening (today's pain). Single-AS3935-hit medium-zone DCs only happen when OM agrees there's a storm. Far-zone hits log unless OM is severe.

2. AS3935 ESP32 bridge — v0.2.0 cmd channel (firmware repo)

Operator built v0.2.0 of `vu2cpl-as3935-bridge` firmware between yesterday's bench bring-up (v0.1.1) and this morning. v0.2.0 adds runtime tunability of every AS3935 register field over MQTT, NVS persistence, and an on-device port of `as3935_tune.py` 's TUN_CAP sweep.

New MQTT topics:

Topic	Direction	Payload
`lightning/as3935/cmd`	Node-RED → ESP32	`{"set": "<key>", "value": …}` or `{"action": "<name>"}`
`lightning/as3935/cmd/ack`	ESP32 → Node-RED	`{"ok": bool, "cmd": "set:<key>"|"action:<name>", "ts": "..."}` (not retained)
`lightning/as3935/status`	ESP32 → Node-RED	retained; republished after every successful set/action so subscribers see fresh state

Tunables (NVS-backed, range-validated): `nf` (0–7) · `wdth` (0–15) · `srej` (0–15) · `tun_cap` (0–15) · `mask_dist` (bool) · `min_num_lightning` (1|5|9|16) · `afe_gb` (`"indoor"` / `"outdoor"`) · `modem_sleep` (`"max"` / `"min"`).

Actions: `republish_status` · `calibrate_tun_cap` (~35 s sweep, MQTT keepalive pumped inside the loop) · `reboot` · `factory_reset_wifi` .

Full HANDOVER entry drafted in this session; pasteable into `vu2cpl-as3935-bridge/HANDOVER.md` separately when the operator gets to it.

3. AS3935 Control Panel — runtime tunables on the shack dashboard

A self-contained admin panel for the v0.2.0 cmd channel, living on its own flow tab (`AS3935 Tuning` , id `fe70cfdcdfa19aa4`) and its own dashboard group (`as3935_ctl_grp`). Three mqtt-in nodes (status / hb / cmd/ack) feed a single `ui_template` ; ui_template's output wires to a single mqtt-out publishing to `lightning/as3935/cmd` . All UI logic and formatting lives inside the template.

Visual design follows the GitHub-dark conventions used by the chrony card and Master Dashboard: `--bg #0d1117` , `--card #161b22` , `--border #30363d` , `--green #3fb950` , `--amber #e3b341` , `--red #f85149` ; LED indicator dots inside status chips; 8 px rounded card corners.

Rows shipped today:

- LED + title + meta (FW · IP · RSSI · uptime)
- Counters (⚡ disturber 📡 IRQ)
- Calib (TRCO · SRCO)

- **Tunables** (NF / WDTH / SREJ / TUN_CAP nudgeable; Mask dist toggleable; **AFE GB toggleable** `cf6816f` / `80a8b40`; Min strikes / Modem sleep dropdowns)
- Actions (Calibrate TUN_CAP · Republish · Reboot · Factory Reset WiFi)
- Command log (last ack)

One nontrivial bug hit and fixed during bring-up (`7d205c1`): The template's IIFE wrapper had been refactored to `(function(){…})()` with no `scope` binding. Inside the IIFE, `scope.$watch` and `scope.send` both reference an undeclared `scope` — the IIFE silently threw a ReferenceError on first run. Heartbeat counters appeared to update only because of cached DOM from a prior working deploy; status never populated, buttons silently dropped clicks. Fix was a two-token change to **Pattern B** (matching the chrony card): `(function(scope){…})(scope)`. The outer `scope` reads from node-red-dashboard's script-wrapper closure where it IS available; the IIFE captures it as a parameter. Restored the `var scope = this;` alternative (Pattern A, used by the original panel and Master Dashboard) was also viable; Pattern B chosen because it's explicit + matches a known-good newer template.

Cosmetic cleanup (`00f4270`):

- Header meta line (FW / IP / RSSI / uptime) recoloured from `--muted` to `--text` (white) — more readable, matches the body text below it.
- Calib row trimmed to `TRCO=… · SRCO=…`; the `· afe_gb=…` segment removed (now a dedicated row in Tunables, deduplicated).

4. Side-quest: rollback-tab pattern

Worth recording as a Node-RED pattern. During v0.2.0 panel development (`f88e965`) the operator imported a fresh clone of the AS3935 control flow onto a brand-new tab id, disabled the **original** tab as single-toggle rollback insurance, deployed. With the new copy verified working, the original disabled tab was deleted on 2026-05-12. Note for the future:

- A disabled flow tab still consumes node IDs and counts toward the flow file's bulk; safe to keep short-term, plan to clean up.
- Two ui_templates with the same `group` ID render into the same dashboard group. Even when the source tab is disabled, the dashboard runtime in node-red-dashboard 1.x may not cleanly stop the disabled instance from participating in group registration — observed-but-uncertain sluggishness while both copies were live. Deleting the disabled tab outright (not just `disabled:true`) is the reliable fix.

5. Post-map-ripout cleanup (`f043a4d`)

Spotted while re-reading the Lightning tab for the distance-graded work: an inject node "Clear map every 30 min" (id `55f94d9dde0dc893`) still firing every 1800 s. The map it referred to was ripped out on 2026-05-08 (HANDOVER 05-08 entry). The trigger wasn't fully dead though — its downstream `clear all` function was still wiping `flow.event_log` every 30 min, just under a stale name. Other state it cleared (`last_strike_km` for the removed gauge, legacy `strikes` array for the removed map dots) had no consumers anymore.

Tightened up:

- Renamed inject to `Clear event log every 24 h`.
- Stretched repeat from 1800 s (30 min) to 86400 s (24 h) — event-log rotation goes from aggressive to daily.
- Dropped the two dead `flow.set` lines from `clear all`; only the `event_log = []` + `{type:'clear'}` emit to Master Dashboard remain.

3 lines changed in flows.json; behaviour now matches the label.

Other ops notes from today

- Real lightning fired at AS3935 distance 1 km mid-session (`energy 219990` , `timestamp 2026-05-12T16:00:06`), 56 disturbers in the previous 4 min prior. Disconnect chain fired correctly per the close-zone rule.
- Indoor noise floor remains high; ESP32-outdoor-install (HANDOVER #1) is the path to material false-positive reduction. The graded matrix helps medium / far zones; close zone false positives remain inherent until the sensor relocates outside.

REBUILD_PI.md impact: none. All changes are inside `flows.json` — a fresh rebuild clones the repo and gets the new logic automatically.

Chrony status card: GitHub-dark palette + vanilla JS DOM (`d9d57e8`)

Brought the GPS NTP chrony card in line with the rest of the dashboard's GitHub-dark conventions. The old card used a custom teal-on-near-black palette (`#5cd0d6` teal section labels, `#0e151e` background, `#e89a4a` orange attention, etc.) that stood out distractingly next to everything else on `/ui` . New palette pins to the shared tokens:

Token	Value	Used for
`--bg`	`#0d1117`	page bg
`--card`	`#161b22`	card surface
`--border`	`#30363d`	outlines + row dividers
`--green`	`#3fb950`	good (PPS, stratum 1, 3D fix)
`--amber`	`#e3b341`	attention threshold + warn chips
`--red`	`#f85149`	bad (no fix, stratum bad)

Other changes folded into the same retheme:

- **Rendering switched** from AngularJS interpolation (`{{data.foo}}` + `ng-class`) to vanilla JS DOM (`getElementById`
    - `classList` , driven by `scope.$watch('msg', …)`). Matches the convention used by other custom widgets across this dashboard; avoids the whole-card re-render that AngularJS bindings trigger on each `scope.data` assignment.
- **Status chips gained color-coded LED dots** (8 px round + a `box-shadow:0 0 4px currentColor` glow). Quick visual triage at a glance — green LED for PPS/stratum 1/3D fix, amber for degraded, red for no fix.
- **Hosting `ui_group` flipped** to `disp: false` (the template carries its own title) and `width: 10` (was width 6 inside the general Network Monitor group).

Pi-side workflow: `pi-gps-ntp-server@6e54f14` landed the canonical template + README upgrade guide + matching preview HTML upstream. Operator then opened the live `Chrony status card` ui_template in the Node-RED editor, pasted the new template body, flipped the `ui_group` properties, Deployed, and `nrsave` d. That commit is the merge here. `nrsave` (now a function — closes HANDOVER #17) handled the DXCC tab extract re-gen automatically; extract was a no-op since the DXCC tab wasn't touched, so the commit is flows.json-only.

CLAUDE.md's "Chrony / GPS Time Server card (gpsntp.local)" section updated: architecture note now reflects vanilla-JS-DOM + the palette + the `disp:false` /width-10 ui_group config; attention thresholds row clarified that the value text renders **amber** (`#e3b341`) not orange when crossed.

2026-05-11

AS3935 publisher: ESP32 bridge takes over from Pi daemon (same day bench bring-up)

After this morning's planning conversation (recorded in HANDOVER #21

- the scaffold for `vu2cpl-as3935-bridge`), the operator wired the ESP-WROOM-32 + AS3935 on a breadboard and got firmware v0.1.1 publishing live the same evening. Key bench results:
- `TRCO=OK` , `SRCO=OK` after `CALIB_RCO` — internal oscillators calibrated cleanly.
- I²C link solid at address `0x03` .
- MQTT pipe live to `192.168.1.169:1883` — retained `status` lands immediately on connect, `hb` ticks every 30 s, retained status auto-refreshes every 5 min.
- WiFi credentials now via WiFiManager captive portal (`vu2cpl-as3935-setup` / `vu2cpl1234` AP on first boot — no `secrets.h` to bake into firmware).
- End-to-end verified: piezo-lighter sparks produce `event:"disturber"` events on `lightning/as3935` . Counters increment. Node-RED Lightning Antenna Protector flow consumed everything without a single config change — the goal of the contract-identical design.
- Indoor Pi daemon `as3935.service` on `noderedpi4` **stopped and disabled**. ESP32 is the sole publisher on `lightning/as3935/*` .

Reflected in this repo (vu2cpl-shack):

- **REBUILD_PI.md Step 7** — no longer enables `as3935.service` on rebuild. Pi daemon stays installed (files in `/home/vu2cpl/` and `/etc/systemd/system/`) but disabled by default. If the ESP32 ever fails, `sudo systemctl enable --now as3935` resurrects the Pi as publisher. Enabling both at once would race the MQTT topic and corrupt retained status — added a row in the failure-modes table for that.
- **REBUILD_PI.md Step 12 verification row #6** — reworded to "AS3935 publishing (from ESP32 bridge)"; still asserts the same `lightning/as3935/hb` topic ticks every 30 s, just from a different publisher.
- **`rebuild_pi.sh` Stage 9** — `enable --now as3935 rpi-agent` → `enable --now rpi-agent` only. Stage 13 verification drops the `systemctl is-active --quiet as3935` check (the `lightning/as3935/hb` topic check below it covers the actual operational signal). Bash syntax-checked.
- **CLAUDE.md Pi-side scripts table** — `as3935_mqtt.py` and `as3935.service` rows now annotated as standby fallback.
- **README.md** — Lightning Antenna Protector subsystem description updated; hardware table row updated; file-tree comments updated.
- **HANDOVER.md "Current system state"** — AS3935 row reflects ESP32 primary, Pi daemon disabled.
- **HANDOVER.md #1** (AS3935 outdoors) — still open; the physical outdoor install (enclosure, power chain, shade mount, post-install TUN_CAP re-tune) remains. Annotated with the bench milestone.
- **HANDOVER.md #21** — updated from "planning" to "v0.1.1 live on bench, indoor daemon retired". Lists v0.2.0+ open items (on-device TUN_CAP cal mode, power chain, enclosure, field install).

No `flows.json` change. The MQTT-contract-identical design paid off — Node-RED keeps consuming `lightning/as3935` , `lightning/as3935/status` , `lightning/as3935/hb` , `lightning/as3935/lwt` exactly as before, just from a different publisher.

HANDOVER #10 (gpsntp + TX RFI watch): closed — no issue at 1 kW

When `gpsntp.local` was first deployed (2026-05-09) with its QLG1 patch antenna tapped off the U3S's 6-way header, the obvious risk was RFI desensitisation of the GPS during shack transmissions — the QLG1 sits physically next to the U3S WSPR TX, and a stratum-1 NTP server losing fix during a transmission is the kind of bug that takes a week of watching to catch.

Operator verified today that `gpsntp` holds stratum-1 PPS lock through full **1 kW** SPE amplifier transmissions — well above the ~200 mW WSPR level the original watch was designed against. If 1 kW doesn't move the needle, the U3S certainly won't either.

Dedicated NEO-M8N + antenna fallback (documented in `pi-gps-ntp-server/HANDOVER.md` as the planned recovery path) remains unused and parked.

CLAUDE.md TODO #1 (AetherSDR MQTT bug): closed — upstream fix shipped

Tracked at ten9876/AetherSDR#1348 — "MQTT: 'Connect failed: Socket is not connected' on plain port 1883 with TLS off (macOS)". Reported against AetherSDR v0.8.11 with the exact symptom Manoj hit (TCP SYN → SYN-ACK from the Pi broker → immediate RST from the Mac client, repeating every 4-5 s without ever reaching the MQTT handshake — Mosquitto logs zero connection attempts).

Root cause was a macOS-specific bug in the bundled libmosquitto's non-blocking connect + immediate packet write path. Fixed in PR #1349, merged 2026-04-15 and shipped in v0.8.15 ~25 minutes after the merge ("Fix MQTT macOS connection failure" in the changelog). Subsequent MQTT polish landed across v0.8.15.1, v0.8.16 (proper TLS with OpenSSL 3.5+), and the 0.9.x line. The project switched from semver to CalVer today; current release is v26.5.1.

Mac-side action (separate from this repo): upgrade AetherSDR on the Mac Mini to v26.5.1, re-test MQTT to `192.168.1.169:1883` with TLS off. If it connects clean, the loop is closed; if it still fails, file a fresh issue with current tcpdump against v26.5.1.

CLAUDE.md TODO #5 (website uploads): closed as already done

Stale carryover. `~/projects/vu2cpl.github.io/` already carried all referenced assets — `shack-desk.jpg` , `shack-workbench.jpg` , `vu7ms_writeup.pdf` , `vu7t_writeup.pdf` — committed and in sync with `origin/main` ; `index.html` correctly references them. The TODO had said `shack.jpg` singular; the page design evolved to a two-image gallery (desk + workbench). No further action.

DXCC TODOs #7 + #8: closed as no-action (already done / already covered)

End-of-day audit pass on the two remaining DXCC backlog items.

TODO #7 — Separate CW/Ph/Data fetch modes. Already in place. `Build Club Log API Request` (`6e60f619acad462e`) constructs four URLs (`mode=0` all-modes, `mode=1` CW, `mode=2` Phone, `mode=3` Data) and the active `Fetch All Modes + Parse lotw only` (`aa7434df62b95ebc`) runs all four in parallel via `Promise.all([fetchURL(m0), fetchURL(m1), fetchURL(m2), fetchURL(m3)])` , parsing each into ew0/ew1/ew2/ew3 and composing `dxccModeWorked` as `{adif: {cw:bool, phone:bool, data:bool}}` consumed by the `NEW_MODE` classifier. Likely the TODO predates this implementation. (The legacy `Fetch All Modes + Parse` variant with `bands[mk]>=2 = confirmed` is disabled; the live LoTW-only variant uses strict `===2` .)

TODO #8 — Non-project folder path support. Already covered by a fallback. Both Fetch functions read `var flowsDir = flow.get('cfg_flows_dir') || os.homedir() + '/.node-red';` — if `cfg_flows_dir` is empty (Projects feature not enabled), the default `~/.node-red/` kicks in. VU2CPL uses Projects so the fallback never fires here, but the defensive code is present for operators who fork the flow into a non-Projects setup.

No code change. CLAUDE.md rows reworded to closed-no-action.

DXCC: filter persistence end-to-end (closes CLAUDE.md TODO #6)

Filter chip state on the DXCC dashboard + the spot TTL slider now survive a Node-RED restart. Verified live: toggled `● MODE` off, restarted nodered, fired the `TEST XE2ABC 20M SSB (new mode)` inject — the `DXCC Prefix Lookup + Alert Classify` node correctly dropped it (no alert table row, no Telegram). Re-enabled `● MODE`, fired the same inject — alert appeared.

Root cause was deeper than the surface bug. Investigation turned up that `Save Alert Filters HTTP` writes filter state with `flow.set('filterX', val, 'file')` but every reader across the tab (`DXCC Prefix Lookup`, `Format Alert for Dashboard Table`, the two `Format Telegram Alert*` functions, `Format FlexRadio Spot Command`) reads with `flow.get('filterX')` — no scope arg = default (memory). That alone would mean reads never see writes. **But** the `'file'` context store wasn't actually configured on this Pi: `enable_file_context.sh` had been part of REBUILD_PI Stage 8 but **had never run** here — its idempotency check (`grep -q '"localfilesystem"'`) matched the commented template block in stock `settings.js`, short-circuiting before any edit. And even if it had run, its substitution block declared a single store named `default` backed by localfilesystem — which would have silently routed all no-scope `flow.set/get` calls across the whole codebase to disk and still left the `'file'`-scoped calls (which look for a store literally named `'file'`) unbacked.

Two-part fix:

1. **`enable_file_context.sh` rewritten** —
    - Idempotency check now anchors on `^\s+contextStorage:` so the commented template doesn't match.
    - Substitution installs **two** named stores plus a string-alias default:

      ```
      contextStorage: {
          default: "memory",
          memory: { module: "memory" },
          file:   { module: "localfilesystem" }
      },
      ```

      This keeps every existing no-scope `flow.set/get` in the codebase in-memory (zero behaviour change for the 100s of such calls outside DXCC) while giving the explicit `flow.set/get(..., 'file')` calls a real file-backed store.
    - Ran on the Pi → `~/.node-red/context/` now populated; flow-scope file for the DXCC tab landed at `~/.node-red/context/d110d176c0aad308/flow.json`.
2. **Readers aligned to `'file'` scope.** 32 mechanical substitutions across 5 reader function bodies, regex `flow\.get\('(filter\w+|spotTTL)'\)` → `flow.get('$1', 'file')`. No structural changes; the regex was tight enough to leave the no-scope reads of `dxccReady`, `entityWorked`, `workedTable`, `dxccBlacklist`, etc. completely untouched. Per-function count:

```
DXCC Prefix Lookup + Alert Classify    9
Format Alert for Dashboard Table       7
Format FlexRadio Spot Command          1
Format Telegram Alert Dedup 10 minute  8
Format Telegram Alert                  7
```

Applied programmatically from Mac side after operator confirmation (per CLAUDE.md rule #1) since 32 manual editor edits across 5 functions had material miss-one-and-it's-silently-half-broken risk. Loaded via `git pull` + `sudo systemctl restart nodered` on Pi.

Surfaced bug, queued as HANDOVER #20 — and later closed as misdiagnosis the same day. I had flagged a worked-table dual-write issue in `Fetch All Modes + Parse` based on the `'file'`-only writes on L91-94. Wrong: a closer read showed the function does *triple* persistence — L85-88 are memory writes (no scope), L91-94 are file context writes (`'file'`), and L96-103 also writes `nr_dxcc_seed.json` to disk directly via `fs.writeFileSync`. Architecture has always been correct; today's enable-the-file-store fix just makes the L91-94 path finally land where it always intended. Lesson, recorded for future audits: filtering scope-by-scope hides parallel writes — scan full function bodies before claiming a dual-write gap.

REBUILD_PI.md impact: none directly — Stage 8 already ran `enable_file_context.sh`; the script fix flows through automatically. A fresh rebuild today would now actually produce a working file context store (previously the install was silently no-oping).

DXCC: Club Log API ban verification — lifted, no flow change (closes CLAUDE.md TODO #10)

Operator confirmed the Club Log API ban from earlier this year is no longer in effect. Verified live: `nr_dxcc_seed.json` has `updated: "2026-05-11T03:27:47.276Z"`, written by the daily `00 02 * * *` cron's `Daily club log refresh (0200)` inject (`c43fbdd61175ce24`). The fetch chain is healthy end-to-end.

No flow change. The current `once: false` posture on `Load Club Log on startup` (`4ebafea5ce2d9d7b`) and `Retry Club Log (90s)` (`9c98f9e7a941e852`) remains correct defence-in-depth, *even with the ban lifted*. Why: those injects fire on every Deploy (not just every Node-RED restart). Flipping to `once: true` would mean a 10-deploy active development session makes 11 Club Log API calls instead of 1. The daily cron + the existing `POST /dxcc/refresh` HTTP endpoint cover every real refresh need:

- Scheduled fetch — daily cron, 02:00 IST, 1 call/day.
- Ad-hoc fetch after a session of QSOs — `POST /dxcc/refresh` (manual button on the DXCC dashboard).
- Restart-time freshness — `Bootstrap Worked Table` (`1a13cd6d9aabaa54`) flips `dxccReady=true` from cached `nr_dxcc_seed.json`, so the tracker is fully operational on cached data within ~2s of Node-RED start.

CLAUDE.md cleanup folded in:

- TODO #10 row reworded to "Closed 2026-05-11 — ban lifted; design retained".
- "Data files (must exist in `cfg_flows_dir`)" list was inaccurate — claimed `nr_dxcc_maps.json` and `nr_dxcc_modes.json` were required. Neither exists on the Pi or in this repo. The modes data lives **inside** `nr_dxcc_seed.json` under key `dxccModeWorked`; the prefix-to-DXCC-entity map is rebuilt in-memory from `cty.xml` on every startup (the file fetched by `Load cty.xml on startup` `3720b5e9691dfb9c`). Corrected.

- BACKUP section's "Critical files" list also referenced both stale files — replaced with the actual two-file truth.

REBUILD_PI.md impact: none. The runbook fetches all DXCC data from a fresh `git clone` + a runtime cty.xml + a runtime Club Log call — never depended on the two phantom files existing.

`nrsave` : auto-regenerate DXCC tab extract (closes HANDOVER #17)

Pi-side `nrsave` was an alias on `~/.bashrc:114` :

```
alias nrsave="cd ~/.node-red/projects/vu2cpl-shack && git add flows.json && git commit -m"
```

It did not regenerate `clublog_dxcc_tracker_v7.json` — meaning CLAUDE.md rule #4 ("on every flows.json commit, extract the DXCC Tracker tab") was being silently violated on every `nrsave` . The extract was only refreshed when something on the Mac side did the python one-liner manually. Today's Parse Strike rebase (e8a2dd4) had to amend in 97/76-line drift accumulated since whenever the extract was last regenerated. Don't want that pattern to recur.

Fix: convert the alias to a function (aliases can't run logic between args, only chain commands), with the extract step threaded in before `git add` :

```
nrsave() {
    cd ~/.node-red/projects/vu2cpl-shack || return 1
    python3 -c 'import json; d=json.load(open("flows.json")); v=[n for n in d if n.get("z")=="d110d176c0aad308" or n.get("id")=="d110d176c0aad308"]; json.dump(v,open("clublog_dxcc_tracker_v7.json","w"),indent=2)' || return 1
    git add flows.json clublog_dxcc_tracker_v7.json
    git commit -m "$1"
}
```

The function fail-fasts if either the `cd` or the extract step returns non-zero, so a malformed flows.json doesn't get committed with a stale extract.

Applied via a python-based in-place edit of `~/.bashrc` (safer than `sed` with the JSON literal's quote-soup in the python one-liner). Backup `~/.bashrc.bak.YYYYMMDD_HHMM` retained on the Pi. Verified live with `type nrsave` .

Documentation alignment:

- **CLAUDE.md rule #4** reworded — now says the regen happens automatically inside `nrsave` , with the manual python one-liner kept as a fallback for non-nrsave commit paths
- **CLAUDE.md "Save changes" + "GIT WORKFLOW"** sections updated to drop the now-redundant manual extract step
- **CLAUDE.md infrastructure table** row reworded from "Git alias" to "Git function"
- **REBUILD_PI.md Step 5** — the "Set up the `nrsave` git alias" block had been carrying a **stale** definition that wrapped a `git save` config alias indirection (a variant that was never actually used on this Pi). Replaced with the current function form so a fresh rebuild lands the same `nrsave` that's running today.
- **`rebuild_pi.sh` Stage 7** — same fix in the automation script. Bash syntax-checked.

Lightning: Parse Strike — Blitzortung dead-code strip (closes HANDOVER #7)

`e8a2dd4` — operator-side flow change, audited + extract-regen-amended + rebased Mac-side.

After the 2026-05-10 Blitzortung drop (HANDOVER #7), the `Parse Strike` function node (`26ddff0cbbfe5fc1` , Lightning Antenna Protector tab) still carried its full Buffer/string parser for Blitzortung's binary-with-embedded-text TCP feed: ~30 lines of `findKey` byte-scanner + `readCoord` digit-parser + a CASE 2/3 branch entered on any non-object `msg.payload` . Nothing was ever wired to feed it — audit of upstream connections returned only:

- 3 TEST inject nodes (object payload)
- `Parse Open-Meteo → Strike` (object payload)

All four upstreams use CASE 1, so CASE 2/3 was unreachable in production. Replaced the dead branch with a one-line "dropped 2026-05-11 (HANDOVER #7) — restore from git history if reinstated" note + `return null` . Net flows.json change: 1 insertion, 1 deletion (function body is a single JSON-encoded string).

Rule #4 catch-up — DXCC tab extract drift fix: the operator's `nrsave` (alias for `git add flows.json && git commit`) did not regen `clublog_dxcc_tracker_v7.json` , so the extract had been drifting behind every nrsave-only commit for an unknown number of days. Today's amend re-ran the extract script and folded the resulting 97-insertion / 76-deletion diff into the Parse Strike commit before pushing. Most of the drift was unrelated (`tx_color` field on FlexRadio slices, AS3935 panel additions, etc. — all already in flows.json from earlier deploys); just nobody had re-run the extract. Worth folding the extract step into `nrsave` itself, but that's a separate Pi-side alias change (HANDOVER #17, queued).

No behaviour change for any payload that actually fires today — CASE 1 retained verbatim. Only difference: a Buffer or string payload (which never arrives) now returns null in 1 line instead of running 30 lines and returning null.

Mac-side rebase note: operator's commit landed on a Pi-side main branch that was 5 commits behind origin (today's doc commits). Stash-seed → amend-extract → rebase-pull → pop-stash → seed-commit → push sequence kept history linear and the seed refresh as its own commit, per repo convention. Operator's authorship preserved through the amend.

No `REBUILD_PI.md` impact — disaster recovery just clones the repo, and the cleaned-up `flows.json` comes along for free.

CLAUDE.md "Key Node IDs" + HANDOVER.md "Key files & IDs to know" caveats updated to drop the "Cases 2/3 dead" note (they're not just dead — they're gone).

LP-700-HID ws tab: Description field cleared (closes HANDOVER #16)

`72fc31e` — operator-side. The tab's sidebar Description carried 419 chars of legacy install instructions (`npm install robertsLando/node-red-contrib-usbhid`

- telepost udev rules + `gpasswd -a $USER telepost` + unplug/replug note), all from the pre-WS-gateway era. Useless after the 2026-05-09 migration and 2026-05-11 HID-package uninstall — anyone following them now would either install a redundant package or modify udev rules for a `/dev/hidraw*` node we don't even talk to directly anymore.

HANDOVER #16 had been closed as won't-do earlier today (purely cosmetic, editor-only sidebar text); operator did it anyway as a zero-risk paste-the-empty-string change. Reopened and closed as done.

LP-700: HID package uninstalled (post-WS-migration cleanup)

The 2026-05-09 LP-700 → WebSocket-gateway migration left `@gdziuba/node-red-usbhid` installed in `~/.node-red/` as a no-longer-used palette package. CLAUDE.md's "uninstall after a week of stable WS operation" check has been met (stable since 05-09, verified end-to-end on the Lightning storm day 05-10 + LP-700 panel auto-scale work). Cleaned up today:

```
cd ~/.node-red && npm uninstall @gdziuba/node-red-usbhid
sudo systemctl restart nodered
```

Node-RED restarted clean (`journalctl -u nodered` showed no errors), LP-700 panel still rendering at ~25 Hz from the WS path.

The accompanying `-dev` build libs (`libudev-dev` , `librtlsdr-dev` , `libusb-1.0-0-dev`) — flagged in CLAUDE.md as needing apt removal — turned out **not to be installed** on the current Pi. Either an earlier cleanup pass removed them or the original `node-red-contrib-usbhid` build linked against system libs without needing the dev headers persistently. Either way, no apt action needed. The runtime counterparts (`libudev1` , `libusb-1.0-0` , `librtlsdr0`) remain — they're transitive deps of many system packages and have nothing to do with the HID node.

Audit before uninstall: `grep -nE 'usbhid|hid-manager|@gdziuba' flows.json` returned only a single stale `info` -description field on the LP-700-HID ws tab (legacy install instructions in the tab's sidebar Description). Cosmetic, zero runtime impact — flagged for the operator to clear via the Node-RED editor when convenient (tab Properties → Description → empty → Deploy → `nrsave`). Not edited here because flows.json edits must come from the Pi-side editor as the source of truth (CLAUDE.md rule #1).

`npm audit` noise: the uninstall surfaced "13 vulnerabilities (12 moderate, 1 high)" from npm's blanket audit across the whole `~/.node-red` install. Unrelated to this change — same advisories were present before. `npm audit fix --force` is **not** safe to run in a Node-RED palette directory (breaks pinned palette versions). Left as-is.

REBUILD_PI.md check: that runbook already did not install `@gdziuba/node-red-usbhid` or its -dev libs in Step 2 (system packages) or "Install required palette packages". No `REBUILD_PI.md` change needed — a fresh rebuild from scratch will land in the same clean state.

CLAUDE.md `## NODE-RED PALETTE PACKAGES` cleaned up: HID row, "When ready to clean up" block, and "Original install reference" archaeology block all removed. Replaced with a brief paragraph recording the migration + uninstall dates for future archaeologists.

`gpsntp.local` : log2ram installed (closes HANDOVER #12)

SD card wear mitigation on the stratum-1 GPS NTP server. Chrony + gpsd run 24/7 and write continuously to `/var/log` ; on a Pi 3B with a consumer SD card this eventually wears out the card. log2ram from the azlux apt repo mounts `/var/log` as tmpfs (128 MB) and flushes to SD hourly + on shutdown.

Procedure followed (`pi-gps-ntp-server/BUILD.md` "Optional — Reduce SD card wear"), with one fix: BUILD.md hardcodes `bookworm main` in the apt sources line, but `gpsntp` is now on Debian 13 (`trixie`). Confirmed the azlux repo has a `trixie/` suite before adding it. Steps:

```
echo "deb http://packages.azlux.fr/debian/ trixie main" | \
  sudo tee /etc/apt/sources.list.d/azlux.list
sudo wget -q -O /etc/apt/trusted.gpg.d/azlux.gpg https://azlux.fr/repo.gpg
sudo apt update
sudo apt install -y log2ram
sudo systemctl enable log2ram
sudo reboot
```

The `enable --now` form from the BUILD.md doc would not actually move `/var/log` to RAM on first install — log2ram's bind-mount has to happen before any service opens a log file, so it only takes effect on next boot. Reboot is mandatory.

Verification after boot:

```
log2ram on /var/log type tmpfs (rw,nosuid,nodev,noexec,noatime,size=131072k,mode=755)
systemctl is-active log2ram      → active
systemctl is-enabled log2ram     → enabled
df -h /var/log                   → 128M total, 612K used
```

Chrony re-locked to stratum 1 with PPS reference within ~60 s of boot. Numbers at first re-lock: system time 181 ns fast of NTP truth, root dispersion 16 µs, skew 0.087 ppm, residual freq −0.168 ppm. Indistinguishable from pre-log2ram numbers (as expected — log2ram only changes where `/var/log` writes land, not anything chrony reads).

Fix landed in the other repo ([`pi-gps-ntp-server@5b115ba`]()): swapped the hardcoded `bookworm` for `${VERSION_CODENAME}` read from `/etc/os-release` (release-agnostic — survives the eventual Debian 14 move), changed `enable --now` to `enable` + explicit `reboot` with a note explaining why `--now` doesn't actually move `/var/log` on first install, and added a verify block.

No REBUILD_PI.md impact — that runbook is for the shack Pi (`noderedpi4`); log2ram is on `gpsntp.local` , a different host with its own (separate) build doc.

N2WQ login pattern — ported to DXClusterAggregator (Swift)

No Node-RED code change. Recording this as a cross-project reference so future-self knows the canonical fix lives in two places (and how to keep them aligned).

The 2026-05-10 N2WQ disconnect cycle (see entries above) was fixed on the Node-RED side via:

- Tightened login regex: short line `endsWith('login:')` style, rejects the banner's `Last login:` mention
- Packet-radio SSID suffix `-1` on the callsign sent to N2WQ to sidestep the cluster's no-duplicate-login enforcement when another LAN client is also logged in as `VU2CPL`

Both patterns have now been ported to the macOS app **DXClusterAggregator** (Swift, separate repo). Notable Swift-side porting issues that Node-RED's `tcp in` node had quietly handled for us:

1. Telnet IAC binary preamble

N2WQ emits 6 bytes of Telnet option negotiation before the prompt:

```
FF FB 03    IAC WILL SUPPRESS-GA
FF FB 01    IAC WILL ECHO
```

These are invalid UTF-8 and corrupt the first chunk if you decode directly. Node-RED's `tcp in` with `datatype: utf8` silently strips/replaces them; a raw-socket client (Swift `NWConnection` , Python `socket.recv` , raw `nc` , `socat`) sees them verbatim.

Swift fix: byte-level `stripTelnetIAC(_:) -> Data` scanner that drops `IAC WILL/WONT/DO/DONT` (3-byte sequences), `IAC SB ... IAC SE` (variable subnegotiation), `IAC IAC` (literal 0xFF), and other 2-byte IAC commands. Applied to every `receive()` chunk before String decoding. No reply needed — clusters tolerate silent partners fine.

2. Hanging prompt without trailing LF

N2WQ sends `login: ` with no LF — a classic Telnet hanging prompt waiting for a live cursor. Code that splits incoming bytes only on newlines never sees the prompt; bytes accumulate in the buffer forever, the server times out, infinite reconnect loop.

Swift fix: after the line-buffer loop drains LF-terminated input, peek at the residual `buffer`. If it trims to a short string (`< 40` chars) ending in a recognized login/password prompt suffix, hand it to the auth handler and consume.

Node-RED's `tcp in` model accidentally handles this fine — each TCP chunk arrives as its own `msg.payload`, treated as a complete unit by our function. No line buffer to drain. Different model; same outcome.

Cross-project alignment

Both implementations use the same prompt suffix list: `login:`, `please login`, `please login:`, `callsign:`, `callsign please:`, `your callsign:`, `enter your callsign:`. Treat this list as a shared canon — if a new cluster needs a different prompt match, update both. The Swift code centralises its list as `static let promptSuffixes`; the Node-RED code keeps it inline in `Login + Parse + Dedup` (`login-parse-dedup-v2`).

Both also rely on operator configuration to set a callsign-SSID (e.g. `VU2CPL-1`) when an LAN-side duplicate is detected. No code change to enable — just edit the cred / source-config field on each side.

Lessons portable to any cluster client

1. **A login prompt is a short line ending with the indicator.** Detecting `login:` as a substring anywhere will eventually match a banner mention. Use `endsWith()` plus a sanity length guard.
2. **Telnet IAC negotiation must be stripped at byte level** for raw socket clients. Anything decoding UTF-8 from `recv()` will either choke or get garbage on the first chunk.
3. **Cluster software enforces uniqueness on the bare callsign**; packet-radio SSIDs (`-1` through `-15`) are accepted as distinct logins by most AR-Cluster forks. `-N` letter suffixes are not always validated cleanly; numeric SSIDs are the safest default.
4. **Hanging prompts vs newline-terminated input** are two different I/O paths. If your client splits on `\n` only, you need an explicit "buffer residual after split, check for prompt" path or you'll deadlock on the very first connection.

No Node-RED code change in this commit. Documentation only.

2026-05-14

Lightning dashboard — label Event Log times as IST

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** Master Dashboard ui_template (`557083037f168b22`)

Event Log header changed from `Event Log` to `Event Log · times in IST (UTC+5:30)`. No code path or logging behaviour changed — purely a UI label so the timestamps in each log row (which come from the

AS3935 bridge's local-formatted `timestamp` field, falling back to `new Date().toLocaleString()` on the Pi) are unambiguously read as Asia/Kolkata local time and not UTC.

The on-disk JSONL historic store (`nr_lightning_events.jsonl` , added 2026-05-13) is unaffected — it stays UTC ISO-8601 via `new Date().toISOString()` for archival use. Dashboard = IST, archive = UTC.

Follow-up tracked as TODO #13 in CLAUDE.md: the AS3935 Local Sensor card duplicates the same local timestamp in both the "Last seen" field and the Disturber / Noise status chip — declutter pending.

Tasmota — set Timezone to IST (+05:30) on all 5 devices

Devices: `powerstrip1` , `powerstrip2` , `powerstrip3` , `4relayboard` , `16Amasterswitch` .

The 16A master switch's `ENERGY.Today` counter on the dashboard was resetting at 05:30 IST instead of local midnight, because the Tasmota device was running on the firmware default `Timezone 0` (UTC). `Today` is computed and rolled over inside Tasmota itself based on its local-clock date — Node-RED just forwards the value (`Parse 16Amasterswitch` → `Energy Aggregator` → `16A Energy Monitor`). No flow change needed; fix is per-device:

```
for t in powerstrip1 powerstrip2 powerstrip3 4relayboard 16Amasterswitch; do
  mosquitto_pub -h 192.168.1.169 -t "cmnd/$t/Timezone" -m "5:30"
done
```

Verified all 5 with empty-payload read-back — each replied `{"Timezone":"+05:30"}` on its `stat/<device>/RESULT` topic. Setting persists in Tasmota NVS across reboots.

One-time anomaly for today (2026-05-14): the timezone change landed at ~15:00 IST. The current `Today` counter (1.120 kWh at the moment of verification) reflects accumulation since the *last* rollover, which happened at 00:00 UTC = 05:30 IST today — so it's only 9.5 h of data, not a full 24 h. The first clean IST midnight- to-midnight cycle begins at 00:00 IST on 2026-05-15.

Documented in CLAUDE.md Hardware Map (Tasmota Power Devices section) and in REBUILD_PI.md Step 11 so a fresh Tasmota reflash doesn't silently revert to UTC. The other 4 devices don't have energy monitoring, but their internal `Timer` actions and log timestamps would have been 5.5 h off — now consistent across the shack.

Lightning dashboard — Event Log survives Node-RED restart

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **New nodes:** `light_bootstrap_inj_01` (inject) + `light_bootstrap_fn_01` (function). Node count 76 → 78.

Problem. After every `sudo systemctl restart nodered` , both the Event Log widget and the AS3935 card's "Last seen" field showed blank. Root cause is two-fold:

1. `flow.event_log` lives in **memory** context (no `'file'` scope on any `flow.get` / `flow.set` call), so restart wipes it. The `Stats refresh every 30s` inject then sends `{type:'log', html:''}` every 30 s, painting `logLines.innerHTML = ''` → blank.
2. The on-disk JSONL store (`nr_lightning_events.jsonl` , added 2026-05-13 in commit `bf480be`) had all the history, but no startup node read it back into `flow.event_log` .

Fix. New inject `Bootstrap Event Log (startup)` fires once at `onceDelay: 2 s` (after Init Defaults at 0.5 s, before Stats refresh at 3 s) into a new function `Bootstrap Event Log from JSONL` . The function:

- Reads `flow.cfg_events_jsonl` (set by Init Defaults).

- Loads the file, splits by newline, takes the last 50 records (mirrors the in-memory cap in `AS3935 Warn Log` / `AS3935 Disconnect Log`).
- Reverses to match the live unshift-built order (newest first).
- Extracts the pre-rendered `rec.html` field from each line (already present — every existing JSONL writer includes the full HTML row alongside the structured fields).
- Sets `flow.event_log` to the result and emits one `{type:'log', html:…}` to Master Dashboard for immediate paint.
- ENOENT (no JSONL yet) → silent grey status; malformed JSON lines skipped silently; other I/O errors → red status + `node.error`.

`libs: [{var:'fs', module:'fs'}]` declared on the function (same pattern as `Append Lightning JSONL`) — function nodes need explicit module imports per the lesson from commit `c8fbcb4`.

What this does not fix. "Last seen" (`a35time`) still blanks on restart until the next AS3935 disturber/noise/strike event. The JSONL records lack the right shape to synthesise a clean `as3935_status` replay (records are `type: warn|disconnect|reconnect`, not `event: disturber|noise`), so bootstrapping it properly needs the AS3935 ESP32 bridge to publish a retained "last event" message — separate larger fix, folded into TODO #13 (AS3935 card declutter) when we tackle it.

Behaviour summary. First Deploy after this commit: bootstrap fires at +2 s, dashboard shows the last 50 events from JSONL within 3 s of startup. Subsequent restarts: same, transparently. No behaviour change to the running flow — pure additive bootstrap.

Lightning dashboard — AS3935 card declutter (TODO #13 closed)

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** Master Dashboard ui_template (`557083037f168b22`)

Two cosmetic problems on the AS3935 Local Sensor card:

1. **`LAST SEEN` showed an absolute timestamp** like `14:32:05` — information that decays in usefulness as time passes ("was that today? yesterday?").
2. **The right-side status chip repeated the same timestamp** during the 60 s disturber/noise window: `⚠ Disturber 14:32:05` — visually noisy and redundant once `LAST SEEN` had the same value.

Fix (pure ui_template edit; no flow change):

- `LAST SEEN` now shows **relative age** — `"4s ago"` / `"3m ago"` / `"1h 14m ago"` / `"2d ago"` — same format used by the existing `alertBox` recap line. Hovering the value reveals the absolute IST timestamp via `title` tooltip, so the precise value isn't lost.
- A 30 s ticker auto-refreshes the relative-age string while a last-event timestamp is held in JS state. Ticker is started lazily on the first event (no work until something happens).
- Three call sites in the disturber / noise / strike branches all funnel through a new `recordAS3935Event()` helper that captures `Date.now()` and repaints. Receive-time is used, not the bridge's timestamp string — avoids parsing the bridge format and the relative-age display is insensitive to the small bridge→broker→browser skew.
- Chip text on disturber/noise dropped the trailing timestamp: `'⚠ Disturber ' + d.timestamp` → `'⚠ Disturber'`. Strike chip (`⚡ X km (Y)`) was already clean.

What this does NOT fix. After a Node-RED + browser restart, `LAST SEEN` is still blank until the next AS3935 disturber/noise/ strike. Bootstrapping it from JSONL doesn't work cleanly because JSONL records are `type: warn|disconnect|reconnect` (per the log writers) — they don't carry the right shape to drive `a35time`. The proper fix is to have the AS3935 ESP32 bridge publish a **retained**

`lightning/as3935/last_event` topic so the broker replays the most recent event on Node-RED reconnect. Tracked as TODO #15 in CLAUDE.md.

Minor immaterial leftover. The disturber / noise / strike branches each still declare `var timeEl = document.getElementById('a35time');` even though the `recordAS3935Event()` helper now gets the element itself. Unused variable; harmless; left as-is to keep this diff focused on the visible change.

Lightning dashboard — Atmospheric CAPE tile + AS3935 chip one-liner

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes:** Master Dashboard ui_template (`557083037f168b22`), Parse Open-Meteo → Strike (`593f22a507b46335`).

Atmospheric CAPE tile. The OM parser already computed `cape` and derived `om_state` (`cold` / `lit` / `severe`) for the distance-graded disconnect matrix, but nothing on the dashboard showed the actual number. New tile under the Reconnect Timer in `#threshBox` :

- Title: `Atmospheric CAPE`
- Value: `<rounded J/kg>` (e.g. `1842 J/kg`), font-size 28 px (matches Disconnect Threshold styling)
- Colour mapping driven by `om_state` , not raw CAPE — because `om_state` already accounts for the WMO-thunderstorm gate (CAPE ≥ 800 alone isn't "lit" without a thunderstorm WMO code):
  - `cold` → green (`--green`)
  - `lit` → amber (`--amber`)
  - `severe` → red (`--red`)

Plumbing: Parse Open-Meteo's output 2 (the "always emit" dashboard log path) now sends two messages instead of one — `[logMsg, capeMsg]` where `capeMsg.payload = {type:'cape', cape, om_state}` . Node-RED dispatches both sequentially to the Master Dashboard. New `paintCape(cape, state)` helper + `{type:'cape'}` handler in `scope.$watch` . Updates every 5 min (OM poll cadence). On boot, tile shows `— J/kg` muted until first OM response (~5 s after Init Defaults).

AS3935 chip one-line + title rename. The status chip (`✓ READY · NF=4 · UP 6H 50M · IRQ=24`) wrapped to two lines because the chip span allowed text wrapping and the header title (`AS3935 LOCAL LIGHTNING SENSOR`) ate too much of the residual flex width. Two small fixes:

- `white-space: nowrap` on `#as3935Evt` inline style — chip text now stays on one line.
- Header title text changed: `AS3935 Local Lightning Sensor` → `AS3935 Sensor` . The parent CSS `text-transform: uppercase` renders it `AS3935 SENSOR` , freeing ~16 chars of horizontal space for the chip. No CSS changes — the title change alone combined with the chip nowrap is enough at the current card width.

CLAUDE.md "Master Dashboard message types" list updated with the new `{type:'cape', cape, om_state}` payload shape.

Lightning dashboard — AS3935 status self-heal (chip rehydrates without manual republish)

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Modified:** `Replay AS3935 State` (`as3935_replay_state`) — outputs 1 → 2, function body augmented. **Added:** `AS3935 Cmd → bridge` (`as3935_cmd_mqtt_out`) — `mqtt out` to `lightning/as3935/cmd` . Node count 78 → 79.

Symptom. After `sudo systemctl restart nodered` , the AS3935 status chip (`✓ READY · NF=4 · TUN=9`) stayed blank until the operator manually triggered `republish_status` from the AS3935 Control

Panel. Heartbeats kept arriving every 30 s (chip handler is hb-aware) but the `as3935_hb` handler is a no-op unless `window._as3935Ready` is set, which only happens after the dashboard receives an `{type:'as3935_ready', ...}` message.

Diagnosis. Three facts narrowed it down:

1. `mosquitto_sub -h 192.168.1.169 -t 'lightning/as3935/status' -C 1 -W 3` from the Pi returned the retained payload instantly — broker has it correctly retained.
2. The MQTT broker config (`f4785be9863eab08`) is plain `cleansession: true` , no clientid, no birth/will — nothing that would suppress retained delivery.
3. A debug node wired to `as3935_mqtt_status` proved that when the bridge does publish (republish_status, or boot), Node-RED receives and processes the message cleanly end-to-end.

The retained-on-subscribe delivery to Node-RED's MQTT client is unreliable across restarts (likely a broker-connect ↔ subscribe ↔ retained-flush timing race that `mosquitto_sub` doesn't hit because it's a single fresh client). Rather than chase the exact race, the self-heal sidesteps it: ask the bridge to republish whenever the cache is empty.

Implementation.

- `Replay AS3935 State` now has 2 outputs:
    - **Output 1** → Master Dashboard (`557083037f168b22`) — unchanged: re-emits cached status + hb every 30 s (the existing page-refresh-survival behaviour).
    - **Output 2** → new `AS3935 Cmd → bridge` mqtt-out (`lightning/as3935/cmd`). When `flow.as3935_status` is null, sends `{action:'republish_status'}` to the bridge. Bridge replies on `/status` , Format AS3935 State caches it, next tick (within 30 s) populates the dashboard.
- **Cooldown:** 5 min, tracked in node-local `context.lastRepublishReq` . Prevents flooding the bridge if the cmd path itself is broken (e.g. bridge offline). Resets on Deploy — fine, by design.
- **Node status:** yellow ring "status null → republish req" when firing; grey ring "status null · cooldown Ns" when waiting; cleared once the cache populates.

Effect. Within 30 s of any Node-RED restart (or any other event that empties the cache), the dashboard chip rehydrates without manual intervention. Same self-heal trips for broker restart, ESP32 reconnect, or any other timing hiccup that causes the retained delivery to be missed.

Bridge firmware (TODO #15-adjacent, not done here). The "right" fix in addition would be a retained `lightning/as3935/last_event` topic from the ESP32 — that solves both the chip rehydration AND the "Last seen" rehydration in TODO #15. The self-heal added in this commit is the dashboard-side complement; it works regardless of the firmware change.

Cleanup for operator. If you added a temporary debug node wired to `as3935_mqtt_status` for diagnostics during this session, delete it (it's not part of the committed flow).

AS3935 Tuning — Control Panel rehydrates within 5 s of opening the page

Tab: AS3935 Tuning (`fe70cfdcdfa19aa4`) **Added:** 5 nodes. **Rewired:** 3 mqtt-in destinations.

Problem. Opening the AS3935 Tuning dashboard page in a fresh browser tab showed empty `/status` data (`FW —` , `IP ?` , `Calib: TRCO=? · SRCO=?`) until the operator manually clicked `Republish Status` . The Control Panel's three mqtt-in nodes (`/status` , `/hb` , `/cmd_ack`) wired directly to the ui_template; `resendOnRefresh` only stores the *last* message, which is always an `/hb` (publishes every 30 s), so refreshes never replayed `/status` (publishes only on bridge boot / settings change / `republish_status`).

Why this is the 2nd attempt — and why a 5 s tick, not `ui_control`. First attempt (commit `6664286`, reverted in `5e0f467`) used `node-red-dashboard`'s `ui_control` node, which emits a message on client connect / tab change. Would have given <100 ms rehydration. Failed because `ui_control` is **not shipped in this `node-red-dashboard 3.6.6` install** — confirmed by `npm install node-red-dashboard@3.6.6 --force`, the `nodes/ui_control.html` and `nodes/ui_control.js` files are genuinely absent from the package. Whether that's a 3.6.6 packaging regression or specific to this install isn't worth chasing; the fast-tick substitute works and stays simple.

Fix — cache + 5 s replay tick.

- Three pass-through cache functions inserted between the mqtt-ins and the Control Panel:
    - `as3935_tuning_cache_status` → `flow.as3935_status`
    - `as3935_tuning_cache_hb` → `flow.as3935_hb`
    - `as3935_tuning_cache_ack` → `flow.as3935_cmd_ack` Each is two lines: `flow.set(key, msg.payload); return msg;`. Live MQTT traffic still reaches the Control Panel unchanged.
- `as3935_tuning_replay_tick` inject, `repeat: 5`, `onceDelay: 1`.
- `as3935_tuning_replay_fn` reads the three caches and re-emits whatever's populated to the Control Panel with original topics preserved (`lightning/as3935/status`, `/hb`, `/cmd/ack`). The Control Panel's existing `scope.$watch('msg', ...)` consumes them with no template change. Node status reports `replay tick · N msg(s)` for visibility.

Effect. Opening the AS3935 Tuning dashboard tab cold (browser restart, fresh tab, new client) — Control Panel fully populates within ≤5 s. No `Republish Status` click. No browser hard-refresh.

Background load. 5 s tick = ~17 k internal Node-RED messages/day per cache populated, each a tiny JSON object. Trivial; no measurable CPU on the Pi.

Audit follow-up (TODO #16 in CLAUDE.md, renumbered from the reverted attempt). Same pattern applies to any other widget that has the same "infrequent publishes + multiple message types" shape. Most likely candidates are tabs with status readouts and config displays (e.g. RPi Fleet Monitor, FlexRadio panel — if those have similar gaps, treat them the same way). High-frequency widgets (LP-700 SWR, FlexRadio meters polled live) probably need nothing — their data flows constantly.

Side note. `node-red-contrib-mdashboard 2.19.4-beta` is also loaded on the Pi (operator is evaluating it). Mdashboard's nodes use `mui_*` prefix — different namespace from Dashboard 1's `ui_*` — and the existing flow uses zero `mui_*` types, so the two coexist without conflict. Mdashboard does **not** cause the `ui_control` gap; that's a Dashboard 1 packaging issue independent of mdashboard's presence.

Lightning tab — Stats refresh tick: 30 s → 10 s

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** Stats refresh inject (`77ec1b216f0c061b`).

Renamed `Stats refresh every 30s` → `Stats refresh every 10s` and `repeat: 30` → `repeat: 10`. One inject config change.

This inject fans out to 5 dashboard-replay functions: `Replay Bypass State`, `Stats → Dashboard`, `Sync Switch State`, `Log → Dashboard`, and `Replay AS3935 State`. All five now refresh 3× faster, so the Lightning chip, bypass state, stats counts, antenna/radio switch indicators, and Event Log all rehydrate within ≤10 s of opening or refreshing the dashboard (instead of ≤30 s).

Background load: each replay reads flow context and emits 1–2 small JSON messages to a ui_template. Three of those (bypass, switch, log) are no-ops most of the time (only emit if there's something to replay). `Replay AS3935 State` self-heal (5-min cooldown) is unaffected — fires no more frequently than before.

Same instant-on philosophy as the AS3935 Tuning fix earlier today, just applied to widgets that already had a replay tick and just needed it sped up. Worth a separate audit pass for tabs without any replay infrastructure yet — TODO #16.

AS3935 card: chip colour + raw distance/energy on disturber & noise

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** Master Dashboard ui_template (`557083037f168b22`).

Two small follow-ups to the AS3935 card behaviour:

1. **Chip text colour now matches event type.** Previously the `as3935_status` disturber/noise branches set `evtEl.textContent` but didn't touch `evtEl.style.color`, so the chip text inherited whatever colour the last `✓ READY` paint left it (green) — even when displaying `⚠ Disturber` / `📡 Noise`. Now:
    - `⚠ Disturber` → `var(--amber)` (matches the LED colour for the same event)
    - `📡 Noise` → `var(--muted)` (matches its LED colour) The `✓ READY` / `⚠ CALIB?` paint logic was already correct and is unchanged.
2. **Raw distance/energy shown on disturber/noise too.** Previously forced to `—` because the AS3935 chip's distance value for non-lightning events isn't physically meaningful. But the raw value (which can be `63` = "out of range", or any other value the chip's detection algorithm outputs) is useful for tuning (`AS3935 Control Panel` calibration work) and for understanding why a particular event was classified the way it was. Now both branches show `d.distance` / `d.energy` if present, falling back to `—` only when the bridge omits the field.

Strike branch (real lightning) was already correct on both counts — left unchanged.

AS3935 LAST SEEN: dashboard side of TODO #15 (firmware to follow)

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Added:** 1 mqtt-in node. **Augmented:** Format AS3935 State, Replay AS3935 State, Master Dashboard ui_template.

Wires up the Node-RED side of the "LAST SEEN survives restart" plan from TODO #15. The bridge firmware change in `vu2cpl-as3935-bridge` will publish a retained `lightning/as3935/last_event` topic with `{event, distance, energy, ts_epoch_ms}` on every disturber / noise / lightning event. This commit prepares the dashboard side; until firmware ships, the new mqtt-in is silent.

Changes:

- **New** `as3935_last_event_mqtt_in` — mqtt in on `lightning/as3935/last_event`, QoS 1, JSON, same broker as the rest. Wires into the existing `as3935_format_state` function.
- **`as3935_format_state`** augmented with a third `else if` branch: matches topics ending in `/last_event`, caches to `flow.as3935_last_event`, emits `{type:'as3935_last_event', event, distance, energy, ts_epoch_ms}` to Master Dashboard. Node status reports `last_event: <event>`.
- **`as3935_replay_state`** augmented to also replay the cached `last_event` on every 10 s Stats refresh tick. Lives alongside the existing status + hb replay and self-heal cmd logic from commits `83f4b20` and earlier.

- **Master Dashboard ui_template** — new handler for `d.type === 'as3935_last_event'`. Seeds `as3935LastTs = d.ts_epoch_ms` and calls `paintAS3935Age()` + starts the 30 s ticker. Inserted before the existing `as3935_status` handler. Handler is no-op when `d.ts_epoch_ms` is missing.

Effect (once firmware ships):

- Node-RED restart → broker replays retained last_event immediately to the new mqtt-in → `as3935_format_state` caches + emits → Master Dashboard seeds `as3935LastTs` and paints `LAST SEEN` with correct relative age within ~100 ms of subscribe.
- Browser refresh / new tab → either the retained payload is replayed (resendOnRefresh on dashboard widget) OR the 10 s replay tick re-emits within ≤10 s.
- Combined: `LAST SEEN` is correct everywhere, always, no need to wait for a live disturber/noise/strike.

Until firmware ships. The new mqtt-in is a quiet subscriber on a topic nobody publishes to. `as3935_format_state` never enters the new branch. Behaviour unchanged from this commit's predecessor.

Master Dashboard message types updated in CLAUDE.md with the new `{type:'as3935_last_event', event, distance, energy, ts_epoch_ms}` shape.

2026-05-15

Lightning — Telegram alerts (Standard scope: disconnect, reconnect, bypass, sensor health)

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Added:** 5 nodes. **Modified:** Init Defaults, Append Lightning JSONL, Bypass Handler wires, AS3935 Status mqtt-in wires. Node count 80 → 85.

Sends Telegram messages on the 6 most operator-relevant lightning events:

Type	Trigger	Message
`disconnect`	`Trigger Disconnect` fires, antenna goes OFF	⚡ ANTENNA DISCONNECT + source + km + time
`reconnect`	`Execute Reconnect` fires, antenna back ON	✅ ANTENNA RECONNECT + reason + time
`bypass_on`	Operator clicks BYPASS on dashboard, mode enters ON	🔕 BYPASS ON + auto-off timer + time
`bypass_off`	Operator clicks BYPASS again or 2-h timer expires	🔔 BYPASS OFF + time
`sensor_offline`	AS3935 ESP32 bridge LWT fires (`event:"offline"` retained on `/status`)	⚠ AS3935 OFFLINE + time
`sensor_online`	Bridge re-publishes `/status` with `event:"ready"` after offline	✓ AS3935 ONLINE + time

Excluded (would spam): disturber + noise events, Open-Meteo polls, heartbeats, TEST injects (filtered on `source` containing `'TEST'`).

Credentials. Reads `TELEGRAM_TOKEN` + `TELEGRAM_CHAT_ID` from the existing systemd env (`/etc/systemd/system/nodered.service.d/secrets.conf` — same source DXCC Credentials uses). Init Defaults reads and sets `flow.cfg_tg_token` + `flow.cfg_tg_chat_id` . No new secret storage, no duplication.

Architecture.

- `Append Lightning JSONL` now has 1 output instead of 0: still appends to `nr_lightning_events.jsonl` first, then forwards the msg downstream to `tg_lightning_router` . JSONL store and Telegram see the same `event_record` s.
- `tg_lightning_router` filters by allow-list, rate-limits per type (5 events in 60 s → one suppression notice → silence until activity slows), formats HTML message with shack callsign + event-specific fields, sets `msg.url` / `msg.method` / `msg.headers` / `msg.payload` for the http-request node. Status surfaces what was last sent.
- `tg_lightning_http` is a stock `http request` node — URL blank (per the Telegram HTTP request convention in CLAUDE.md), POST, JSON body, 8 s timeout. Wires to `tg_lightning_debug` for response visibility in the editor sidebar.

Transition detectors added to surface events that don't currently emit `event_record` :

- `bypass_xition_detector` — tapped off Bypass Handler output 1 (which fans out `bypass_state` + `log` messages to Master Dashboard). Filters on `bypass_state` , detects ON↔OFF via `context.lastBypassOn` , emits `event_record { type: 'bypass_on' | 'bypass_off', expires_min, source: 'operator', html }` . First sample after flow start suppressed (can't distinguish "transition" from "initial state").
- `as3935_health_xition` — tapped off AS3935 Status (retained) mqtt-in (parallel wire alongside `as3935_format_state`). Watches the `event` field (`ready` vs `offline`), detects transitions via `context.lastOffline` , emits `event_record { type: 'sensor_offline' | 'sensor_online', source: 'AS3935', html }` . First sample suppressed. Note: the retained payload always replays on subscribe, so the suppression is essential — otherwise every Node-RED restart would falsely "transition" to whatever the current state is.

Both transition detectors wire to `light_jsonl_append_01` , which both persists to JSONL and forwards to the Telegram Router. So bypass + sensor-health events now also live in the historic store — useful for retrospective analysis.

Rate-limit semantics. Per `rec.type` , a sliding 60 s window allows up to 5 events. The 6th arrival within the window emits one suppression notice (`"5 disconnect events in 60s — further duplicates suppressed until activity slows"`) and then stays quiet. Once the window drains below 5, the counter resets and normal sending resumes. Prevents storm-day noise; the operator still gets the first 5 disconnects of a cluster and a clear marker that more are happening.

Sister-channel cross-ref. DXCC Tracker has its own Telegram path for NEW DXCC alerts; same bot, same chat, same shack signature prefix. Both channels coexist — no plumbing collision because they use separate routers + separate http-request nodes.

SPE (WS) — ON_SPE routes through WebSocket, exec node removed

Tab: SPE (WS) (`spe_ws_tab_01`) **Modified:** `Button → WS command` (`ws_btn_router`). **Removed:** `SPE power-on script` exec node (`ws_spe_power_on_exec`).

The dashboard's `ON_SPE` button was the only one taking a different code path: a Node-RED `exec` node ran `python3 /home/vu2cpl/power_spe_on.py` on the Pi, which toggled the FTDI's DTR/RTS to cold-start the amp. Every other button (MODE, TUNE, INPUT, ..., OFF_SPE) sent a command string over the WebSocket to the `spe-remote` server.

Reviewing the `spe-remote` source revealed that the WebSocket server **already handles `power_on` internally** via the same DTR/RTS toggle — `spe/power_control.py` `_power_on_sync()` does exactly the sequence `DTR=1 → DTR=0 → RTS=1 → wait 1 s → DTR=1 → RTS=0`, the same one in `power_spe_on.py`. The amp's CPU being off doesn't matter because the FTDI hardware lines are controlled at the host end via ioctl, independent of whatever's on the other end of the serial cable.

The old `ws_btn_router` comment justifying the exec detour was wrong: it conflated the *serial data link* (which is dead when the amp is off — no CPU on the other end) with the *FTDI hardware lines* (which remain controllable regardless). Cleaned up.

Changes:

- `ws_btn_router` simplified: `outputs: 2 → 1`, special-case `if (msg.payload === 'ON_SPE') return [null, msg]` removed, `ON_SPE: 'power_on'` added as a regular entry in the command map. Function now mirrors every other button.
- `ws_spe_power_on_exec` exec node deleted entirely.
- `ws_pwr_result_dbg` debug node **kept** — `ws_parse_node` still routes `power_result` responses (the structured ack from `spe-remote` on `power_on` / `power_off`) to it, which is actually *more* useful than the script's stdout was. Provides visible confirmation in the editor sidebar that the DTR sequence ran.

`power_spe_on.py` stays on the Pi as a standalone fallback if the `spe-remote.service` is down (Pi-side cron / manual script could still hit the FTDI directly). Just not in the dashboard's runtime path anymore.

Sister-repo handover: see `spe-remote/handover.md` for the canonical documentation of this consolidation from the server side — clarifies that clients should use `power_on` over WS, not spawn the standalone script.

Dashboard rehydration audit (TODO #16 closed)

Tabs touched: Solar (`590e889d44815afb`), RBN Skimmer (`f9a0e3ad0e019052`), RPi Fleet (`d5fec2fea3dd37f4`). **Modified:** 3 function nodes (the state aggregators on each tab).

The audit walked all 10 remaining dashboard tabs after yesterday's AS3935 Tuning + Lightning chip fixes. Encouraging finding: 11 of 12 widgets already have `storeOutMessages: true` + `resendOnRefresh: true` AND are fed by a single "State Aggregator" function pattern that emits the **full state object** on every update (not partial diffs). On browser refresh, Node-RED dashboard replays the last full-state message → widget paints completely. The pattern just works without any of the cache + replay-tick scaffolding we built for AS3935 Tuning.

(The AS3935 Tuning Control Panel was the exception precisely because its widget dispatched on `msg.topic` with three separate input topics — `resendOnRefresh` could only store one. Aggregator widgets bypass that limitation.)

Real gaps found — 3 aggregators on slow-poll sources that lose state on Node-RED restart because their `flow.set` was on memory context:

- `Solar State Aggregator` — 5–15 min HTTP polls (NOAA, Open-Meteo, prop.kc2g.com)
- `RBN State Aggregator` — RBN spots, event-driven and bursty
- `RPi State Aggregator` — 60 s MQTT telemetry from each Pi

If you restart Node-RED between polls, the dashboard sits half-populated until the next poll cycle. For Solar that's up to 15 min of stale display.

Fix: swap both the `flow.get` and the `flow.set` to use `'file'` scope. State persists to disk (via the `localfilesystem` context store enabled by `enable_file_context.sh` per TODO #6), survives Node-RED restart, and the dashboard paints from the last-saved state within seconds.

```
// Before:
var st = flow.get('solarState') || {};
...
flow.set('solarState', st);

// After:
var st = flow.get('solarState', 'file') || {};
...
flow.set('solarState', st, 'file');
```

Each aggregator's state object is plain JSON (no circular refs, no non-serializable types), so the file-store serialisation Just Works. Write frequency is low — Solar writes ≤ once per 5 min, RBN on each spot batch, RPi every 60 s. No perf concern.

Verified no key conflicts. Each of `solarState`, `rbn_dash`, `rpi_dash` is touched ONLY by its respective aggregator (no other node reads or writes them), so the scope swap is self-contained and can't desync with another reader.

Power Strips checked but not modified. The `Poll Tasmota state every 10s` inject re-queries every device every 10 s, so `flow.power_states` and `flow.energy_state` stay fresh continuously regardless of restart. Worst-case rehydration ≤10 s, meets the bar without changes.

Tabs already fine (no changes): Rotor, FlexRadio, LP-700 (WS), SPE (WS), DXCC Tracker (bootstraps from disk), GPS NTP (retained MQTT), Internet/network (30 s pings), Power Strips (10 s poll).

FlexRadio — split-mode slice coloring (TODO #2 closed)

Tab: FlexRadio (`a0a882f85c89cffc`) **Nodes:** `Flex State Aggregator` (`de6b988cbc7182ca`), `FlexRadio Panel` (`bf129ed26ea2ca5f`).

Problem. In FlexRadio split mode, both the RX and TX slices report `tx==1` (because both are "transmit-capable"). The discriminator is the `active` field: the RX slice (currently focused for listening) has `active==1` and the TX slice (where the mic + key go) has `active==0`. The dashboard's previous condition `tx==1 && active==1` therefore picked the RX as the "TX slice" — painted colours backwards on every split-mode operation.

Fix. Pre-compute a per-slice `isTx` boolean in the aggregator, where we have visibility across all slices, then have the dashboard consume just that boolean.

`Flex State Aggregator`: new block before the existing `activeSlices` loop counts `tx==1 && in_use==1` slices, then sets `sl.isTx` on each:

- `tx != 1` → `isTx = 0`
- multiple `tx==1` → `isTx = 1` only if `active == 0` (split mode)
- single `tx==1` → `isTx = 1` (normal mode)

`isTx` is propagated into each `activeSlices` push so the dashboard's `ng-repeat` sees it.

`FlexRadio Panel` : two `ng-class` expressions updated:

- `activeSlices` loop: `s.tx==1 && s.active==1` → `s.isTx==1`
- `slices[s]` grid (the `['A','B','C','D']` loop, appearing twice in the same ng-class ladder for "transmitting" + "active" states): `msg.payload.slices[s].tx==1 && msg.payload.slices[s].active==1` → `msg.payload.slices[s].isTx==1`

Single source of truth. The aggregator owns the "which slice is actually transmitting" decision. If FlexRadio changes split-mode semantics in the future, only the aggregator needs touching — the dashboard just reads the boolean.

Verified by inspection of the aggregator + panel logic. Live verification: next time you split-tune (e.g. listen on the DX station's frequency, transmit on the split — common during pile-ups), the RX-side slice should be the green one and the TX-side slice should be the yellow/amber one. Previously the colours would have been swapped.

2026-05-17

AS3935 SENSOR card — Distance / Energy / Event tiles persist across refresh

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** Master Dashboard ui_template (`557083037f168b22`).

The AS3935 SENSOR card's `DISTANCE KM` / `ENERGY` / `EVENT` tiles were only updated by live event handlers (`as3935_status` for disturber/noise, `strike` for lightning). On Node-RED restart or browser refresh, they reverted to `—` / `⏳` and stayed blank until the chip fired the next event — which could be hours.

The bridge's retained `lightning/as3935/last_event` topic already carries `{event, distance, energy, ts_epoch_ms}` (it's used to rehydrate `LAST SEEN`). The fix: have the same handler that seeds `LAST SEEN` also populate Distance / Energy / Event icon from the same payload.

Change: extended the `as3935_last_event` handler in the Master Dashboard ui_template. After setting `as3935LastTs` , the handler now reads `d.event` / `d.distance` / `d.energy` and updates the three DOM elements:

- `iconEl` : ⚡ (lightning) / ⚠ (disturber) / 📡 (noise) / ⏳ (default)
- `distEl` : `d.distance` if present, else `—` ; colour scoped by event type (blue / amber / muted)
- `engEl` : `d.energy` if present, else `—`

Effect:

- Browser refresh / fresh tab / Node-RED restart → broker replays retained `last_event` within ~100 ms → mqtt-in → `Format AS3935 State` → handler runs → tiles populate from the most recent event the bridge ever published. Stay populated until a live event arrives.
- Live event handlers (`as3935_status` , `strike`) unchanged — they still overwrite the tiles on fresh events.
- 5 s replay tick on the AS3935 Tuning tab also re-emits `as3935_last_event` via `as3935_evt_replay_fn` → reinforces persistence.

Unchanged: chip header (`a35Evt`) and LED colour. Those reflect *current sensor health* (driven by `/status` + `/hb`), not last event. Showing stale last-event text in the chip after a refresh would be misleading — current health is the right semantic there.

Telegram alerts — stop misleading "ANTENNA DISCONNECT" for near-miss strikes + show actual state

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes modified:** `AS3935 Disconnect Log` (`611667d11de744ed`), `Telegram Alert Router` (`tg_lightning_router`).

Problem reported: operator received a Telegram alert reading "⚡ VU2CPL: ANTENNA DISCONNECT · Source: AS3935 · Distance: 31 km" — but the antenna was NOT actually disconnected. The distance-graded matrix (medium-zone 10–25 km, OM=cold, no corroboration in 5-min window) decided not to fire the disconnect. Two issues compounded:

1. **`AS3935 Disconnect Log` was claiming `type:'disconnect'` for every within-threshold strike** — it runs in parallel with `Trigger Disconnect` off the same switch output, so it doesn't know whether the matrix downstream will actually fire. Logging a `disconnect` event_record before the matrix has run = false-positive Telegram alerts.
2. **The Telegram message text never mentioned the actual antenna state** — operator couldn't tell from the message whether the antenna was really off or not.

Fixes:

1. **`AS3935 Disconnect Log` is now observational.** Event_record `type` renamed from `'disconnect'` to `'as3935_observed'` (non-bypass case). `'as3935_observed'` is not in the Telegram allow-list → no false alerts. The dashboard log line is also updated: `'DISCONNECT triggered'` (ev-err / red) → `'within disconnect zone (matrix runs)'` (ev-warn / amber). Reflects what this node actually knows — the chip saw a strike within range, downstream matrix decides outcome. Bypass branch unchanged (already correctly labelled `'disconnect_suppressed'`).
2. **`Telegram Alert Router` now reads actual state at send time.** New `stateLine()` helper inside the router reads `flow.antenna_off` / `flow.radio_off` / `flow.radio_enabled` and emits a line like `Antenna: OFF\nRadio: OFF` (radio only included when enabled). Injected into both `disconnect` and `reconnect` message templates. By the time the event_record reaches the router, `Trigger Disconnect` / `Execute Reconnect` / `Send Radio Command` have already updated the flow flags, so the state read is post-action.

Single source of truth for actual disconnects: `Format Log` (source= `'system'` , type= `'disconnect'`), which is fed only by `Trigger Disconnect` 's output. Real disconnects → Format Log → event_record → JSONL → Telegram fires (correctly). Within-threshold observations → AS3935 Disconnect Log → event_record `'as3935_observed'` → JSONL (still persisted for analysis) → Telegram skips (not in allow-list).

Telegram message shape now:

> ⚡ ***VU2CPL: ANTENNA DISCONNECT*** *Source: system Distance: 5 km Antenna:* ***OFF*** *Radio:* ***ON*** *(only shown when* `radio_enabled=true` *) Time: 02:18:42*

...where `Antenna:` line reflects `flow.antenna_off` reality, not just the event's intent.

AS3935 dashboard — merge IRQ counters + session counters into one row

After the Control + Events merge (one ui_template, one card), there were still **two counter rows visible**: the compact text counter near the top (driven by bridge `/hb` `counters: {lightning, disturber, noise, irq}` — lifetime counts since ESP32 boot) and the verbose-styled session counters near the bottom (JS-incremented per browser tab, reset on refresh).

Operator wanted one row in the session-counter visual style.

Merge details:

- CSS: `#a35ev` prefix stripped from the 6 countstrip-related selectors (`.countstrip` , `.cnt` , `.cnt .v` , `.cnt.lightning .v` , `.cnt.disturber .v` , `.cnt.noise .v`) so they apply in either parent container.
- Top HTML: the placeholder `<div class="card counters" id="a35counters">awaiting heartbeat...</div>` replaced with a full `.countstrip` row containing **four** `.cnt` spans — lightning, disturber, noise, IRQ. Same word-label + coloured-value style as the deleted session counter row.
- JS: the `cn.textContent = '⚡ … ⚠ … 📡 … IRQ …'` string-build replaced with four `setSpanContent` calls that write `c.lightning` , `c.disturber` , `c.noise` , `c.irq` into their individual `.v` spans.
- Events section: the `Session counters (since this browser opened)` `.card` HTML block removed entirely; the JS that backed it (`var counts = {…}` declaration, the `counts[p.event]++` increment inside the event handler, the `renderCounts()` function, and its two call sites — `renderLast(); renderCounts(); renderLog();` → `renderLast(); renderLog();`) all removed.

Effect: one counter row at the top of the AS3935 card, driven by the bridge's lifetime counters (more authoritative than browser session counts — survives browser refresh, matches the IRQ count which the chip reports independently), styled exactly like the deleted session counter row.

Source of truth: bridge `/hb` `counters` object. Updated every 30 s.

AS3935 dashboard — merge Control + Events panels into one ui_template

Tab: AS3935 Tuning (`fe70cfdcdfa19aa4`) **Commits:** `abdd424` (re-parent into shared ui_group — visually didn't fuse), `43da30e` (height tweak), and today's merge.

`node-red-dashboard` wraps every `ui_template` in its own card frame regardless of `group` membership — so just dropping both into the same `as3935_ctl_grp` still rendered them as two visually-separate cards with a gap. Operator wanted one continuous card. The only structural fix: **one widget = one card**, so the two `ui_template` `format` strings got merged into one.

Merge details:

- `<style>` blocks concatenated. Class-name collisions audited beforehand: only `.card` (identical between the two) and `.sect` (differs by 4 px vs 6 px `margin-bottom` — imperceptible). The Control Panel's `:root{}` block defines the `--card` , `--border` , etc. custom properties; the Events Panel's CSS references them via `var(--…)` and the cascade keeps them in scope after merge.
- HTML body: Control Panel's `<div id="a35tune">…</div>` first, then the Events Panel's `<div id="a35ev">…</div>` second — each retains its own root container so the JS handlers (which use `document.getElementById` to find their elements) keep working unchanged.
- `<script>` block: two IIFEs in series, each registers its own `scope.$watch('msg', …)` . Both fire on every message; each filters by `msg.topic` (topics are disjoint between the two panels — `/status` , `/hb` , `/cmd/ack` for the Control side; `lightning/as3935` , `/last_event` for the Events side).
- 3 nodes that wired to `as3935_evt_panel` (the now-deleted Events ui_template) were rewired to `223cb2ce733c5d3f` (the surviving Control Panel, which now owns the merged content). Deleted: `as3935_evt_panel` ui_template.

- Control Panel height bumped 12 → 22 to accommodate the combined content. Tunable later if needed.

Effect: AS3935 Tuning dashboard tab now shows **one continuous card** containing (top-to-bottom): AS3935 Bridge header, ⚡ Δ📡 IRQ counters, Calib, 🔋 Battery, Tunables, Actions row, ack box, Last Event card, Session Counters, Recent Events log. No inter-card gap. Same data, same MQTT inputs, same buttons.

Why not CSS-trick the inter-widget gap: considered but rejected. `md-card { margin: 0 !important; box-shadow: none !important }` would fight `node-red-dashboard`'s baseline styling globally and risk affecting unrelated widgets on a dashboard version bump. Merging templates is the structural fix: one widget, one card, by design.

AS3935 dashboard — import v0.3.0 (battery telemetry + Events panel + TEST injects)

Tab: AS3935 Tuning (`fe70cfdcdfa19aa4`) **Source:** `vu2cpl-as3935-bridge/nodered/as3935-control-flow.json` (built from the bridge repo's `nodered/build-flow.py`, commit `8084ad9`). **Added:** 13 nodes. **Updated:** Control Panel format, `as3935_tuning_replay_tick` wires.

Two pieces from the bridge's v0.3.0 build artifact ported into the shack's actual flows.json:

1. **`AS3935 Control Panel` (`223cb2ce733c5d3f`) format updated** — picks up the new 🔋 battery row (mV reading + derived %SOC from a piecewise-linear LUT, green ≥ 3.90 V / amber 3.70–3.90 V / red < 3.70 V, plus `(divider not wired?)` hint when reading < 500 mV), the new **Query Battery** action button, and the `vbat_offset_mv` tunable (±500 mV per-chip Vref trim). Existing node ID + position
    - wires preserved; only `format` field swapped.
2. **New `AS3935 Events` panel** alongside the Control Panel on the same dashboard tab (`c55b930b17a24bb1` , AS3935 Tuning). Shows a 30-row rolling event log (from `lightning/as3935`) plus session counters (lightning / disturber / noise) plus a "Last Event" card backed by retained `lightning/as3935/last_event` (rehydrates on Node-RED restart within ~5 s). 5 TEST inject buttons publish synthetic events directly to `lightning/as3935` so the panel can be exercised end-to-end without involving the ESP32 — useful for styling tweaks, debouncing, etc.

Dedupe handled by the importer. Bridge build artifact also contains the cache + replay infrastructure on the Tuning side (`as3935_tuning_cache_status/hb/ack` + `as3935_tuning_replay_tick/fn`) — these have identical IDs and contents to what's been in the shack since 2026-05-14. Importer skipped them. Same for the Tuning-side mqtt-in/out nodes which the shack has with its own pre-existing IDs.

Tick fan-out extended: `as3935_tuning_replay_tick` now wires to both `as3935_tuning_replay_fn` (existing, Control Panel) AND the new `as3935_evt_replay_fn` (Events panel). One 5 s tick keeps both panels populated.

Operator action remaining (not in this commit): the bridge's ESP32 firmware needs the v0.3.0 binary flashed for `vbat_mv` to actually arrive. **Done by the operator on 2026-05-17** before this import. Battery row should populate within 30 s of dashboard load (first `/hb` after restart) or immediately on Query Battery click.

Lightning dashboard — hide RADIO tile when `radio_enabled` is false

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** Master Dashboard ui_template (`557083037f168b22`) **Commit:** `df8ea41`

After yesterday's `RADIO_ENABLED = false` fix, the auto-disconnect logic correctly stops touching the radio plug — but the Master Dashboard's top switchBox still showed the **RADIO ON/OFF tile + its RECONNECT button**. Misleading: the visible tile suggested the radio was still being managed by lightning logic when it wasn't.

Fix. Two surgical edits to the dashboard ui_template:

1. HTML: the radio `<div class="sw-row">` (containing `#radioStatus` + the RECONNECT button) gets `id="radioRow"` .
2. JS: in `setStats(d)` (called every 10 s from the Stats refresh tick), prepend a display toggle:

```
var rr = document.getElementById('radioRow');
if (rr) rr.style.display = d.radio_enabled ? '' : 'none';
```

`d.radio_enabled` is already populated by `Stats → Dashboard` (`d1dca3df391cdfb8`), which reads `flow.get('radio_enabled')` set by Init Defaults. No new wiring, no new nodes — just an HTML id + 2 lines of JS.

Net effect when `RADIO_ENABLED = false` :

- Top switchBox: only ANTENNA OFF/ON tile visible. RADIO tile + its RECONNECT button hidden. Bypass button stays — flex layout collapses the gap.
- Stats panel "Flex Radio" row was already gated by `if (d.radio_enabled) rows.push(...)` — also hidden. No change needed there.

Net effect when `RADIO_ENABLED = true` : unchanged from before.

Worst-case rehydration after a Deploy or browser refresh: up to 10 s (the Stats refresh tick interval). The row may briefly flash visible at page load before the first tick runs. Acceptable; if it ever becomes annoying, the toggle can be moved into the static inline `<style>` block driven by a body class set on first message arrival.

`setRadio()` still runs on MQTT events even when the row is hidden — it updates the DOM element's class + text in the background. So flipping `radio_enabled = true` back at runtime shows the row immediately with current Tasmota state on the next 10 s tick — no stale flash.

VU2CPL skimmer — login handlers send `VU2CPL-1` on port 7300

Tabs: RBN Skimmer Monitor (`f9a0e3ad0e019052`), DXCC Tracker (`d110d176c0aad308`) **Nodes:** `Telnet VU2CPL :7300` tcp-in (`df7d1786eab4d5a2`), `Login Handler VU2CPL` (`fa367f22588d17bc`), new tcp-out `rbn_vu2cpl_tcp_reply` , `Login + Parse + Dedup` (`login-parse-dedup-v2`).

Trigger. VU2CPL skimmer's TCP port moved from **7550** (CwSkimmer's local "raw" telnet, no auth, streams spots on connect) → **7300** (the proper telnet cluster port with `Please enter your callsign:` prompt + login). Connection still TCP-succeeded so both tcp-in nodes showed green "connected", but no spots flowed — the cluster was silently waiting for a login that Node-RED never sent. Symptom: RBN Skimmer Panel showed 0 spots; DXCC Tracker had stopped getting VU2CPL spots (other clusters unaffected).

Diagnosis. `nc -v vu2cpl.ddns.net 7300` confirmed the cluster is fully operational — banner + prompt + spots after manual `vu2cpl-1` login. So the upstream is fine; the issue is each tab's login-handling pipeline.

Fix 1 — DXCC Tracker (`login-parse-dedup-v2`): the existing login code already had prompt-detection wired into a `tcp out` reply node (`2a20b140b97c35b0`), but a `lc.length < 40` safety guard (added to avoid false-matching N2WQ's long "Last login: ..." welcome banner) was rejecting CwSkimmer's prompt.

CwSkimmer sends the banner + prompt **concatenated as one TCP chunk** (~150 chars) because the tcp-in there uses `newline: ""` (raw chunks). Trimmed chunk ends in `…callsign:` so `endsWith('callsign:')` matched — but length > 40 → safety guard blocked the send. Fix: apply the length check to **just the trailing line of the chunk**, by splitting on `\r?\n` and taking the last segment. False-positive guard against `Last login: … from <ip>` still holds (that line ends with an IP, not `login:`).

Fix 2 — RBN Skimmer Monitor: three separate problems on this tab.

- **No login code existed at all.** The function ironically named "Login Handler VU2CPL" only filtered DX spot lines — never sent a callsign. Worked previously because port 7550 never asked for one.
- **No tcp-out reply node existed on this tab.** Added a new one (`rbn_vu2cpl_tcp_reply` , `beserver: "reply"`) wired from output 0 of the Login Handler.
- **Login Handler upgraded** to 2 outputs:
    - Output 0 → tcp-reply with `VU2CPL-1\r\n` and `_session` preserved (so the reply rides the same TCP socket).
    - Output 1 → existing `Parse DX Spot` (`7d2c70d8935ef86f`). Function now splits the incoming chunk on `\r?\n` , runs prompt-detection on the trailing line, and emits one msg per `DX de` line so the downstream parser still sees clean single-line input.
- **`tcp-in` `newline` setting flipped from `"\n"` to `""` (raw chunks).** This was the silent killer — CwSkimmer's prompt has **no trailing newline** (it's waiting for input), so a tcp-in configured to split on `\n` buffers it forever. Login Handler never saw a message; login never fired; no spots. Switching to raw chunks lets the prompt arrive immediately, at the cost of the Login Handler having to split chunks into lines internally (which it now does). The DXCC tab's tcp-in was already configured `newline: ""` — that's why DXCC was rescuable with just Fix 1.

Login SSID = `-1` on both tabs. Two parallel sessions to the same CwSkimmer use the same callsign. `nc` testing confirms CwSkimmer's telnet port doesn't enforce per-callsign uniqueness — both sessions accepted, both stream spots. The DXCC tab loads its SSID from `cfg_cl_login_ssid` (set to `-1` in the Credentials node). The RBN tab hardcodes `VU2CPL-1` inline (RBN has no credentials node).

Effect. Both DXCC Tracker's VU2CPL feed and the RBN Skimmer Monitor panel show live spots again, with the same Login Handler architecture mirrored on both tabs. Login-fire shows as a blue `… → login sent` flash on the node status, followed by green `… +N spots` ticks as spots flow (need `Show node status` ticked under hamburger → User Settings → View).

Future-proofing: if a fourth/fifth cluster gets added with a CwSkimmer-style un-terminated prompt, the lastLine-split + endsWith approach handles it without further changes. The prompt-fragment list inside both handlers covers `login:` , `please login` , `please login:` , `callsign:` , `callsign please:` , `your callsign:` , `enter your callsign:` — extend as needed.

AS3935 Bridge UI — Tune Cap 30 s countdown + Query Battery instant update

Tab: AS3935 Tuning (`fe70cfdcdfa19aa4`) **Node:** AS3935 Control Panel ui_template (`223cb2ce733c5d3f`).

Two operator-affordance polish fixes shipped together (one commit `4dd4bab`).

1. Tune Cap button — live 30 s countdown.

Previously the action button rendered as `Tune Cap (35 secs)` — a static hint telling the operator the calibration sweep takes ~35 s, with no feedback once pressed. Replaced with a live countdown:

- Default label: `Tune Cap` .
- On click: fires `a35.action('calibrate_tun_cap')` (unchanged), then disables the button and starts a `setInterval` ticker. Label updates each second: `Tune Cap (30s)` → `Tune Cap (29s)` → … → `Tune Cap (1s)` . At zero the interval is cleared, button re-enabled, label restored to `Tune Cap` .
- Re-clicks during the countdown are ignored (`button.disabled` guard) — protects against a fat-finger firing two simultaneous calibrate cycles.
- HTML: button gains `id="a35tunebtn"` so the JS can grab it. New `a35.calibrate()` method on the window object wraps the existing action call.

Visible timer is 30 s; firmware sweep is ~35 s. The 5 s gap is intentional — the cmd/ack badge below still shows the real `calibrate_tun_cap` result when the bridge finishes, so the operator knows the chip is actually done. Lengthening the visual timer to match the firmware would force the operator to wait pointlessly if the chip finishes early.

2. Query Battery — 🔋 row refreshes instantly (was up to 30 s lag).

The bridge firmware's `query_vbat` action publishes the fresh reading two ways: embedded in the ack's `cmd` string (`action:query_vbat vbat_mv=NNNN`) AND as a structured `vbat_mv` field in a republished `/status` . Both arrive within ~100 ms of the cmd.

Bug: the dashboard's render logic preferred `hb.vbat_mv` (from the 30 s heartbeat) over `state.vbat_mv` (from `/status`), to keep the display tracking the freshest cadence under normal operation. But this meant the just-arrived fresh `/status` value was shadowed by the still-cached heartbeat value from up to 29 s ago. Display only updated when the next `/hb` tick came in — earlier 2026-05-17 changelog entry that claimed *"immediately on Query Battery click"* was aspirational, not actually realised.

Fix: `cmd/ack` handler now regex-extracts `vbat_mv=(\d+)` from the ack's `cmd` string and assigns the parsed integer directly to `hb.vbat_mv` . The `render()` call that runs right after the watcher fires picks up the fresh value within the same tick (~50 ms total).

Touch-up only on the dashboard — no firmware change. Updated SHACK_CHANGELOG entry from 2026-05-17 (above) corrected indirectly: it's now correctly *immediate* on Query Battery click, finally.

DXCC Telegram alerts — decode HTML entities in country names

Tab: DXCC Tracker (`d110d176c0aad308`) **Nodes:** `Format Telegram Alert Dedup 10 minute` (`94b77826079bad57` , live) + `Format Telegram Alert` (`d5eca40d3503035d` , disabled — patched for consistency).

Telegram alert for V49B on 30M Data arrived rendering the entity name as `SAINT KITTS &amp; NEVIS` instead of `Saint Kitts & Nevis` .

Root cause: cty.xml stores ampersands as `&amp;` per XML spec (also `&lt;` , `&gt;` etc. for any other reserved characters). `seedNames` therefore carries the HTML-entity-encoded form, which is fine for the dashboard alert table (rendered via innerHTML, browser auto-decodes), but the Telegram message uses `parse_mode: 'Markdown'` — Markdown does NOT decode HTML entities, so the literal `&amp;` lands in the user's chat.

Fix: added a local `htmlDecode()` helper inside the Telegram formatter functions. Decodes `&amp;` , `&lt;` , `&gt;` , `&quot;` , `&#39;` , `&apos;` back to their literal characters just before composing the Markdown text. Applied to `a.entity` only — `a.call` / `a.spotter` aren't HTML-encoded in any realistic path. seedNames left untouched in memory so the dashboard HTML table behaviour is unchanged.

Other DXCC entities the same fix now covers (any ampersand-bearing country): Trinidad & Tobago, Antigua & Barbuda, Turks & Caicos Is., St. Vincent & the Grenadines, São Tomé & Príncipe, St. Pierre & Miquelon, St. Helena, Ascension & Tristan da Cunha, etc.

Doc note added to CLAUDE.md ("Entity names from cty.xml are HTML-entity-encoded…") so future maintainers adding a new Telegram-bound formatter mirror the `htmlDecode()` pattern instead of re-tripping over this.

DXCC Telegram alerts — 🔵 / 🟡 icons for `? BAND` and `? MODE` unconfirmed

Tab: DXCC Tracker (`d110d176c0aad308`) **Nodes:** `Format Telegram Alert Dedup 10 minute` (`94b77826079bad57` , live) + `Format Telegram Alert` (`d5eca40d3503035d` , disabled — kept in sync).

The dashboard's alert table distinguishes confirmed vs unconfirmed band/mode alerts via dim colour variants: `NEW_BAND` = `#58a6ff` (bright blue) → `NEW_BAND_UNCONF` = `#2d6aad` (dim blue); `NEW_MODE` = `#e3b341` (amber) → `NEW_MODE_UNCONF` = `#9a7030` (dim amber). The colour-coding is the at-a-glance visual cue.

Telegram had no equivalent: the icons map only covered `NEW_DXCC` / `NEW_BAND` / `NEW_MODE` / `NEED_QSL` , so the two `_UNCONF` variants fell through to the default `📻` (generic radio fallback). Operator received `📻 ? MODE … EH90ALL — Ceuta & Melilla` instead of a colour-tagged message — losing the band/mode dimension at a glance.

Fix: extended the icons map in both formatter functions:

```
var icons = {
    NEW_DXCC: '🔴',
    NEW_BAND: '🔵', NEW_BAND_UNCONF: '🔵',
    NEW_MODE: '🟡', NEW_MODE_UNCONF: '🟡',
    NEED_QSL: '🟣'
};
```

Design choice — option 1 of two discussed: reuse the *same* emoji as the confirmed variant. The text label (`? BAND` / `? MODE`) already carries the unconfirmed signal — adding a second visual indicator (orange, light blue, etc.) would over-encode the same fact in two ways. The emoji's job is the band/mode/DXCC dimension; the `?` prefix is the confirmed/unconfirmed dimension. Two orthogonal signals, kept that way.

Future: if a `NEED_QSL_UNCONF` or similar variant gets added, extend the map with `NEED_QSL_UNCONF: '🟣'` for consistency.

2026-05-16

Init Defaults — `RADIO_ENABLED = false` , `THRESHOLD_KM = 40` (operator preferences)

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** Init Defaults (`ec1fd4dece8c4dc0`)
Commits: `41c1708` (RADIO_ENABLED), `81ad69e` (THRESHOLD_KM)

Operator wanted only the antenna plug to be auto-disconnected on lightning events (radio plug left alone), and the disconnect threshold bumped from the 25 km default to 40 km. Two one-line edits.

The road to `false` — a JavaScript-typing gotcha worth memorialising. First attempt was `const RADIO_ENABLED = 'false';` — the **string** `'false'` , not the boolean `false` . Any non-empty string is truthy in JavaScript, so:

```
if (!flow.get('radio_enabled')) return null;
```

…evaluated `!'false'` → `false` → the gate stayed open and the radio relay still got switched off on every disconnect. Audited every code path; this was the only failure mode (no other node in the flow publishes to the radio Tasmota's cmd topic). Fix: drop the quotes — boolean literal. The Init Defaults blue-dot status text also relies on truthy evaluation (`+(RADIO_ENABLED ? ' radio:'+... : '')`), so with the string `'false'` the status text would still show the `radio:...` segment — a useful diagnostic signal that the bug exists.

Hardening considered but skipped: tighter coercion in Init Defaults (`flow.set('radio_enabled', RADIO_ENABLED === true)`) would have caught the typo at compile time. Decided against it because the existing call sites are clear, and the cost of strict coercion is that future-Manoj typing `RADIO_ENABLED = 'true'` (string) would *silently* disable the radio — same class of bug, different direction. Better: rely on operator awareness + the "how-to-edit-Init-Defaults" mental model.

AS3935 sensor health alerts — 3-minute grace period before firing

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** `as3935_health_xition` (transition detector added 2026-05-15) **Commit:** `ed260fe`

Operator was receiving paired Telegram alerts (`⚠ OFFLINE` , `✓ ONLINE`) within a minute on routine network blips — WiFi re-association, MQTT keepalive timeout + reconnect, brief ESP32 reboot. Real outages need alerts; brief flaps are noise.

Asymmetric debounce added to the transition detector:

- **Offline transition** → `setTimeout(graceMin × 60_000)` to fire the alert. If the sensor comes back online before the timer expires, `clearTimeout` and no alert fires.
- **Online transition** → cancels any pending offline timer. Only fires the recovery alert *if* the offline alert was actually sent (tracked via `context.alertedOffline`). Recovery from a non- reported outage is just noise.

Net effect:

Outage duration	Alerts sent
< 3 min flap	0 (silent)
3 min and up	1 OFFLINE at the 3-min mark, 1 ONLINE on recovery

The grace period is **3 min** by default (`cfg_sensor_offline_grace_min` in Init Defaults). Rationale: WiFi reconnect / MQTT keepalive (60 s) + reconnect / ESP32 reboot + DHCP all complete in ≤ 2 min in this shack. 3 min covers the common false-positive cases without delaying real-outage notification beyond actionable time. Easy to tune later if real-world experience shifts the optimal value.

Node statuses surface what's happening visually:

- Yellow ring `offline · pending 3m grace` → timer running, no alert sent yet
- Grey ring `flap recovered · no alert` → returned before timer
- Red dot `offline alert sent` → timer fired with sensor still down
- Green dot `online recovery alert sent` → paired recovery sent after a real outage

In-memory timer state (kept in `context.set('offlineTimer', t)`) is wiped on Node-RED restart, which is fine: the `lastOffline === undefined` first-sample guard at the top of the function suppresses the first post-restart transition. No spurious alerts on flow restart.

Operational lessons captured

Two recurring causes of confusion in this session, both written into docs for next time:

1. **Editor view vs disk truth.** After a `git pull` while Node-RED is running, the editor reflects in-memory flow state, not the file on disk. New nodes pulled from origin don't appear in the editor until `sudo systemctl restart nodered`. Caught us today when the Telegram nodes were "missing" from the editor view even though grep confirmed they were on disk.
2. **Safari ≠ Chrome/Firefox shortcuts.** Wrote `Cmd+Shift+F` as a fit-to-view suggestion — that's Safari's fullscreen, not Node- RED's view-all. Added a universal rule to `~/.claude/CLAUDE.md` ("Browser + keyboard environment") to default to mouse/menu actions over keyboard shortcuts, and to verify any suggested shortcut against Safari on macOS specifically. The Node-RED navigation that always works: scroll the canvas with the trackpad. The minimap is via hamburger → View, when the version exposes it.

2026-05-26

SPE (Vue /shack) — auto-ranging power bar replaces L/M/H scale

The Vue `/shack` SPECard's output-power bar was driven by `state.pwrMax`, which `ws_format_state` derives from the amp's selected power level (`p_level: L | M | H` → `pwrMaxMap = {L:500, M:1000, H:1500}` W, bumped to 700/1400/2000 for the 20K model). Operator clarified that "auto-scale per README" was meant to be the *detected-power-driven* auto-ranging behaviour the legacy `/ui` `SPE Panel (WS)` ui_template has had for years, not the L/M/H ↔ pwrMax mapping.

Fix — port the auto-ranging ladder, with finer low rungs

Vue SPECard now ignores `state.pwrMax` and computes its own `pwrBandMax` locally:

```
const PWR_LADDER = [5, 10, 25, 50, 100, 250, 500, 1000, 1500, 2000, 5000];
const pwrBandMax = computed(() => {
  const p = parseFloat(state.pwr) || 0;
  for (let i = 0; i < PWR_LADDER.length; i++) {
    if (p <= PWR_LADDER[i]) return PWR_LADDER[i];
  }
  return PWR_LADDER[PWR_LADDER.length - 1];
});
```

`pwrPct` divides current power by `pwrBandMax.value` for the fill width. A new `pwrBandLabel` computed pretty-prints the rung as `5 W`, `100 W`, `1k W`, `1.5k W`, `5k W`.

Ladder differs from the legacy `/ui` Panel (`[5,25,50,100,500,1000, 1500,2000,5000]`) by inserting `10` and `250` and bumping `25→50` so low-power tune carriers are actually readable:

- 3 W tune fills ~60 % of the **5 W** bar
- 18 W exciter pre-drive fills ~72 % of the **25 W** bar
- 80 W barefoot fills ~80 % of the **100 W** bar
- 700 W mid-drive fills 70 % of the **1k W** bar

- Full output sits at the **1.5k W** rung

The amp's L/M/H setting is still rendered as the `PWR LVL` tile (amber for Middle, red for Maximum) — just no longer drives the bar scale. `ws_format_state` is unchanged (still emits `state.pwrMax` for D1 compat; Vue ignores it).

CLAUDE.md SPE section + README "SPE Amplifier" both rewritten to make the new semantics explicit and to call out "don't recouple L/M/H to pwrMax" as a deliberate design rule.

Commit `debb454`.

/shack PWA — apple-touch-icon, favicon, manifest icons

For iPad/iPhone "Add to Home Screen" and a browser-tab favicon that isn't Node-RED's red dot. Previous `index.html` referenced uibuilder's shipped `uib-world.svg` which is generic and doesn't survive PWA install.

Hand-drew two source SVGs in `uibuilder/shack/src/`:

- `icon.svg` (180×180) — dark `#0d1117` rounded square with a subtle `#1f6feb` accent border, three concentric green signal arcs (`#3fb950`) over a vertical antenna mast (`#c9d1d9`) with a blue feed dot, and **"VU2CPL"** + **"SHACK"** lettering centred below. Used for browser favicon + apple-touch-icon.
- `icon-maskable.svg` (192×192) — full-bleed background, mast and callsign re-positioned into the centred 80 % safe zone so Android adaptive icons can crop without amputating anything. Used for manifest `purpose:"maskable"`.

Rasterised via `rsvg-convert` to 6 PNGs: `favicon-{16,32,48}.png`, `apple-touch-icon-180.png`, `icon-{192,512}.png`. Built `favicon.ico` (multi-resolution 16+32+48) via Python PIL — this is the key one because **Safari prefers `.ico` over `<link rel="icon">` SVG**, and falls back to the *site root*'s `/favicon.ico` (which is Node-RED's red dot) if `/shack/favicon.ico` doesn't exist.

`index.html` link tags reordered most-compatible-first so each browser picks something it understands:

```
<link rel="shortcut icon" href="./favicon.ico?v=4">
<link rel="icon" type="image/x-icon" href="./favicon.ico?v=4">
<link rel="icon" type="image/png" sizes="16x16" href="./favicon-16.png?v=4">
<link rel="icon" type="image/png" sizes="32x32" href="./favicon-32.png?v=4">
<link rel="icon" type="image/png" sizes="48x48" href="./favicon-48.png?v=4">
<link rel="icon" type="image/svg+xml" sizes="any" href="./icon.svg?v=4">
<link rel="mask-icon" href="./icon.svg?v=4" color="#1f6feb">
<link rel="apple-touch-icon" href="./apple-touch-icon-180.png?v=4">
<link rel="apple-touch-icon" sizes="180x180" href="./apple-touch-icon-180.png?v=4">
```

`apple-mobile-web-app-title` meta added so iOS home-screen label shows **"Shack"** not the full title. `manifest.json` lists raster + SVG, 192/512 carry `purpose:"any maskable"` for Chrome PWA install.

Cache-buster suffix `?v=N` — bump on each icon redesign so browsers re-fetch.

Commits `debb454` (initial SVG + apple-touch-icon links) + `5897fab` (rasterised PNGs + multi-res `favicon.ico` + reordered link tags + maskable PNGs in manifest).

Gotcha — Safari favicon cache is sticky

Safari maintains a per-host favicon cache in `~/Library/Safari/Touch Icons Cache/` that survives `Cmd+R`, Develop → Empty Caches, AND `rm -rf` (macOS Sequoia TCC blocks `~/Library/Safari/*` writes even with `sudo` — you'd need Full Disk Access). After deploying a new favicon set on the Pi and verifying via DevTools that `/shack/favicon.ico` returns 200 with the new icon, the browser-tab icon may still show the cached old one (Node-RED's red dot in this case).

The reliable nuke: **`Cmd+Q` the Safari process and relaunch.** The in-process favicon cache rebuilds from disk on next page load. Secondary path: Safari → Settings → Privacy → Manage Website Data → search `192.168.1.169` → Remove → reload.

This matters because anyone who refreshes the new favicon and sees the old one will assume the deploy is broken. It isn't — Safari is. Logged here so the next refresh doesn't waste an afternoon debugging a non-bug.

Lightning Vue card — fix CAPE always-null bug (writer/reader key mismatch)

`Build Lightning state for Vue` (`vue_lightning_builder_01`) had:

```
s.cape = flow.get('cape_now') ?? null;
```

…but the only writer on the tab, `Parse Open-Meteo → Strike` (`593f22a507b46335`), emits the value as:

```
flow.set('om_cape_latest', cape);
```

So `flow.cape_now` was never set anywhere and the Vue Lightning card's CAPE tile always rendered as `—` regardless of actual atmospheric conditions. `omState` worked fine for the same reason — its key matched between writer and reader.

Same class of bug as the 2026-05-10 DXCC `spot.dxCall` vs `spot.call` field-name mismatch. Pattern: when a downstream node renders nothing while upstream is clearly alive, the first thing to check is **whether the keys actually match end-to-end**, not whether the downstream code has a bug.

Fix (`vue_lightning_builder_01`) — read the actual key the writer emits, plus an inline comment naming the writer and warning future-self not to rename either side without touching the other:

```
// Writer is `Parse Open-Meteo → Strike` (593f22a507b46335) on this
// same tab. It writes om_cape_latest + om_state (+ om_wc_latest).
// Don't change these key names without updating that node too —
// Vue dashboard goes silent.
s.cape    = flow.get('om_cape_latest') ?? null;
s.omState = flow.get('om_state')       ?? null;
```

Commit `662bb7b`. Verified live: CAPE tile populates within ~5 min (next Open-Meteo poll) or immediately if `Poll Open-Meteo (5 min)` inject is hand-fired.

Lightning Vue card — drop redundant display rows from stats grid

The expanded card's stats `<dl>` had four rows that just duplicated data already visible elsewhere on the page:

Removed row	Where it actually lives

Callsign	Top bar (`VU2CPL` chip)
Grid	Top bar subtitle (`MK83TE · Bengaluru · Shack Control`)
Threshold	Thresholds collapsible — right beside its slider, where it's interactive
Reconnect	Thresholds collapsible — right beside its slider
Antenna	Collapsed header summary chip (`ANT ON` / `ANT OFF`, green/red)

Result: the stats grid is now three operational rows that live nowhere else on the page — `Total strikes`, `<40 / <50 / >50` zone breakdown, `Closest`. State fields (`state.callsign / grid / thresholdKm / reconnectMin / antennaOn`) kept in the reactive state object because Thresholds sliders, builder defaults, and the collapsed-header chip still read them.

Commits `3497bea` (Callsign/Grid/Threshold/Reconnect) + `916a789` (Antenna). Front-end only — no Node-RED restart needed.

Open follow-up: when the Lightning card is *expanded*, the collapsed-summary header (which carries the `ANT ON / ANT OFF` chip) is hidden via `v-if="!expanded"`. So antenna state isn't currently visible at the top of the open card either. If we want it always visible, move the chip out of the `v-if` so it sits permanently in the header row alongside the title. Deferred — operator hasn't asked for it.

DXCC Vue card — surface seed age from nr_dxcc_seed.json mtime

`Build DXCC state for Vue` (`vue_dxcc_builder_01`) had `seedAge: null` hardcoded, so the **Seed** field on the DXCC card always rendered as `—` instead of the actual age of the last Club Log fetch.

Fix: `fs.statSync` the seed file's mtime and format as a short relative-time string. The seed file is rewritten by `Fetch All Modes + Parse` on the daily 02:00 Club Log cron (and on any manual `POST /dxcc/refresh`), so mtime cleanly equals "last successful fetch" — exactly what the Seed field is meant to mean.

Format ladder (same shape as Last-seen tiles elsewhere on the dashboard for consistency):

Age	Renders as
< 1 min	`just now`
< 1 h	`Nm ago`
< 24 h	`Nh ago`
< 30 d	`Nd ago`
else	`Nmo ago`

Path probe order mirrors `Bootstrap Worked Table` (the canonical reader): `flow.cfg_flows_dir + '/nr_dxcc_seed.json'` first, then `~/.node-red/nr_dxcc_seed.json` as fallback. Required adding `libs: [{var:'fs',module:'fs'}, {var:'os',module:'os'}]` to the function node (per the CLAUDE.md gotcha that `fs` / `os` are not free globals in Node-RED function nodes).

Also emits `stats.seedMtime` (epoch ms) for future-proofing — Vue can do `toLocaleString` or stale-data colour-coding off it without a flow round-trip.

Commit `ea0e361`. Verified live: post-restart, Seed field flipped from `—` to `3h ago` within the next 3-second tick (matched the ~05:00 IST time of day versus the 02:00 IST cron run).

Dashboard 2 (FlowFuse) — retired; uibuilder + Vue 3 wins

The Dashboard 2 POC installed on 2026-05-24 (one Solar widget on `/dashboard`) is gone. The decision to stop investing in D2 was clear once the uibuilder + Vue 3 `/shack` migration delivered the actual goal — a dashboard that reflows cleanly from iPhone portrait through 4K desktop, with PWA install on iPad / iPhone — at significantly lower porting cost per card.

Why D2 didn't survive

In ~22 commits of porting effort, the D2 POC surfaced architectural blockers that compound across 12 cards:

- **D2's grid is fixed**, not responsive. Each widget declares `width: N, height: N` units. The "responsiveness" we got from D2 was the same hand-tuned breakpoints D1 already had — moving to D2 bought zero layout intelligence we didn't already control with CSS on D1.
- **`ui_control` not shipped in `node-red-dashboard` 3.6.6** (and verified absent on `--force` reinstall — not a packaging fluke, genuinely absent from the npm tarball). This kills the instant-on rehydrate pattern; you have to fall back to periodic ticks anyway, which is what D1 already does.
- **`ui-gauge` can't render value text at `width < 2`** — so any compact card has to use custom SVG anyway. Custom SVG inside D2 costs the same as custom SVG inside D1; the abstraction over the primitive is the only thing D2 was selling, and it was leaking.
- **Cross-tab wires silently drop in D2.** `flow.solarState` is tab- scoped, so fanout had to move onto the source tab regardless. No improvement over D1.
- **Default `gtype` is `gauge-half`** (not the unqualified `gauge`). Minor, but the kind of paper-cut you hit per widget.
- **`ui-text` format is `{{msg.payload}}`** not `{{value}}`. Different templating dialect than D1.
- **No native collapsible groups.** No dynamic widget-height messages on `ui-template`. So custom layout JS still needed.

Set against `/shack`'s wins — CSS column masonry with `clamp()` typography that *actually* reflows phone → tablet → laptop → 4K, collapsed-by-default cards with click-to-expand, PWA install with home-screen icon, single-file Vue app with no build step — D2 has no remaining purpose.

What got removed

flows.json — 9 nodes deleted (1 tab, 1 ui-base, 1 ui-page, 1 ui-theme, 1 ui-group, 1 function, 1 inject, 2 ui-templates), all prefixed `d2_` for clean grep:

```
d2_tab_solar_01          D2 Solar (flow tab)
d2_ui_base_01            Shack D2 (ui-base, served /dashboard)
d2_ui_page_shack_01      Shack (ui-page)
d2_ui_theme_01           Shack Dark (ui-theme)
d2_ui_grp_solar_01       Solar Conditions (ui-group)
d2_solar_fanout_01       Solar State → D2 widgets (function)
d2_solar_tick_01         Refresh D2 every 30s (inject)
d2_solar_unified_01      Solar — Unified Compact (ui-template)
d2_global_css_01         Global CSS overrides (ui-template)
```

Plus one stray wire on the Solar-tab inject that fed `d2_solar_fanout_01` as its second downstream — stripped via a Python pass over the JSON. After cleanup: 511 → 502 nodes, zero `d2_*` references anywhere in the serialised flow (regex-verified post-write).

Pi-side — `npm uninstall @flowfuse/node-red-dashboard` ran cleanly; package.json dropped the dep + 25 transitive deps + ~80 MB of `node_modules/`. `/dashboard` URL returns 404 (was 200 before).

Misleading code comment — the `Replay AS3935 state` function (`as3935_tuning_replay_fn` on the `AS3935 Tuning` tab) carried a comment "Tick is the workable substitute until either Dashboard 1 ships ui_control again or we migrate widgets to Dashboard 2 (FlowFuse)." The D2 hint is now stale; rewrote it to "D1 /ui rehydrate path; the /shack Vue card uses its own uibuilder tick (vue_as3935_tick) and doesn't depend on this one."

What got kept

The D2 history stays in this changelog and in HANDOVER — the lesson ("a 'responsive dashboard' that doesn't reflow isn't responsive") is worth keeping for the next time someone (including me, six months from now) wonders whether D2 has matured enough to revisit.

Verification

```
$ curl -s -o /dev/null -w '%{http_code}' http://localhost:1880/dashboard
404
$ curl -s -o /dev/null -w '%{http_code}' http://localhost:1880/shack/
200
$ curl -s -o /dev/null -w '%{http_code}' http://localhost:1880/ui
301
```

`/ui` (D1) and `/shack` (Vue) keep working; `/dashboard` (D2) is gone.

Commit `13c9ac6`.

New doc — `FORK_GUIDE.md` for different-operator forks

A 350-line top-level runbook for a different operator who wants to run this stack at **their own QTH**. Pattern matches the existing docs: REBUILD_PI = same-Pi rebuild, DEPLOY_PI = fleet member, and now FORK_GUIDE = different station.

Why a separate doc. CLAUDE.md is full of VU2CPL-specific operational lore (the GPIO17-vs-GPIO4 AS3935 saga, the SPE WS migration history, the cluster login regex saga) which a forker doesn't need to read end-to-end. REBUILD_PI assumes VU2CPL-specific values everywhere (`192.168.1.169`, `VU2CPL`, `MK83TE`, `gpsntp.local`). A clean per-site customisation pass needed its own runbook.

Stages covered: A=network/MQTT broker, B=Lightning Init Defaults (callsign / grid / antenna power switch / radio enable / thresholds), C=DXCC + Telegram credentials + cluster selection, D=FlexRadio, E=SPE amplifier, F=rotator, G=LP-700, H=Tasmota devices (topic renames + TZ + power-card layout), I= `/shack` PWA rebadge with full icon-regeneration command list (`rsvg-convert` + PIL `favicon.ico` multi-res), J=RPi fleet, K=network monitor, L=GPS NTP card, M=Open-Meteo location, N=backups + secrets hygiene.

Hardware compatibility table up front tells a forker exactly which flow tabs to delete if they don't own the matching hardware, so the "I followed the runbook and now I have a broken FlexRadio tab that's bothering me" path closes off early.

Troubleshooting table at the end indexes the usual "I skipped a stage" failure modes (`/ui` shows only `—` → MQTT broker IP wrong somewhere; DXCC silent → cluster IPs unreachable; etc.).

REBUILD_PI.md prologue gets a callout pointing forkers at FORK_GUIDE *before* they walk the rebuild runbook. README.md "Documentation map" gains a FORK_GUIDE row + repository-layout tree includes the new file. CLAUDE.md unchanged — it's already labelled as "VU2CPL-specific operational lore" and the forker is now correctly steered away from it.

Verification: clean git clone + `rebuild_pi.sh` end-to-end remains the same first step for both same-Pi rebuild AND forker (the docs just diverge after step 13).

Commit `13c9ac6` .

DXCC — Login + Parse + Dedup now handles multi-line TCP chunks

In the Vue `/shack` DXCC card the **VU2CPL** cluster pill was showing offline while N2WQ / VU2OY / VE7CC were green. The cluster server (`vu2cpl.ddns.net:7300` , CwSkimmer) was alive and serving spots — verified by manual `nc` login + watching spots arrive. So the TCP side was fine.

Root cause: `login-parse-dedup-v2` treated each incoming TCP chunk as a single line. The tcp-in nodes use `newline: ""` (raw chunks) because CwSkimmer's `Please enter your callsign:` prompt has no trailing newline (and tcp-in's newline-split would buffer the prompt forever). But CwSkimmer also **batches** multiple lines into one TCP segment — `cluster ack + 2-3 DX spots in one chunk` is the normal post-login traffic pattern. Two failure modes inside the function:

1. `line.startsWith('DX de')` failed when the chunk began with the post-login ack `VU2CPL-1 de SKIMMER 2026-05-26 ...Z CwSkimmer >` . The embedded `DX de` lines were silently dropped.
2. Even when a chunk did start with `DX de` , the parse regex's `$` anchor failed because of embedded `\n` between consecutive spots.

Both paths returned `[null, null, null]` , so `cluster3` (VU2CPL) never received a `lastSpotTs` bump. Cluster Watchdog (every 30s) saw `lastSpotTs === 0` and held `connected: false` . Vue rendered "offline".

N2WQ / VU2OY / VE7CC happen to deliver one spot per TCP chunk (line-buffered output from traditional cluster software) so the single-line assumption held for them and they showed green.

The RBN tab's `Login Handler VU2CPL` got this same fix back on 2026-05-17 ("loops over all lines, emits one msg per DX de") but the DXCC tab's unified handler never got it. The two now match.

Fix: refactor `Login + Parse + Dedup` to split the chunk on newlines and iterate per-line, emitting one spot msg per `DX de` line via `node.send()` , plus one final `cluster_status` update per chunk. Login detection already operated on the **last** fragment of the chunk and stays as-is.

Also tightened: mute-check fallback now derives `_name` from `msg.topic` (clusterN) when `_session.host` / `msg.host` aren't populated. TCP-in in client mode doesn't reliably set `_session` on inbound msgs, so the previous mute check could silently skip the host string and miss mute commands. Topic-based fallback closes the gap.

Same class of bug as the 2026-05-10 DXCC `spot.dxCall` vs `spot.call` field-name mismatch. Pattern: when a downstream node renders nothing while upstream is clearly alive, the first thing to check is **whether the parsing assumptions actually match what the wire is delivering**, not whether the downstream code has a flag bug.

Status badge format also clearer now: `[clusterN] X/Y spots (chunk)` shows emitted vs total DX lines so the dedup ratio is visible at a glance.

Verified by manual `nc vu2cpl.ddns.net 7300` → login as `VU2CPL-1` returns the cluster ack + 2 spots all in one `recv()` call.

Commit `e188e9d`.

Vue `/shack` LIVE/OFFLINE pill — multi-iteration saga

The TopBar's connection indicator went through five iterations in one afternoon before landing on something that works on every browser / PWA combo. Logged here so the rationale for the final architecture isn't lost.

Symptom: pill stuck on red OFFLINE even though cards were visibly receiving live data. Mac Safari sometimes worked, iPad / iPhone Safari / dock-icon PWAs / home-screen PWAs did not.

Iteration 1 (`0834b41`) — read `uibuilder.ioConnected` boolean directly on a 1s interval. Mac green; iPad red. uibuilder v7's `ioConnected` property is inconsistently populated across WebKit variants and reconnect cycles.

Iteration 2 (`3822489`) — install a wrapper on `uibuilder.onTopic` so every card's per-topic callback bumps a `window.__shackLastMsgAt` heartbeat; pill = green if any msg within last 8s. Conceptually right approach (every card IS using `onTopic` and they ARE getting data, so the wrap should catch every msg). Mac green; iPad still red. The wrap installed correctly but something further down still broke the binding.

Iteration 3 (`479ef10`) — added no-cache HTTP-equiv meta tags to `index.html` on the theory that iPad Safari was caching the old HTML. Helped future-proof updates but didn't fix the immediate red pill (Safari Website Data clear was needed once to drop the old cached HTML; from that point on the meta tags do their job).

Iteration 4 (`77e9059`) — added VISIBLE diagnostics to the OFFLINE pill itself: a build-stamp + per-signal counters strip showing `v5 · uib? · wrap=Y · oT28 oM29 dU0 dS28 · ioC=Y · 0s`. This is what cracked it: the screenshot from Mac Safari showed EVERY signal firing correctly (`oT28` = 28 msgs via wrapped `onTopic`, `oM29` = 29 via `onChange('msg')`, `dS28` = 28 via DOM event `uibuilder:stdMsgReceived`, `ioC=Y` = `ioConnected` true, `0s` = last msg 0 seconds ago) — but pill still said OFFLINE. So the data collection was fine; the bug was in the state-to-pill propagation, not in the heartbeat.

Iteration 5 (`85ab62f`) — the actual fix. State `connected` was being set in the App component's `setInterval` (`connected.value = true`) and passed to TopBar via `:connected` prop. The prop binding was failing silently — Vue's reactive ref-to-prop chain didn't propagate the change. Meanwhile TopBar had its own working `setInterval` (`tick()`, the same one that updated the clocks and the diag-strip last-msg counter — proven to work because we watched the `0s` counter increment). Moved the `connected`-state derivation INTO `tick()`, dropped the parent-to-child prop chain entirely. Pill works on every device since.

Lesson: when you can't reason about which of three competing signals is breaking, **make every signal visible on the device**. The diag-strip iteration was ugly UI but it diagnosed the bug in one screenshot's worth of feedback. Without it, this could have been another full afternoon of guessing.

Architecture left in place:

- `window.__shackLastMsgAt` heartbeat (single global, bumped from 4 parallel paths in `installAllPaths()`)
- `window.__shackPaths` per-path counters (used by the diag-strip when offline — but inert when connected, so they're cheap to leave wired up for future debugging)

- `window.__shackBuild = 'v6 · …'` build stamp, visible in the diag-strip — distinguishes "code didn't load" (no diag-strip at all) from "code loaded but signal broken" (diag-strip with stale counters)
- TopBar's `tick()` reads `__shackLastMsgAt` + `__shackPaths.ioConnectedSeen` on a 500ms interval; `connected.value` = OR-gate of `(age < 8000ms) || ioConnectedSeen`

Commits: `0834b41`, `3822489`, `479ef10`, `77e9059`, `85ab62f`.

Vue `/shack` PWA — defeat HTML caching on iOS Safari / Mac dock icons

While debugging the LIVE/OFFLINE pill (above), discovered that iOS Safari and the macOS Sonoma "Add to Dock" webviews both cache the `/shack` HTML aggressively and don't honour `Cache-Control: max-age=0` reliably. Cards still received data (the cached JS worked fine), but my new HTML referencing `?v=N` cache-busters was never fetched — so subresource updates didn't propagate.

Fix (`479ef10`): added three meta tags to `index.html` head:

```
<meta http-equiv="Cache-Control" content="no-cache, no-store, must-revalidate">
<meta http-equiv="Pragma" content="no-cache">
<meta http-equiv="Expires" content="0">
```

These are interpreted by WebKit at HTML-load time and instruct the browser/PWA not to cache the HTML itself for future use. Won't help the PWA whose cached HTML is from BEFORE these tags landed (those have to be deleted + re-added once); but every PWA installed from this commit onward auto-refreshes on each open. Operator never again has to ask users to delete + reinstall after a code push.

Per-device-cleanup procedure documented in **FORK_GUIDE.md Stage O** troubleshooting table.

TopBar — responsive layout on iPhone portrait

After pill fix landed, iPhone portrait (~390px wide) was showing the sunrise + sunset clocks overflowing off-screen to the right. Topbar was a single-row flexbox with 4 clocks (UTC, IST, Sunrise, Sunset) pinned right via `margin-left: auto`.

Fix (`22d75c8`):

- `.topbar { flex-wrap: wrap }` so left group (callsign + pill + subtitle) and right group (4 clocks) stack vertically when they can't fit horizontally.
- `.topbar__left { flex: 1 1 auto; min-width: 0 }` lets the left group shrink rather than pushing clocks off-screen.
- `.clocks { flex-wrap: wrap }` so the 4 clocks can wrap into a 2×2 if needed.
- `@media (max-width: 480px)`: smaller callsign font, 4 clocks splitting full width via `justify-content: space-between` + `flex: 1 1 0` per clock, tighter padding overall.

Mac and iPad-landscape (>480px) layouts unchanged — same single-row look as before.

FORK_GUIDE — new Stage O for per-device PWA install

A forker's users (family / club mates / co-ops) need a clean roll-out path separate from operator-side customisation. Stages A–N cover making `/shack` work; Stage O covers handing it out to humans.

Added under Stage N:

- **Operator pre-flight:** `curl` smoke test + browser verify before handing the URL out to users

- **Hand-out template:** the 2-line message a forker actually sends ("URL: http://… · Tip: install as home-screen app")
- **Per-platform PWA install steps:**
    - Mac Safari 17+: File → Add to Dock
    - iPad/iPhone Safari: Share → Add to Home Screen
    - Android Chrome: ⋮ → Install app
    - Windows / Linux Chrome / Edge: address-bar install icon
- **What users see after install:** LIVE/OFFLINE pill semantics (8s heartbeat window, auto-reconnect, tooltip with last-msg age), auto-update behaviour via no-cache meta tags, responsive column-masonry layout, collapsed-by-default cards
- **Per-device troubleshooting table** covering the 5 bugs we hit this afternoon during the actual rollout (stale Safari favicon cache on iOS, Mac dock-icon webview holding pre-no-cache HTML, iOS Safari ignoring `max-age=0`, sunrise/sunset overflow on iPhone, pill permanently OFFLINE = real Node-RED issue)

Cross-linked from Stage I (operator-side rebadging) so anyone reading the dashboard section sees the user-facing install path.

Commit `3b909df`.

Lightning — FlexRadio TX inhibit on disconnect + Telegram retrigger alert

Two interlocking features landed together because both touch the disconnect/reconnect chain, and the second materially improves the operator's experience of the first.

TX inhibit chain (safety enhancement)

When the matrix fires `Trigger Disconnect`, we already cut antenna power via Tasmota. We now ALSO send the FlexRadio a TX inhibit command (`transmit set inhibit=1`) over its TCP API. The radio stays powered and RX-capable, but key-up does nothing — the operator can't accidentally transmit into a disconnected antenna. On reconnect / force-reconnect / bypass-on / manual override / HTTP ANT ON, we send `transmit set inhibit=0` to clear it.

> ***Command form discovered via spray-and-pray*** *— see the "Command lineage" sub-section below. First two iterations used* `interlock tx1_inhibit=N` *which FLEX-6600 firmware rejects.*

Why TX inhibit and not full radio power-off: cutting radio power (the existing `RADIO_ENABLED=true` mode in Init Defaults) also kills RX — operator loses storm-band situational awareness + DX cluster spots during the very period they most want to be listening. TX inhibit gives the same safety guarantee without the RX blackout. SmartSDR has this exact button on the front panel as the red "TX Inhibit" toggle; we're driving the same property over the API.

Wiring (Lightning Antenna Protector tab):

New nodes:

- **`tx_inhibit_setter_01`** (function "TX Inhibit Setter") — reads `msg.cmd` from any incoming msg:
    - `'DISCONNECT'` → emit `payload: ['xmit 0', 'transmit set inhibit=1']` (universal PTT release first, then inhibit future TX)
    - `'RECONNECT'` / `'BYPASS_ON'` → emit `['transmit set inhibit=0']`
    - else → `return null`
    - Idempotent — same value re-fired is harmless, so DC retriggers during the 20-min reconnect window safely re-send `=1`.

- **`flexradio_tx_inhibit_01`** (`flexradio-request` "FlexRadio TX Inhibit") — reuses the existing `flexradio-radio` config node (`94d0df28ae5cccfc`) the FlexRadio tab already uses. Config nodes are cross-tab; no new TCP connection.

New wires (each additive — no existing wires removed):

- `Trigger Disconnect` (out 0, msg.cmd='DISCONNECT') → Setter
- `Execute Reconnect` (out 0, msg.cmd='RECONNECT') → Setter
- `Force Reconnect` (out 0, msg.cmd='RECONNECT') → Setter
- `HTTP → Antenna ON` (out 1, Tasmota msg) → Setter (after adding `cmd: 'RECONNECT'` to that msg — one-line edit to the existing function)
- `Manual Override` (out 0, cmd='RECONNECT' or 'DISCONNECT') → Setter

Bypass ON path doesn't get a direct wire — it routes through `Execute Reconnect` (Bypass Handler's output 3 fires Execute Reconnect's standard ON), so it picks up `cmd: 'RECONNECT'` automatically.

Failure modes — handled gracefully:

- Radio not connected: `flexradio-request` emits its standard `radio not available or not connected` error. Disconnect chain to Tasmota is on a parallel wire so antenna still goes OFF.
- Multiple DCs in quick succession: each fires `=1` ; radio is idempotent.
- Bypass timer expires with no real strike: matrix prevents DC while bypass active, so inhibit never set, no clear needed.

Telegram alert — retrigger differentiation

Pre-fix: every disconnect during a storm sent the same `⚡ ANTENNA DISCONNECT` alert. A single 30-minute storm with strikes every 5 minutes during the 20-min reconnect window produced 5+ duplicate DISCONNECT alerts in Telegram. Noisy and operationally useless — the antenna was already OFF, the operator already knew, the duplicate alerts buried any other shack notifications.

Fix path: thread a `retrigger` boolean from Trigger Disconnect through to the Telegram formatter:

1. **`Trigger Disconnect`** now sets `msg.retrigger = already` (where `already = flow.get('antenna_off') || false` at function entry). True iff antenna was off at the moment this DC fired.
2. **`Format Log`** propagates `retrigger: !!msg.retrigger` into the `event_record` that flows through `light_jsonl_append_01` and on to the Telegram router.
3. **`tg_lightning_router`** branches on `rec.retrigger` inside the `disconnect` case:
    - retrigger=false (first DC of this storm): existing alert format, plus a new "TX **inhibited**" line.
    - retrigger=true (subsequent DC, antenna already off): NEW alert: `⚡ STORM CONTINUES · Source: ... · New strike at: N km · Antenna already <b>OFF</b>; TX still inhibited. · Reconnect timer reset to <b>NN min</b> from now.` NN reads `flow.reconnect_min` live so it reflects whatever the operator has the slider set to.

`reconnect` alert also gains a "TX **allowed** again" line so the on/off symmetry is visible to the Telegram reader.

Net behaviour: one initial DISCONNECT alert per storm, then a STORM CONTINUES alert for every additional strike during the reconnect window (each useful — it confirms the storm hasn't ended), then one RECONNECT alert when the all-clear timer finally fires. Rate-limiting at 5-per-60s per type still applies, so very dense storm activity still gets capped to one summary notice.

What needed editing

- **`Trigger Disconnect`** (`d62fb0c3c40f03b7`): added `retrigger: already` to the output-0 msg.
- **`Format Log`** (`5bfc6db2af9dd24c`): added `retrigger: !!msg.retrigger` to event_record.
- **`HTTP → Antenna ON`** (`f2092c6e0d932c7b`): added `cmd: 'RECONNECT'` to the output-1 Tasmota msg so the Setter can recognise the operator-override path.
- **`tg_lightning_router`** (`tg_lightning_router`): disconnect case now branches on rec.retrigger; reconnect case adds TX allowed line.
- Two new nodes inserted (Setter + flexradio-request).
- Five new wires from existing sources to the Setter.

502 → 504 nodes. clublog_dxcc_tracker_v7.json regenerated per CLAUDE.md rule #4 (no DXCC nodes touched but the regen is mandatory on every flows.json commit). CLAUDE.md Lightning Antenna Protector section grew two new subsections documenting the TX inhibit chain + Telegram retrigger logic.

Verification path (operator-side, post-deploy):

1. Fire `TEST ⚡ 6 km DISCONNECT` inject → antenna goes OFF via Tasmota AND radio refuses TX (try keying — nothing happens). SmartSDR shows the red TX Inhibit indicator. Telegram fires the first-DC format alert with "TX inhibited" line.
2. Within 20 min, fire another `TEST ⚡ 6 km` → no new Telegram `DISCONNECT`. Instead get `STORM CONTINUES` with the timer- reset wording. Radio still TX-inhibited.
3. Wait for reconnect timer to fire (or click Force Reconnect). Antenna ON, radio TX inhibit clears, SmartSDR's TX Inhibit indicator goes dark, RECONNECT alert with "TX allowed again".

Commits `d87a708` (initial implementation), `e16646e`

- `34c1db4` (debug iterations), `0722d41` (working form locked in).

Command lineage — how `transmit set inhibit=N` was discovered

The initial implementation `d87a708` used `interlock tx1_inhibit=N` based on what looked like a sensible API form. It didn't work — 3 commands sent, 3 empty replies, SmartSDR's TX Inhibit indicator stayed dark. Two debug iterations followed:

`e16646e` — sprayed 3 known forms (`xmit 0` , `interlock tx1_inhibit=N` , `radio set tx_inhibit=N`) and added a diagnostic logger downstream of `flexradio-request` . The logger dumped `msg.payload` of the radio's reply, which came back EMPTY for all three. Inconclusive — either all 3 commands were ack'd with no payload (silent success), or the reply lived in a different msg field that we weren't reading.

`34c1db4` — expanded to 5 command variants (added `transmit set inhibit=N` and `interlock tx1_inhibit_value=N`) AND rewrote the logger to dump EVERY msg field (not just `msg.payload`). One test inject later, journalctl revealed the actual reply shape:

```
flex reply msg: payload="" · _txInhibitValue=1 ·
                request="xmit 0" · sequence_number=9 · status_code=0
flex reply msg: payload="" · _txInhibitValue=1 ·
                request="interlock tx1_inhibit=1" ·
                sequence_number=10 · status_code="0x5000002D"
flex reply msg: payload="" · _txInhibitValue=1 ·
                request="radio set tx_inhibit=1" ·
                sequence_number=11 · status_code="0x5000002D"
flex reply msg: payload="" · _txInhibitValue=1 ·
                request="transmit set inhibit=1" ·
                sequence_number=12 · status_code=0
flex reply msg: payload="" · _txInhibitValue=1 ·
                request="interlock tx1_inhibit_value=1" ·
                sequence_number=13 · status_code="0x5000002D"
```

`status_code=0` is ACK, `0x5000002D` is "command not recognised". The data was hiding in `msg.status_code`, not `msg.payload`.

Lineage table (FLEX-6600, SmartSDR v3.x firmware, 2026-05-27):

Command	`status_code`	Verdict
`xmit 0`	`0`	✓ universal PTT release (works everywhere)
`interlock tx1_inhibit=N`	`0x5000002D`	✕ rejected
`radio set tx_inhibit=N`	`0x5000002D`	✕ rejected
`transmit set inhibit=N`	**`0`**	**✓ THE working form**
`interlock tx1_inhibit_value=N`	`0x5000002D`	✕ rejected

`0722d41` trimmed the spray array to just the two ACKing commands (`xmit 0` + `transmit set inhibit=N`) and removed the debug logger entirely. CLAUDE.md "Lightning Antenna Protector" gained the lineage table so a future firmware update that breaks this can be re-derived quickly — re-apply the spray-and- pray pattern from `34c1db4`, grep journalctl for `status_code=0`, identify the new working form.

Verification — confirmed end-to-end 2026-05-27

Operator verification after `0722d41`:

1. Fire `TEST ⚡ 6 km DISCONNECT` inject → antenna OFF via Tasmota AND SmartSDR TX Inhibit indicator lit red AND key-up refused. Telegram fired the first-DC alert with "TX inhibited" line. ✓
2. Press BYPASS ON in `/shack` dashboard → antenna ON, SmartSDR TX Inhibit clears (BYPASS_ON path routes through Execute Reconnect which emits `cmd:'RECONNECT'`, picked up by the Setter), key-up works. ✓
3. Fire another `TEST ⚡ 6 km DISCONNECT` while bypass active → Trigger Disconnect early-exits on `bypass_active`. Antenna stays ON, TX inhibit stays clear, only an info log entry appears. Telegram silent (matrix didn't fire DC). ✓

Bypass mode correctly shadows the disconnect+inhibit chain; manual operator override fully restores radio TX capability.

Side note — minor cosmetic dead code

The `TX Inhibit Setter` has an explicit `cmd === 'BYPASS_ON'` branch but it's never hit, because the BYPASS ON path routes through `Execute Reconnect` which rewrites `msg.cmd = 'RECONNECT'` before the message reaches the Setter. The `RECONNECT` branch handles bypass-on correctly. The `BYPASS_ON` case is harmless (does the same thing) — leaving it as documentation of intent in case Bypass Handler's wiring ever changes to skip Execute Reconnect.

2026-05-27

Lightning — Open-Meteo no longer emits a synthetic 0-km strike

Closes HANDOVER follow-up #25.

The bug (cosmetic, not safety): `Parse Open-Meteo → Strike` (`593f22a507b46335`) used to map current-hour thunderstorm (WMO weather_code ∈ {95, 96, 99}) to `km = 0` , plus a graduated ladder for rising CAPE (km=10 for ≥2500 J/kg, km=40 for ≥1500, km=100 for ≥800). It emitted a strike-shaped payload at that synthetic distance to the Parse Strike chain.

The 2026-05-12 distance-graded matrix-fix already prevented these from firing disconnects (Trigger Disconnect early-rejects `source === 'Open-Meteo'`), so no false DC fired — but the event recorder ("Tap on Haversine output") still caught the strike message and wrote a `type:'strike' source:'Open-Meteo' km:0` line to `nr_lightning_events.jsonl` . The dashboard event log read "Open-Meteo: TS NOW → 0 km @ ..." which looks like "lightning directly overhead" but actually means "Open-Meteo forecast says probability of thunderstorms is high." Two different things; not what the operator expected to see.

The fix (option (a) from the TODO):

`Parse Open-Meteo → Strike` no longer emits a strike-shaped payload at all. Only emits:

- `type: 'cape'` — feeds the CAPE tile on Master Dashboard / Vue Lightning card (was already wired, unchanged)
- `type: 'log'` — descriptive text line, wording rewritten to NOT include a fake distance

Outputs reduced 2 → 1. Wire to Parse Strike (`26ddff0cbbfe5fc1`) removed.

The matrix corroboration logic in Trigger Disconnect is unaffected — it reads `flow.om_state` directly, and that's still set on every OM poll (alongside `om_state_until` , `om_cape_latest` , `om_wc_latest`). The om_state derivation logic (cold / lit / severe based on tsActive + CAPE threshold) didn't change.

New log-line wording (mode → text mapping):

Condition	Pre-fix log line	New log line
Current-hour TS (wc=95/96/99)	`Open-Meteo: TS NOW (wc=95) → 0 km @ 14:00`	`Open-Meteo: Thunderstorm NOW (wc=95) → SEVERE @ 14:00` (suffix when om_state≠cold)
Forecast-hour TS	`TS hour (wc=96) → 0 km`	`Thunderstorm forecast (wc=96) → LIT @ 14:00`
Showers + high CAPE	`Shower+CAPE 1200 → 20 km`	`Showers + high CAPE 1200 J/kg @ 14:00`
Severe CAPE only	`Severe CAPE 2700 → 10 km`	`Severe CAPE 2700 J/kg @ 14:00`
Strong CAPE	`Strong CAPE 1800 → 40 km`	`Strong CAPE 1800 J/kg @ 14:00`
Moderate CAPE	`Moderate CAPE 900 → 100 km`	`Moderate CAPE 900 J/kg @ 14:00`
Calm	`Calm CAPE 200 (no action)`	`Calm · CAPE 200 J/kg @ 14:00`

The `→ STATE` suffix on TS conditions surfaces the matrix's view of severity directly in the log without using a misleading number. Node status badge also colour-codes by om_state now (red/severe, yellow/lit, green/cold) instead of by the fake distance.

Downstream effects — semantically correct, no UX loss:

- **Vue Lightning card** — CAPE tile + om_state were already driven by the `type:'cape'` message via the existing builder; no change.
- **D1 `/ui` Master Dashboard** — the dashboard's `type:'cape'` handler renders the CAPE tile; the `type:'strike'` handler no longer sees OM events. Event log shows the descriptive log lines instead.
- **JSONL event log** (`nr_lightning_events.jsonl`) — OM events no longer appear as `type:'strike'` entries. They never made it past Trigger Disconnect anyway since 2026-05-12, so this just drops a cosmetic noise stream. Real AS3935 strikes continue to be logged correctly.
- **Telegram alerts** — already ignored OM-only events (router allowlist excludes `'strike'` type). No change.

Files touched:

- `flows.json` — `Parse Open-Meteo → Strike` func body fully rewritten + outputs 2→1 + wires trimmed
- `CLAUDE.md` — "Key Node IDs" table entry for `593f22a507b46335` updated; new "Behaviour change vs pre-2026-05-27" paragraph appended to the "Distance-graded disconnect" section
- `HANDOVER.md` — TODO #25 struck through with closure note
- `clublog_dxcc_tracker_v7.json` regenerated (no DXCC nodes touched but mandatory per CLAUDE.md rule #4)

Commit `3ea8d33` .

D1 `/ui` — five Vue-parity backports (closes #26–#30)

Five HANDOVER follow-ups closed together — D1 `/ui` now matches Vue `/shack` on the substantive UX deltas. Did in three commits sized by risk.

`b9b727c` — #28 SPE finer power-bar rungs

Trivial array-literal change in `ws_panel_node` 's SPE_SCALE: `[5, 25, 50, 100, 500, 1000, 1500, 2000, 5000]` → `[5, 10, 25, 50, 100, 250, 500, 1000, 1500, 2000, 5000]` . Adds 10 W and 250 W rungs. Low-power tune carriers (1–10 W) now read meaningfully on D1 instead of all collapsing into the 5 W rung. Matches Vue's PWR_LADDER exactly.

`0153c9a` — #27 cluster click-to-mute + #30 seed-age display

Both touch the DXCC Dashboard ui_template (`38a6451a95a57685`).

#30 (seed-age display) — new "Seed age" row in the WORKED STATS card backed by `dxcc_seed_age_fn_01` function node (`fs.statSync` on `nr_dxcc_seed.json` , `libs:[fs, os]` declared) driven by a 30 s inject `dxcc_seed_age_tick_01` . Watch handler in the template's `scope.$watch` now branches on `topic === 'seed_age'` and updates `#st-seed` text + amber colour for `>30d` ages. Mirrors `vue_dxcc_builder_01` 's implementation for the /shack DXCC card.

#27 (cluster mute toggle) — each of the 4 cluster nodes (VU2CPL/VU2OY/VE7CC/N2WQ) now has an `onclick` handler that fires `scope.send({topic:'dxcc_mute', payload:{name, mute}})` . New downstream function `dxcc_mute_handler_01` reads that topic and updates `global.dxcc_cluster_mute` — the SAME global already honoured by `login-parse-dedup-v2` 's mute check, so **D1 and Vue share mute state**. Visual: muted cluster nodes render at 0.4 opacity with `.cl-info` text replaced by amber "MUTED" label. Cluster Watchdog (`c68f81fda8c7f015`) extended to attach `payload._muted` (current global state) alongside per-cluster status so the dashboard rehydrates correctly on page open instead of starting from local stale state.

DXCC Dashboard output now fans out to TWO targets (existing Blacklist Filter + new mute handler). Both ignore msgs they don't recognise.

`38bf3fb` — #26 AS3935 Bridge + Lightning merge + #29 stats trim

The big one. Master Dashboard (`557083037f168b22`) absorbs the standalone AS3935 Control Panel (`223cb2ce733c5d3f`).

Motivation — Vue `/shack` Lightning card already shows AS3935 sensor state, Bridge tunables, controls, test injects, and event log all together. D1 had them split across two dashboard tabs (Shack tab for Lightning Master Dashboard; Shack Monitoring tools tab for AS3935 Control Panel). Operator wanted them unified to match Vue.

Merge mechanics:

- Master Dashboard `format` absorbs the AS3935 panel piecewise:
  - **CSS** (`#a35tune *` , `#a35ev *` namespaces) appended inside the Master Dashboard's `<style>` block. Namespaces don't collide with Master's `#dash` namespace.
  - **HTML** wrapped in a `<details><summary>AS3935 Bridge — Tuning · Diagnostics · Test injects</summary>...</details>` collapsible inserted right after the existing `as3935Card` mini-display and before the Event Log. Preserves the compact distance/energy/event/ last-seen tiles already on Master; controls below are click-to-expand so the card isn't overwhelming.
  - **Two IIFE scripts** (Pattern-B `(function(scope){...})(scope)` form) appended inside Master Dashboard's `<script>` . Both bind their own `scope.$watch` and coexist cleanly with Master's existing IIFE — three watch handlers on the same scope is fine, each just inspects msg fields they care about and returns.
- Master Dashboard's `height` bumped **12 → 22** to fit the merged content even with the AS3935 details collapsible closed.

- **7 incoming wires** that previously targeted `223cb2ce733c5d3f` (AS3935 status, hb, cmd-ack, event, last_event mqtt-in subscriptions plus replay-tick fan-out) re-pointed to `557083037f168b22` . The merged template's IIFEs receive everything the standalone panel used to receive.

- Master Dashboard's **output** now also wires to `82f732a0dac14945` (AS3935 cmd MQTT-out). AS3935 IIFE scope.send calls for `{set: key, value: v}` tunable updates and `{action: name}` maintenance commands now reach the bridge via this fan-out wire — alongside the existing Manual Override + Save Threshold wires which ignore non-matching topics.

- **`223cb2ce733c5d3f` ui_template + `as3935_ctl_grp` ui_group deleted.** Regex-verified zero remaining references. The "Shack Monitoring tools" dashboard tab keeps its other four groups (RBN Skimmer, RPi Fleet, Network Monitor, Solar Conditions) — only the orphan AS3935 group goes away. The tab itself stays populated.

#29 (stats grid trim) done in the same commit — removed Callsign

- Grid Square entries from Master Dashboard's `rows` array in the stats render function. Both already render in the Header card. Kept Threshold + Reconnect + Antenna rows (useful at-a-glance and Vue keeps them too in its Thresholds collapsible).

507 → 505 nodes net.

Risk + mitigation — this is a substantial single-template change (~22 KB of HTML/CSS/JS pasted in). If a script error breaks the new IIFEs the Master Dashboard wouldn't render at all (panel-wide JS failure stops Angular). Mitigation: (a) the two IIFEs are unchanged from the standalone panel that worked for weeks; only their host template changed, (b) restart-then-30s journalctl shows no error/warn lines from Node-RED, (c) revert path is `git revert 38bf3fb` which restores both templates atomically.

Operator verify path after deploy:

1. Open `/ui` Lightning Master Dashboard card → AS3935 mini-display tiles still render with live data (distance/energy/event/last seen). ✓
2. Click the `▶ AS3935 Bridge — Tuning · Diagnostics · Test injects` summary → opens. NF/WDTH/SREJ/TUN_CAP rows populate; battery row shows current value; counters tick; test-inject buttons present.
3. Hit a tunable nudge (e.g. NF +) → check journalctl for `lightning/as3935/cmd` publish with the right payload.
4. DXCC Dashboard → click a cluster pill → goes 0.4 opacity + "MUTED" amber text. Confirms global mute is set.
5. DXCC Dashboard → WORKED STATS row "Seed age" reads actual age (e.g. `3h ago`).
6. SPE Panel → push a low-power tune carrier (~3 W) → bar reads reasonable percentage on the 5W rung not stuck near 0.

If any step fails: `git revert 38bf3fb` restores AS3935 Control Panel standalone; `git revert 0153c9a` restores DXCC pre-mute-pill; `git revert b9b727c` restores the coarser SPE ladder.

DXCC Dashboard D1 — layout restructure (settings collapsible, full-width spots)

Operator request to reshape the DXCC Dashboard so the spots table (the thing you actually look at) gets the full card width instead of being squeezed into the right column of a two-column layout.

Old structure (two-column `dx-wrap` grid):

```
[Cluster header pills]
[dx-side: Stats | Refresh/Clear | Filters | Blocked]   [dx-main: spots table]
 ^^^ 220 px fixed sidebar                              ^^^ remainder
```

New structure (vertical stack):

```
[<details>: Stats + Filters + Blacklist (collapsed by default)]
[Cluster header pills (always visible)]
[Spots table — FULL card width]
```

HTML changes:

- Old `dx-wrap` grid container removed entirely
- `dx-side` now lives inside `<details class="dxcc-settings">` with a collapsible `<summary>` "STATS · FILTERS · BLACKLIST"
- `dx-main` (the spots table) is now a flat sibling, full card width
- Cluster header (with the 4 mute-toggle pills from #27) moved DOWN to sit between the collapsed settings and the table — always visible since it's the operational read-out (which cluster is connected / muted)

CSS changes:

- Dropped `.dx-wrap` rule

- `.dx-side` changed from `flex-direction: column` to `display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));` so the four settings cards arrange side-by-side on wide screens and stack on narrow
- `.dx-main` gets `width: 100%`
- New rules for `details.dxcc-settings` — matches GitHub-dark card theme with custom `::before` triangle marker that swaps ▶/▼ on the `[open]` attribute; default `::-webkit-details-marker` hidden

No JS changes — all element IDs preserved (`st-ent` , `st-conf` , `st-seed` , `dx-band-btns` , `bl-input` , `cl-grid` , `dx-tbody` , ...) so existing `scope.$watch` handlers + onclick wiring continue to fire correctly. Cluster mute pills + seed-age display + filter buttons all still work as before.

Commit `ccc05d7` .

Lightning Master Dashboard D1 — Vue-parity trim + narrow-card friendly

Mirror the trims we did to the Vue `/shack` Lightning card on 2026-05-26 (callsign/grid/threshold/reconnect/antenna rows out of the stats grid) over to D1, and reshape the layout so the card lives comfortably at width 8 (≈ 384 px) instead of width 10 (≈ 480 px).

Removed:

- Hidden `switchBox` legacy block (was `display: none` dead code carrying the old vertical antenna/radio/bypass UI; replaced by the `row2` button row months ago but never deleted)
- Stats row: **Threshold** (visible in the Disconnect slider value)
- Stats row: **Reconnect** (visible in the Reconnect slider value)
- Stats row: **Antenna** (already shown by the ANTENNA ON button and by the bypass banner)
- Big **Atmospheric CAPE** display in the right column (was a 28 px number with a label; replaced by a one-line statusline)
- Three separate zone rows (`<thr km` / `<50` / `>50`) merged into one slash-separated row matching Vue

Restructured:

- `row3` was a 2-column grid (`statsBox` 220 px sidebar | `threshBox` right). Now stacked vertically:

  ```
  [stats grid (3 rows: Total / zone-slash / Closest)]
  [CAPE statusline: "CAPE 240 J/kg · OM state: lit"]
  [<details> THRESHOLDS · Disconnect · Reconnect]
  ```

- Sliders now in a collapsible `<details>` (matches Vue's "Thresholds" section)
- CAPE compact statusline replaces big-number display; **OM state** surfaces inline as colour-coded sub-text (cold → grey, lit → amber, severe → red)
- `as3935Body` AS3935 mini-tiles get `repeat(auto-fit, minmax(80px, 1fr))` so the four tiles wrap to 2×2 on narrow card widths instead of overflowing

JS:

- `setStats` rows array trimmed from 8 → 3 (+ Radio if enabled)
- `paintCape` extended to also paint `#omStateVal` sub-text alongside the CAPE value; both live on the new compact statusline

Card geometry: `width 10 → 8` , `height 22 → 18` (removed content frees vertical room).

CSS additions (~60 lines):

- `.status-line` — compact one-line data display, used by CAPE
- `details.thr-details` — Thresholds collapsible matching the GitHub-dark card theme
- `#as3935Body` grid override for responsive tile wrap

Commit `3a68f19` .

Follow-up fine-tuning — three small fixes after operator screenshot

Screenshot showed two glitches in the expanded AS3935 Bridge collapsible:

1. **Double arrow on the summary:** "▼ ▶ AS3935 BRIDGE — TUNING ..." — browser's default disclosure triangle (▼ when open) was rendering alongside my literal `▶` baked into the summary text.
2. **`IRQ 436` clipped** at the right edge — `.countstrip` was `display: flex; gap: 18px` with 4 items, didn't fit at width 8.

Commit `4a2f62b` :

- AS3935 Bridge `<details>` switched from inline-styled (with the literal `▶`) to `class="thr-details"` — picks up the existing CSS that hides the default marker
- Literal `▶` removed from the summary text
- `.countstrip` gained `flex-wrap: wrap` + tighter `gap: 6px 12px` so counters reflow to 2 rows on narrow cards instead of clipping
- `#a35tune` and `#a35ev` inner-panel padding tightened from `10px` → `6px 0` to buy back ~8 px of usable width for the tunable rows

But that commit had a follow-on bug — **no arrow at all** because `.thr-details` only hid the default marker without adding a `::before` triangle replacement (the THRESHOLDS collapsible had a literal `▸` in its summary text which papered over the missing CSS triangle, but the AS3935 Bridge migration dropped that literal as part of un-doubling the arrow).

Fix commit `ff50b2a` :

- Added `::before` triangle to `.thr-details > summary` that swaps `▶ ↔ ▼` on the `[open]` attribute (same pattern as `.dxcc-settings` above)
- Stripped the literal `▸` from the THRESHOLDS summary so it doesn't double up with the new CSS triangle

Net: both collapsibles on the Lightning card (THRESHOLDS + AS3935 Bridge) now show a single proper open/closed triangle that animates correctly. The `.thr-details` class is now the canonical pattern for narrow-card collapsibles in this dashboard — reusable for any future section.

HANDOVER #8 — DXCC backlog audit closure

Per-item audit confirmed every sub-item in #8's row text was already done weeks ago via the parent #18/#19 rows or the original 2026-05-10/11 work. Row was just stale:

Sub-item	Status
Filter persistence	Closed 2026-05-11 (#19)
Separate CW/Ph/Data fetches	`Fetch All Modes + Parse` already `Promise.all`s modes 0..3

Non-project-folder path	`Bootstrap Worked Table` falls back to `~/.node-red/nr_dxcc_seed.json`
README+PDF	DXCC.md + .pdf last committed together `cfea28f`
Club Log API ban verification	Closed 2026-05-11 (#18)
Daily 02:00 inject wiring	Inject `c43fbdd61175ce24` "Daily club log refresh (0200)" with crontab `00 02 * * *` verified live

Closed via `218a11a` with the audit detail captured in the row's closure note for future reference. Open TODOs **5 → 4**.

Dashboard auth — always-on via `httpNodeAuth` + `ui.auth.users` (closes #32)

Original sketch in HANDOVER #32 called for **Caddy + Let's Encrypt + LAN-bypass + reverse proxy** — estimated 2–4 hours of work. Operator clarified the actual ask is much simpler: just **require username/password always**, no LAN-bypass nuance, no off-LAN access yet. That collapsed the design from "reverse proxy with conditional auth" to "flip the two switches Node-RED already exposes."

The two switches:

`~/.node-red/settings.js` ships with two auth blocks commented out (install default):

```
//httpNodeAuth: {user:"user",pass:"$2a$08$..."},
//ui: { path: "ui" },
```

`httpNodeAuth` protects **all `http in` endpoints** — the `/lightning/*`, `/dxcc/*`, `/rotor/*` controls the Vue cards fetch against. `ui:` configures the D1 dashboard, including its own `auth.users` array for credentials.

Implementation (10 min):

Uncomment both blocks, populate with credentials reusing the existing `adminAuth.users[0].password` bcrypt hash (single credential across editor + dashboards keeps Safari's password manager happy):

```
httpNodeAuth: { user: "vu2cpl", pass: "$2y$08$..." },  // reuse adminAuth hash

ui: {
    path: "ui",
    auth: {
        type: "credentials",
        users: [
            { username: "vu2cpl",
              password: "$2y$08$...",  // same hash
              permissions: "*" }
        ]
    }
},
```

Restart Node-RED. Done.

Verification (post-restart):

```
$ curl -s -o /dev/null -w '%{http_code}\n' http://localhost:1880/ui
401         # ← prompts for password ✓
$ curl -s -o /dev/null -w '%{http_code}\n' http://localhost:1880/shack/
401         # ← Vue dashboard also protected ✓
$ curl -s -o /dev/null -w '%{http_code}\n' http://localhost:1880/lightning/threshold
401         # ← HTTP control endpoints also protected ✓
$ curl -s -o /dev/null -w '%{http_code}\n' -u vu2cpl:WRONGPASS
http://localhost:1880/ui
401         # ← wrong password rejected ✓
```

uibuilder `/shack` works without setting `useSecurity: true` on the uibuilder node — its static files + socket.io endpoint are both served through Node-RED's HTTP layer, which `httpNodeAuth` covers uniformly. Once Safari has authenticated for `/shack`, subsequent `fetch()` calls from Vue cards (to `/lightning/*` etc.) automatically include the auth header (browser scopes basic-auth per origin).

Backup on Pi: `~/.node-red/settings.js.bak.20260527_222201` — copy back + restart to roll back atomically if needed.

What this trades off vs the original Caddy plan:

Caddy approach (deferred)	Always-on approach (shipped)
No prompt on LAN, prompt off-LAN	Prompt always (Safari remembers)
HTTPS via Let's Encrypt	Plain HTTP — fine on LAN, **NOT safe over public internet**
Reverse proxy moving part	No new software
~2–4 hours setup	10 min edit + restart

Upgrade path for off-LAN access (not actually needed today but captured for future): add Caddy on top for TLS termination only, no auth logic — Node-RED already does auth. Router forwards 443 → Pi. Strictly upgrade-compatible with what shipped tonight.

Documentation updates:

- **REBUILD_PI.md Step 4** gains a new "Enable dashboard auth" sub-section with the exact `httpNodeAuth` + `ui.auth.users` blocks to paste, the `node-red admin hash-pw` command for generating a fresh bcrypt hash, and the curl-based post-restart verification steps. So a future bare-metal Pi rebuild ships with auth enabled by default, not as an afterthought.
- **FORK_GUIDE.md Stage N** gains a new item #4: forkers must generate THEIR OWN bcrypt hash for their own credentials — `settings.js` is `.gitignore`d so the hash never leaks even if they push the fork publicly.
- **HANDOVER #32** struck through with closure note explaining the design collapse from Caddy → just-flip-the-switches, and the upgrade path if Caddy is ever actually wanted later.

Commit `c53ea0a`.

DXCC Vue card — fix misleading "Worked / Confirmed" stat labels

Operator spotted that the `/shack` DXCC card showed **"Worked 319 / Confirmed 2395"** — which is impossible if both are the same unit (you can't have more confirmed than worked). The two numbers were different quantities mislabelled as the same:

Club Log field	Value	What it actually is
`ws.entities`	319	distinct DXCC **entities** worked
`ws.bandSlots`	2432	total entity-band **slots** worked
`ws.confirmed`	2395	band **slots** confirmed (QSL/LoTW)
`ws.unconfirmed`	37	band slots unconfirmed (2395+37 = 2432 ✓)

The card displayed `entities` under "Worked" and `confirmed` (slots) under "Confirmed" — apples-to-oranges. The D1 dashboard had this right all along (four separate labelled rows: Entities / Band slots / Confirmed ✓ / Unconfirmed ✕); only the Vue card conflated them.

Fix (`88f0f99`):

- `vue_dxcc_builder_01` now emits semantically-named fields: `entities` , `bandSlots` , `confirmedSlots` , `unconfirmedSlots` (legacy `totalWorked` / `totalConfirmed` aliases kept so the collapsed-summary header's "319 worked" still works — entities IS the right unit for that one-glance metric).
- Card statusline relabelled to: `Entities 319 · Slots 2395 / 2432 ✓ · Seed 21h ago` with a hover tooltip spelling out the ratio: "Of 2432 band slots worked, 2395 are confirmed via QSL/LoTW."

Now the two numbers are coherent: 319 entities account for 2432 entity-band combinations, of which 2395 are formally confirmed. Build stamp bumped v7 → v8.

Terminology — "Rotor" → "Rotator" repo-wide (brand "Rotor-EZ" kept)

Operator request for locale/technical consistency. Strictly: a *rotor* is the spinning element inside a motor; a *rotator* is the device that turns an antenna. Hams use both colloquially, but the technical term ages better in shared docs. The Idiom Press **Rotor-EZ** brand name stays as-is.

Substitution (`bc23255`) — safe because "Rotator" (R-O-T-A-T-O-R) doesn't contain "Rotor" (R-O-T-O-R) as a substring, so a plain string-replace doesn't recurse:

```
Rotor-EZ  → KEEP (brand)
ROTOR     → ROTATOR
Rotor     → Rotator
rotor     → rotator
```

Applied to (105 lines across 9 files):

- `flows.json` (38) — tab label, dashboard group, function-node IDs (`rotator_go_fn` , `rotator_stop_fn` , `rotator_lpsp_fn` , `rotator_builder_NN` , `rotator_tick_NN` , `rotator_*_http_in` , `rotator_go_pwr_mqtt` , ...), HTTP-in URL paths, MQTT bridge nodes, variable names, comments, and all `wires` references (renamed atomically by global string-replace so no dangling refs).
- `uibuilder/shack/src/index.js` (22) — Vue component `RotorCard → RotatorCard` , registration + `<RotatorCard/>` template instance, card header text, CSS class refs, and all five `fetch('/rotator/*')` URLs.
- `uibuilder/shack/src/index.css` (27) — `.rotator-stage` , `.rotator-compass` , `.rotator-aside` , `.rotator-timer` , `.rotator-pwr-pill[--on/--off]` , `.rotator-preset-chip[__lbl/__deg/ --active]` , `.rotator-presets-row` , `.rotator-manual` . Selectors stay in sync with the JS class assignments.

- `README.md` , `CLAUDE.md` , `REBUILD_PI.md` , `FORK_GUIDE.md` , `rebuild_pi.sh` — current-state doc references.

NOT applied to: HANDOVER.md "What landed this week" historical rows + SHACK_CHANGELOG.md historical entries — they describe past work using contemporary names; rewriting in place would erase archeological context. Only HANDOVER's current TODO #31 cell was renamed (it describes a future task: now "Rotator → WebSocket gateway", and the future repo will be `vu2cpl/rotator-remote`).

Bonus — endpoint consistency resolved: previously `/rotor/{go,stop,lpsp}` were inconsistent with `/rotator/power-toggle` (a discrepancy flagged in CLAUDE.md for weeks). All four are now `/rotator/*` . Verified post-restart: all four return 401 (auth-gated, route exists); Vue fetch calls + http-in node paths in lockstep.

Mid-task bug recovered: the rename script's sentinel `BRAND_ROTOREZ_KEEP` (used to protect the brand name during substitution) itself contained "ROTOR", so the uppercase pass rewrote it to `BRAND_ROTATOREZ_KEEP` and the final unwind couldn't match the original sentinel — leaving 15 literal sentinel strings across 6 files where "Rotor-EZ" should be. A follow-up pass restored all 15. Lesson: pick sentinels that can't collide with the very patterns you're substituting.

Build stamp v8 → v9, `index.js?v=9` cache-buster. Commit `bc23255` .

DXCC cty.xml — persist parsed maps + bootstrap-from-cache (resilience fix)

Operator caught a real silent-failure mode: cty.xml prefix maps were held in flow context **memory only**, never written to disk. The failure mode:

1. Node-RED restarts (deploy, reboot, package update, anything)
2. `Load cty.xml on startup` inject fires at +5s post-deploy
3. Live Club Log fetch from `cdn.clublog.org/cty.php` fails — DNS hiccup, CDN flap, rate-limit ban, home internet down
4. `Retry cty.xml (60s)` keeps trying every minute in background
5. Meanwhile: `DXCC Prefix Lookup + Alert Classify` early-exits on every spot with status `Waiting for cty.xml...`
6. From the outside the system looks healthy — cluster TCP sessions alive, spots arriving in logs, DXCC dashboard says "online" — but **every spot is silently dropped at the prefix-lookup stage** and no alerts fire

Worst class of failure: no banner, no Telegram alert about the problem, just silent omission. The system is half-resilient — `nr_dxcc_seed.json` (worked/confirmed data) IS persisted and has bootstrap-from-file fallback (closed in #18 weeks ago); cty.xml had the same pattern available but never got it.

Two complementary changes

1. `Parse cty.xml → Prefix Maps` (`5f26764c3416b75e`) now writes parsed maps to `nr_cty_maps.json` after each successful Club Log fetch. JSON cache: `prefixMap` + `exceptions` + `entityNames` + `parsedAt` timestamp + stats. Adds `libs:[fs,os,path]` declaration to the function node. Pattern matches the existing `Fetch All Modes + Parse` → `nr_dxcc_seed.json` write that's been working for weeks.

2. New `Bootstrap cty.xml from file` (`cty_bootstrap_fn_01`) + inject (`cty_bootstrap_tick_01`) at `onceDelay:1` — well before the live fetch at `onceDelay:5` . Reads `nr_cty_maps.json` if present, populates `flow.dxccPrefixMap` / `dxccExceptions` / `dxccSeedNames` immediately. Sets node status to `cached N ent (age X)` with relative-time age display.

Result: **the system always boots with a usable prefix map** (from cache), and the live fetch refreshes it. If the live fetch fails, cached data keeps serving while the retry loop runs in background. First-time deploys with no cache yet: bootstrap is a no-op; live fetch populates from scratch and writes the cache for next time.

Bug recovered mid-deploy

Commit `6a1caa5` shipped with a `require('os').homedir()` call in the bootstrap function's fallback path. Node-RED function nodes don't have `require()` available — the `libs:[]` declaration is the substitute. ReferenceError caught in journalctl:

```
[error] [function:Bootstrap cty.xml from file]
        ReferenceError: ReferenceError: require is not defined (line 22, col 5)
```

This was caught **by the very condition the resilience fix is meant to address** — Club Log was unreachable at restart, the live fetch failed (expected), but the bootstrap also failed (the bug), so DXCC alerts went silent. Fix in `58f7761` : just use the already- imported `os` reference from the libs declaration.

Verified post-fix: next restart fired bootstrap successfully at +1s, read 168 KB cache, populated all three maps. Live fetch at +5s refreshed normally. Spot check from cache: VU → ADIF 324 (India ✓), K → 291 (USA ✓), G → 223 (England ✓). 2934 prefixes, 8987 callsign exceptions, 340 entity names.

Commits `6a1caa5` (resilience fix) + `58f7761` (bug fix).

FORK_GUIDE — full rewrite for clarity

Operator feedback: the existing 605-line guide (Stages A through O) was too complicated to comprehend on a first read. It assumed comfort with git, Node-RED, systemd internals — most ham operators wanting to run this stack don't have that background.

Rewrite shifts (`dd0cfa3`):

- **Drop the "fork on GitHub" framing.** Plain `git clone` is enough for personal use; GitHub account only needed if you want to contribute back. Filename kept as `FORK_GUIDE.md` for backwards compat with existing doc-map links.
- **15 Stages A-O collapsed to 4 numbered Steps** with three named sub-steps. Reader can now hold the structure in head.
- **New "What you'll end up with" intro** so reader knows the destination before the journey starts.
- **Customisation reframed as "The Big Three" edits**: 90% of what you need to change lives in 2 Node-RED nodes (Lightning Init Defaults + DXCC Credentials) plus one settings file (TopBar callsign in `index.js`).
- **Advanced material pushed below.** PWA icon regeneration with `rsvg-convert` + PIL, per-tab hardware deletes, per-cluster edits — all moved to clearly-labelled "Optional" sections that beginners can ignore until they want them.
- **Friendlier language throughout.** "Stage B — Lightning Antenna Protector (Init Defaults)" becomes "3a — Lightning + station identity." Stage-letter system gone.

605 → 401 lines (−204, −33%). New structure:

```
1. What you'll end up with
2. What you need before starting
3. The four steps
   ├─ Step 1 — Get the code (5 min)
   ├─ Step 2 — Install Node-RED and the rest (60 min, one-time)
   ├─ Step 3 — Tell it about YOUR station (30 min, one-time)
```

```
    │   ├─ 3a — Lightning + station identity
    │   ├─ 3b — DXCC credentials (skippable)
    │   └─ 3c — Tell the dashboard your callsign
    └─ Step 4 — Use it (PWA install per platform)
4. Optional — hardware you DON'T have
5. Optional — make the dashboard look like YOUR station
6. Power switching — your Tasmota devices
7. Other clusters / radios / hardware
8. Updating later (just git pull + restart)
9. Auth — set YOUR password, not VU2CPL's
10. Troubleshooting
11. Doc map
12. Where to ask for help
```

The "Where to ask for help" section frames future doc bugs as issues to open against the repo — feedback loop closes naturally.

REBUILD_PI + rebuild_pi.sh — fork-friendly updates + data-files audit

Audit prompt from operator: do REBUILD_PI.md and rebuild_pi.sh need updates given the recent cty.xml resilience fix, and can they be made usable by a forker (not just VU2CPL rebuilding the same Pi)?

REBUILD_PI.md changes

1. **New "If you're following this for YOUR station" callout** at the top with a substitution table — Pi IP, hostname, user, timezone, repo URL. Keeps the runbook concrete (literal values are easier to read than abstract `${USER}@${IP}` placeholders) but makes the substitution rule explicit. Points readers at FORK_GUIDE.md for the deeper customisation pass after the rebuild completes.

2. **"Files referenced" section split into two tables.** The original listed only deployed Pi-side scripts. The new structure:

    - **Pi-side scripts + systemd units** (deployed by Step 7) — the existing list, unchanged.
    - **Runtime data files** (auto-generated; live in flows dir) — new section listing `nr_dxcc_seed.json`, `nr_dxcc_blacklist.json`, `nr_cty_maps.json` (added today by the resilience fix), and `nr_lightning_events.jsonl`. Each row has writer, purpose, rough size, and a note on regeneration vs unrecoverable history.

3. **New verification check #16** under Step 12 — confirms `nr_cty_maps.json` exists post-first-fetch with size > 100 KB and stats matching ~340 entities / ~2900 prefixes / ~9000 exceptions. Validates the resilience path actually wrote its cache; gives a concrete `python3 -c '...'` one-liner for content spot-check. "15-point checklist" → "16-point".

rebuild_pi.sh changes

1. **New "Fork configuration" block** at the top of the script with four `readonly` constants:

    ```
    readonly EXPECTED_USER='vu2cpl'
    readonly EXPECTED_HOSTNAME='noderedpi4'
    readonly REPO_URL='git@github.com:vu2cpl/vu2cpl-shack.git'
    readonly REPO_NAME='vu2cpl-shack'
    ```

Defaults stay VU2CPL's so script behaviour is unchanged for the primary use case. Comments above the block explain what each variable controls (Pi user, hostname, fork URL, project dir name).

2. **Script body now uses these variables** instead of hardcoded strings. 8 substitutions: preflight `id -un` /hostname checks, ssh-keygen comment, `cd ~/.node-red/projects/<repo>`, cp/chown of Pi-side scripts to `/home/<user>/`, chmod, sudoers entry, crontab entry, `usermod -aG telepost`. For a forker, configuring the script is now a 4-line edit at the top of the file.

3. **Left as-is:** preamble docstring (showing VU2CPL defaults descriptively), brand-name reference to the `vu2cpl-as3935-bridge` upstream repo (different repo, brand name).

No code-flow changes — this is a refactor for clarity. The script is still only tested against VU2CPL's setup; forkers should still run with `--status` and `--stage N` to step through carefully on first use. Everything genuinely generic (apt packages, mosquitto LAN config, Node-RED palette install, udev rules for FTDI/HID, file context store, LP-700 server install) applies to any station with zero changes.

Commit `cf109fd`.

2026-06-03 — Manual disconnect = sticky-off (HANDOVER #33)

Operator-flagged safety hole closed. Before this fix: clicking Manual Disconnect on the dashboard correctly cancels the reconnect timer — antenna stays off. **But** if any auto-strike arrives during the manual-off state, `Trigger Disconnect` runs the matrix, fires the disconnect chain, and `Reconnect Timer` re-arms its 20-min countdown behind the operator's back. 20 minutes later → antenna auto-reconnects mid-storm. The 2026-05-26 retrigger-text differentiation masked the bug — Telegram said "STORM CONTINUES" but the timer was still being reset under the hood.

Design (operator-clarified)

- Manual DC is a **sticky** state — `flow.manual_off = true`. Distinct from `antenna_off` (which is "any kind of off"). Distinct from Bypass switch (which is "ignore lightning, keep antenna ON" — the opposite intent).
- Survives Node-RED restart via the `file` context store. Closes a safety hole where pre-emptive DC before any AS3935 hit could be silently cleared by a Node-RED hiccup, letting the operator TX into a disconnected antenna.
- Auto-strikes while `manual_off` is true do **alert-only** with a new event type `auto_strike_while_manual_off`. No MQTT OFF re-send, no timer manipulation, no TX inhibit re-fire. Telegram message uses a new 🟣 MANUAL HOLD format distinct from the existing ⚡ DISCONNECT / ⚡ STORM CONTINUES / ✅ RECONNECT alerts.
- Bypass-ON **clears** `manual_off` — operator hitting Bypass is explicitly overriding the manual hold. Falls out naturally from Execute Reconnect clearing manual_off (Bypass Handler output 3 wires to Execute Reconnect).

Implementation — 6 surgical edits

Node	Change
`Manual Override` (`22e5df9713499f53`)	Adds `flow.set('manual_off', !on, 'file')` next to the existing antenna_off set.
`Trigger Disconnect` (`d62fb0c3c40f03b7`)	New early-exit after the bypass guard. When `manual_off === true`, emits `event_record` of type `auto_strike_while_manual_off` on a **new output 2** wired

	to `light_jsonl_append_01`. Fire-disconnect return updated to 3-output form `[msg, {payload:'reset'}, null]`. Node `outputs` bumped 2 → 3.
`Force Reconnect (dfad237cceca95b4)`	Adds `flow.set('manual_off', false, 'file')`.
`HTTP → Antenna ON (f2092c6e0d932c7b)`	Adds `flow.set('manual_off', false, 'file')`.
`Execute Reconnect (bfbe99e98a8c6ce8)`	Adds `flow.set('manual_off', false, 'file')`. Covers both timer-expiry and Bypass-ON paths.
`Telegram Alert Router (tg_lightning_router)`	Allow-list adds `'auto_strike_while_manual_off'`. Formatter renders 🟣 MANUAL HOLD with distance / source / "Antenna remains OFF (manual); TX still inhibited" / time.

No new nodes, no new endpoints, no new dashboard buttons. The existing Manual Override toggle already drives the new behaviour.

Scope alignment

Per HANDOVER #19 (DXCC filter persistence) lesson — all `manual_off` reads use `flow.get('manual_off', 'file')` and all writes use `flow.set('manual_off', value, 'file')`. The `file` context store is already configured in `settings.js` from the same #19 work. `flow.get(...)` on a never-written key returns `undefined`, which is treated as falsy → falls through to the matrix. Safe on a fresh install.

Test plan

1. Click Manual Disconnect on the dashboard → antenna OFF, timer shows "Cancelled", `flow.manual_off === true` in file context.
2. Fire TEST close-zone strike inject → Trigger Disconnect status shows "MANUAL OFF · TEST 5km — alert only", no MQTT to powerstrip (verify on `cmnd/powerstrip1/POWER5` topic), no timer countdown, Telegram fires with 🟣 MANUAL HOLD message.
3. Click Manual Reconnect → antenna ON, `manual_off === false`.
4. **Key safety test:** set Manual Disconnect → `sudo systemctl restart nodered` → check `flow.get('manual_off','file')` still `true` after restart → fire TEST inject → still alert-only.
5. Set Manual Disconnect → click Bypass-ON → antenna ON, `manual_off === false`, TX uninhibited.

Commit `82c85ea`. Pi restarted 2026-06-03 08:05 IST, all flow connectors (4 cluster TCPs, 2 MQTT brokers, rotator serial, FlexRadio discovery) came back up clean. cty.xml bootstrap-from-cache also fired (yesterday's resilience work proving itself across a restart).

2026-06-03 — Dashboard ANT toggle drives manual_off (second pass)

Yesterday morning's manual_off plumbing was correct but **unreachable from the UI** — the only existing antenna control was the one-way "ANTENNA ON" button (D1) and a read-only "ANT ON / ANT OFF" status chip (Vue collapsed view). There was no way for the operator to manually disconnect the antenna from either dashboard. The orphan `Manual Override` function node had been in flows.json since some prior UI cleanup, but nothing was emitting the `topic:'manual_override'` message it required, so it was dead code.

This commit makes the manual_off path reachable.

Design

Per operator's "keep it simple" constraint: **2 controls only** on the Lightning Master Dashboard — `[ANT toggle]` + `[BYPASS toggle]`. No third "Manual Override" button (the historical UI that was removed). The orphan Manual Override function node is deleted.

The single ANT toggle:

- **D1 Master Dashboard:** existing "ANTENNA ON" button replaced with bidirectional `antToggle`. Green border + "ANTENNA ON" when antenna is on; red border + "ANTENNA OFF" when off. Click → confirm dialog → fetch.
- **Vue /shack LightningCard:** the **collapsed-view ANT chip** in the card header is now clickable (`@click.stop` so it doesn't also expand the card). The **expanded-view ANTENNA ON button** also becomes bidirectional (`btn--green` ↔ `btn--red`). Both call the same `action('antennaToggle')` handler.

Confirm dialog wording:

- Going OFF → "Disconnect antenna?"
- Going ON → "Reconnect antenna?"

Matches the existing SPE + Power Control button confirm pattern — one extra tap prevents accidental disconnect mid-QSO from a stray mobile tap.

Backend — symmetric ant-on / ant-off chain

Endpoint	What it does
`POST /lightning/ant-on` (modified)	Sets `antenna_off=false`, `manual_off=false`, fires Tasmota ON, cancels Reconnect Timer, clears TX inhibit. **NEW:** emits `manual_on` event_record on output 3 → JSONL → Telegram. Outputs bumped 3 → 4.
`POST /lightning/ant-off` (NEW)	Mirror of ant-on. Sets `antenna_off=true`, `manual_off=true` (file scope, sticky), fires Tasmota OFF, cancels Reconnect Timer, fires TX inhibit. Emits `manual_off` event_record on output 3 → JSONL → Telegram.

3 new nodes added on Lightning tab: `lightning_antoff_http_in_01` , `lightning_antoff_fn_01` , `lightning_antoff_res_01` . Same shape as the existing ant-on chain.

Telegram alerts — 2 new event types

- `manual_off` → 🟣 "ANTENNA OFF (manual) · Operator disconnected via dashboard · Antenna: OFF · TX inhibited · Time …"
- `manual_on` → ✅ "ANTENNA ON (manual) · Operator reconnected via dashboard · Antenna: ON · TX allowed again · Time …"

The 🟣 MANUAL HOLD alert (for auto-strikes while operator has manually disconnected) shipped in the first pass and is unchanged — distinct from the `manual_off` alert (which fires *when* operator hits the toggle) vs MANUAL HOLD (which fires when a strike arrives *during* manual-off state).

Deletions

- `Manual Override` function node `22e5df9713499f53` deleted from the Lightning tab.

- Master Dashboard's outgoing wires updated — `22e5df9713499f53` removed from its output 0 wire list (now wires to `88ce7a0b9c27747d`
  - `82f732a0dac14945` only).
- D1 Master Dashboard: dead `_antOn` JS handler removed. Orphan `antStatus` element references already gone since the 2026-05-27 switchBox cleanup (`setAnt()` was writing into the void); now `setAnt()` drives the live `antToggle` element.

Files touched

- `flows.json` — 1 node deleted + 3 nodes added + 3 nodes modified (ant-on, Master Dashboard, Telegram Router).
- `uibuilder/shack/src/index.js` — LightningCard template (collapsed chip + expanded button) + action handler. Build stamp v9 → v10.
- `uibuilder/shack/src/index.css` — new `.ant-chip` hover style.
- `uibuilder/shack/src/index.html` — cache buster `index.js?v=9 → 10` and `index.css?v=3 → 4`.

Test plan

1. **D1 dashboard load** — Lightning card shows green "ANTENNA ON" button.
2. **D1 click while ON** → confirm "Disconnect antenna?" → confirm OK → button flips to red "ANTENNA OFF" within ~1s (waiting for stat_ant message echo). Tasmota plug actually turned off (verify via Power Control tab or `mosquitto_sub` on `stat/powerstrip1/POWER5`). Telegram: 🟣 ANTENNA OFF (manual) message.
3. **D1 fire TEST close-zone strike** → status badge shows "MANUAL OFF · TEST 5km — alert only". Telegram: 🟣 MANUAL HOLD message. No second disconnect; no MQTT to powerstrip; no timer.
4. **D1 click while OFF** → confirm "Reconnect antenna?" → OK → button flips back to green "ANTENNA ON". Tasmota plug on. Telegram: ✅ ANTENNA ON (manual).
5. **Vue collapsed view** — close the LightningCard (click chevron). The "ANT ON" / "ANT OFF" chip in the header is now clickable. Hover shows the `.ant-chip` highlight + tooltip. Click on collapsed chip → confirm → toggles state. Card does **NOT** also expand (the `@click.stop` does its job).
6. **Vue expanded view** — same toggle behaviour on the big button.
7. **Restart safety test** (the safety-critical case): set ANT OFF on dashboard → `sudo systemctl restart nodered` → after restart, button should still render red "ANTENNA OFF" (state.antennaOn stays false because `flow.manual_off` is `true` in file context). Fire TEST inject → still alert-only.

Edge cases handled

- Initial state on D1 dashboard load (before first stat_ant message arrives): button defaults to green "ANTENNA ON" — matches pre-change behaviour.
- Vue `@click.stop` on the collapsed chip prevents the click from also bubbling up to the `card__header` 's `@click="expanded = !expanded"` handler.
- `confirm()` dialog uses native browser modal — works in Safari (operator's daily-driver per global CLAUDE.md). Blocks until responded; cancel does nothing.

Commit `beb82d9` . Pi restarted 2026-06-03 08:53 IST, all flow connectors back up clean.

2026-06-03 — FORK_GUIDE: path bug fix + upgrade-from-old-version scenario

Operator-reported feedback from forkers: "even after rewriting, other users not able to update the dashboard."

Diagnosed bug — path mismatch

FORK_GUIDE Step 1 told forkers to clone to `~/vu2cpl-shack/` (in their home directory), but `rebuild_pi.sh` clones to `~/.node-red/projects/vu2cpl-shack/` — and **that's where Node-RED actually serves `/shack` from**. All the editing instructions in the guide referenced the wrong path (`~/vu2cpl-shack/uibuilder/...`).

Forkers ended up with two copies of the repo on their Pi: one in `~/` they edited and one in `~/.node-red/projects/` that Node-RED actually used. Refreshing `/shack` after editing the home-dir copy showed no change, and forkers concluded "dashboard updates don't work" — the silent footgun.

Fix — single-path everywhere

Section	Change
Step 1	Clone now targets `~/.node-red/projects/vu2cpl-shack/` directly (via `mkdir -p ~/.node-red/projects && git clone … ~/.node-red/projects/vu2cpl-shack`). Added a callout for existing forkers who cloned to the wrong location, with `mv` / `rm -rf` recovery commands. The "to update later" sub-block in Step 1 now points at the correct path.
Step 3c (TopBar callsign)	Editing path updated.
Optional rebrand (icon.svg)	Two path references updated.
Updating later	Cleaned up — was telling forkers to `cd ~/vu2cpl-shack` first, then later `ssh + cd ~/.node-red/projects/...` (the dual-path nonsense). Now one block, one path, with the restart-only-if-flows-changed note and the hard-refresh hint. Cross-references the new upgrade-scenario section for conflict handling.

`rebuild_pi.sh` itself didn't need any changes — its `[[ -d "$REPO_DIR/.git" ]]` check (line 359) already handles the pre-cloned case idempotently.

New section — "Already running an older version? Just want the latest dashboard"

Operator's second ask: a forker is running an older version of the stack and wants to upgrade just the dashboard / flow code without redoing the whole REBUILD_PI / Step 3 customisation cycle.

The new section walks through five steps:

1. **Find where your repo lives** — covers both the correct location and the wrong-old-location (`~/vu2cpl-shack/`) recovery with `sudo systemctl stop nodered && mv … && start nodered` .
2. **Note your local edits** — `git status` + conditional `git stash` if anything's unstaged. Distinguishes "never customised" (30s pull) from "customised" (5-min conflict-resolution flow).
3. **`git pull`** — happy path (clean merge → jump to step 5) vs conflict path (`CONFLICT (content): Merge conflict in flows.json`).

4. **Resolve the flows.json conflict** — recommends the simplest path: `git checkout --theirs flows.json` (accept upstream wholesale), then re-apply customisations via the Node-RED editor's Init Defaults node. Optional `git stash pop` workflow for forkers who want to recover their old values for reference.
5. **Restart Node-RED + hard-refresh** — `sudo systemctl restart nodered`, then Safari Shift+reload for desktop / Settings → Safari → Website Data → delete for mobile / re-add to home screen for PWA. Verifies the upgrade via the new build stamp in the dashboard footer.

Plus a "Things to verify after upgrade" 5-row checklist (flow loaded clean / Init Defaults values stuck / dashboard shows your data / new endpoints registered / Tasmota topics intact) and a "When NOT to use this short path" callout pointing forkers at REBUILD_PI follow-up when the upgrade includes new palette nodes or new systemd services, with `git log --oneline @{1}..@` to inspect what landed.

Why the new section helps

The original FORK_GUIDE assumed forkers were doing a fresh install. Forkers using a stable-ish setup don't want to redo Step 3 every time VU2CPL pushes a feature — they want a 5-minute upgrade path. The new section closes that gap while being explicit about when the short path isn't enough (palette / service changes need REBUILD_PI follow-up).

Doc-only change — no Pi restart needed; the Pi will pull this as part of the normal doc CDP cycle.

2026-06-03 — FORK_GUIDE: comprehensive end-to-end rewrite

Operator escalation: "WHAT RUBBISH, I NEED A COMPREHENSIVE GUIDE. NOT BITS AND PIECES, I HAVE ASKED AT LEAST 3-4 TIMES IN AS MANY DAYS." Across multiple recent sessions I'd been patching FORK_GUIDE incrementally — path bug fixed earlier today, then upgrade scenario added, then palette gap surfaced when the operator spot-checked. Each pass exposed another gap because the document wasn't designed to be exhaustive from the start. Time to stop bandages and ship one self-contained document.

Approach

Rewrote FORK_GUIDE.md as a **single end-to-end guide** that no longer punts to REBUILD_PI.md for the meat of the install. A forker can read top-to-bottom and complete: fresh install, upgrade, customization, troubleshooting — without bouncing to other files (except E4 doc map for deeper reference).

401 lines → 807 lines, but the depth is what was missing. Sections are gated ("skip if not relevant") so the actual reading path for any single user is shorter than 807 lines.

New structure

```
# Running this shack stack at your station
## Quick start (3-line cheat sheet for impatient experts)
## What you end up with (feature inventory)
## Before you start (hardware + credentials + decisions checklists)

# Part A — Fresh install (90 minutes)
  A1. Get the Pi ready (Pi OS, SSH, network)
  A2. Clone the repo to the correct location
  A3. Edit rebuild_pi.sh for your station (4-line fork config)
  A4. Run the install script (13 stages, each described)
```

```
      + "What to do if a stage fails" subsection
  A5. Tell it about YOUR station
      A5.1 Init Defaults (Lightning + identity)
      A5.2 DXCC credentials (skip if no DX clusters)
      A5.3 Dashboard callsign (Vue TopBar)
      A5.4 Tasmota device topics
  A6. Install the PWA on each device
  A7. Verify the install

# Part B — Upgrade an existing install
  B1. Find where your repo lives (correct vs wrong path recovery)
  B2. Save your customizations
  B3. Pull the latest
  B4. Refresh palette nodes (bash rebuild_pi.sh --stage 4)
      + WHY palette ≠ flows.json explanation
  B5. Refresh Pi-side scripts and systemd units (if needed)
  B6. Resolve flows.json conflicts
  B7. Restart + hard-refresh
  B8. Verify after upgrade

# Part C — Hardware variations
  C1. Hardware you DON'T have (per-card removal table)
  C2. Different cluster hosts, FlexRadio model, etc.

# Part D — Optional customization
  D1. Rebrand: icons, name, colors
  D2. Add your own card

# Part E — Reference
  E1. What lives where (cheat sheet) — file paths inventory
  E2. Auth & secrets — what's safe to push
  E3. Troubleshooting (13-row symptom→fix table)
  E4. Doc map
  E5. Where to ask for help
```

What's new vs the previous version

- **Palette refresh gap closed** — Part B4 now explicitly explains that palette nodes don't come from flows.json, gives the exact command (`bash rebuild_pi.sh --stage 4`), and documents how to detect when it's needed (`journalctl … grep "Unknown type"`).
- **Pi-side scripts / systemd / udev refresh gap closed** — Part B5 covers the rarer upgrades that need stages 9/10/11 re-run.
- **Quick start cheat sheet** at the top for experienced installers who just want the 3 commands.
- **Hardware + credentials + decisions checklists** up front so the forker can prep before starting.
- **13 install stages each described** in Part A4 in 2-3 sentences with typical time, instead of punting to REBUILD_PI.md.
- **"What lives where" cheat sheet** (E1) with every file path including runtime data files (`nr_dxcc_seed.json` , `nr_cty_maps.json` , etc.) — answers the "where's my data?" question forkers always have.
- **13-row troubleshooting table** (E3) covering the symptoms operators have actually reported: dashboard shows —, auto-DC doesn't fire, DXCC cluster red, Telegram silent, PWA icon stale, Power-strip state doesn't sync, /shack 404, editor login fails, HTTP 401, D1 vs /shack auth split,

`Unknown type` errors, `ENOENT` runtime data, antenna stuck OFF after reboot (manual_off sticky behaviour — points at the dashboard toggle), stage failures, Cmd+Shift+R doesn't work in Safari.
- **manual_off sticky behaviour documented** in troubleshooting — operators will hit this after a Pi reboot if they had manually disconnected. The 'antenna stuck OFF' entry explains it's intentional and points at the dashboard toggle.
- **Hard refresh instructions per browser** — Safari Shift+click, Chrome `Cmd/Ctrl+Shift+R`. Explicit "Cmd+Shift+R doesn't work in Safari" troubleshooting row.

What this guide deliberately does NOT cover

- Adding new flows from scratch in Node-RED (referred to CLAUDE.md for developer reference).
- Modifying existing flows beyond editing constants in Init Defaults.
- Vue component development beyond the "copy an existing card as a template" pattern (D2).

These are out of scope for a forker who just wants to run the stack. The doc map (E4) routes them to CLAUDE.md and SHACK_CHANGELOG.md.

Why this took so long

The previous rewrites (605 → 401 lines, then incremental patches) were optimizing for brevity. But forkers needed *exhaustive*, not brief — they need every step spelled out because they can't tell which step is the one they're missing. The 807-line version is longer but each forker reads only the part relevant to them (fresh install OR upgrade OR rebrand), so the *effective* length they encounter is similar to the previous version while the coverage is much more comprehensive.

Doc-only change. No Pi restart needed.

2026-06-03 — rebuild_pi.sh: NEW Stage 13 — interactive station customization

Follow-on to the comprehensive FORK_GUIDE rewrite (`97a942c`). Forker friction in Part A5 (Init Defaults editing in the Node-RED editor + TopBar editing via SSH+nano) was the largest remaining manual step after `rebuild_pi.sh` finished. This commit closes that gap into the script.

What was wrong

Stages 1–12 of `rebuild_pi.sh` brought a forker from blank Pi to a running Node-RED with the full stack of palettes, services, udev rules, and secrets. Stage 13 (verify) reported pass/fail. **But the dashboard still showed VU2CPL's callsign, grid, and MQTT broker IP everywhere** because Init Defaults values are in `flows.json` (committed) and TopBar callsign is in `index.js` (committed). Forkers had to open the editor, find the right node, edit constants, deploy — then SSH back in, open `index.js` , find TopBar, edit it, hard-refresh. The FORK_GUIDE walked through it but it was still 15 minutes of error-prone manual work per fresh install.

Design

Inserted a new **Stage 13 — station customization** between secrets (12) and verify (now 14). Stage 13 is interactive, runs as part of the main pipeline (so forkers experience it right after stage 12 without thinking about it), and is idempotent so re-runs are safe.

Step	Behaviour

Idempotency check 1	Reads current `CALLSIGN` from Init Defaults via Python. If it's already non-VU2CPL (forker already ran this), prints "already customised" and marks done.
Idempotency check 2	If the script's CONFIG block (`EXPECTED_USER`, `EXPECTED_HOSTNAME`) still matches VU2CPL's defaults AND `CALLSIGN` is `VU2CPL`, this is VU2CPL's own Pi — no customization needed. Marks done and skips. **This is what prevents the upstream operator from being prompted to re-customize their own Pi when running the updated script.** Forkers who edited the CONFIG block (which the new FORK_GUIDE A3 instructs them to do) will not match this and will get the prompts.
Prompts	Callsign (alphanumeric + slash, 3–10 chars, force uppercase) · 6-char Maidenhead grid (regex-validated, force `XX99xx` casing) · MQTT broker IP (default: Pi's own LAN IP via `hostname -I`) · Tasmota antenna power-strip topic · Antenna `POWER1..POWER5` channel · Disconnect threshold km (default 40) · Reconnect timer minutes (default 20) · QTH text for the TopBar sub line
Validation	Bash `while` loops reject invalid input with red error message and re-prompt. Defaults shown in the prompt text; pressing Enter accepts the default.
Confirmation	Summary of all values printed, then `[Y/n]` confirm before any file is touched.
Backups	`flows.json` and `uibuilder/shack/src/index.js` copied to `.bak.YYYYMMDD_HHMMSS` before patching.
Patching	Python heredocs (env-var bridge for safety) perform regex substitution on the Init Defaults function body (7 const declarations) and the Vue TopBar (`<span class="callsign">` + `<div class="sub">`). Optional patch of `manifest.json` `name` / `short_name` for the PWA install.
Cleanup	Patch env vars unset. Final messages remind the operator to `sudo systemctl restart nodered` once the rest of the install completes.

FORK_GUIDE.md follow-up

- **A4 stage table** — row 13 is now "station customisation" with detailed description; old verify row renumbered to 14.
- **A5 preamble** — new callout: "Most of this section is now automated by Stage 13" pointing forkers at A5.2 (DXCC creds — still manual since secrets.conf is its own path) and A5.4 (Tasmota topics for non-antenna devices — too device-specific to prompt for during install) as the remaining manual steps.
- **A5.1** and **A5.3** headers gain "— manual fallback" suffix with a callout: "Stage 13 does this automatically. Read on only if you skipped Stage 13, or want to change values later." Sections themselves are unchanged so forkers who hit edge cases (script failed at stage 13, or they want to change values months later) still have the manual instructions intact.

Migration for existing installs

Where you are	What happens when you `git pull` + re-run `bash rebuild_pi.sh`
VU2CPL's own Pi (operator: this includes `noderedpi4`)	Stage 13 detects defaults match + callsign is VU2CPL → marks done + skips. Stage 14 (verify) re-runs once (state file has the old `13_verify` entry; new STAGES array has `14_verify` which is unmarked). Verify is idempotent; takes 2 minutes. No prompts. No file changes.

Forker who already customised manually (callsign != VU2CPL in Init Defaults)	Stage 13 detects already-customised → marks done + skips. Stage 14 re-runs verify. No prompts.
Forker on a fresh install with new script	Stage 13 prompts → patches → done. The painful A5.1+A5.3 manual steps are gone.

Safety + reversibility

Backups always written before any change. If the patch goes wrong (values rejected by Node-RED, or operator picks bad values):

```
cd ~/.node-red/projects/vu2cpl-shack
ls -lt flows.json.bak.* | head -3           # find your backup
cp flows.json.bak.<timestamp> flows.json
cp uibuilder/shack/src/index.js.bak.<timestamp> uibuilder/shack/src/index.js
sudo systemctl restart nodered
```

Or just re-run `bash rebuild_pi.sh --stage 13` with corrected values — the new values overwrite (after one more backup).

Why this matters

A forker who used to spend ~15 minutes manually editing Init Defaults in the Node-RED editor + opening SSH+nano to patch TopBar now spends ~90 seconds answering Stage 13's prompts. The friction that triggered "WHAT RUBBISH, I NEED A COMPREHENSIVE GUIDE" was exactly this — the gap between "the script finished" and "the dashboard says my callsign." With Stage 13, the script genuinely finishes the install end-to-end for the common case.

Commit `9be2bde`. Doc + script change; no Pi restart needed.

2026-06-03 — REBUILD_PI.md: fix "Faster path" clone target (was /tmp/, now correct)

Spotted by the operator while re-reading docs after the Stage 13 rollout: the REBUILD_PI.md "Faster path — the rebuild script" section told forkers to clone to `/tmp/vu2cpl-shack/` and run the script from there. Wrong on two counts:

1. **`/tmp/` is wiped on reboot.** Any local edits the forker made to `rebuild_pi.sh` (the new FORK_GUIDE Part A3 fork-config block, in particular) would vanish at first reboot.
2. **The script's stage 7 clones AGAIN to `~/.node-red/projects/vu2cpl-shack/`.** So the forker ends up with **two copies** of the repo, with the `/tmp/` clone's edits sitting in a different directory than the canonical clone. Subsequent stages `cd $REPO_DIR` operate on the projects-dir clone, not the `/tmp/` one — so anything the forker thought they'd customised wouldn't apply.

This directly contradicted the new FORK_GUIDE A2 (clone to `~/.node-red/projects/vu2cpl-shack/`), exactly the kind of doc-drift the comprehensive rewrite was meant to eliminate.

Fix

`REBUILD_PI.md` "Faster path" block now reads:

```
mkdir -p ~/.node-red/projects
git clone https://github.com/vu2cpl/vu2cpl-shack.git \
  ~/.node-red/projects/vu2cpl-shack
bash ~/.node-red/projects/vu2cpl-shack/rebuild_pi.sh
```

Added two callouts:

1. **"Why this exact path?"** — explains the second-clone-from-stage-7 trap and why single-path-from-the-start matters.
2. **"Forking?"** — points at FORK_GUIDE A3 for the 4-line CONFIG block edits needed before running the script. Without those edits Stage 13 auto-skips and the dashboard stays branded as VU2CPL's.

Also bumped the "script pauses for **two** interactive steps" line to three, adding Stage 13 (station customisation) to the list with the auto-skip-on-VU2CPL-Pi note.

Single doc edit, no other files. CDP cycle wraps with the existing HANDOVER tip pointer at `9be2bde` (still the most recent code change); this commit moves it to whatever the upcoming hash is.

2026-06-03 — Cross-doc consistency audit + sweep

Operator escalation (rightly): "why are we having these kind of errors still?????" — after the `/tmp/` REBUILD_PI fix landed. The pattern across 24 hours had been "user reports bug → I patch one file → user spot-checks → finds next bug → repeat." Each patch was tactically correct but I never did a holistic sweep of ALL docs after the big changes (Stage 13, ANT toggle, manual_off, ant-off endpoint, Manual Override deletion, FORK_GUIDE rewrite).

What the audit found

#	Bug	Where	Cause
1	`Manual Override` listed as live source of TX inhibit setter	`CLAUDE.md` "FlexRadio TX inhibit chain" sources list	Node deleted today (commit `beb82d9`) but the architectural doc not updated; missing `HTTP → Antenna OFF` as the new 5th source
2	`manual_off` cleared by "Manual Override ON"	`CLAUDE.md` "Manual disconnect = sticky-off" subsection	Same root cause — Manual Override deleted but the clearing-paths sentence still listed it
3	OPEN BUGS table only listed #1	`CLAUDE.md` "OPEN BUGS / PENDING TODO"	Stale ever since #6 and #31 were opened (May/June 2026); never re-synced with HANDOVER #1/#6/#31
4	`13 stages` should be `14 stages`	`FORK_GUIDE.md` line 194	Stage 13 added today; the prose count next to the stage table wasn't updated
5	`Stages 2–13` should be `Stages 2–14`	`REBUILD_PI.md` line 50	Same root cause as #4
6	`/tmp/vu2cpl-shack` clone target	`REBUILD_PI.md` (fixed in earlier commit `cb3403b`)	Pre-existing wrong path that survived two recent rewrites

7	`~/vu2cpl-shack/` editing paths	`FORK_GUIDE.md` (fixed in earlier commit `2caceeb`)	Pre-existing wrong path

Bugs 1–5 fixed in this commit. Bugs 6–7 were fixed earlier today.

Structural cause

Three contributing factors:

1. **No automated cross-doc drift checker.** Every doc-related commit could in principle leave another doc stale.
2. **Multiple docs cover overlapping territory.** FORK_GUIDE, REBUILD_PI, CLAUDE, and README all reference paths, stages, nodes, and endpoints — every change to one is a drift candidate for the other three.
3. **High change velocity in the last 24 hours.** Stage 13 added, Manual Override deleted, ANT toggle added, `manual_off` made reachable via dashboard, FORK_GUIDE rewritten end-to-end, the `/tmp/` path bug fixed — six structural changes in one day, each one a drift-creator.

Going forward — audit checklist as standard CDP step

Adding a **cross-doc consistency audit** to the CDP rule for any substantive change. Before pushing docs, grep across all `*.md` files for:

Audit	grep target	Why
Path consistency	`/tmp/vu2cpl-shack`, `~/vu2cpl-shack` (without `.node-red`), other ad-hoc paths	Catch wrong-location clone instructions
Stage count drift	`13 stages`, `Stages 2.13`, `14 stages`, `Stages 2.14`	Catch stage-count text/table mismatch when Stage 13 customisation was added
Deleted nodes still referenced	Node ID literals (e.g. `22e5df9713499f53`), node names (e.g. `Manual Override`)	Catch architectural docs referencing nodes that no longer exist
Stale interaction-count	`two interactive`, `pauses for two`	Catch interactive-prompt count drift when Stage 13 (3rd prompt point) landed
New features under-documented	New endpoint URLs (e.g. `/lightning/ant-off`), new node IDs (e.g. `lightning_antoff_fn_01`), new event types (e.g. `manual_on`, `manual_off`, `auto_strike_while_manual_off`)	Catch CLAUDE.md sections that summarize the architecture not getting the new entry
TODO list drift	`^| [0-9]+ |` in `CLAUDE.md` "OPEN BUGS" vs `HANDOVER.md` "Open follow-ups"	Catch the two TODO lists getting out of sync

The exact grep patterns are recorded in this entry so future sessions can re-run the audit programmatically. When a major change lands, this audit runs as the last step of the CDP cycle **before** the final `git push`.

Self-correction

To the operator: I should have done this audit when you said "comprehensive" for FORK_GUIDE. Going forward, "comprehensive" means the entire doc set, not just one file. The CDP rule's "Document" step is now codified to include this cross-doc grep before push.

2026-06-03 — rebuild_pi.sh: drop EXPECTED_USER + EXPECTED_HOSTNAME (auto-detect)

Operator: "why insist on vu2cpl username and hostname as noderedpi4? in the rebuild script?"

Real design bug, not a typo. The script's preflight() **failed** hard if `id -un != 'vu2cpl'`, blocking any forker from running the script until they edited the CONFIG block. The fix the operator's earlier Stage 13 design relied on (`EXPECTED_USER == 'vu2cpl'` + `EXPECTED_HOSTNAME == 'noderedpi4'` → "upstream's own Pi, skip prompts") was conflating two concerns: what user/hostname the Pi *actually* uses (should auto-detect) versus whether this is upstream's own Pi (relevant only to Stage 13 idempotency).

What changed in the script

Before	After
`readonly EXPECTED_USER='vu2cpl'` (forker must edit before script will run)	**dropped**
`readonly EXPECTED_HOSTNAME='noderedpi4'` (used for preflight + Stage 13 idempotency)	**dropped**
`readonly ACTUAL_USER="$(id -un)"`	**new — auto-detected**
`readonly ACTUAL_HOSTNAME="$(hostname)"`	**new — auto-detected**
preflight: `fail` if `id -un != EXPECTED_USER`	preflight: `fail` only if running as root; otherwise pass through
preflight: `warn` if `hostname != EXPECTED_HOSTNAME`, prompt continue	preflight: log hostname for info; new best-effort Raspberry Pi detection via `/proc/cpuinfo` + `/proc/device-tree/model` warns (doesn't block) if not on a Pi
11 internal references to `$EXPECTED_USER` (sudoers, ssh-key comment, file copy, chown, chmod, crontab, telepost group, etc.)	All substituted to `$ACTUAL_USER`
Stage 13 idempotency: `EXPECTED_USER == 'vu2cpl' && EXPECTED_HOSTNAME == 'noderedpi4' && CALLSIGN == 'VU2CPL'` → skip	Simplified: `ACTUAL_USER == 'vu2cpl' && CALLSIGN == 'VU2CPL'` → skip. The hostname check went away — if a forker happens to also have hostname `noderedpi4` (unlikely with custom Pi OS imager hostnames) but a different user, the user-mismatch is enough to trigger prompts. If a forker has user `vu2cpl` AND callsign still `VU2CPL` (i.e., they haven't customised), the prompt skip is still incorrect but they'll notice the dashboard branding and re-run `--stage 13`.

CONFIG block now only has `REPO_URL` + `REPO_NAME` — and the comments are explicit: "**you do NOT need to edit user or hostname**. The script auto-detects whatever user you're SSH'd in as." Effectively only

`REPO_URL` is meaningful, and only for forkers who have their own GitHub fork (most don't bother — `git pull` from upstream works fine).

Preamble docstring also updated — the old "Pre-requisites: Hostname `noderedpi4`, user `vu2cpl`, SSH access" line replaced with "SSH access as a regular (non-root) user. Whatever username and hostname your Pi has is what this script configures the Pi for."

What changed in the docs

Doc	Change
`FORK_GUIDE.md` "Decisions you'll make during install"	Was "Change them in the script's CONFIG block BEFORE running" with 4 variables shown. Now: "You don't need to pre-configure the script. It auto-detects your Pi's username and hostname … One optional edit if you happen to have your own GitHub fork: `REPO_URL`."
`FORK_GUIDE.md` Part A3	Was "Edit `rebuild_pi.sh` for your station (2 minutes)" with 4-line edit walkthrough. Now "(Optional) Edit `rebuild_pi.sh` for your GitHub fork (1 minute)" with a single `REPO_URL` edit; opens with "**Most forkers skip this step entirely.**"
`REBUILD_PI.md` "Forking?" callout	Was "edit the **Fork configuration** block near the top of `rebuild_pi.sh` (4 `readonly` constants: …)". Now "The script auto-detects your Pi's username and hostname — no pre-config edits required. … The only reason to edit `rebuild_pi.sh` is if you have your own GitHub fork."

Backwards compatibility

For VU2CPL (operator on `noderedpi4` as user `vu2cpl`):

- Auto-detected `ACTUAL_USER='vu2cpl'`, `ACTUAL_HOSTNAME='noderedpi4'`
- preflight passes silently (not root)
- Internal paths unchanged (`/home/vu2cpl/...`, etc.)
- Stage 13 idempotency: `ACTUAL_USER == 'vu2cpl'` AND callsign still `VU2CPL` → skip. **Same behaviour as before** for the operator.

For a forker on their own Pi:

- Auto-detected `ACTUAL_USER='kc1abc'` (or whatever), `ACTUAL_HOSTNAME='myshack'`
- preflight passes (was failing before forcing them to edit CONFIG)
- Internal paths: `/home/kc1abc/...`, sudoers for `kc1abc`, etc. **All derived automatically.**
- Stage 13 prompts fire (user != vu2cpl → idempotency check fails → prompt)

The script is now genuinely "git clone + bash rebuild_pi.sh" for forkers, no pre-edits required.

Commit `56b0c87`. Doc + script change; no Pi restart needed.

2026-06-03 — rebuild_pi.sh: Pi-OS-Imager `pi` user trap + state file relocation

Operator hit two real fresh-install failures on `noderedpi5` while testing the `--auto-detect-user` script changes. Both are now covered in script.

Failure 1 — Node-RED installed for the wrong user

Symptom (verbatim from operator):

```
✗ /home/vu2cpl/.node-red/settings.js not found — Stage 3 should have created it
```

Root cause: **Pi OS Imager pre-creates a `pi` user** even when you customise the imager to add a different username (`vu2cpl` in this case). The Node-RED installer (`update-nodejs-and-nodered`) then chose `pi` as the systemd `User=` instead of the operator's actual user. Result: Node-RED was writing to `/home/pi/.node-red/`, but the script + the repo + everything else lived under `/home/vu2cpl/`. Stage 5 (settings.js patcher) couldn't find settings.js because it was in the wrong user's home dir, and the script's `[[ -f ... ]]` test also failed due to permission masking on `pi`'s home subtree.

Fix — new `ensure_nodered_user_matches()` helper

Reads `systemctl show nodered -p User --value`. If the systemd User= doesn't match `ACTUAL_USER`, prompts the operator `[Y/n]` to auto-fix:

1. `sudo systemctl stop nodered`
2. Backs up `/home/<wrong-user>/.node-red/` to `.node-red.bak.<ts>`
3. Writes a systemd drop-in:

   ```
   /etc/systemd/system/nodered.service.d/user.conf
   [Service]
   User=<correct-user>
   WorkingDirectory=/home/<correct-user>
   ```

4. `daemon-reload` + restart
5. Polls up to 60 s for `/home/<correct-user>/.node-red/settings.js` to appear (first-run bootstrap on a slow SD card)

Idempotent — does nothing when already aligned. Called from:

- **Stage 3** (right after `systemctl enable --now nodered`, before the `:1880` reachability check) — catches the fresh-install case on the first pass.
- **Stage 5** (start of stage) — defense-in-depth for resumed installs where Stage 3 was already marked done in the state file from a prior buggy run.

Backwards-compat for VU2CPL on noderedpi4 (`User=vu2cpl` and `ACTUAL_USER=vu2cpl`): the helper returns immediately. Identical behaviour.

Failure 2 — Stage 3 `:1880` timeout too tight

The old 5-second timeout for first-ever Node-RED start was failing intermittently on fresh installs on slow SD cards (Node-RED's first-run userDir bootstrap can take 30–60 s). Bumped to 90 s with a 2 s poll interval. Plus the failure message now points at `sudo journalctl -u nodered -n 50` so the operator knows where to look.

Failure 3 — state file in `/tmp/`

Pi reboot mid-install would wipe `/tmp/rebuild_pi.state`, forcing the operator to replay stages 1-N from scratch. State file moved to `$HOME/.rebuild_pi.state`. Persists across reboots. Same file-locking semantics; same `--reset` flag clears it.

Test result

Operator confirmed on `noderedpi5` after applying the immediate recovery (manually writing the drop-in then re-running `--stage 5`):

```
✓ settings.js found at /home/vu2cpl/.node-red/settings.js (Node-RED user: vu2cpl)
✓ Projects feature already enabled
→ Restart Node-RED to pick up the change
✓ Node-RED back up
✓ Stage 5 complete
```

The script-level auto-fix (commits `ce80438` + `304d504`) means a future forker hitting the same Pi-OS-Imager `pi` user trap gets a `[Y/n]` prompt and self-recovery instead of a `fail`.

Commits

- `ce80438` — Stage 5 derives settings.js path from systemd User= + retries 30 s for bootstrap lag.
- `304d504` — `ensure_nodered_user_matches` helper; called from Stage 3 + Stage 5; state file moved to `$HOME`; Stage 3 timeout 5 s → 90 s.

Doc-only change here. No Pi restart needed.

2026-06-03 — rebuild_pi.sh: comprehensive robustness sweep — Stage 6 + timeout audit

Operator hit two more fresh-install failures on `noderedpi5`:

```
Stage 6:  ✗ GitHub auth failed — pubkey not added or wrong account
            (but `ssh -T git@github.com` directly: "Hi vu2cpl! …successfully
authenticated")

Preflight: curl: (28) Resolving timed out after 5000 milliseconds
            ✗ github.com unreachable — check network
            (but git pull works fine)
```

Both were the same class of bug: **too-tight timeouts and fail-on-first-attempt behaviour** in the script. Once they surfaced, I did a comprehensive audit of every `timeout`, `max-time`, and `fail "..."` call point in the script to catch the rest of the class.

Fix 1 — Stage 6 (GitHub SSH key): opt-in, retry, diagnostic

Previous behaviour: generate key → display → prompt → one SSH test → fail hard on first miss with no diagnostic. Three problems:

1. **Most forkers don't push to GitHub.** They `git pull` from upstream (HTTPS — no SSH needed). For them, Stage 6 was gratuitous friction that could fail spuriously.
2. **The fail discarded the actual GitHub response**, so operators couldn't tell whether the key was on the wrong account, the paste truncated, or the network blocked SSH.
3. **No retry window.** Freshly added keys can take 5-30 seconds to propagate at GitHub. The script's instant test could miss that window.

Rewrote Stage 6:

- **Opt-in prompt at the top:** "Do you plan to push changes back to GitHub? [y/N]" — default N. If skipped, marks the stage done with the note "git pull from upstream works without an SSH key. If you need push later: bash rebuild_pi.sh --stage 6"
- **Captures `ssh -T` output** and prints it on every failure along with a 3-cause hint list (key not added / wrong account, paste truncated, firewall) AND the pubkey re-displayed so the operator can re-copy without scrolling.
- **3-attempt retry loop** with `[r]etry / [s]kip / [q]uit` at each failure. Skip behaves like the opt-out path.
- **On success**, parses `Hi <user>!` out of the SSH response and reports `authenticated as GitHub user: <name>` — catches the "I added it to the wrong account" case immediately.

Fix 2 — preflight curl github.com: retry loop with diagnostics

Previous: `curl -sS --max-time 5 -o /dev/null https://github.com` — fail on first attempt. 5 s is too tight for a DNS-cold first request on a Pi with slow DNS or WiFi (operator's case: DNS resolution alone took >5 s).

New:

- **3 attempts**, `--max-time 15 --connect-timeout 10` each, 2 s sleep between.
- On final fail, prints diagnostics: default route, `/etc/resolv.conf` nameservers, `ping -c1 -W2 github.com` output. Gives the operator actionable information.
- Reports number of attempts on success (operator can see "took 2 attempts" and know DNS was slow but recovered).

Fix 3 — Stage 5 restart wait: same 5 s timeout bug

After the audit, found that Stage 5's "Restart Node-RED to pick up the change" wait used the same broken `timeout 5 bash -c '...'` pattern that bit Stage 3 earlier. It hadn't been hit yet, but would have been on any fresh-Pi install where settings.js patching triggered a restart on a slow SD card.

Bumped to `timeout 60` with 2 s polls between attempts. Same journalctl hint on fail.

Comprehensive timeout audit — full state after fixes

Spot	Timeout / behaviour	Why this is appropriate
Preflight curl github.com	3 × 15 s w/ retry + diagnostics	DNS-cold first request + WiFi can take >5 s; retries handle propagation
Stage 3 `:1880` listen	90 s, 2 s polls, journalctl hint	Fresh Node-RED bootstrap on slow SD card can take 30-60 s
Stage 3/5 ensure_nodered_user_matches	Auto-fix via systemd drop-in, 60 s bootstrap wait	Drop-in install + restart + first-run dir creation
Stage 5 settings.js detection	systemd-derived path + 30 s polling retry + restart attempt	First-run can lag listen on `:1880`; retry covers it
Stage 5 restart `:1880` listen	60 s, 2 s polls, journalctl hint	Same class as Stage 3; restart bootstrap can be slow

Stage 6 GitHub SSH	Opt-in, 3 retries, actual GitHub error shown, skip option	Key propagation + wrong-account + paste truncation cases
Stage 9 telemetry smoke test	5 s warn-only (not `fail`), with "wait 60 s for cron tick" hint	monitor.sh runs every minute; first publish may not yet be there
Stage 14 MQTT verifications	3 s localhost / 35 s (AS3935 30 s cadence + 5 s buffer) / 65 s (1-minute cron + buffer)	Each matches its source's publish cadence

Every fail-on-timeout path now has either a retry loop or an actionable error message + diagnostic. No more "X failed" with no indication of why or what to do next.

Commits

- `39c9490` — Stage 6: opt-in, retry loop, GitHub-response diagnostic, skip option
- `00f7774` — Preflight curl: 3 × 15 s retry + diagnostics; Stage 5 restart: 5 s → 60 s

Doc-only change here. No Pi restart needed.

Class-of-bug lesson

Every `timeout X` / `--max-time X` / `fail` call point in a setup script is a potential failure on the slowest plausible runtime environment. Slowest plausible for this script: fresh Pi 4B on WiFi, slow Class 10 SD card, DNS-cold, ~50 km from the GitHub edge. Today: every previously-too-tight timeout has been audited and bumped if appropriate, or wrapped in retry+diagnostic if not.

For the operator on `noderedpi5`, the take-away: every previously spurious "X failed" message would now self-recover or print enough information to act on.

2026-06-03 — rebuild_pi.sh: the noderedpi5 saga — Stages 9, 11, 7, 14, 5 ×3

Multi-commit thread closing out the fresh-install bring-up on `noderedpi5`. Every step taught the script something it didn't know before. Recording the whole arc here so this pattern doesn't have to be re-discovered next time.

The failures, in order

#	Commit	Stage	What failed	Why
1	`f332ba8`	9	`/etc/sudoers.d/debian_frontend: bad permissions`	`sudo visudo -c` (no `-f`) validates the WHOLE `/etc/sudoers` tree; failed on a pre-existing system file with 644 instead of 440 perms. Fix: validate ONLY our `rpi-agent` file. Surface pre-existing issues elsewhere as `warn`, not block.

2	`9f6a71c`	11	`./redeploy.sh: No such file or directory`	LP-700-Server upstream repo restructured: `redeploy.sh` moved into `deploy/` AND changed to a dev-machine SSH push tool. Fix: opt-in prompt (most forkers don't own one), arch-detect (`uname -m`), download pre-built binary from GitHub release, run `sudo ./deploy/install.sh`.
3	`4745b8d`	7 (1st attempt)	"all stages done. but no /ui or shack"	Stage 7 cloned the repo but never told Node-RED which project to activate. Fix: write `.config.projects.json` with `activeProject` set.
4	`7e6e863`	14	Stage 14 reported success even when dashboards didn't load	Stage 14 was checking infrastructure (Node-RED running, MQTT alive), not the end-user goal. Fix: 6-bug rewrite — split critical vs optional checks, drop hardcoded `192.168.1.169`, add /shack check, fix `mark_stage` `"13_verify"` → `"14_verify"` typo, add project-active + flow-parsed checks, add hardware-skippable variants.
5	`e77dad0`	7 (2nd attempt)	"still no ui or shack"	My `.config.projects.json` was written to `~/.node-red/projects/.config.projects.json`. **Wrong path.** Node-RED reads it from `~/.node-red/.config.projects.json` (sibling of settings.js, NOT under projects/). Verified against working noderedpi4 install. Fix: correct path + add `credentialSecret: false` (matches noderedpi4 exactly).
6	`7a54616`	5 (v2 attempt)	"still no ui or shack"	The v1 Stage 5 patcher inserted `projects: { enabled: true }` at the **top level** of `module.exports`. Node-RED reads `editorTheme.projects.enabled`, not top-level `projects.enabled`. Fix: detect uncommented `editorTheme: {` block and insert projects inside it.

7	`5c58c27`	5 (v3, the real fix)	"still no ui or shack" — `node -e ...editorTheme.projects.enabled` returned `false`	v2's "insert at top of editorTheme" was overridden by Node-RED's DEFAULT `projects: { enabled: false }` block FURTHER DOWN inside editorTheme. JS object-literal "later-key-wins" semantics. Fix: don't insert. Regex-match any `projects: { ... enabled: false }` block and **flip the boolean**. `[^}]*?` non-greedy ensures the `enabled` is inside the same projects block. Idempotent: multi-flips all matching blocks to `true`.
8	`e5f799e`	5 (extension)	`/ui` works but `/shack` 404	uibuilder v7.6.2 (noderedpi4) auto-detects Node-RED Projects → reads from `projects/<name>/uibuilder/`. uibuilder v7.7.1 (noderedpi5) **dropped this auto-detection** → falls back to `userDir/uibuilder/` → no `shack/` source there → 404. Fix: explicitly set `uibuilder.uibRoot` in settings.js to `__dirname + '/projects/<REPO_NAME>/uibuilder'`. Makes the path explicit, doesn't depend on auto-detection.

The recurring pattern

Three of the eight bugs followed identical structure: **the script's "I'm done" check was less rigorous than what real end-to-end success required.** Each stage was checking "did I write the file" or "did the command exit 0" rather than "is the thing the operator cares about actually working."

Stage 5 was the worst case — three iterations because:

- v1's grep-based detection (`if grep -q 'projects:' ...`) couldn't tell that "projects: exists but at the wrong nesting level" was the same as "broken"
- v2's "insert inside editorTheme" assumed the editorTheme block didn't already have a competing `projects:` block (it did — Node-RED's default template)
- v3 was the first to use `node -e ...` to **evaluate** settings.js the way Node-RED actually does, and to verify the value after the patch

The lesson is now codified in Stage 5 (and Stage 14): **patch → re-evaluate → fail loudly if the actual evaluated state doesn't match expectations.** No more "stage marked done but the thing it was supposed to enable isn't actually enabled."

Class-of-bug lesson — feature flags via deeply-nested config

Node-RED's `editorTheme.projects.enabled` is a feature flag buried three levels deep in a JavaScript object literal that also accepts arbitrary top-level keys for forward-compat. **You cannot safely patch this with grep+sed.** The right tools:

1. **Evaluate**, don't grep: `node -e 'console.log(require("./settings.js").editorTheme?.projects?.enabled)'` gives you the value Node-RED actually reads.
2. **Verify after patch**: re-evaluate, fail if value doesn't match expectation.
3. **Surgical flips, not insertions**: when a default value already exists, flipping it is safer than inserting a parallel one (JS later-wins makes parallel-insert silently broken).
4. **Explicit configs over auto-detection**: uibuilder's project auto-detection went away in a minor version bump. Anything the script depends on that "just works" via auto-detection is a future bug. Make it explicit.

Same lesson applies to LP-700-Server's repo restructure (commit 2): upstream changed without breaking the install API, but did break our script. Lesson: prefer release-artifact downloads (stable URL via the releases API) over assumptions about repo structure.

Commits

- `f332ba8` — Stage 9 sudoers: validate only our file
- `9f6a71c` — Stage 11 LP-700: adapt to restructured upstream + opt-in
- `4745b8d` — Stage 7: activate project (wrong path)
- `7e6e863` — Stage 14: actually verify end-to-end
- `e77dad0` — Stage 7: correct .config.projects.json path
- `7a54616` — Stage 5 v2: editorTheme nesting (still broken)
- `5c58c27` — Stage 5 v3: flip-not-insert + verify-after-patch (the real Projects fix)
- `e5f799e` — Stage 5: explicit uibuilder.uibRoot

Operator outcome

`http://noderedpi5-ip/shack` confirmed loading. All Stage 14 critical checks pass: project active, flows parsed, `/shack` + `/ui` respond, `rpi-agent` active. Optional checks (LP-700, AS3935, GPS-NTP) skip gracefully where the hardware isn't on this Pi.

Doc-only change here. No Pi restart needed.

2026-06-03 — rebuild_pi.sh: Stage 13 always-prompt + Stage 9 crontab fix

Three commits closing out remaining UX issues on the `noderedpi5` install after the main project-activation saga.

`1e3e69a` — Stage 13 drops `ACTUAL_USER=='vu2cpl'` auto-skip

Operator: "why is this not offering any customisation options?"

The old Stage 13 had a "Running as user 'vu2cpl' with callsign=VU2CPL → skip" path that fired even when the operator clearly wanted to customize. The reasoning was "this must be VU2CPL's primary Pi (`noderedpi4`) where defaults are correct." But the same operator can have multiple Pis (`noderedpi4` = primary, `noderedpi5` = test/backup) that need different MQTT broker IPs, hostnames, Tasmota topics, etc.

Fix: replaced with an opt-in prompt. *The operator knows their context; the script just asks.*

`514af17` — Stage 13 also drops `callsign != VU2CPL` auto-skip

Operator: "it allowed customisation only once. after that it just says its already customised?"

Caught a second auto-skip in the same stage. After the operator customized once (callsign now ≠ VU2CPL), `--stage 13` re-runs hit a DIFFERENT auto-skip: "Init Defaults already customised, re-run with `--stage 13`." But they WERE running with `--stage 13`. Catch-22.

Root cause: two layers of idempotency were redundantly fighting each other:

Layer	Purpose	Behaviour
State-file marker	Main pipeline skips already-done stages	Correctly bypassed by `--stage N` (which calls `unmark_stage` first)
Stage function's value-based auto-skip	"Belt-and-suspenders"	Fired after the state-file marker was unmarked, blocking re-runs

The state-file marker alone is sufficient. The value-based auto-skip was redundant *and* wrong.

Fix: removed the value-based auto-skip. Stage 13 now **always** prompts when it runs, showing the current callsign in the prompt text so the operator knows the current state. Default answer is N, so the main-pipeline case is still a fast Enter-tap to no-op.

`75dfe77` — Stage 9 crontab survives "no existing crontab"

Operator: Stage 9 exited silently right after `→ Schedule monitor.sh in user crontab` — no error, no completion message. Script just returned to the prompt.

Root cause: a `set -euo pipefail` interaction in this pipeline:

```
(crontab -l 2>/dev/null | grep -v 'monitor.sh' ; echo "* * * * *  …") | crontab -
```

On a fresh Pi, `crontab -l` exits 1 (no crontab). The inner pipe's pipefail returns non-zero. `set -e` exits the subshell **before** the `echo` runs. Subshell output is empty. Outer `crontab -` installs an empty crontab and exits non-zero. Pipefail catches the subshell's exit. Script aborts silently because `crontab -l`'s stderr was redirected to `/dev/null` and `set -e` doesn't print on abort.

Fix: capture the existing crontab with `|| true` first, then process:

```
existing_cron=$(crontab -l 2>/dev/null || true)
{
    echo "$existing_cron" | grep -v 'monitor.sh' || true
    echo "* * * * *  /home/$ACTUAL_USER/monitor.sh"
} | crontab -
```

Same class of bug as Stage 14's earlier `13_verify` typo: silent failure that left the install half-finished. Audited for similar patterns elsewhere in the script — no other pipelines have the "first command might fail on fresh install + pipefail kills us" shape unprotected.

Doc updates landing alongside

REBUILD_PI.md: the "Faster path" callout's bullet for Stage 13 updated to describe the new opt-in prompt behaviour. State-file path also updated from `/tmp/rebuild_pi.state` (stale; moved to `$HOME` earlier) to `$HOME/.rebuild_pi.state`.

FORK_GUIDE.md A4 stage table: Stage 13 row updated to reflect "always asks, default N" semantics. Stage 14 row updated to mention the critical-vs-optional split that landed in `7e6e863`.

REBUILD_PI.md Step 12 "16-point checklist" note clarified — the manual checklist is wider than what `--stage 14` runs programmatically (script's 7 critical + 4 optional vs the operator's full-stack view), and that's by design.

Doc CDP wrap covers all three code commits + the three doc fixes.

2026-06-04 — flows.json audit: 8 dead nodes pruned + 2 unnamed groups renamed

Audit pass on `flows.json` (509 nodes scanned) for stale code, disabled-but-still-wired functions, orphan nodes whose producers were deleted in earlier cleanups, and never-instantiated subflows. Result: 8 nodes deleted (509 → 501), 3 dead wires pruned, 2 unnamed groups given descriptive names. Zero behaviour change — every deletion either runtime-disabled (`d:true`) or had no upstream producer reaching it.

Deletions

Node ID	Type	Name	Why dead
`66ca1554.e4c85c`	subflow def	red trigger	Never instantiated
`a7edb3ab6224c12f`	subflow def	red trigger (1)	Never instantiated
`f0932848.b39158`	function	send X TRIGGER	Inside dead subflow #1
`82b5c109e09d7a14`	function	send X TRIGGER	Inside dead subflow #2
`d5eca40d3503035d`	function (d=true)	Format Telegram Alert	Superseded by `94b77826079bad57` (Format Telegram Alert Dedup 10 minute); upstream `b981643f37259f89` (DXCC Classify) and `1000e598fc8e322e` (🔔 Test Telegram) already wire to the live one
`9fd52c02a8486dce`	function (d=true)	Fetch All Modes + Parse	Superseded by `aa7434df62b95ebc` (lotw only); upstream `6e60f619acad462e` (Build Club Log API Request) already wires to the live one
`8778c5c78701ce17`	function	Flex Frequency Display	Orphan 7-segment-LED formatter, zero upstream wires
`0c1611c32c969490`	function	Skimmer Config (VU2CPL + VU2OY)	Orphan flow-var setter (`flow.set('sk1_call', …)`); all 4 consumers (`Parse Calibration CSV`, `Watchdog Check`, `RBN State Aggregator`, `Update Spot Counters`) carry inline `flow.get('sk1_call') || 'VU2CPL'` fallbacks, so the setter never running was already harmless and now stops being misleading

Wire prunes (3)

- `b981643f37259f89` (DXCC Classify) output 0 → dropped `d5eca40d3503035d`
- `1000e598fc8e322e` (🔔 Test Telegram) output 0 → dropped `d5eca40d3503035d`
- `6e60f619acad462e` (Build Club Log API Request) output 0 → dropped `9fd52c02a8486dce`

Group renames

Two groups on the Solar and Internet/Network tabs had `name: "?"` — purely cosmetic but they print as `?` in the editor sidebar and hurt grep-ability. Renamed to describe their actual contents:

- `498240cb5d374d24` (Solar tab, 33 nodes — NOAA / F10.7 / Geomag / GOES X-ray / MUF–foF2 fetch + parse + aggregator + Solar Panel ui_template) → **"Solar fetch + parse + dashboard"**
- `0cffa681c4abed74` (Internet and network monitor tab, 15 nodes — 5 pings + 5 timestamp stampers + State Aggregator + Network Monitor Panel) → **"Network ping + dashboard"**

What was investigated and left alone (so we don't re-flag next audit)

- 3 config nodes (`serial-port 677e2b7f2c916183` , `websocket-client ws_spe_client` , `websocket-client lp7wsclient00001`) looked orphan to the wire-graph but are heavily used — they're referenced via `serial:` / `client:` fields on serial-in/out + websocket-in/out nodes, not via flow wires. False alarm.
- Cross-tab MQTT fan-in on `lightning/as3935{,/hb,/last_event,/status}` and `stat/powerstrip1/POWER2` — intentional and documented in CLAUDE.md (AS3935 Tuning tab piggy-backs on Lightning tab's subscriptions; Rotator tab piggy-backs on All Power Strips' Tasmota state).
- Vue push ticks at 1–3 s cadence — intentional UI refresh.
- `catch` nodes and `ui_template` nodes without incoming wires — by design.
- Active debug nodes (`lp7ackdbg00001 ack` , `b5c2ed3182e180 Callsign Sent` , `8fe63e652f1feb Debug Alerts`) — daily-driver troubleshooting, operator's call to keep or strip.

Doc updates

- **CLAUDE.md** "KEY NODE IDs" → DXCC Tracker table: the row for `9fd52c02a8486dce` (Fetch All Modes + Parse) was pointing at the now-disabled node as if it were active. Replaced with `aa7434df62b95ebc` (Fetch All Modes + Parse lotw only), with an inline note about the supersession date so an archeology pass can find the old ID via this commit.
- **clublog_dxcc_tracker_v7.json** — re-extracted via the rule #4 script. Was 80 nodes (included the disabled Fetch + Format Telegram Alert); now 78 nodes.

Audit method (in case a future audit wants to repeat it)

Single Python pass over `flows.json` :

1. Build incoming-wire map: for every node ID, list which other nodes wire INTO it (via `wires[i][j]` arrays).
2. For each non-config node: flag as "no incoming" if not in the producer-types allow-list (`inject` , `mqtt in` , `http in` , `tcp in` , `serial in` , `ping` , `flexradio-*` , `uibuilder` , …) AND nothing wires into it.
3. For each `d:true` node: list its upstream + downstream wires; if both ends also reach a non-disabled sibling, the disabled node is pure visual debt and can be dropped.
4. For each `subflow` node: search for `type: "subflow:<id>"` elsewhere — zero instantiations = dead.

5. For each `function` whose body has `flow.set(KEY, …)` : search every other node's body for `flow.get(KEY)` — if no consumer (or all consumers have `|| default` fallbacks), the writer can be dropped.
6. For each "orphan-looking" config node (`serial-port` , `websocket-client` , `mqtt-broker` , etc.): the wire-graph won't catch references; instead scan every other node for string fields equal to the config's ID. If zero hits, truly orphan.

The full audit script is preserved in conversation log; re-runnable if the flow grows enough to invite another pass.

2026-06-04 — rebuild_pi.sh Stage 1: I²C verify softened (warn, not fail)

Operator report: Stage 1 failing on a fresh Pi at `[[ -e /dev/i2c-1 ]] || fail "/dev/i2c-1 not present after raspi-config"` .

Root cause

Two interacting issues, both pre-existing brittleness:

1. **`raspi-config nonint do_i2c 0` sets the `dtparam=i2c_arm=on` line in `/boot/firmware/config.txt` , but on some Pi-OS images the `i2c-dev` kernel module isn't auto-loaded** until the next boot — so `/dev/i2c-1` doesn't materialise immediately even though raspi-config returned 0.
2. **`udev` is async** — even when the module IS loaded, the device node can take a beat to appear.

The check fired before either path completed, killed Stage 1, and the operator had no clue what to do next (the fail message didn't mention the AS3935-daemon-is-fallback context).

Fix — Stage 1

Three changes:

1. **Try `sudo modprobe i2c-dev`** before checking. Covers the "raspi-config flipped the dtparam but module isn't loaded yet" case without needing a reboot.
2. **Retry the device-node check** up to 5 × 1 s. Covers the udev-lag case.
3. **Downgrade fail → warn on the miss**, with actionable diagnostics:
    - `lsmod` filter for `i2c_*` to show which modules are loaded
    - `dtparam` grep against `/boot/firmware/config.txt` (Bookworm+) and `/boot/config.txt` (older) to confirm raspi-config actually wrote the setting
    - `/dev/i2c-*` listing to confirm whether ANY i2c device node showed up
    - Plain-English likely causes (need a reboot; or no I²C hardware on this Pi)
    - **Plain-English impact**: only the legacy `as3935.service` Pi-side daemon needs `/dev/i2c-1` , and that daemon has been **standby fallback** since 2026-05-11 (ESP32 bridge is the live publisher). Normal shack operation is unaffected.
    - Recommended manual recovery: finish the rebuild, reboot, re-check.

Stage 1 now marks complete and lets the install continue. If a forker is rebuilding a fleet-monitor-only Pi (no AS3935 hardware at all) they no longer get blocked at all.

Same philosophy as the noderedpi5-saga fixes: every `fail` that isn't strictly needed for end-user goal becomes `warn` with diagnostics + the operator can decide whether to chase it.

Doc updates landing alongside

REBUILD_PI.md Step 2 — the manual `ls /dev/i2c-1` check now prefaces with `sudo modprobe i2c-dev` to cover the just-set-but-not-loaded case, and a callout note explains the "can be skipped on a fleet-monitor-only Pi" semantic so an operator who hits the failure manually has the same context the script's `warn` block prints.

FORK_GUIDE.md Stage 1 row already said "if you have AS3935 hardware" — text is consistent, no change needed.

2026-06-04 — rebuild_pi.sh Stage 1: detect pending-reboot after dist-upgrade

Follow-up to the I²C-verify softening earlier today. Operator hit a **different** failure mode on the next attempt:

```
modprobe: FATAL: Module i2c-dev not found
```

Root cause

Stage 1 does `apt -y dist-upgrade` as its very first step. On a fresh Pi-OS install that step **almost always picks up a new `raspberrypi-kernel*` package** — the upstream firmware/kernel moves faster than the imager's bundled snapshot. After the upgrade lands:

- New kernel's modules are written to `/lib/modules/<new-kernel-version>/`
- Old kernel is still running (`uname -r` returns the OLD version)
- `/lib/modules/$(uname -r)/` is therefore stale or empty
- `modprobe` looks under that path → finds nothing → returns `FATAL: Module i2c-dev not found`

Same mechanism breaks ANY post-dist-upgrade `modprobe`, not just `i2c-dev`. Pi OS drops a marker file at `/var/run/reboot-required` (or `/run/reboot-required`) to signal exactly this condition.

Fix

Stage 1 now checks for the reboot-required marker **immediately after `dist-upgrade`**, before any later step can hit the confusing modprobe FATAL. If the marker exists:

- Print a red ✕ explaining the kernel-was-updated state
- Spell out what's broken (modprobe, dtparam from raspi-config, any udev-managed device) so the operator doesn't waste time chasing individual symptoms
- Print the exact recovery sequence (`sudo reboot` → `git pull
    - bash rebuild_pi.sh`)
- **Exit 0 without marking Stage 1 complete** — so the resume after reboot replays Stage 1 (dist-upgrade is idempotent; nothing new to install second time round)

Net effect: instead of running on through `step "Install runtime packages"` and hitting the modprobe FATAL further down with no useful context, the operator gets a single clear "reboot then resume" message right where it makes sense — at the end of the dist-upgrade step.

Why not auto-reboot?

Considered. Rejected because:

- Stage 1 is interactive (the operator is watching the install banner); pulling the rug out is rude
- The operator may have other tasks open in the SSH session

- The script's design philosophy is "narrate clearly, let the operator decide" — same philosophy as Stages 6 / 11 / 13's opt-in prompts

The recovery sequence is a copy-paste; explicit > clever.

Companion: the I²C-verify softening landed earlier today still applies

When the operator follows the recovery path (reboot, re-run Stage 1), the post-reboot run will:

1. Skip the reboot-required check (marker file is gone)
2. Re-run dist-upgrade idempotently (no new packages)
3. Run raspi-config's dtparam flip (idempotent — already set)
4. Reach the I²C verify step. Now `modprobe i2c-dev` succeeds (modules dir matches running kernel) AND `/dev/i2c-1` materialises (dtparam takes effect on the running kernel tree).
5. Mark Stage 1 complete, proceed to Stage 2.

For a fleet-monitor-only Pi without AS3935 hardware: same as before — the I²C verify warns-not-fails on the missing device node, prints the legacy-daemon-is-fallback context, and Stage 1 still marks complete.

Documentation drift checked

- REBUILD_PI.md Step 2 — manual path: the same reboot-required case applies to a forker doing the manual rebuild. Added a callout right after `dist-upgrade` documenting the check + the fix, matching the script's behaviour.
- FORK_GUIDE.md — Stage 1 row text ("5 min") could be misleading if a reboot is required. Left as-is; the wall-clock impact is marginal (reboot ≈ 30 s) and the row already covers the common case. The "Troubleshooting" table at the end already has a kernel-update-pending row pattern; not re-touched this pass.

2026-06-04 — rebuild_pi.sh Stage 1: bullseye fallback for raspi-config do_serial; preflight shows OS

Third Stage-1 fix today, this time on a different Pi — `adersh` forking the repo onto a Pi OS Bullseye host. The script ran clean through preflight + apt steps, then died here:

```
  → Enable I²C + hardware UART
/usr/bin/raspi-config: 2909: do_serial_hw: not found
```

Root cause

`raspi-config nonint do_serial_hw 0` is **bookworm+**. On bullseye (and buster), the equivalent is `do_serial` with combined arg semantics:

arg	login shell over serial	hardware UART
0	on	on
1	off	off
2	off	**on** ← what we want
3	on	off

The script issued `do_serial_hw 0`, raspi-config's bash function table didn't have that name, returned `127 "not found"`, and `set -e` killed the run.

Fix — Stage 1

Try the modern call first; on failure, fall back to the bullseye form; on both failing, warn with a manual recovery sequence (operator runs `sudo raspi-config` interactively → Interface Options → Serial Port → No / Yes). Same warn-not-fail pattern as today's other Stage-1 fixes — the script keeps going and marks the stage complete, because the hardware-UART enable is only needed for the rotator + SPE amplifier paths (a fleet- monitor-only Pi without serial hardware doesn't need it at all).

`do_i2c 0` also got a `|| warn …` guard for symmetry — it's been stable across releases but if a future Pi OS renames it the script now degrades gracefully instead of dying silently.

Bonus — preflight surfaces Pi-OS version

While in the area, added an `/etc/os-release` lookup right after the Raspberry-Pi detection line. Preflight now prints:

```
✓ Pi OS:      Raspbian GNU/Linux 11 (bullseye) (bullseye)
```

— so the next debugging pass can SEE the OS version at a glance instead of inferring it from package-version timestamps after a failure. If the codename is `bullseye` or `buster`, an extra soft warning fires noting these are past Pi OS's current-stable line; the script has fallbacks where it can but the operator should be aware that edge cases may surface.

This is the kind of cheap, high-leverage observability that should have been in the script from day one. Same lesson the noderedpi5 saga kept reinforcing: when a `fail` fires with no context, every minute the operator spends inferring "WHY did this fail?" is a minute the script should have spent telling them up front.

Documentation drift check

REBUILD_PI.md Step 2 currently uses `do_serial_hw` in the manual path. Most forkers will be on bookworm/trixie now (bullseye reached end-of-life late 2024), so the manual path is fine for the common case. Left as-is to avoid bloating with version- gating that the script already handles for the rare bullseye forker.

What the forker should do next

```
cd ~/.node-red/projects/vu2cpl-shack
git pull
bash rebuild_pi.sh    # resumes from Stage 1
```

Stage 1 will replay (apt steps are idempotent), the bullseye fallback will fire and proceed, and Stages 2–14 continue normally. Expect a soft warning about bullseye being past Pi OS current-stable — informational only, not blocking.

2026-06-04 — Fork tooling: MQTT broker config-node patch + hardware-tailored dashboard cards

Operator question that kicked this off: "the rebuild script — MQTT broker is not being filled in many instances on the flow? And if a forker has no particular module, that card should be removed from the Vue /shack dashboard, and even the D1 dashboard where there are no conflicts or shared tabs."

Two distinct things. One real bug, one new feature.

Bug — Stage 13 patched the wrong MQTT target

Stage 13's flows.json patcher rewrote `const MQTT_BROKER = '…'` inside the **Init Defaults function node** only. But a function node can't reconfigure a broker *config* node at runtime — the 37 `mqtt in` / `mqtt out` nodes connect via the two `mqtt-broker` **config nodes** (`f4785be9863eab08` "Tasmota MQTT Broker" + `mqttbroker.shack`), whose `broker` field is what actually decides where they dial. Those were never patched, so on a forker's Pi every mqtt node kept dialing `192.168.1.169` (the upstream Pi) and showed disconnected. That's the "broker not filled in many instances."

Fix: the patcher now also iterates every `mqtt-broker` config node and sets `broker` = the forker's MQTT IP. Verified in a dry-run: both config nodes flip to the entered IP; re-running with a different IP flips them again (idempotent).

Feature — hardware-tailored dashboard (scoped: Vue all + safe D1 subset, dependency-locked)

Goal: a forker who has no FlexRadio / no rotator / no DXCC interest shouldn't see dead cards. Two design decisions taken with the operator before building:

- **Scope** = tailor the Vue `/shack` dashboard for all 12 cards, PLUS auto-disable the **five stand-alone D1 `/ui` tabs** that are cleanly isolated (SPE, LP-700, Solar, DXCC, RBN). The other seven D1 cards are left alone — they're entangled (Flex/Rotator/ Lightning/Power switch outlets through each other) or share a D1 `ui_group` (GPS-NTP + Network share the `Network Monitor` group), so removing them from D1 risks the layout or orphans a control.
- **Entanglement handling** = dependency-locked prompts (below).

Mechanism (Vue): a single `const CARDS = { flex:true, … }` block near the top of `uibuilder/shack/src/index.js` ; each card in the root template gets `v-if="CARDS.<key>"` ; `App.setup()` returns `CARDS` . Flip a flag to `false` → card unrendered. **Nothing is deleted** — the component definition + its data plumbing stay intact, so flipping back to `true` fully restores it. Far safer than regex-deleting card code. Build stamp bumped `v10 → v11` .

Mechanism (D1 safe subset): for the 5 isolated subsystems, the patcher sets `"disabled": true` on the **flow tab** (not just the ui_template). This both removes the `/ui` card AND stops the tab's background polling — no more pointless Club Log fetches / RBN telnet / error spam for hardware the forker doesn't have. The 5 tabs and the fact that each is single-subsystem (zero shared groups) were confirmed by ID before wiring this:

Subsystem	Flow tab ID	Tab label
SPE	`spe_ws_tab_01`	SPE (WS)
LP-700	`18fb42443172f33c`	LP-700-HID ws
Solar	`590e889d44815afb`	Solar
DXCC	`d110d176c0aad308`	DXCC Tracker

RBN	`f9a0e3ad0e019052`	RBN Skimmer Monitor

(This is a refinement of the originally-approved "disable the ui_template" — whole-tab disable is strictly cleaner for these 5 confirmed-isolated tabs and the operator signed off.)

Stage 13 prompts: after the existing identity prompts, a new "Which subsystems does THIS station have?" block asks 12 Y/n questions (all default Yes). Then two dependency rules:

- **R1 (hard):** if Power = no but any of Lightning / Rotator / Flex = yes → **force Power on**, with an explanation that Power is the switching layer for the antenna disconnect / rotator auto-off / Flex power relay. Hiding it would orphan those.
- **R2 (warn):** if Flex = no but Lightning = yes → warn that the disconnect chain's TX-inhibit step targets a radio you don't have. Harmless (the flexradio request fails silently) but it won't do anything. Don't block — just inform.

The summary block now lists "Cards ON" / "Cards OFF" before the final confirm.

What is deliberately NOT touched

- No flow nodes deleted; no wires cut. Hidden Vue cards keep their builder/cmd-router nodes (harmless when the card isn't shown).
- D1 for Flex / Rotator / Lightning / Power / RPi / Network / GPS — Vue-only hiding. RPi/Network/GPS are isolated enough that a future pass could add them to the D1 safe subset; held back this round to honour the agreed 5-tab scope (Network + GPS share a D1 group, so they need care).
- Stage 13 still backs up `flows.json` + `index.js` before patching, so every run is reversible by restoring the `.bak` or re-running with the cards turned back on.

Verification

Dry-ran both patchers against throwaway copies (extracted the two embedded Python heredocs verbatim from the script so the test ran the *real* logic, not a paraphrase):

- Run 1 (Flex/SPE/DXCC off, broker `10.0.0.5`): both broker config nodes → `10.0.0.5`; SPE + DXCC tabs `disabled:true`; LP-700 + Solar + RBN tabs untouched; `CARDS.flex/spe/dxcc=false`, rest `true`; flows.json still valid JSON; index.js still parses (`node --check`).
- Run 2 (all on, broker `192.168.50.1`, on the already-patched copy): broker re-patched; all 5 tabs re-enabled (`disabled` removed); all 12 CARDS back to `true` → **idempotent**.
- Node count unchanged 501 → 501 (nothing added/removed).

Files changed

- `uibuilder/shack/src/index.js` — CARDS block + 12 `v-if` + setup return + build stamp.
- `rebuild_pi.sh` — Stage 13 hardware prompts, dependency rules, broker config-node patch, safe-subset tab disable, CARDS flip, exports + unset.
- Docs: this entry, HANDOVER row + tip, FORK_GUIDE Stage 13 row + A5 section, REBUILD_PI Stage 13 note, CLAUDE.md uibuilder section, README card line.

2026-06-06 — Rotator → WebSocket gateway (TODO #31 closed, all 3 phases)

Lifted the Idiom Press Rotor-EZ serial control out of Node-RED into a standalone Python WebSocket gateway, mirroring `spe-remote` and `lp700-server`. The Node-RED Rotator tab no longer owns the FTDI port — it's a thin ws-client of `rotator-remote.service` (`:8090`). This is the last of the three shack USB devices to be lifted into a gateway; it unblocks multi-client access (browser + future Mac app) and removes the "restart Node-RED to free the serial port" friction.

Planned in plan mode with two operator decisions locked up front: **new standalone repo** (vs in-shack scripts) and **serial/heading only** (vs gateway-owns-power). Executed phase-by-phase, checking in on the Pi between phases.

Phase 1 — the gateway (new repo `vu2cpl/rotator-remote`)

github.com/vu2cpl/rotator-remote (public, MIT), ~1190 lines. Copied the `spe-remote` skeleton and stripped to the (much simpler) rotator needs:

- `rotator/protocol.py` — Rotor-EZ command bytes + reply parser. **Bytes extracted verbatim from the pre-refactor flow**, not paraphrased: `AI1;` query · `AP1NNN\r` set (CR terminator) · `;` stop · replies framed by `;` , first 3 digits = azimuth. The set-vs-query terminator asymmetry is real DCU-1 and load-bearing — the Explore-agent summary had glossed it, so reading the actual `serial-port` config + `Build Rotator String` output strings before writing code was essential.
- `rotator/serial_handler.py` — single port owner: a daemon reader thread hands bytes to the asyncio loop via `call_soon_threadsafe` ; two loop tasks (poll `AI1;` every 1 s; drain a command queue) share one write-lock so the poll and a client command never interleave on the wire. Parsing + broadcast run on the loop thread (safe for Tornado `write_message`).
- `rotator/websocket_handler.py` — multi-client broadcast with state-dedup
    - heartbeat gate (copied from spe-remote).
- `server.py` / `app.py` / `config.py` — Tornado on `:8090` , `/ws` + `/healthz` , 5 s presence heartbeat.
- `setup.sh` / `run.sh` / `install-service.sh` / `uninstall-service.sh` / `systemd/rotator-remote.service.template` — adapted from spe-remote (renamed, dialout group, `{{USER}}` / `{{WORKDIR}}` substitution).

WS API — server pushes `{type:state,heading,target,moving,ts}` + `{type:heartbeat,serial,ts,clients}` ; clients send `{type:command,action:goto|stop|lpsp[,heading]}` (LP-700-style JSON; `lpsp` is computed gateway-side as `(heading+180)%360`).

Install rationale (documented in the repo README): like spe-remote / lp700-server this is a **Pi-only service** (no rotator on the Mac), so it ships `setup.sh` + `install-service.sh` rather than the global macOS/Pi-branching `install.py` pattern (which targets cross-platform fresh-machine bootstraps).

Verification: offline unit tests in a real venv (command bytes byte-exact, reply parsing, `moving` incl. wrap, command translation, split-frame buffering, bad-command tolerance, broadcast fires) — all green. Then **bench-verified on the real rotor**: `goto 90` turned it clockwise through north (heading streamed 325→358→5→23, the 358→5 wrap handled correctly), `stop` / `lpsp` /multi-client/heartbeat all confirmed. First bring-up showed `serial:"down"` until the rotator was powered on via Tasmota MQTT — a benign reminder that power is separate.

Phase 2 — Node-RED Rotator tab → ws-client

flows.json [`626ccc0`](626ccc0) , 501 → 497 nodes. The wiring graph revealed a cleaner refactor than first planned: **both `Build Rotator String` and `Click → Heading` already compute the heading and emit raw `AP1NNN\r` strings**, so instead of rewriting their internals I swapped the transport at the boundary.

- **DELETE (8):** Poll 1s inject, serial-in (Idiom Rotor-EZ), Receive Rotator, rbe, trigger, Slice heading, serial-out, serial-port config (`677e2b7f2c916183` , referenced only by the two serial nodes).
- **MODIFY (1):** `Send Rotator` → `Send Rotator → WS` (`3cfe1d67d0490107`). Body now translates the raw strings (`AP1NNN\r` goto, `;` / `stop` stop) into command JSON; output rewired from serial-out to the new websocket-out. Heading-computing logic in Build Rotator String / Click → Heading **untouched**.
- **ADD (4):** `websocket-client` config (`rotator_ws_client` , `ws://localhost:8090/ws`), `websocket out` , `websocket in` , and `Parse Rotator WS` — JSON-parses the gateway state, caches `flow.rotator_heading` , and emits the heading number to `Rotator fmt + SVG update` + `Raw HDG to display` (exact replacement for Slice heading; `Rotator fmt` uses `parseFloat` so a numeric heading is fine). `target_hdg` stays owned by the command functions (instant feedback + arrival-clear in Rotator fmt).
- **UNCHANGED:** Build Rotator String, Click → Heading, compass SVG, all 4 HTTP endpoints, the entire power path (Tasmota `powerstrip1/POWER2` over MQTT + auto-off timer on the All Power Strips tab), and the Vue builder.

The apply was scripted in Python with verify-before-write; a first run correctly aborted on a **pre-existing** dangling wire (`Raw HDG to display` → a long-deleted node `27a6286454485995` , which Node-RED ignores at runtime) — the check was relaxed to only fail on refs to nodes *this* change deleted, and the pre-existing dangle was preserved as-is. Both new function bodies were `node --check` 'd.

Deploy ordering (critical): the ws-client flow deploys FIRST (Node-RED restart releases the port), THEN the gateway starts. Verified on the Pi: `/ui` compass + `/shack` RotatorCard both track heading, GO/STOP/LP-SP work, power-toggle still flips the outlet, multi-client confirmed.

Phase 3 — deployment automation + docs

- `rebuild_pi.sh` : new **Stage 13b** (`13b_rotator_remote` , opt-in), inserted between customize (13) and verify, mirroring the LP-700 stage — prompts "Do you have a Rotor-EZ rotator?", clones the repo, lists `/dev/serial/by-id/` and prompts for the rotor's device (patches `config.yaml`), runs `setup.sh` + `install-service.sh` , checks `/healthz` . Stage 14 (verify) gained an optional `rotator-remote /healthz` check and is now positionally `--stage 15` .
- Docs: CLAUDE.md (INFRASTRUCTURE table row, new **Rotator** flow-notes section, USB-serial table note, OPEN BUGS #31 closed, flow-tab count 24→28), HANDOVER (#31 closure with the original spec preserved + What-landed + Current-state rows), REBUILD_PI.md (interactive-pauses list + verify `--stage 15` note), FORK_GUIDE.md (Stage 13b row, verify optional checks, "what lives where" serial-path/systemd/repos entries), README.md (transport list + Rotator section), and this entry. PDF regenerated.

Net effect

Three USB devices (SPE, LP-700, rotator) now all live behind WebSocket gateways with a uniform architecture. Node-RED touches no serial/HID port directly. No more "restart Node-RED to free a port"; multi-client everywhere; the Mac app (#6) is unblocked for all three.

2026-06-06 — D1 Rotator compass: remove 4 decorative corner brackets

Cosmetic. The D1 `/ui` Compass SVG (`cae633df9457557f`) had 4 cyan (`#00e5ff`) L-shaped corner brackets at the four corners of the compass box — pure HUD framing from the 2026-05-23 aviation-HUD rebuild, no click handlers / bindings / functional value. On the dark dashboard the bright cyan reads as light/white, and the operator found them a distraction. Removed the 4 `<path d="M … L … L …" fill="none">` elements from `svgString` (path count 6 → 2). **Preserved** the functional `<polygon id="rot-arrow">` heading pointer (also a triangle) and the `rotator-power-*` knob paths. No behaviour change. Verified the heading arrow + power knob survive and the SVG envelope is intact before writing.

2026-06-06 — rebuild_pi.sh: spe-remote install stage + ask-hardware-once refactor

Operator caught a real gap: "rebuild script is not installing web socket servers on pi?" The script installed `lp700-server` (Stage 11) and `rotator-remote` (Stage 13b) but **never** **`spe-remote`** — a from-scratch rebuild left the SPE amplifier gateway uninstalled (it was only on the Pi because it had been set up by hand). Worse, FORK_GUIDE already *listed* `spe-remote.service` as if it were installed. Fixed, plus the deeper "one fix for good" the operator asked for: eliminate the redundant double-prompting.

Two things were wrong

1. **No spe-remote stage.** `vu2cpl/spe-remote` (public, installs like rotator: `setup.sh` + `install-service.sh`, `:8888`, no `/healthz` — serves its dashboard at `/`) had no rebuild stage.
2. **Redundant prompting.** Stage 13 asked a 12-question "which subsystems do you have?" round (for card visibility), and *each gateway stage* asked again ("Do you have an LP-700? / a rotator?"). Same questions, twice.

The fix — ask once, up front, persist, everyone reads it

- New **`hw_inventory()`** runs right after pre-flight (before any stage): asks the 12 Y/n questions once, applies the dependency rules (R1 force Power if Lightning/Rotator/Flex stay; R2 warn if Flex dropped while Lightning stays), and persists to `$HOME/.rebuild_pi.hw`. On resumes and `--stage N` single runs it loads the file **silently** — no re-prompt. `--reset` wipes it alongside the state file. `ask_card` was hoisted from Stage 13 to a top-level helper.
- **Gateway stages read the inventory, don't prompt.** Stage 11 (`lp700-server`), Stage 13b (`rotator-remote`), and the new Stage 13c (`spe-remote`) each install if `HW_<x>=1`, skip silently if `0` (default-on if a key is somehow absent — safe). The two serial gateways still prompt for the device's `/dev/serial/by-id/…` path.
- **New Stage 13c (** **`13c_spe_remote`** **)**: clones `vu2cpl/spe-remote`, patches `config.yaml` serial port, `setup.sh` + `install-service.sh`, liveness check `curl :8888/` (no `/healthz` on spe-remote).
- **Stage 13** keeps its identity opt-in (callsign/grid/broker/…) but its "which subsystems" round is gone — it now reads the inventory `HW_*` for the CARDS flip + safe-subset tab disables. Stage 14 verify gained an optional `spe-remote :8888` check (now 7 critical + 6 optional). Verify is positionally `--stage 16` (13b + 13c precede it).

Net: all three USB gateways are installable by the rebuild, driven by one hardware answer set, asked exactly once. Verified: `bash -n` clean; STAGES order correct; dry-ran the persist→load→gate logic (lp700=0 → skip, rotator/spe=1 → install, unset key → safe default install).

Docs

CLAUDE.md (INFRASTRUCTURE row for `spe-remote.service` @ `:8888`), REBUILD_PI.md (pauses list rewritten around the up-front inventory; verify `--stage 16`), FORK_GUIDE.md (new "hardware inventory" table row; Stage 13c row; 11/13/13b reworded to "driven by inventory"; verify `--stage 16` + spe check). PDF regenerated.

2026-06-06 — rebuild_pi.sh: MQTT broker IP moved to mandatory inventory + always-patched in Stage 7

Operator report: "MQTT is not working on a new installation using the script despite giving the correct IP (different site, different from the .169 IP)."

Root cause — broker IP buried in an opt-in stage

The broker IP was collected and applied **only inside Stage 13's opt-in station-customize flow** ("(Re-)customize station identity? [y/N]"). A bare Enter past that prompt marks Stage 13 done and returns — so the two `mqtt-broker` config nodes keep their shipped value (`192.168.1.169` , the upstream Pi) and every one of the 37 mqtt nodes dials a broker that doesn't exist on the fork's LAN. The broker config nodes themselves are otherwise clean (port 1883, no TLS, no auth, empty client-id), so the *only* thing that needed setting was `broker` — and it was skippable. (Couldn't confirm on the operator's Pi — no access at the time — so this was fixed at the script level from the identified fragility.)

Fix — ask once (mandatory), apply always

- The broker IP now lives in the **mandatory up-front inventory** (`hw_inventory` , new `ask_broker_ip` helper), default = this Pi's own LAN IP (Stage 2 installs Mosquitto locally, so that's almost always right). Persisted as `MQTT_IP` in `$HOME/.rebuild_pi.hw` . Can't be skipped the way the opt-in prompt could. Upgrade path: an inventory file written earlier today (no `MQTT_IP`) triggers a one-time prompt + append on next load.
- **Stage 7 (clone_repo, always runs) now patches both `mqtt-broker` config nodes** to `MQTT_IP` and restarts Node-RED — so MQTT connects out of the box on a fork regardless of whether the operator runs the Stage 13 customize. On VU2CPL's own rebuild this is a no-op (Pi IP == `.169` == shipped value).
- **Stage 13** no longer re-prompts for the broker; it reads `MQTT_IP` from the inventory (its own broker-config-node patch stays, idempotent).

Verification

`bash -n` clean. Dry-ran the Stage 7 patch python against a copy of flows.json (`MQTT_TARGET=10.0.0.5` → both broker nodes → `10.0.0.5`). Dry-ran the inventory persist/load round-trip (`MQTT_IP` survives write→source).

Docs

CLAUDE.md broker note (now "inventory + Stage 7 always"), REBUILD_PI.md + FORK_GUIDE.md inventory/Stage-7 rows + the "why the broker patch matters" callout rewritten. PDF regen.

To recover the operator's already-broken Pi (when access returns)

`bash rebuild_pi.sh --stage 7` (re-patches the broker from the inventory), or just pencil-edit the broker in the Node-RED editor → Server = the Pi's IP, then restart Node-RED.

2026-06-08 — Fork fix: sweep ALL `.169` broker hardcodes (not just the config nodes)

Follow-up to the 2026-06-06 broker fix. That one made Stage 7 rewrite the two `mqtt-broker` *config nodes*, but the shipped `192.168.1.169` literal still survived in several other spots, so a fork at a different

IP had broken telemetry, a broken LP-700 ws-client, and a misleading Init Defaults value. Swept all of them. Full write-up in `FORK_BROKER_HARDCODE_FIX.md`.

Hardcodes swept

- **`monitor.sh`** — `BROKER=192.168.1.169` → reads `$MQTT_BROKER` → `/etc/default/vu2cpl-shack` → `127.0.0.1`. (Cron can't use systemd EnvironmentFile, so it sources the file itself.)
- **`as3935_mqtt.py`** — `MQTT_BROKER = "192.168.1.169"` → `os.environ.get("MQTT_BROKER","127.0.0.1")` (+ `import os`).
- **`as3935.service`** — added `EnvironmentFile=-/etc/default/vu2cpl-shack` (the `-` makes it optional) + `Environment=PYTHONUNBUFFERED=1` (the daemon's own docstring required this for logs to reach journald — it was missing).
- **`flows.json` LP-700 ws-client** (`lp7wsclient00001`) — was `ws://192.168.1.169:8089/ws`; Stage 7 now rewrites the host to `localhost`, matching the SPE (`:8888`) and rotator (`:8090`) ws-clients, which already used `localhost`. Gateways run on the *same Pi*, so they must NOT inherit the (possibly external) broker IP.
- **`flows.json` Init Defaults `MQTT_BROKER` const** — Stage 7 now rewrites it to the broker IP too.

Stage 7 — broadened sweep + fail-loud residual check

The Stage-7 patcher now handles broker config nodes (→ broker IP), `websocket-client` paths (`.169` → `localhost`), and the Init Defaults const (→ broker IP), then runs a **fail-loud residual sweep**: if any `192.168.1.169` survives and the target isn't itself `.169`, the stage aborts (a missed hardcode = broken MQTT). On VU2CPL's own rebuild (target == `.169`) the sweep is a near-no-op and the residual check is skipped — only the ws-clients flip to `localhost` (an improvement).

Stage 9 — env file + unit retargeting

- Writes `/etc/default/vu2cpl-shack` (`MQTT_BROKER=<inventory IP>`), consumed by `as3935.service` (EnvironmentFile) and `monitor.sh` (sourced).
- `sed` -retargets both `as3935.service` and `rpi-agent.service` from the upstream `vu2cpl` user/home to `$ACTUAL_USER` at install — they shipped with `User=vu2cpl` + `/home/vu2cpl/…` hardcoded, doubly broken on a fork with a different user. The sed patterns are user/home-specific so `vu2cpl-shack` in the EnvironmentFile path is untouched.

Verification

`bash -n` (rebuild_pi.sh, monitor.sh) + `py_compile` (as3935_mqtt.py) clean. Dry-ran the Stage-7 sweep against a copy of flows.json: fork IP `10.0.0.5` → both brokers + Init const → `10.0.0.5`, all three ws-clients → `localhost`, **0 residual** (exit 0); VU2CPL IP `.169` → ws-clients → `localhost`, residual check skipped. Dry-ran the sed retarget (`vu2cpl` → `pi`): paths + `User=` flipped, `vu2cpl-shack` intact.

The repo's `flows.json` is intentionally left at `.169` (VU2CPL's own value); Stage 7 rewrites it at install. No `flows.json` commit here.

2026-06-08 — rotator-remote: standalone web UI at :8090/ (in the rotator-remote repo)

Part B of the 2026-06-08 handover. Gave `rotator-remote` its own self-contained control page at `http://<pi>:8090/`, exactly like `spe-remote` serves its UI at `:8888/` — independent of Node-RED's `/shack`. Same single-serial-owner / multi-client model: Node-RED stays a ws-client on the same `/ws`; the page is just another client.

Done in the `vu2cpl/rotator-remote` repo (commit `7fc5411`), not this one:

- `rotator/app.py` serves `web/` via a `NoCacheStaticFileHandler` (copied from spe-remote) at `/`, keeping `/ws` + `/healthz` as their own routes (so Stage 13b's `curl :8090/healthz` still passes).
- `web/{index.html,app.js,style.css}` — dark compass UI: live heading readout, SVG compass (needle = heading, marker = target, click-to-slew), numeric azimuth + GO, preset bearings, prominent STOP, LP/+180°, and a status line (connection pill + rotor up/down + clients). WS connect/reconnect lifted from spe; commands use the existing JSON protocol (goto/stop/lpsp). No build step.

Power stays out of scope (Tasmota/MQTT in Node-RED). Verified locally — the server serves `/`, `/healthz`, and the assets with no-cache headers.

Shack-repo change here is only the CLAUDE.md infra-row cross-reference.

2026-06-08 — Vue `/shack`: brand the remaining VU2CPL hardcodes for forks

Audit of `uibuilder/shack/src/` for fork-unfriendly hardcodes, after the broker sweep. **Headline: the Vue app has no *network* hardcodes** — every `fetch()` is a relative path (`/lightning/*`, `/dxcc/*`, `/rotator/*`) and all data/commands go through `uibuilder.onTopic` / `send` / `start` (Socket.IO to `location.host`). No absolute `ws://` / `http://`, no `.169`. So a fork's `/shack` automatically talks to its own Node-RED — none of the MQTT-class breakage exists here. What remained was purely **cosmetic / branding**.

Stage 13 now also brands (it already did TopBar + manifest name)

- `index.html` `<title>` → `<callsign> Shack` (new patch step).
- `manifest.json` `description` → `<callsign> amateur radio shack control`.
- `index.js` `LightningCard` reactive-state defaults `callsign` / `grid` (overwritten by live data ~3 s after load, but the first paint is now correct). Targeted regex — there's exactly one `callsign:` / `grid:` key.
- Bumped the `index.js` cache-buster `?v=10` → `?v=11` to match the build stamp (they'd drifted apart since the CARDS change, so a cached browser could serve a stale `index.js`).

Two that can't be auto-patched (fork-specific gear) — now flagged

These depend on the forker's own hardware/clusters, which the script can't know, so they're marked with `// FORK:` comments in `index.js` and documented in FORK_GUIDE A5.3:

- **`NetworkCard` host list** (`index.js`, ~2480): device labels + IPs. `addr` is display-only; `key` must match the network-monitor stamp functions in Node-RED.
- **DXCC `clusterNames`** (`index.js`, ~815): must match the `cluster_status` names emitted by the DXCC tab's `login-parse-dedup`.

Verification

`bash -n` + `node --check` clean. Dry-ran the branding patches with `CALLSIGN=K1ABC GRID=FN42aa`: title → "K1ABC Shack", manifest name + description → K1ABC, LightningCard defaults → K1ABC/FN42aa; cluster names + device IPs left intact; patched `index.js` still valid JS.

No `flows.json` change. The repo keeps VU2CPL's own values; Stage 13 brands forks at install (callsign=VU2CPL → no-op on VU2CPL's own rebuild).

2026-06-19 — Fork fix: mosquitto `status=3` from duplicate `persistence_location`

A forker (`adersh@NodeRed`) ran `rebuild_pi.sh` and `mosquitto.service` failed to start: `Active: failed` , `code=exited, status=3` , systemd "Start request repeated too quickly". Foreground run gave the exact cause:

```
Error: Duplicate persistence_location value in configuration.
Error found at /etc/mosquitto/conf.d/lan.conf:5.
Error found at /etc/mosquitto/mosquitto.conf:13.
```

Root cause — the "two locations" problem. Mosquitto reads config from the stock `/etc/mosquitto/mosquitto.conf` **and** from every file pulled in by its `include_dir /etc/mosquitto/conf.d` . On Raspberry Pi OS Bookworm the stock file *already* sets `persistence` , `persistence_location /var/lib/mosquitto/` , and `log_dest file /var/log/mosquitto/mosquitto.log` . Our Stage 2 drop-in (`conf.d/lan.conf`) **re-declared the same three** on top of the listener lines. mosquitto 2.x treats a duplicate `persistence_location` as **fatal**, not a warning → it exits before binding → `status=3` , and systemd's rapid-restart backoff then reports "repeated too quickly". (VU2CPL's own Pi never hit this — its broker predates the script and was hand-configured, so the duplicate only surfaces on a *clean* fork install where Stage 2 writes the drop-in over a pristine stock conf.)

Fix — `rebuild_pi.sh` `stage_02_mosquitto()`

The drop-in now carries **only** the two directives the stock conf lacks:

```
# VU2CPL shack broker — LAN-only, no auth
listener 1883
allow_anonymous true
```

`persistence` / `persistence_location` / `log_dest` are inherited from the stock `mosquitto.conf` . **Defensive branch:** if a minimal/non-standard stock conf is *missing* `persistence_location` (so retained messages — LWT, retained `shack/gpsntp/chrony` , etc. — would be lost on reboot), the script appends the three persistence lines to `lan.conf` *only then*, via a `grep -qE '^\s*persistence_location' /etc/mosquitto/mosquitto.conf` guard. So it's correct whether or not the stock conf already persists — and never duplicates.

Self-diagnosing verify step: the "Verify mosquitto active" check now, on failure, dumps `mosquitto -c … -v` parse output + the last 8 journal lines before calling `fail` , so the *next* forker sees the real error instead of a bare "not active".

Docs

- `REBUILD_PI.md` Step 3 trimmed to the 2-line `lan.conf` + a callout box explaining the duplicate-key trap, plus the `mosquitto -c … -v` diagnostic one-liner; new row in the "Common failure modes" table for the `status=3` symptom.

Immediate unblock (given to the forker)

```
sudo tee /etc/mosquitto/conf.d/lan.conf >/dev/null <<'EOF'
# VU2CPL shack broker — LAN-only, no auth
listener 1883
allow_anonymous true
EOF
sudo systemctl restart mosquitto
```

No `flows.json` change.

2026-06-19 — SPE Tune: front-panel TUNE-LED indicator on both dashboards (follow-up #34)

Surfaced the SPE amplifier's front-panel **TUNE LED** on both the D1 `/ui` SPE Panel and the Vue `/shack` SPECard, so the operator can see when the amp is *in TUNE mode* (waiting for carrier / ATU sweeping) without looking at the rig.

Gateway side was already done (no spe-remote change needed). `spe-remote` decodes the LED from the RCU LCD frame — `serial_handler.py:639` `self._last_tune_active = (payload[4] & 0x40) == 0` (byte 4 bit 6; CLEAR = LED on) — stamps `state.tune_active` on every CSV state broadcast (`serial_handler.py:577`), and RCU mode is auto-enabled at connect + kept alive by `_rcu_tick_loop`. So `tune_active` (bool) already ships in the WS state JSON to every client. This change is **Node-RED + Vue only**.

Data path: `ws_format_state` builds one flat payload consumed by *both* UIs — D1 `ws_panel_node` reads it directly; `vue_spe_bridge_01` forwards the same object to the Vue card (which `Object.assign`s it into `state`). So one new field feeds both.

Changes (flows.json + uibuilder)

- **`ws_format_state`** (flows.json): emit `tune: !!d.tune_active` in the panel payload.
- **`ws_panel_node`** (D1 `/ui`): the existing `Tune` button gains `id="spew-tune-btn"` + an inline `●` LED (`#spew-tune-led`). Render handler lights the LED amber (`var(--gh-amber)`) with a glow + an amber inset box-shadow on the button while `d.tune` is true; dims to `#30363d` otherwise. The amp-gone (`!d.usb`) wipe block resets the LED so it can't stick on after the heartbeat drops (heartbeat-down msgs don't carry `tune`).
- **Vue SPECard** (`uibuilder/shack/src/index.js`): added `tune:false` to reactive state; the amber `Tune` button glows + shows a `●` while `state.tune` ; the **collapsed card header** shows a `⚡ TUNE` chip so it's visible without expanding. Build stamp → `v12 · 2026-06-19 SPE tune LED` , `index.html` cache-buster `?v=11` → `?v=12` .

The "make Tune a button" half of #34 was already satisfied (both UIs were `<button>` s); the LED is the new work.

Verification: flows.json round-trips byte-identical except the two node strings; `node --check` clean on index.js. **On-amp confirmation still pending** — needs a real TUNE cycle (button → low-power carrier) to watch the LED light amber on both UIs and clear when tuning ends. #34 stays open until that's seen on the bench/live amp.

Same-day revision (button colour, not glow). The first cut left the Tune button **permanently amber** (`btn--amber` / `gh-btn-a`) and only toggled a box-shadow — visually a no-op, so it read as "no tune indication" on test. Revised so the button's **fill colour itself toggles** to mirror the amp's TUNE LED: neutral when the LED is off (D1 `gh-btn`; Vue `btn--blue`), solid amber + glow + a `●` when it's on (D1 sets `background:var(--gh-amber)` with dark text; Vue swaps to `btn--amber`). The amp-gone (`!d.usb`) wipe and the Vue idle state both return the button to neutral. Verified by WS probe that the gateway sends `tune_active=false` while the amp is in Standby/RX (correct). Build stamp → `v13`, cache-buster `?v=13`.

Confirmed working on-amp 2026-06-19 — operator ran a real TUNE cycle; the button goes amber on both UIs while the front-panel TUNE LED is lit and returns to neutral when it clears. **Follow-up #34 closed.**

Confirm-dialog copy fix (`v14`). The Vue `confirmTune()` dialog said "The amp will transmit a low-power tuning carrier for a few seconds" — which is wrong: the amp doesn't key itself, the *operator* sends the carrier so the ATU can tune. Reworded to "Transmit a low-power tuning carrier within a few seconds to start tuning." Build stamp → `v14`, cache-buster `?v=14`.

Standard Commit Sequence (reminder)

Updated 2026-07-13 — the DXCC Tracker tab extract (`clublog_dxcc_tracker_v7.json`) referenced here was retired 2026-07-01 (CLAUDE.md rule #4, kept its number, retired in place); `nrsave` on the Pi now just stages + commits `flows.json` directly, no separate extract step. Current sequence, after a Deploy in the Node-RED editor:

```
# On the Pi:
nrsave "description of change"
git push
```

If a `flows.json` change needs to be made from the Mac instead (rare — the browser editor + `nrsave` on the Pi is the normal path for flow edits):

```
cd ~/.node-red/projects/vu2cpl-shack   # on the Pi, or the Mac's local clone
git add flows.json
git commit -m "<description>"
git push
```

Then on the other machine:

```
cd ~/.node-red/projects/vu2cpl-shack   # Pi
# or ~/projects/vu2cpl-shack           # Mac
git pull
sudo systemctl restart nodered         # Pi only, always after a flows.json change
```